
Java avancé

(avec communications réseau
et persistance)

Table des matières

I - Essentiel sur le multi-threading.....	7
1. Concept de threads.....	7
2. Gestion des threads avec java.....	8
3. Synchronisation des threads.....	10
4. Attente et rendez-vous (wait & notify) java 1.0.....	12
5. Concurrent api (Future, ExecutorService).....	14
1. Concurrency (Lock, Semaphore, ...).....	22
X - Rappels sur lambdas et streams.....	40
1. Java multi-paradigme "objet + fonctionnel"	40
2. Lambda expressions et prog. fonctionnelle.....	41
3. Streams (java 8 et plus).....	50
XI - MultiThreading et asynchronisme.....	54
1. ForkJoin.....	54

2. CompletableFuture , streams asynchrones.....	63
3. Reactive Streams (depuis java 9).....	70

XII - Fondamentaux réseau et sockets.....78

1. Rappels fondamentaux (réseaux).....	78
2. Api des sockets originelle (langage c).....	79
3. Vue d'ensemble sur les sockets java.....	82
4. Socket du coté Serveur.....	83
5. Socket du coté client.....	84
6. DatagramSocket en mode non connecté/UDP.....	85
7. Transfert de données binaires.....	86

XIII - Application réseau (patterns , ...).....88

1. Patterns de communication.....	88
2. Gestion des erreurs réseaux.....	90
3. Sécurité réseau.....	91
4. Performance et optimisation (réseau).....	91

XIV - Remote Procedure Call en java (RMI).....92

1. Présentation api RMI (Remote Method Invocation).....	92
2. Code type d'une api rmi (interface).....	94
3. Code type de l'application serveur.....	94
4. Code type de l'application cliente (Rmi-JRMP).....	96
5. RMI over IIOP (comme "EJB Remote").....	98

XV - Mapping Objet-Relationnel (ORM) et JPA.....100

1. Problématique "O.R.M.".....	100
2. JPA (Java Persistence Api).....	101

XVI - Architecture d'Hibernate (historique,évolution).....103

1. Hibernate: ancienne architecture & configuration.....	103
2. Hibernate en tant qu'implémentation de JPA>=2.....	105

XVII - JPA : architecture & configuration.....106

1. Présentation de JPA (Java Persistence Api).....	106
2. Unité de Persistance (META-INF/persistence.xml).....	106
3. JPA3 et JarkartaEE.....	110
4. Configuration JPA/Hibernate sans spring ni jakartaEE.....	111

5. Configuration du mapping JPA via annotations.....	114
XVIII - EntityManager et entités persistantes.....	116
1. Entity Manager et son contexte de persistance.....	116
2. Transaction JPA.....	118
3. Différents états - objet potentiellement persistant.....	119
4. Cycle de vie d'un objet JPA/Hibernate.....	119
5. Synchronisation automatique dans l'état persistant.....	120
6. objet persistant et architecture n-tiers.....	120
7. Principales méthodes JPA / EntityManager et Query.....	121
8. Contexte de persistance et proxy-ing.....	122
XIX - Langage de requêtes JPQL.....	124
XX - O.R.M. JPA (généralités).....	133
1. Vue d'ensemble sur les entités, valeurs et relations.....	133
2. Identité d'une entité (clef primaire).....	135
3. Propriétés d'une colonne.....	137
4. Relations (1-1, n-1, 1-n et n-n).....	137
XXI - O.R.M. JPA – détails (1-n , 1-1 , n-n, ...).....	139
2. Relations d'héritage & polymorphisme.....	149
XXII - JPA : aspects divers et avancés.....	153
1. Quelques spécificités de JPA.....	153
2. EntityGraph (depuis JPA 2.1).....	156
3. Cycle de persistance (pour @Entity) et annotations/callbacks associées pour "Listener".....	158
4. Annotations diverses de JPA.....	159
5. Variantes pour le mapping JPA.....	161
6. Précautions à prendre pour ne pas trop détériorer les performances.....	162
7. Map et apports de JPA 2.0.....	166
8. Api "Criteria" de JPA 2.....	171
XXIV - Cache niveau 2 Hibernate (JPA/SpringBoot).....	175
1. Caches de niveau 1 et 2 (Jpa/Hibernate).....	175
2. Cache de niveau 2 Hibernate.....	176

XXV - Logs , Tests unitaires , JUnit,	182
1. TDD (Test Driven Development).....	182
2. Tests unitaires avec JUnit 5.....	182
XXVI - Profiling JVM, gestion mémoire ,	200
1. JMX (<i>Java Management Extensions</i>).....	200
2. Présentation de HotSpot JVM.....	203
3. Vue extrêmement simplifiée du "Garbage Collector"	203
4. Générations.....	205
5. Indicateurs de performances et mesures.....	207
6. Dimensionnement du tas global (total java heap).....	208
7. Paramétrage (tuning) du comportement recherché.....	209
8. Processus d'optimisation (tuning).....	212
9. Outils pour l'optimisation des exécutions java.....	212
10. Essentiel de Java VisualVM.....	213
11. Détection d'une fuite de mémoire.....	213
XXVII - Annexe – Tests avancés (JaCoCo, Mockito,...). .	219
1. Analyse de la couverture des tests.....	219
2. Positionnement et intérêts des "mocks"	221
3. Mockito.....	222
4. Présentation de JMeter.....	230
5. Configuration et démarrage du produit JMeter.....	232
6. Configuration élémentaire d'un plan de test JMeter.....	233
7. Très important: tester tout d'abord le test !!!.....	237
8. Gestion correcte des sessions HTTP (cookies).....	238
9. Successions de requêtes http liées entre elles.....	238
10. Configurations diverses.....	241
XXVIII - Introspection, annotations ,	245
1. Rappels sur énumérations.....	245
2. Introspection (java.lang.reflect).....	245
3. Annotations : Présentation et intérêts.....	250
4. Annotations et méta-annotations prédéfinies.....	250
5. Création de nouvelles annotations.....	252
6. Insertion d'annotations au sein d'un code source.....	253
7. Accès aux annotations (de rétention RUNTIME) via l'introspection de java >=5	253

8. Analyse et traitement des annotations de rétention « SOURCE ».....	255
XXIX - Switch expression, pattern matching, record.....	261
1. Switch expressions.....	261
2. Pattern Matching instanceof.....	263
3. TextBloc.....	264
4. Record (classe de données simplifiée pour DTO).....	266
5. Sealed classes.....	272
6. Pattern matching sur switch/case of object.....	274
7. utilitaire jpackage.....	275
8. Autres apports des versions récentes (12,...,17).....	276
9. Nouveau type de licence depuis java 17.....	276
XXX - ClassLoader , weak references , modules.....	277
1. ClassLoader.....	277
2. WeakReference et SoftReference.....	283
3. Modules java 9+ (essentiel).....	287
XXXI - Threads virtuels , etc.....	304
1. Pattern and record matching.....	304
2. Virtual Thread.....	306
3. Structured Concurrency (preview java 21).....	310
4. Foreign Function and Memory (preview java 21).....	313
5. Vector api (incubator java 21).....	314
XXXII - Annexe – JShell.....	316
1. JShell (RPEL java).....	316
XXXIII - Annexe – Optional , DateTime,	321
1. Optional , DateTime ,	321
XXXIV - Annexe – Lombok.....	326
1. Lombok.....	326
XXXV - Annexe – MapStruct.....	329
1. MapStruct (efficient Dto converter).....	329
XXXVI - Annexe – NIO.....	333

1. NIO2 (new IO).....	333
2. Principales classes et interfaces de nio2.....	333
3. Gestion des chemins (Path).....	335
4. Classe utilitaire Files (" <i>helper</i> " avec 50 méthodes statiques).....	339
5. Parcours des éléments d'un répertoire.....	341
6. Parcours d'une hiérarchie de répertoires (visiteur).....	343
7. FileSystem (par défaut et "personnalisé").....	343
8. Lecture et écriture dans un fichier.....	344

XXXVII - Process Api, try with resource,Http2 api,346

1. Process Api.....	346
2. try with resources (AutoClosable).....	349
3. Http2 Client.....	351

XXXVIII - Communication par messages / JMS.....356

1. Présentation de JMS.....	356
2. Queue & Topic.....	358
3. Exemples (fragments) de code en JMS 1.1.....	359
4. Champs des entêtes de message.....	362
5. Acquittement des messages reçus.....	362
6. Liste des principaux "Provider JMS".....	363
7. ActiveMQ / Artemis.....	364

XXXIX - Annexe – énoncés des Tps (java avancé).....368

1. TP sur les threads (java avancé).....	368
2. TP sur les communications réseau (java avancé).....	371
3. TP sur la persistance (JPA/Hibernate) (java avancé).....	373
4. TP sur coeur langage java (fonctionnalités avancés).....	378
5. TP sur fonctionnalités récentes (java avancé).....	379
6. TP sur Tests,Mock,Profiling,... (java avancé).....	383

I - Essentiel sur le multi-threading

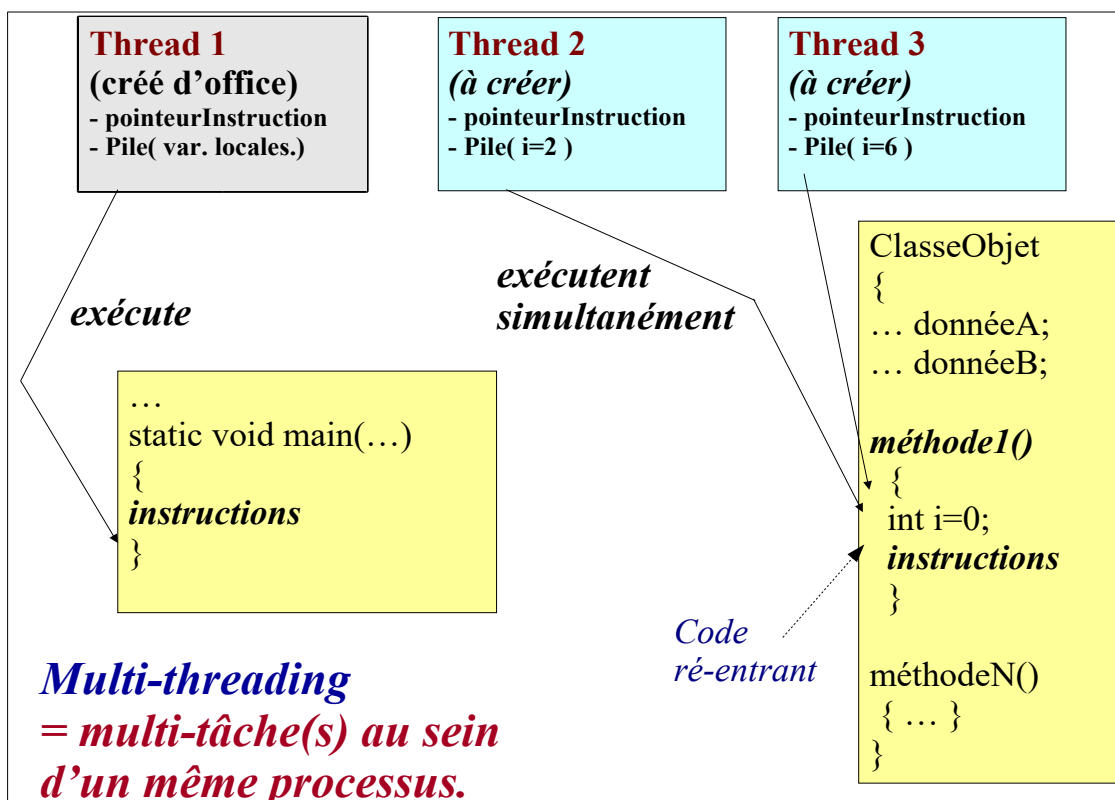
1. Concept de threads

Concept de Thread

Un **thread** est un **fil d'exécution**. Il s'agit d'une **entité dynamique qui peut exécuter un ensemble d'instructions**. L'association de *plusieurs threads* permet de mettre en œuvre **des traitements parallèles au sein d'un même processus**.

Sous Java, la gestion des threads fait intervenir trois entités fondamentales:

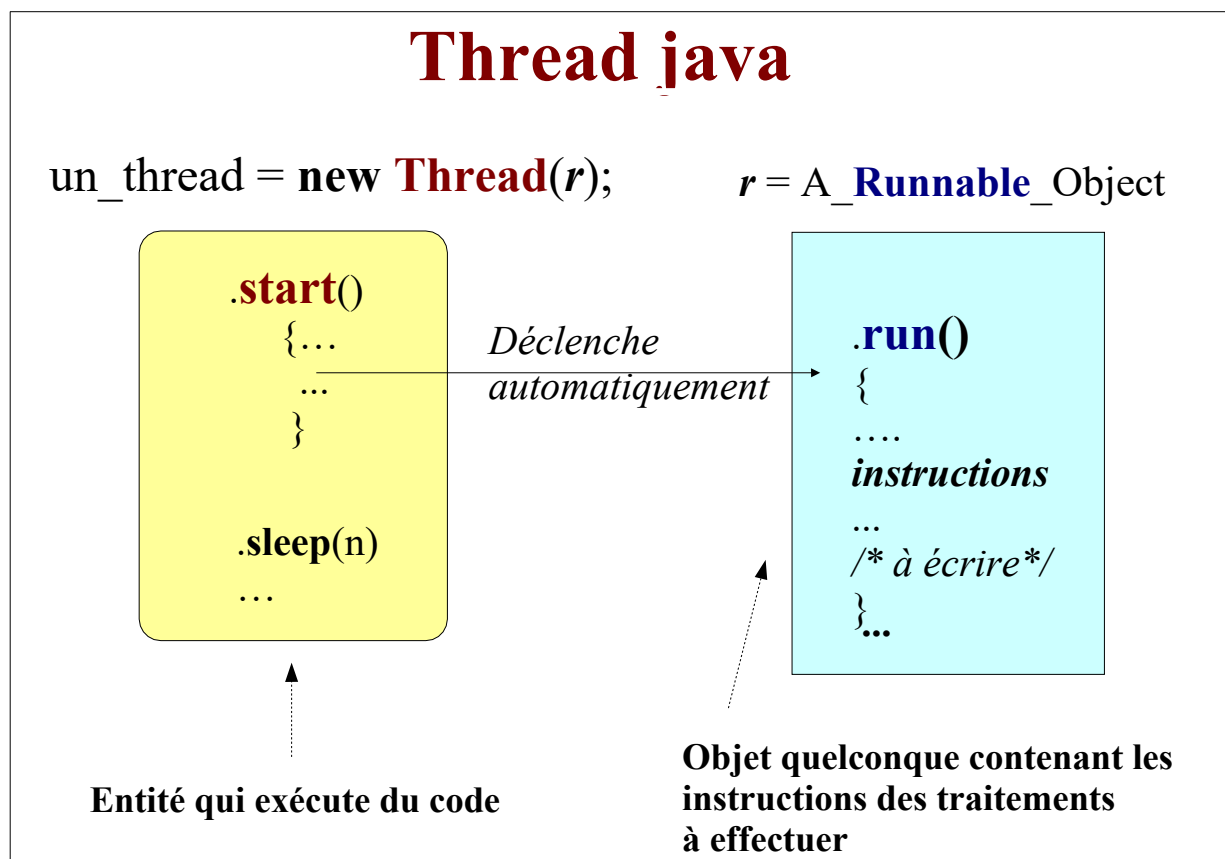
- Le **moniteur de threads** (gestionnaire central et caché dans les API de JAVA qui *gère les exécutions concurrentes des différents threads*).
- Une instance de la classe **Thread** (*entité qui exécute du code* et que l'on peut lancer, arrêter,...)
- Un objet implémentant l'interface **Runnable** et dont la méthode **run** correspond au point d'entrée de *l'ensemble des instructions à exécuter* par un nouveau thread.



Remarque: Il existe toujours au moins un thread au sein d'une application ou applet JAVA. Ce thread principal est créé automatiquement, il s'occupe de la gestion des événements associés à l'interface graphique ou bien de l'enchaînement des instructions partant de la méthode statique `main()` en mode texte. Si on fait exécuter par ce thread principal et implicite des traitements longs (boucle infinie, accès réseau, calculs, ...), on risque alors de bloquer la gestion de l'interface graphique et l'utilisateur aura alors l'impression que le programme ne répond plus.

Il est donc nécessaire de créer un ou plusieurs threads supplémentaires dès que l'on veut effectuer des opérations qui peuvent être longues ou bloquantes (Animations, Accès réseau, ...).

2. Gestion des threads avec java



La méthode statique `sleep()` est souvent employée dans les animations, elle permet d'insérer des temporisations entre deux affichages d'image par exemple.

```
try{
    Thread.sleep(n); //méthode statique qui endort Thread.currentThread()
} catch(InterruptedException ex) { ex.printStackTrace(); }
```

NB (Alternative) : Au lieu de créer une classe implémentant l'interface Runnable, on peut également sous classer la classe Thread et programmer directement la méthode `run()` dans cette sous classe (Thread spécifique).

Exemple de code: (comptage croissant et/ou décroissant)

```

package tp.basic;

public class CountTask implements Runnable {
    int from;
    int to;

    CountTask(int from,int to){
        this.from=from; this.to=to;
    }
    CountTask(){
        this(1,5);
    }

    public void display_with_pause(int i) {
        System.out.println(">>> i=" + i + " - " + Thread.currentThread().getName());
        try {
            Thread.sleep(1000); //1000ms de pause
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public void run() {
        if(this.from <= this.to){
            for(int i=from;i<=to;i++){
                display_with_pause(i);
            }
        } else {
            for(int i=from;i>=to;i--){
                display_with_pause(i);
            }
        }
    }
}

```

```

package tp.basic;

public class BasicThreadExample {
    public static void main(String[] args) {
        System.out.println("start of main() executed by " + Thread.currentThread().getName());
        CountTask countTask = new CountTask(0,5);
        CountTask countDownTask = new CountTask(5,0);
        Thread t1= new Thread(countTask); t1.setName("countUpThread");
        Thread t2= new Thread(countDownTask); t2.setName("countDownThread");
        t1.start(); t2.start();
        System.out.println("end of main() executed by " + Thread.currentThread().getName());
    }
}

```

```
}
}
```

⇒ résultats :

start of main() executed by main

end of main() executed by main

>>> *i=5 - countDownThread*

>>> *i=0 - countUpThread*

>>> *i=4 - countDownThread*

>>> *i=1 - countUpThread*

>>> *i=3 - countDownThread*

>>> *i=2 - countUpThread*

>>> *i=3 - countUpThread*

>>> *i=2 - countDownThread*

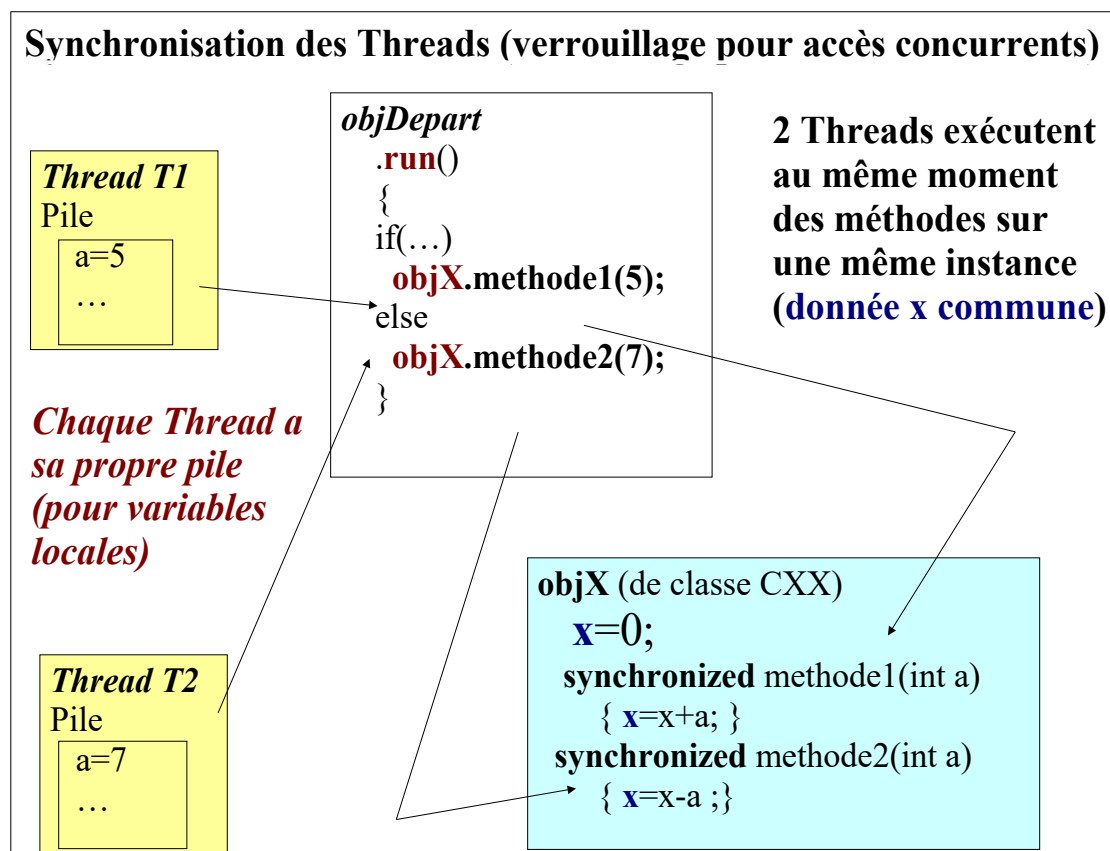
>>> *i=4 - countUpThread*

>>> *i=1 - countDownThread*

>>> *i=0 - countDownThread*

>>> *i=5 - countUpThread*

3. Synchronisation des threads



Accès concurrents / synchronisation des threads

Dans certains cas , il se peut que *plusieurs threads* puissent *exécuter simultanément différentes méthodes sur une même instance et donc manipuler les mêmes données internes de l'objet*.

Pour que toutes ces opérations puissent s'effectuer sans interférence, il faut temporairement verrouiller ces données pour qu'un seul thread puisse les modifier à la fois.

Le langage java a choisi de gérer cette synchronisation via un mot clef **synchronized** qui *permet au moniteur central de la machine virtuelle de gérer la concurrence entre les threads* :

Dès qu'un thread exécute une méthode synchronisée sur un objet java, tous les autres threads qui souhaitent exécuter une (même ou autre) méthode synchronisée sur la même instance seront automatiquement bloqués (mis en attente).

Le mot clef **synchronized** place donc une sorte de **verrou** sur un **objet complet** (pour protéger l'accès aux données internes).

Syntaxe:

Lorsque le mot clef **synchronized** est utilisé pour préfixer une méthode , l'objet (this) de la classe courante est alors automatiquement verrouillé durant tout le temps d'exécution de la méthode:

```
public class Cx {
protected int x=0;
synchronized public void methode1(int a)
{
    x=x+a;
    ...
}
...
}
```

Cas particulier des méthodes statiques:

Lorsque le mot clef **synchronized** est placé devant une méthode "**static**" , le verrou porte sur les membres statiques de la classe .

Lorsque la syntaxe **synchronized(Object) { ... }** est utilisée à l'intérieur d'une méthode , seul l'objet (ou le sous objet) précisé est verrouillé et la durée du verrou correspond au temps nécessaire à un thread pour exécuter le bloc d'instructions délimité par les accolades :

```
public class Cy {
```

```

protected CPartie objPartX = new CPartie();
protected int y=0;

public void methodeOrdinaire(...)
{
    ...
    synchronized( objPartX )
    {
        objPartX.x++;
        ...
    }
    ...
    synchronized( this )
    {
        this.y++;
        ...
    }
}

```

4. Attente et rendez-vous (wait & notify) java 1.0

4.1. mécanismes

La classe **Object** qui est la racine de toutes les autres classes de Java comporte (entre autres) les fonctions **wait(timeout)**, **notify()** et **notifyAll()**.

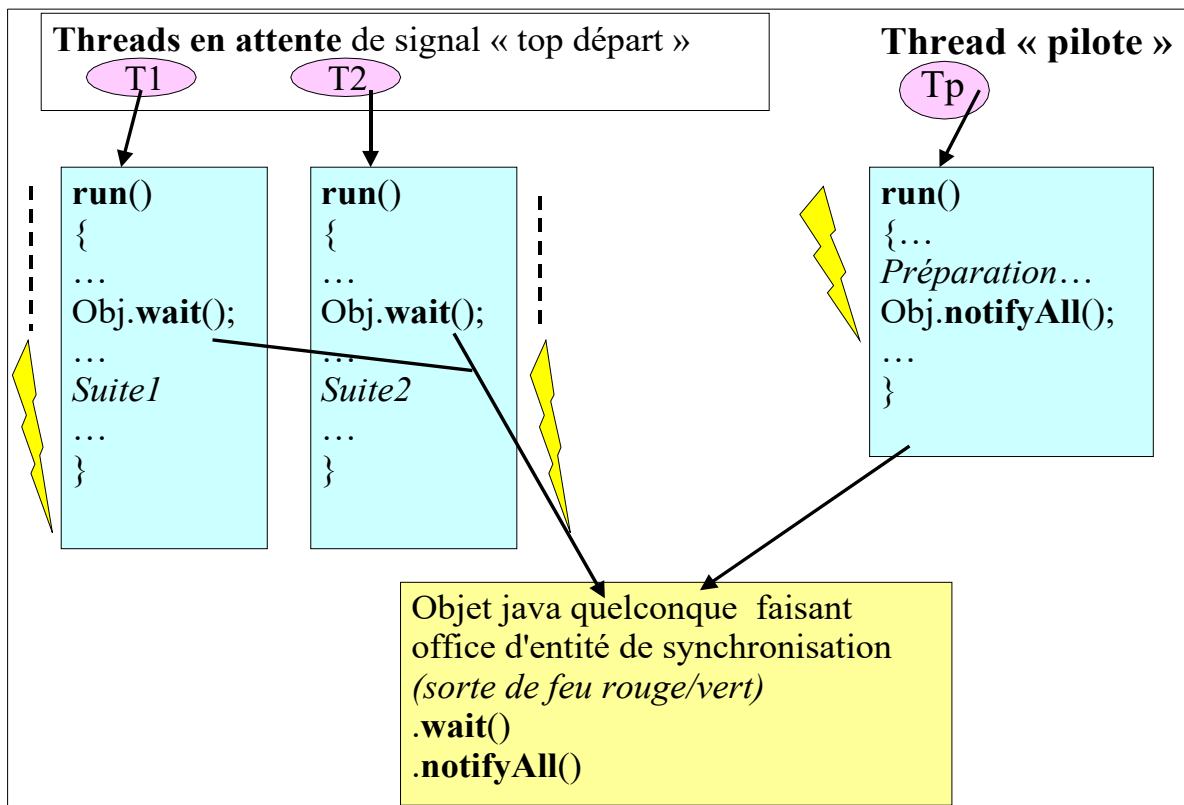
NB: Les méthodes **wait()** et **notify()** ne peuvent être appelées sur une instance que si elles sont placées dans un bloc "**synchronized**".

Comportement induit:

Un thread qui appelle la méthode **wait(timeout)** sur une instance sera mis en sommeil (pas de temps CPU consommé) jusqu'à ce que:

- La durée du **timeout** (en ms) ait expirée.
ou bien
- Un autre thread a appelé la méthode **notify()** sur le même objet.
ou bien
- Le thread en question a été interrompu (**InterruptedException**).

NB: Lorsqu'un thread appelle la méthode **notify()** sur une instance, Le superviseur des threads ne réveille alors qu'un seul autre thread (qui avait appelé wait()). La méthode **notifyAll()** permet de réveiller d'un coup tous les threads qui ont appelé la méthode wait() sur cette même instance.



4.2. Exemple concret ==> Gestion d'un pool de ressources

`getRessource()` avec un `wait()` sur un sous objet "Attente" s'il faut attendre une ressource disponible
`libérerRessource()` avec un `notify()` sur un sous objet "Attente" pour débloquer une seule des attentes

4.3. Autres aspects avancés sur les threads

- On peut donner un nom explicite (chaîne de caractères) à un Thread. Celui-ci peut être passé au niveau du constructeur.
- `join()` de la classe `Thread` permet d'attendre la fin de l'exécution d'un thread.
- Les Threads peuvent être organisés en groupe(s) et sous groupe(s).
- On peut modifier la **priorité** d'un thread : `un_thread.setPriority(n);`
- La classe **ThreadLocal** permet de gérer des variables qui sont accessibles globalement par une grande partie du code source et qui sont locales vis à vis des Threads (chaque Thread a sa propre valeur).

Le **jdk 1.5** a apporté un bon nombre de nouvelles fonctionnalités pointues sur les threads (ex: **sémaphore** du package `java.util.concurrent`).

5. Concurrent api (Future, ExecutorService)

5.1. interface Runnable (java.lang)

Depuis java 8 , l'interface fondamentale Runnable est devenue une interface (uni-)fonctionnelle.

```
public interface Runnable {  
    public void run();  
}
```

```
Runnable r = () -> { /*code de la tâche*/ }
```

5.2. interface Callable<T> (java.util.concurrent)

```
public interface Callable<T> {  
    public T call();  
}
```

```
Callable<T> c = () -> { /*corps de la tâche*/ return t; }
```

Callable<T> existe depuis le jdk 1.5 et est depuis java 1.8 vue comme une interface (uni-)fonctionnelle .

II - interface Future<T> (java.util.concurrent)

Disponible depuis le jdk 1.5, un objet **Future<T>** en **java** correspond partiellement à la notion de **Promise** en *javascript* .

Un objet technique de ce type (*recupéré immédiatement lors d'un lancement de traitement long*) **permettra de récupérer un résultat en différé** (dans le futur) .

boolean isDone()	teste la terminaison du thread pour savoir si la donnée résultante de son exécution est disponible. Retourne true si ce thread s'est bien terminé ou s'il a malheureusement levé une exception pendant son exécution, ou enfin s'il a été suspendu.
T get()	retourne le résultat (instance de T) de la tâche exécutée par le thread. Si le thread n'a pas terminé son exécution, alors l'appel de cette méthode est bloqué en attente active jusqu'à ce qu'il termine.
boolean cancel(true)	Demande à arrêter la tâche si elle n'est pas finie , le paramètre d'entrée "mayInterruptIfRunning" est souvent fixé à true . La valeur de retour (souvent ignorée) est à true si tâche interrompue ou false si tâche déjà terminée .
boolean isCancelled()	renvoi true si tâche "interrompue" avant la fin .
T get(long timeout, TimeUnit unit)	variante avec timeout de la méthode .get() Si par exemple après un timeout de 1500 TimeUnit.MILLISECONDS la tâche n'est toujours pas finie , cette méthode remonte une exception de type TimeoutException à rattraper via un try/catch pour nous signaler que cette tâche n'est pas terminée .

III - ExecutorService

Au coeur du package `java.util.concurrent` l'interface **ExecutorService** comporte les principales méthodes suivantes :

<code>void execute(Runnable command)</code>	lance l'exécution (via un thread créé ou disponible) d'une tâche de type <code>Runnable</code> , ne retourne rien.
<code>Future<T> submit(Callable<T> task)</code>	lance l'exécution d'une tâche de type <code>Callable<T></code> , retourne un objet de type <code>Future<T></code> permettant de récupérer ultérieurement (en différé) un résultat de type <code>T</code> .
<code>Future<?> submit(Runnable task)</code>	Comme <code>execute()</code> mais retournant <code>Future<?></code> pour attente du résultat ou de la fin
<code>T invokeAny(Collection<? extends Callable<T>> tasks)</code> ou bien <code>T invokeAny(... tasks, long timeout, ...)</code>	Lance (via des threads) une liste de tâches et attend <u>en mode bloquant</u> la première réponse , les autres tâches moins rapides sont automatiquement annulées/stoppées.
<code>List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)</code> <i>existe en version avec timeout</i>	lance <u>en mode bloquant</u> plein de tâches et récupère une liste de résultats encapsulés dans des <code>Future<T></code> quand tout est prêt/fini .
Autres méthodes	<code>shutdown()</code> , <code>awaitTermination(timeout,...)</code> , ...

Une instance d'une classe implémentant l'interface `ExecutorService` pourra être créée via

```
ExecutorService executor = Executors.newSingleThreadExecutor();
```

ou bien

```
ExecutorService executor = Executors.newFixedThreadPool(3/*nThreads*/)
```

ou bien

```
ExecutorService executor = Executors.newCachedThreadPool();
```

ou bien d'autres façon encore .

<code>newSingleThreadExecutor()</code> ;	Un seul nouveau thread pouvant lancer plusieurs tâches alors exécutées séquentiellement les unes après les autres .
<code>newFixedThreadPool(3/*nThreads*/)</code>	via un pool de threads (en //) de taille maxi à paramétrer (si plus de tâches à exécuter que de threads dispos --> attente automatique via queue) . chaque thread ne sera arrêté que si appel explicite à <code>.shutdown()</code> .
<code>newCachedThreadPool()</code>;	via un pool de threads dont la taille est automatiquement ajustée en fonction des besoins (à la hausse ou à la baisse si rien à faire durant 60s)
autres	voir javadoc Executors

1.1. Exemple "EssaiExecutors"

MyRunnableCode.java

```
package tp.langage.thread;

public class MyRunnableCode implements Runnable {
    private String prefix;

    public MyRunnableCode() {super(); this.prefix = ""; }
    public MyRunnableCode(String prefix) { super(); this.prefix = prefix; }

    @Override
    public void run() {
        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {}
        System.out.println(prefix + Thread.currentThread().getName());
    }
}
```

EssaiExecutors.java

```
package tp.langage.thread;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

//ExecutorService depuis jdk 1.5
public class EssaiExecutors {

    // Un "ExecutorService" (à fabriquer via Executors.new...Executor()) démarre automatiquement des
    // Threads (rangés dans des "pools") pour exécuter des instances de Callable<T> ou de Runnable
    public static void main(String[] args) {

        MyRunnableCode myRunnableCode = new MyRunnableCode("");

        ExecutorService singleThreadExecutor = Executors.newSingleThreadExecutor();
                                                    //exécution séquentielle en background

        singleThreadExecutor.submit(myRunnableCode);
        singleThreadExecutor.submit(myRunnableCode);
        singleThreadExecutor.submit(myRunnableCode);
        singleThreadExecutor.shutdown();//automatiquement différé
    }
}
```

```
MyRunnableCode myRunnableCode2 = new MyRunnableCode("#");

ExecutorService multiThreadExecutor = Executors.newFixedThreadPool(3);
//exécutions multiples (en //) en background

multiThreadExecutor.submit(myRunnableCode2);
multiThreadExecutor.submit(myRunnableCode2);
multiThreadExecutor.submit(myRunnableCode2);
multiThreadExecutor.shutdown();//automatiquement différé , .shutdownNow() pour arrêt immédiat

MyRunnableCode myRunnableCode3 = new MyRunnableCode("@");

ScheduledExecutorService scheduleExecutor =
    Executors.newSingleThreadScheduledExecutor();
System.out.println("Lancement d'un thread/tâche (@) en différé (2000ms)");
scheduleExecutor.schedule(myRunnableCode3, 2000, TimeUnit.MILLISECONDS);
scheduleExecutor.shutdown();//automatiquement différé , .shutdownNow() pour arrêt immédiat
}
}
```

Résultats :

```
Lancement d'un thread (@) en différé (2000ms)
#pool-2-thread-3
#pool-2-thread-2
*pool-1-thread-1
#pool-2-thread-1
*pool-1-thread-1
*pool-1-thread-1
@pool-3-thread-1
```

IV - Exemple "TestFuture" (avec Executors et Callable<T>)

LongTask.java

```
package tp.langage.thread;

public class LongTask {

    public static void printThread() {
        System.out.println(Thread.currentThread().getName());
    }

    public static void simulateLongTask(String msg, long nbMs) {
        try {
            System.out.println(">>(begin)" + msg + " / by " + Thread.currentThread().getName());
            Thread.sleep(nbMs);
            System.out.printlnprintln("** interrupted **");
        }
    }
}
```

CallableComputing.java

```
package tp.langage.thread;
import java.util.concurrent.Callable;

public class CallableComputing implements Callable<String> {
    private double x;

    @Override
    public String call() throws Exception {
        LongTask.simulateLongTask("long computing task (in background) ...", 5000);
        return String.valueOf(Math.sqrt(x));
    }

    public CallableComputing() {super(); this.x = 0; }
    public CallableComputing(double x) {super(); this.x = x; }

    public double getX() { return x; }
    public void setX(double x) {this.x = x; }
}
```

EssaiFuture.java

```

package tp.langage.thread;

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public class EssaisFuture {

    public static void main(String[] args) {
        //NB: Callable<T>/call() ressemble un peu à l'interface Running/run()
        //mais permet de récupérer (ultérieurement) un résultat via Future<T> .
        Callable<String> c = new CallableComputing(9);
        String result=null;

        ExecutorService executor = Executors.newSingleThreadExecutor();

        Future<String> futureRes = executor.submit(c);
        while(result==null){
            LongTask.simulateLongTask("other works ...",2000);
            if(futureRes.isDone()){
                try {
                    result = futureRes.get();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } catch (ExecutionException e) {
                    e.printStackTrace();
                }
            }
        }
        System.out.println("result=" + result);

        System.out.println("-----");
        Future<String> futureRes2 = executor.submit(c);
        LongTask.simulateLongTask("other works ...",2000);
        if(futureRes2.isDone()){
            try {
                result = futureRes2.get();
                System.out.println("result2=" + result);
            } catch (InterruptedException e) {
                e.printStackTrace();
            } catch (ExecutionException e) {
                e.printStackTrace();
            }
        }
        else {

```

```

    futureRes2.cancel(true);
    if(futureRes2.isCancelled()){
        System.out.println("background computing was cancelled");
    }
}
System.out.println("-----");
Future<String> futureRes3 = executor.submit(c);
LongTask.simulateLongTask("other works ...",2000);
try {
    result = futureRes3.get(1500,TimeUnit.MILLISECONDS);
    System.out.println("result3=" + result);
} catch (InterruptedException e) {
    e.printStackTrace();
} catch (ExecutionException e) {
    e.printStackTrace();
} catch (TimeoutException e) {
    System.err.println("tâche 3 toujours pas terminée au bout de 1500ms");
    System.out.println("after 1500ms,futureRes3.isDone()="+futureRes3.isDone());
    System.out.println("after 1500ms,futureRes3.isCancelled()="+futureRes3.isCancelled());
    //e.printStackTrace(); }
}
    executor.shutdown();//automatiquement différé , .shutdownNow() pour arrêt immédiat
}

```

Résultats :

```

>>(begin)other works ... / by main
>>(begin)long computing task (in background) ... / by pool-1-thread-1
<<(end)other works ... / by main
>>(begin)other works ... / by main
<<(end)other works ... / by main
>>(begin)other works ... / by main
<<(end)long computing task (in background) ... / by pool-1-thread-1
<<(end)other works ... / by main
result=3.0
-----
>>(begin)other works ... / by main
>>(begin)long computing task (in background) ... / by pool-1-thread-1
<<(end)other works ... / by main
** interrupted **
background computing was cancelled
-----
>>(begin)other works ... / by main
>>(begin)long computing task (in background) ... / by pool-1-thread-1
<<(end)other works ... / by main
after 1500ms,futureRes3.isDone()==false
after 1500ms,futureRes3.isCancelled()==false
tâche 3 toujours pas terminée au bout de 1500ms
<<(end)long computing task (in background) ... / by pool-1-thread-1

```

1. Concurrence (Lock, Semaphore, ...)

V - Verrous (Lock)

Un verrou (Lock) est un objet de synchronisation permettant à un thread d'effectuer un travail sans être gêné par d'autres threads .

Variantes de verrouillage :

`lock.lock()` ; //en mode bloquant

`boolean isLockAcquired = lock.tryLock(1, TimeUnit.SECONDS);`

Exemple :

```
package tp.concurrent.basic;
import java.util.concurrent.locks.ReentrantLock;

public class SharedObjectWithLock {
    public static SharedObjectWithLock uniqueInstance = new SharedObjectWithLock();

    ReentrantLock lock = new ReentrantLock();
    int counter = 0;

    public int incrementCounter(int delay,boolean withLock) {
        int currentCounter=0;
        if(withLock)
            lock.lock();
        try {
            // Critical section here :
            Thread.sleep(delay); //delay ms pause
            counter++;
            currentCounter=counter;
        }catch(InterruptedException ex){
            ex.printStackTrace();
        }
        finally {
            if(withLock)
                lock.unlock();
        }
        return currentCounter;
    }
}
```

```

package tp.concurrent.basic;
public class IncrementTask implements Runnable{
    int delay;
    IncrementTask(int delay){ this.delay=delay; }
    IncrementTask(){ this(500); }

    @Override
    public void run() {
        //boolean withLock = false;
        boolean withLock = true;
        for(int i=1;i<=10;i++){
            int currentIncrementedCounter =
                SharedObjectWithLock.uniqueInstance.incrementCounter(this.delay,withLock);
            System.out.println("currentIncrementedCounter="+currentIncrementedCounter
                + " in thread " + Thread.currentThread().getName());
        }
    }
}

```

```

package tp.concurrent.basic;
public class EssaiLock {
    public static void main(String[] args) {
        (new Thread(new IncrementTask(100))).start();
        (new Thread(new IncrementTask(30))).start();
        //NB: relancer cet exemple avec ou sans withLock = true au sein de IncrementTask.java
    }
}

```

<i>Résultats avec withLock=false</i>	<i>Résultats avec withLock=true</i>
currentIncrementedCounter=1 in thread Thread-1	currentIncrementedCounter=1 in thread Thread-0
currentIncrementedCounter=2 in thread Thread-1	currentIncrementedCounter=2 in thread Thread-1
currentIncrementedCounter=3 in thread Thread-1	currentIncrementedCounter=3 in thread Thread-0
currentIncrementedCounter=4 in thread Thread-0	currentIncrementedCounter=4 in thread Thread-1
currentIncrementedCounter=5 in thread Thread-1	currentIncrementedCounter=5 in thread Thread-0
currentIncrementedCounter=6 in thread Thread-1	currentIncrementedCounter=6 in thread Thread-1
currentIncrementedCounter=7 in thread Thread-1	currentIncrementedCounter=7 in thread Thread-0
currentIncrementedCounter=8 in thread Thread-0	currentIncrementedCounter=8 in thread Thread-1
currentIncrementedCounter=9 in thread Thread-1	currentIncrementedCounter=9 in thread Thread-0
currentIncrementedCounter=10 in thread Thread-1	currentIncrementedCounter=10 in thread Thread-1
currentIncrementedCounter=11 in thread Thread-1	currentIncrementedCounter=11 in thread Thread-0
currentIncrementedCounter=12 in thread Thread-0	currentIncrementedCounter=12 in thread Thread-1
currentIncrementedCounter=13 in thread Thread-1	currentIncrementedCounter=13 in thread Thread-0
currentIncrementedCounter=14 in thread Thread-0	currentIncrementedCounter=14 in thread Thread-1
currentIncrementedCounter=15 in thread Thread-0	currentIncrementedCounter=15 in thread Thread-0
currentIncrementedCounter=16 in thread Thread-0	currentIncrementedCounter=16 in thread Thread-1
currentIncrementedCounter=17 in thread Thread-0	currentIncrementedCounter=17 in thread Thread-0

currentIncrementedCounter=18 in thread Thread-0	currentIncrementedCounter=18 in thread Thread-1
currentIncrementedCounter=19 in thread Thread-0	currentIncrementedCounter=19 in thread Thread-0
currentIncrementedCounter=20 in thread Thread-0	currentIncrementedCounter=20 in thread Thread-1

NB: il existe si besoin ReadLock et WriteLock pour effectuer des lectures multiples et écriture unique :

Exemple partiel :

```
public class SynchronizedMapWithReadWriteLock {  
  
    Map<String,String> syncHashMap = new HashMap<>();  
    ReadWriteLock lock = new ReentrantReadWriteLock();  
  
    Lock writeLock = lock.writeLock();  
  
    public void put(String key, String value) {  
        try {  
            writeLock.lock();  
            syncHashMap.put(key, value);  
        } finally {  
            writeLock.unlock();  
        }  
    }  
  
    public String remove(String key){  
        try {  
            writeLock.lock();  
            return syncHashMap.remove(key);  
        } finally {  
            writeLock.unlock();  
        }  
    }  
  
    Lock readLock = lock.readLock();  
  
    public String get(String key){  
        try {  
            readLock.lock();  
            return syncHashMap.get(key);  
        } finally {  
            readLock.unlock();  
        }  
    }  
    ...  
}
```


}

NB : Il existe également **StampedLock** avec la possibilité d'effectuer des lectures optimistes (on tente une lecture rapide sans verrou et si ça ne marche pas on retente en verrouillant) :

Exemple partiel :

```
/*
StampedLock works as read/write lock with stamp/optimistic features
the returned "stamp" (as long) is a sort of lock version number
*/

public class SynchronizedMapWithStampedLock {

    Map<String,String> syncHashMap = new HashMap<>();
    StampedLock lock = new StampedLock();

    public void put(String key, String value) {
        long stamp = lock.writeLock();
        try {
            syncHashMap.put(key, value);
        } finally {
            lock.unlockWrite(stamp);
        }
    }

    public String readWithOptimisticLock(String key) {
        long stamp = lock.tryOptimisticRead();
        //The tryOptimisticRead() method returns a stamp
        // only if the lock is not currently held in write mode.

        String value = syncHashMap.get(key);

        if(!lock.validate(stamp)) {
            stamp = lock.readLock();
            try {
                return syncHashMap.get(key);
            } finally {
                lock.unlock(stamp);
            }
        }
        return value;
    }
}
```

VI - CyclicBarrier

Une barrière cyclique (**CyclicBarrier**) est un objet de synchronisation qui permet à un ensemble de threads de s'attendre mutuellement (les uns les autres) jusqu'à atteindre un point d'exécution appelé « barrière ». Cette barrière est qualifiée de cyclique car elle peut être réutilisée une fois que tous les threads ont été libérés (attente débloquée).

Au sein du constructeur de CyclicBarrier il faut préciser le nombre de threads devant se synchroniser. Et un des constructeurs surchargés permet d'enregistrer (sous forme de référence Runnable) une tâche à automatiquement déclencher une fois que tous les threads ont terminé leur travail et qu'ils ont tous appelé cyclicBarrier.await() .

Cette tâche de « poursuite du travail » une fois que toutes les préparations sont faites est exécutée par un des threads ayant appelé cyclicBarrier.await() , souvent le dernier .

Exemple (simulation d'une consolidation de chiffre d'affaire):

Trois threads vont récupérer des chiffres d'affaires partiels et vont stocker cela dans un tableau « partialSalesArray » .

Quand tout est prêt , la barrière cyclique sera atteinte et une tâche de calcul global sera alors automatiquement déclenchée :

```
package tp.concurrent.basic;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class CollaborativeWork {
    //ex: stats consolidées (sur chiffres d'affaires) à partir de 3 sources complémentaires
    private int NB_SOURCES=3;
    private double[] partialSalesArray=new double[NB_SOURCES];
    private double totalSales =0;
    private CyclicBarrier cyclicBarrier = null;

    public class PartialSourceTask implements Runnable{
        int sourceIndex;

        PartialSourceTask(int sourceIndex){
            this.sourceIndex=sourceIndex;
        }

        @Override
        public void run() {
            double partialSales=Math.random()*10000;
            int delay = (int) (Math.random()*2000);
            System.out.println("at index=" + sourceIndex + " partialSales =" +partialSales + " by "
                + Thread.currentThread().getName() + " with random delay="+delay);
            try { Thread.sleep(delay); } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

```

partialSalesArray[sourceIndex]=partialSales;
try {
    cyclicBarrier.await();
    System.out.println("at index=" + sourceIndex + " it's done (waiting) by " +
        Thread.currentThread().getName());
} catch (InterruptedException e) { throw new RuntimeException(e); }
catch (BrokenBarrierException e) { throw new RuntimeException(e); }
}
}

```

```

public class GlobalSalesTask implements Runnable {

```

```

    @Override
    public void run() {
        totalSales=0;
        for(int i=0;i<NB_SOURCES;i++){
            totalSales += partialSalesArray[i];
        }
        System.out.println("GlobalSalesTask , totalSales =" + totalSales + " by " +
            Thread.currentThread().getName());
    }
}

```

```

public void consolidateSales(){
    cyclicBarrier=new CyclicBarrier(NB_SOURCES,new GlobalSalesTask());
    for(int i=0;i<NB_SOURCES;i++){
        Thread t = new Thread(new PartialSourceTask(i));
        t.start();
    }
}

```

```

public static void main(String[] args) {
    CollaborativeWork collaborativeWork= new CollaborativeWork();
    collaborativeWork.consolidateSales();
    try { //une seconde fois (pour montrer l'aspect cyclique/réutilisable)
        Thread.sleep(5000);
    } catch (InterruptedException e) { throw new RuntimeException(e); }
    collaborativeWork.consolidateSales();
}
}

```

⇒ résultats :

at index=1 partialSales =7014.787946376747 by Thread-1 with random delay=1550
at index=2 partialSales =450.10944370323824 by Thread-2 with random delay=1956
at index=0 partialSales =3188.598636930733 by Thread-0 with random delay=203
GlobalSalesTask , totalSales =10653.496027010719 by Thread-2
at index=2 it's done (waiting) by Thread-2
at index=1 it's done (waiting) by Thread-1
at index=0 it's done (waiting) by Thread-0
... idem avec Thread 3,4,5 ...

VII - Semaphore (Synchronisation de Threads)

Un **Semaphore** est (en java depuis le jdk 1.5) un objet technique de synchronisation entre différents threads .

Un **sémaphore** correspond conceptuellement à un **ensemble de jetons disponible** .

Un thread doit **acquérir un jeton disponible** (via un **appel bloquant** à semaphore.acquire() ou bien semaphore.tryAcquire(timeout...) *pour pouvoir ensuite travailler seul sur une ressource partagée* .

En appelant la méthode symétrique semaphore.release() , un thread peut rendre un jeton dans le semaphore (souvent après avoir terminé un certain travail en mode "exclusivité") . Cette action va immédiatement débloquer d'éventuelles attentes exprimées via semaphore.acquire() .

Plus précisément, un **sémaphore** encapsule un entier, avec une contrainte de positivité, et deux opérations atomiques d'incrément et de décrémentation :

- via le constructeur : variable entière (toujours positive ou nulle) ;
- opération -- (**acquire()**) : décrémente le compteur s'il est strictement positif ; bloque s'il est nul en attendant de pouvoir le décrémentation ;
- opération ++ (**release()**) : incrémente le compteur.

NB : Depuis java 1.5 , les **Semaphores** constituent un mécanisme de synchronisation **plus souple et plus fiable** que le mécanisme .wait()/notify() disponible dès java 1.0 sur la classe Object .

EssaiSemaphore.java

```
package tp.langage.thread;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class EssaiSemaphore {
    public static void main(String[] args) {
        Semaphore semaphore = new Semaphore(0);

        Thread t = new Thread(()-> { LongTask.simulateLongTask("background thread work ...", 3000);
                                semaphore.release()});

        t.start();
        System.out.println("... faire autre chose ...");

        //attendre la disponibilité du sémaphore:
        try {
            //semaphore.acquire();
            if(semaphore.tryAcquire(5, TimeUnit.MINUTES)){
                System.out.println("semaphore acquis");
            }else{
                System.out.println("after 5mn (semaphore toujours pas disponible)");
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
}
```

Résultats :

```
... faire autre chose ...  
>>(begin)background thread work ... / by Thread-0  
<<(end)background thread work ... / by Thread-0  
semaphore acquis
```

La notion de "***Mutex***" (mutuelle exclusion) peut s'implémenter en java avec **un sémaphore à un seul jeton** .

VIII - Considérations générales sur mécanismes de synchronisation

Différences importantes entre Lock et Semaphore :

- Un sémaphore peut être « acquired » par un thread et libéré (« released ») par un autre thread et ceci n'est pas possible sur un verrou (Lock).
- Un sémaphore (éventuellement binaire) est un mécanisme de signalisation et n'appartient pas à un thread à un instant t.
- Un verrou (Lock) est un mécanisme de verrouillage qui appartient à un certain thread précis à un instant t.

Autrement dit, les sémaphores sont plus souples .

Autre problème potentiel avec les verrous : inter-blocage mutuel (dead lock).

Exemple de deadLock :

```
package tp.concurrent.basic;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class DeadLockExample {
    int a=5;
    int b=6;
    private Lock lockA= new ReentrantLock();
    private Lock lockB = new ReentrantLock();

    public static void main(String[] args) {
        DeadLockExample deadlock = new DeadLockExample();
        new Thread(deadlock::operationSomme, "Tx").start();
        new Thread(deadlock::operationMult, "Ty").start();
    }

    public void operationSomme() {
        int somme=0;
        lockA.lock();
        somme = this.a;
        System.out.println("lockA acquired, waiting to acquire lockB.");
        try { Thread.sleep(50);} catch (InterruptedException e) { e.printStackTrace(); }
        lockB.lock();
        System.out.println("lockB is acquired, ready for somme(a,b)");
        somme=somme +this.b;
        System.out.println("somme="+somme);
        lockB.unlock();
        lockA.unlock();
    }

    public void operationMult() {
        int produit=0;
```

```

lockB.lock();
produit = this.b;
System.out.println("lockB acquired, waiting to acquire lockA.");
try { Thread.sleep(50);} catch (InterruptedException e) { e.printStackTrace(); }
lockA.lock();
    System.out.println("lockA is acquired, ready for produit(a,b)");
    produit=produit * this.a;
    System.out.println("produit (a*b)="+produit);
lockA.unlock();
lockB.unlock();
}
}

```

⇒ comportement :

lockA acquired, waiting to acquire lockB.

lockB acquired, waiting to acquire lockA.

Et l'application s'est bloquée/plantée tout seule (deadlock) !!!

Autre exemple/analogie : à force de politesse tout le monde se bloque dans un carrefour (avec des priorités à droite de tous les cotés) .

Element de solution au problème d'interblocage/deadlock :

```

public void operationSomme() {
    int somme=0;
    try {
        boolean lockAacquired = lockA.tryLock(60, TimeUnit.MILLISECONDS);
        if(lockAacquired) {
            somme = this.a;
            System.out.println("lockA acquired, waiting to acquire lockB for operationSomme.");
            Thread.sleep(50);
            boolean lockBacquired = lockB.tryLock(60, TimeUnit.MILLISECONDS);
            if(lockBacquired) {
                System.out.println("lockB is acquired, ready for somme(a,b)");
                somme = somme + this.b;
                System.out.println("somme=" + somme);
                lockB.unlock();
            } else { System.out.println("cannot acquire lockB for operationSomme"); }
            lockA.unlock();
        } else { System.out.println("cannot acquire lockA for operationSomme"); }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

```

Nouveau comportement :

lockA acquired, waiting to acquire lockB for operationSomme.

lockB acquired, waiting to acquire lockA for operationMult.

cannot acquire lockA for operationMult

cannot acquire lockB for operationSomme

cannot acquire lockB for operationMult

Certains traitement ne sont pas effectués et l'application n'est plus bloquée/plantée .

1.1. Bonnes pratiques sur la gestion des applications multi-threads

Petite nuance :

Deadlock et **livelock** ont tous deux pour conséquence de bloquer la progression de l'exécution normale de l'application mais il y a une différence clef : Des threads en "*livelocked*" continuent à utiliser du temps CPU (RUNNABLE), tandis que des threads en mode "*deadlocked*" sont en attentes de déverrouillage (BLOCKED).

Pattern "collect & removeAll after loop"

Si l'on tente de supprimer un élément dans une collection (ex : ArrayList, LinkedList) en plein milieu d'un parcours → <i>ConcurrentModificationException</i>	<u>Solution</u> : ajouter les éléments à supprimer dans une collection temporaire durant le parcours. Supprimer d'un coup tous les éléments après le parcours en invoquant <code>collection.removeAll(listeTemporaireEltsASupprimer)</code>
--	---

Minimiser la durée des sections critiques:

- La durée des sections critiques (blocs de code synchronisés) doit être aussi courte que possible pour minimiser les temps d'attente des autres threads.
- Ne pas trop abuser des "synchronized ou autres" imbriqués les uns dans les autres car ça peut augmenter le risque de deadlock .

4 conditions de "Coffman" pour qu'un "deadlock" puisse se produire :

Exclusion mutuelle	Si "synchronized" ou bien "ReentrantLock"
Hold and wait	Un thread détient un verrou et tente d'en obtenir un autre
No preemption	La VM et son moniteur de thread n'a pas prévu de mécanisme de préemption / déblocage automatique, il faut s'assurer que le code est fiable
Attente circulaire	Par exemple : t1 attend t2 , t2 attend t3 et t3 attend t1

Théoriquement , en faisant en sorte qu'une de ces 4 conditions ne soit pas remplie on devrait être tranquille (sans deadlock) .

Spécificité liée aux threads virtuels (de java >=21) :

Il vaut mieux utiliser un **ReentrantLock** que le mot clef `synchronized` au sein d'un bloc de code exécuté par un thread virtuel car ça peut quelquefois bloquer l'ensemble du réel thread porteur (phénomène appelé "virtual thread pinning") .

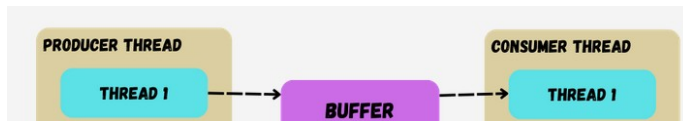
Utiliser des techniques de détection et de prévention des deadlocks:

Implémenter des techniques de détection et de prévention des deadlocks, telles que le **contrôle de la détention des verrous** et l'utilisation de **timeouts**.

Exemple : `if (semaphoreXy.tryAcquire(5, TimeUnit.MILLISECONDS)) { ... }`

Bonnes conceptions (multi-threading):

- **Approche producteur-consommateur:** L'approche producteur-consommateur est un modèle de conception pour synchroniser la production et la consommation de données entre les threads.



- **File d'attente bloquante:** Les files d'attente bloquantes permettent aux threads de stocker et de récupérer des éléments de manière synchrone, ce qui peut être utile pour synchroniser la communication entre les threads.

Exemple :

ValuesProducerTask	ValuesConsumerTask
<pre> package tp.thread; import java.util.concurrent.BlockingQueue; public class ValuesProducerTask implements Runnable{ private long delay; private BlockingQueue<Double> queue; public ValuesProducerTask(BlockingQueue<Double> queue,long delay) { this.queue = queue; this.delay = delay; } @Override public void run() { try { for (int i = 0; i < 10; i++) { double val = Math.random() * 100; queue.put(val); // Produces an item and puts it into the queue System.out.println("Produced and sent value: " + val); Thread.sleep(delay); } queue.put(-1.0); // Indicates end of prod by adding "-1.0" sentinel value //nb: cannot put "null" value : NullPointerException } catch (InterruptedException e) { Thread.currentThread().interrupt(); } } } </pre>	<pre> package tp.thread; import java.util.concurrent.BlockingQueue; public class ValuesConsumerTask implements Runnable { private long delay; private BlockingQueue<Double> queue; public ValuesConsumerTask(BlockingQueue<Double> queue,long delay) { this.queue = queue; this.delay=delay; } @Override public void run() { try { while (true) { Double value = queue.take(); // Consumes an item from the queue if (value <= -1.0) { break; // If sentinel value is encountered, stop consuming } System.out.println("extracted/consumed value: " + value); Thread.sleep(delay); } } catch (InterruptedException e) { Thread.currentThread().interrupt(); } } } </pre>

package tp.thread;	Produced and sent value: 78.93209636056928
import java.util.concurrent.BlockingQueue;	extracted/consumed value: 78.93209636056928
import java.util.concurrent.LinkedBlockingQueue;	Produced and sent value: 83.96266020359143
	Produced and sent value: 21.673959835890322
public class ProducerConsumerExampleApp {	extracted/consumed value: 83.96266020359143
	Produced and sent value: 70.14724269831746
public static void main(String[] args) {	Produced and sent value: 59.427941960992484
BlockingQueue<Double> queue = new LinkedBlockingQueue<>(); // Shared queue	extracted/consumed value: 21.673959835890322
Thread producerThread = new Thread(new ValuesProducerTask(queue,2)); //delay=2ms	Produced and sent value: 62.42772767921545
Thread consumerThread = new Thread(new ValuesConsumerTask(queue,4)); //delay=4ms	Produced and sent value: 51.98628055724737
producerThread.start(); consumerThread.start(); //same order et chacun son rythme	extracted/consumed value: 70.14724269831746
}	extracted/consumed value: 59.427941960992484
}	extracted/consumed value: 62.42772767921545
	extracted/consumed value: 51.98628055724737
	...

Utiliser des outils de débogage et de profiling:

Utiliser des outils de débogage et de profiling pour identifier et résoudre les problèmes de synchronisation des threads. Outils possibles : jstack , jconsole, **JProfiler**, **VisualVM** , et via ajout de code "ThreadMXBean"

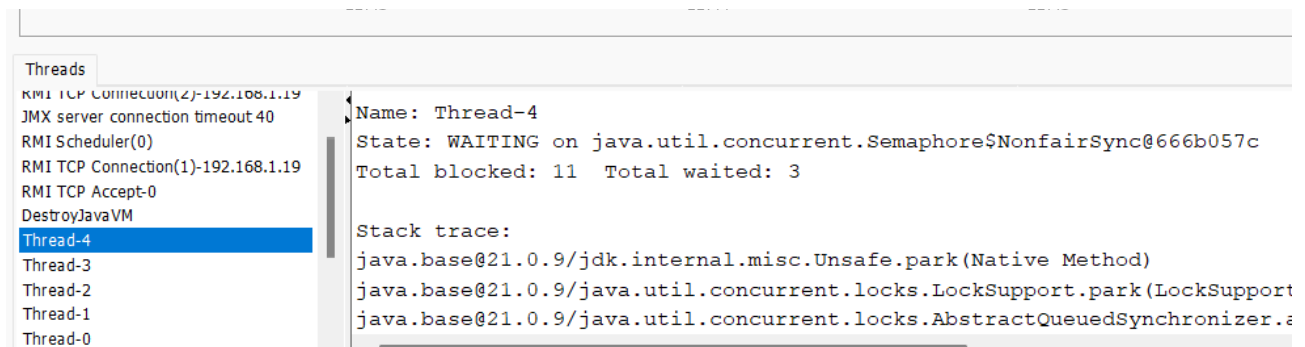
Exemple :

Après avoir lancé l'application *DinerPhilosophesApp* de numéro 12884 retourné par la commande **jps** , on peut lancer la commande **jstack** 12884 qui affiche l'état de tout les threads de l'application avec plein de choses de ce type en cas de "deadlock" :

```
"Thread-0" #30 [3860] prio=5 os_prio=0 cpu=15.62ms elapsed=525.78s tid=0x0000002121e8fec30 nid=3860
waiting on condition [0x00000072d99fe000]

java.lang.Thread.State: WAITING (parking)
    at jdk.internal.misc.Unsafe.park(java.base@21.0.9/Native Method)
    - parking to wait for <0x000000071cfd33b0> (a java.util.concurrent.Semaphore$NonfairSync)
    at java.util.concurrent.locks.LockSupport.park(java.base@21.0.9/LockSupport.java:221)
```

si **jconsole** à la place de jstack , alors via on voit cela dans l'onglet "Threads" :



NB : le bouton "detect deadlock" fonctionne bien avec des "ReentrantLock" (appartenant à un thread précis) mais ne fonctionne pas pour détecter un deadlock avec des sémaphores .

Eventuelle détection par programmation (basée sur ThreadMXBean):

```
//ex : appel en fin de main() après avoir démarré les threads
// detectDeadlockAfterDelay(10*1000);
private static void detectDeadlockAfterDelay(long msDelay) {
    ThreadMXBean bean = ManagementFactory.getThreadMXBean();

    try {
        Thread.currentThread().sleep(msDelay);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }

    long[] deadlockedIds = bean.findDeadlockedThreads();

    if (deadlockedIds == null) {
        System.out.println("No deadlock detected.");
        return;
    }

    ThreadInfo[] infos = bean.getThreadInfo(deadlockedIds, true, true);
    System.out.println("Deadlock detected! Threads involved:");
    for (ThreadInfo info : infos) {
        System.out.printf(
            " Thread '%s' (id=%d) is waiting on a lock held by '%s' %n",
            info.getThreadName(),
            info.getThreadId(),
            info.getLockOwnerName()
        );
    }
}
```

⇒ **résultats :**

Deadlock detected! Threads involved:

Thread 'Thread-0' (id=30) is waiting on a lock held by 'Thread-1'

Thread 'Thread-1' (id=31) is waiting on a lock held by 'Thread-2'

Thread 'Thread-2' (id=32) is waiting on a lock held by 'Thread-3'

Thread 'Thread-3' (id=33) is waiting on a lock held by 'Thread-4'

Thread 'Thread-4' (id=34) is waiting on a lock held by 'Thread-0'

IX - CompletionService (attendre et consommer résultats produits)**EssaiCompletionService.java**

```

package tp.langage.thread;

import java.util.concurrent.Callable;
import java.util.concurrent.CompletionService;
import java.util.concurrent.ExecutorCompletionService;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

//CompletionService depuis jdk1.6 avec méthode take() retournant un Future
//logique producteur/consommateur

public class EssaiCompletionService {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(4); //jusqu'à 4 threads en //
        CompletionService<String> completionService =
            new ExecutorCompletionService<String>(executor);

        double[] tabVal = { 4, 9, 16, 25, 36, 49, 64, 81, 100 };
        int taille = tabVal.length;
        for(int i=0;i<taille;i++){
            Callable<String> c = new CallableComputing(tabVal[i]);
            //CompletionService<String> encapsule l'executor et est typé comme Future<T>
            completionService.submit(c); //lancement asynchrone d'un "producteur"
        }
        for(int i=0;i<taille;i++){
            try {
                Future<String> futureRes = completionService.take(); //attente du PREMIER TERMINE
                System.out.println(futureRes.get()); //consommateur simple
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
        System.out.println("fin-main");
        //arrêter l'executor (si besoin en différé) :
        executor.shutdown();
    }
}

```

Résultats :

```

>>(begin)long computing task (in background) ... / by pool-1-thread-2
>>(begin)long computing task (in background) ... / by pool-1-thread-3
>>(begin)long computing task (in background) ... / by pool-1-thread-4
>>(begin)long computing task (in background) ... / by pool-1-thread-1
<<(end)long computing task (in background) ... / by pool-1-thread-2

```

```
<<(end)long computing task (in background) ... / by pool-1-thread-4
<<(end)long computing task (in background) ... / by pool-1-thread-3
<<(end)long computing task (in background) ... / by pool-1-thread-1
>>(begin)long computing task (in background) ... / by pool-1-thread-3
3.0
4.0
2.0
>>(begin)long computing task (in background) ... / by pool-1-thread-1
>>(begin)long computing task (in background) ... / by pool-1-thread-2
>>(begin)long computing task (in background) ... / by pool-1-thread-4
5.0
<<(end)long computing task (in background) ... / by pool-1-thread-3
<<(end)long computing task (in background) ... / by pool-1-thread-1
<<(end)long computing task (in background) ... / by pool-1-thread-2
>>(begin)long computing task (in background) ... / by pool-1-thread-1
6.0
<<(end)long computing task (in background) ... / by pool-1-thread-4
8.0
7.0
9.0
<<(end)long computing task (in background) ... / by pool-1-thread-1
10.0
fin-main
```

X - Rappels sur lambdas et streams

1. Java multi-paradigme "objet + fonctionnel"

1.1. Evolution importante dès java 8

Les premières versions de java (1.0 à 1.7) étaient centrées sur le paradigme orienté objet .

A partir de la version 8 , le langage java s'est fortement inspiré du langage scala et est devenu un langage multi-paradigme "objet" et "fonctionnel" .

Sans renier ses fondements orientés objets, le langage java a introduit à partir de la version 8 un **nouveau style complémentaire de programmation** appelée **"programmation fonctionnelle"** .

1.2. Programmation fonctionnelle

Au sein d'un contexte de programmation purement orienté objet , les objets comportent des données dont les valeurs (à l'instant t) représentent un certain état .

En appelant une méthode sur un objet , il y a souvent un changement de valeur(s) effectué et donc un changement d'état .

Exemple :

```
p1 = new Personne("jean", "Bon", 40) ;// où 40 signifie ici 40 ans
p1.incrementerAge() ; //premier changement d'état (40 ans → 41 ans)
p1.incrementerAge() ; //second changement d'état (41 ans → 42 ans)
System.out.println("nouvel age de p1=" + p1.getAge()) ; //affiche 42
```

A l'inverse en appliquant le style de programmation fonctionnelle , on manipule avant tout des fonctions pures (sans contexte , sans état contextuel) qui permettent d'appliquer des transformations (ou des calculs) .

Exemple :

```
Personne p1 = new Personne("jean","Bon",40);
System.out.println("p1="+p1);
Function<Personne,Personne> avecUnAnDePlus ;
avecUnAnDePlus = (Personne p) -> new Personne(p.getPrenom(),
                                     p.getNom(),
                                     p.getAge() + 1);
Personne p1Bis = avecUnAnDePlus.apply(avecUnAnDePlus.apply(p1));
//NB: l'objet (de données) p1 n'est pas modifié.
//la double application de la fonction de transformation avecUnAnDePlus
//a permis de créer un nouvel objet p1Bis ayant 2 ans de plus que p1.
System.out.println("p1Bis="+p1Bis);//42 ans
```


NB : La syntaxe java suivante

```
avecUnAnDePlus = (Personne p) -> new Personne(p.getPrenom(),  
                                                p.getNom(),  
                                                p.getAge() + 1);
```

est nouvelle depuis java 8 et correspond à une *lambda expression* .

Ça correspond ici à une fonction admettant en entrée un objet de type *Personne* et retournant en retour un autre objet *Personne* .

1.3. Programmation fonctionnelle accrochée à un squelette objet

Etant donné que java est un langage avant tout orienté objet , les nouveaux éléments ayant un style "fonctionnel" doivent s' accrocher à un squelette de code un peu orienté objet pour pouvoir exister , être compilé et être exécuté par une machine virtuelle java.

2. Lambda expressions et prog. fonctionnelle

2.1. Interfaces fonctionnelles , "lambda expression" et références

Une **interface fonctionnelle** (que l'on peut facultativement explicitement marquer avec la nouvelle annotation **@FunctionalInterface** du **jdk >= 1.8**) est **une interface qui ne comporte qu'une seule méthode ordinaire** (sans "static" ni "default").

Son rôle est de permettre une gestion simple d'une certaine méthode/fonction abstraite .

(*SAM*= **S**ingle **A**bstract **M**ethod → interface de type "SAM")

Exemple (V1) :

```
package tp.langage.v8.sam;  
/* L'annotation @FunctionalInterface (du jdk >= 1.8) est facultative  
   elle permet au compilateur de vérifier que l'interface comporte une seule méthode (ordinaire) */  
@FunctionalInterface  
public interface SamPredicate<E> {  
    boolean test(E e); //retourne true si l'entité e (de type E) satisfait certains critères  
}
```

Exemple (V2) :

```
package tp.langage.v8.sam;
```

@FunctionalInterface

```
public interface SamPredicate<E> {
    boolean test(E e); //retourne true si l'entité e (de type E) satisfait certains critères
    default String getAuthor() { return "developpeur fou"; } //méthode par défaut
    // (alias "extension method" alias "defender method" ). Les méthodes par défaut ont permis
    // d'ajouter de nouvelles fonctionnalités à java8 sans remettre en question les interfaces java7
    static void methodeStatiqueAutoriseeSurInterfaceDepuisJava8(String msg){
        System.out.println(msg);
    }
    //les méthodes statiques au sein des interfaces permettent d'éviter la programmation de
    // nombreuses mini classes utilitaires périphériques
}
```

NB: `java.util.function.Predicate` (depuis java 8) correspond à un équivalent prédéfini de la version 1 de l'interface fonctionnelle `SamPredicate`.

Classe "Person" (pour la compréhension de l'exemple) :

```
...
public class Person {
    String firstname; //+get/set
    String lastname; //+get/set
    int age; //+get/set

    public Person(String firstname, String lastname, int age) { ...}
    public Person(){}
    public String toString() {return "Person (" + firstname + "," + lastname + ", age=" + age + ")"; }

    public static int compareP1P2ByAge(Person p1, Person p2) {
        if (p1.getAge() > p2.getAge()) {
            return 1;
        } else if (p1.getAge() < p2.getAge()) {
            return -1;
        } else {
            return 0;
        }
    }

    public int compareThisP2ByAge(Person p2) {
        if (this.getAge() > p2.getAge()) {
            return 1;
        } else if (this.getAge() < p2.getAge()) {
            return -1;
        } else {
            return 0;
        }
    }
}
```

Syntaxe lourde (java 7) avec classes anonymes imbriquées :

```

...
public class FilterSortListJava7TestApp {
    ...
    public static List<Person> extractSubListBySamPredicate(List<Person> pList,
                                                            final SamPredicate<Person> samPredicate) {
        final List<Person> sublist = new ArrayList<Person>();
        for (Person p : pList) {
            if (samPredicate.test(p)) {
                sublist.add(p);
            }
        }
        return sublist;
    }
    ...
    public static void mainWithInnerAnonymousSamPredicateImplementations() {
        List<Person> pList = StaticPersonList.initList();

        System.out.println("person sublist with age from 20 to 25:");
        List<Person> subList1 = extractSubListBySamPredicate(pList,
                                                             new SamPredicate<Person>(){
                                                                 @Override
                                                                 public boolean test(Person p) {
                                                                     return p.getAge() >= 20 && p.getAge() <= 25;
                                                                 }
                                                             });
        System.out.println(subList1);

        System.out.println("person sublist with lastName starting with letter p");
        List<Person> subList2 = extractSubListBySamPredicate(pList,
                                                             new SamPredicate<Person>(){
                                                                 @Override
                                                                 public boolean test(Person p) {
                                                                     return p.getLastName().startsWith("p");
                                                                 }
                                                             });
        System.out.println(subList2);
    }
    ...
}

```

Syntaxe (très allégée) avec lambda expression depuis de jdk 1.8 :

```

package tp.langage.v8.lambda;
....
import java.util.function.Predicate;

public class FilterSortListJava8TestApp {
    ...
    public static List<Person> extractSubListByJava8Predicate(List<Person> pList, final
    Predicate<Person> predicate) {
        final List<Person> sublist = new ArrayList<Person>();
        for (Person p : pList) {
            if (predicate.test(p)) {
                sublist.add(p);
            }
        }
    }
}

```

```
    }  
    return sublist;  
}  
public static void mainWithLambdaExpressions() {  
    List<Person> pList = StaticPersonList.initList();  
  
    System.out.println("person sublist with age from 20 to 25:");  
    List<Person> subList1 = extractSubListByJava8Predicate(pList,  
        (person) -> person.getAge() >= 14 && person.getAge() <= 25 );  
    System.out.println(subList1);  
  
    System.out.println("person sublist with lastName starting with letter p");  
    List<Person> subList2 = extractSubListByJava8Predicate(pList,  
        (person) -> person.getLastName().startsWith("p") );  
    System.out.println(subList2);  
}
```

2.2. Lambda expressions

La nouvelle syntaxe **(arguments) -> corps de la fonction** apportée par le jdk 1.8 est appelée "**lambda expression**".

Derrière cette syntaxe très concise se cache un gros travail de déduction / résolution de la part du compilateur :

- fabrication automatique d'une implémentation respectant l'interface uni-fonctionnelle précisée par le contexte (ex : fonction englobante invoquée)
- mise en rapprochement des arguments de la "lambda expression" avec les paramètres d'entrées de la méthode abstraite (de l'interface fonctionnelle)
- utilisation du corps de la fonction précisé au sein de la "lambda expression" avec éventuelle interprétation contextuelle (lexicale) du mot clef this .
- ...

→ les types des arguments peuvent ainsi être déduits (et leurs compatibilités vérifiées) par le compilateur .

Exemples (variations syntaxiques) :

```
Collections.sort(pList,
(Person p1, Person p2) -> { return Integer.compare(p1.getAge() , p2.getAge()) ; })
```

ou bien avec return implicite (si une seule instruction dans { }) :

```
Collections.sort(pList, (Person p1, Person p2) -> Integer.compare(p1.getAge() , p2.getAge()) )
```

ou bien (avec déduction/inférence des types de p1 et p2 selon le contexte) :

```
Collections.sort(pList, (p1, p2) -> Integer.compare(p1.getAge() , p2.getAge()) )
```

Exemple dans son contexte :

```
public static void sortListComparatorInnerAnonymousImplementation() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList, new java.util.Comparator<Person>(){
        @Override
        public int compare(Person p1, Person p2) {
            if (p1.getAge() > p2.getAge()) { return 1; }
            else if (p1.getAge() < p2.getAge()) { return -1; }
            else { return 0; }
        }
    });
    System.out.println(pList);
}
```

simplifié en

```
public static void sortListWithLambdaExpression() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList,
        /* (Person p1, Person p2) -> p1.getAge() == p2.getAge() ? 0 : (p1.getAge() < p2.getAge() ? -1 : 1) */
        (Person p1, Person p2) -> Integer.compare(p1.getAge(), p2.getAge())
        /* (p1,p2) -> p1.getLastname().compareTo(p2.getLastname()) */
    );
    System.out.println(pList);
}
```

NB:

- D'un point de vue "typage de données / signature" , une lambda expression peut être vue par le compilateur comme un équivalent d'une implémentation d'une classe anonyme imbriquée implémentant une interface unifonctionnelle.
- D'un point de vue "performance/implémentation réelle" , une lambda expression est une construction spécifique du compilateur (qui est plus légère et plus performante qu'une classe anonyme imbriquée) . Conceptuellement, on est pas très loin d'un code "inline" du c++ ou d'une référence de fonction fléchée de javascript.

2.3. Référence de fonctions

NB : la (nouvelle) syntaxe **NomClasse::nomFonction** correspond à une **référence de fonction** .
→ les arguments et le corps de la fonction référencée sont alors utilisés de la même façon que ceux d'une "lambda expression" .

NB2 : une référence de fonction peut éventuellement **référer un "constructeur"** tel que le montre cet exemple :

```
....map(person -> new Student(person));
```

pouvant être ré-écrit en

```
...map(Student::new);
```

NB :

A la place d'une lambda expression à n paramètres en entrée, on pourra placer :

- soit une référence de fonction statique avec n paramètres en entrée
- soit une référence de fonction ordinaire (pas statique) avec n-1 paramètres en entrée sachant que lorsque cette méthode sera exécutée , le mot clef **this** sera utilisé en faisant office de premier paramètre d'un équivalent en forme lambda .

Exemple :

```
...
public class Person {
    ...
    public static int comparePIP2ByAge(Person p1, Person p2) {
        return p1.getAge() - p2.getAge() ;
    }

    public int compareThisP2ByAge(Person p2) {
        return this.getAge() - p2.getAge() ;
    }
}
```

```
public static void sortListWithFunctionReference() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList, Person::comparePIP2ByAge);
    //Person::comparePIP2ByAge() have same code as java.util.Comparator.compare()
    //but with different name and without explicit interface implementation
    System.out.println("via static function reference, pList=" + pList);

    pList = StaticPersonList.initList();
    Collections.sort(pList, Person::compareThisP2ByAge);
    System.out.println("via no static function reference, pList=" + pList);

    System.out.println("liste triée par nom puis par prénom (with function reference:");
    pList.sort(Comparator.comparing(Person::getLastName)
               .thenComparing(Person::getFirstName));
    System.out.println(pList);
}
```

2.4. Scope visibility in lambda expression :

```
public class Java8TestApp {
    private String message="Hello World (V8)";
    public static void main(String[] args) {
        (new Java8TestApp()).testRun();
    }
    public void testRun(){
        //lambda expression (compatible run()) utilisant "message" du scope parent:
        /* Runnable r1 = () -> System.out.println(this.message); */
        Runnable r1 = () -> System.out.println(message);
        Thread t1 = new Thread(r1); t1.start();
    }
}
```

NB : En langage **java** , une lambda expression est formulée par $(p) \rightarrow p.getXy() \geq this.seuil$
 en langage **javascript** , une fonction fléchée est formulée par $(p) \Rightarrow p.xy \geq this.seuil$
 Dans les 2 cas le mot clef **this** (utilisé dans une lambda) fait référence au contexte objet englobant.

2.5. Function, Predicate, Supplier, Consumer, ...

Avec un peu plus de recul et plus formellement :

Une **interface fonctionnelle** correspond à un **type (de traitement fonctionnel) abstrait**.

Une **"lambda expression"** (ou bien une référence de méthode) correspond à une **implémentation concrète d'une interface fonctionnelle**.

→ on peut écrire

```
Runnable r1 = () -> System.out.println("message");
```

D'autre part, le package **java.util.function** comporte un paquet d'**interfaces uni-fonctionnelles génériques** :

Function<T,R> - takes an object of type T and returns R.

Supplier<T> - just returns an object of type T.

Predicate<T> - returns a boolean value based on input of type T.

Consumer<T> - performs an action with given object of type T (no return)

BiFunction - like Function but with two parameters.

BiConsumer - like Consumer but with two parameters

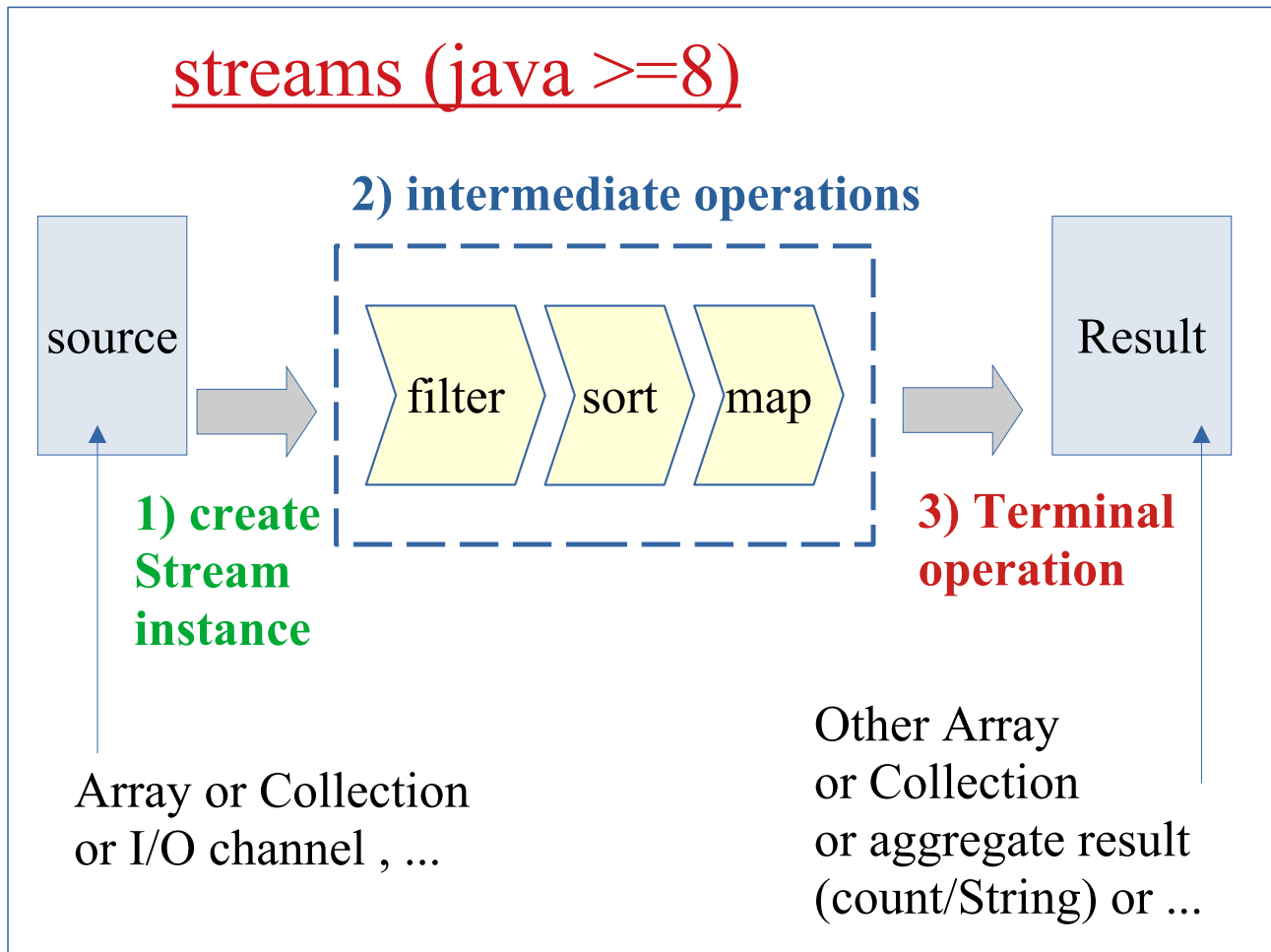
Interface Fonctionnelle	signature de la fonction	exemple
UnaryOperator <T>	T apply(T t)	String::toLowerCase
BinaryOperator <T>	T apply(T t1, T, t2)	BigInteger::add
Predicate <T>	boolean test(T t)	Collection::isEmpty
Function <T, R>	R apply(T t)	Arrays::asList
BiFunction <T, U, R>	R apply(T t, U u)	(x, y) -> x * y
Supplier <T>	T get()	Instant::now
Consumer <T>	void accept(T t)	System.out::println
autres (variantes , ...)	...	

Exemple d'utilisation (explicite et directe) :

```
List<String> liste = initListOfString();
Function<String, String> atrFct = (name) -> {return "@" + name;};
Function<String, Integer> lengthFct = (name) -> name.length() ;
//ou bien Function<String, Integer> lengthFct = String::length ;
for (String s : liste) {
    System.out.println( atrFct.apply(s) + " , length=" + lengthFct.apply(s));
}
```

L'utilisation des types fonctionnels abstraits et génériques (de **java.util.function**) est souvent effectuée indirectement en tant que paramètres des opérations (sort, filter, map, ...) sur les "Streams".

3. Streams (java 8 et plus)



3.1. Stream et opérations (sort , filter , map , reduce , collect , ...)

`java.util.stream.Stream` (depuis le jdk 1.8) permet d'effectuer **des opérations de tri , filtrage , ... avec une syntaxique très concises sur des flux de données en vrac** (*bulk data operations in english*).

Exemple:

```
public static void mapWithStream(){ // map() is for "transformation"

    List<Person> persons = StaticPersonList.initList();
    Stream<Student> studentsStream = null;
    /*
    //v1 (explicit):
    studentsStream = persons.stream().filter(p -> p.getAge() <= 30)
        .map(new Function<Person, Student>() {
            @Override
            public Student apply(Person person) {
                return new Student(person);
            }
        });

    //v2 (with lambda expression):
    studentsStream = persons.stream().filter(p -> p.getAge() <=30 )
        .map(person -> new Student(person));
    */

    //v3 (with function/constructor reference):
    studentsStream = persons.stream().filter(p -> p.getAge() <=30 )
        .map(Student::new);

    System.out.println("liste of students:");
    studentsStream.forEach(System.out::println);

    //with collect() terminal operation (at end of operation stream) :
    List<Student> students = persons.stream().filter(p -> p.getAge() <=25 )
        .map(Student::new)
        .collect(Collectors.toList()); //or .collect(Collectors.toCollection(ArrayList::new));

    System.out.println("liste of (youngs) students:" + students);
}
```

```
List<Person> listePersonnesFiltreesTrieesEtTransformees =
    persons.stream()
    .filter( (p)->p.getAge()>=18 )
    .sorted( (p1,p2)->Integer.compare(p1.getAge(), p2.getAge()))
    .map( (p) -> { p.setLastname(p.getLastname().toUpperCase()); return p; } )
    .collect(Collectors.toList());
```

```
public static void skipAndLimitOnStream(){
    final List<Integer> demoValues = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
    //limit the input -> [1, 2, 3, 4] :
    System.out.println(demoValues.stream().limit(4).collect(Collectors.toList()));
    //jumping over the first 4 elements -> [5, 6, 7, 8, 9, 10] :
    System.out.println(demoValues.stream().skip(4).collect(Collectors.toList()));
}
```

```
public static void reduceStreamToASingleResult(){
    final List<String> demoValues = Arrays.asList("1_2_3","-4_5_6","-7_8_9");
    System.out.println("stream reduced to as single optional result: "
        + demoValues.stream().reduce(String::concat));

    final List<Integer> demoValues2 = Arrays.asList(5,8,12);
    System.out.println("stream reduced to as single result (somme des valeurs): "
        + demoValues2.stream().reduce(0 , (x,y) -> x+y));
    //.reduce(identity, accumulator) where identity is initialValue
    // (or default result if stream is empty)
}
```

```
public static void filteringWithStreamFilterAndLambda(){
    List<Person> pList = StaticPersonList.initList();
    pList.stream().filter( (p) -> p.getAge() >= 32 ).forEach( (p) -> System.out.println(p) );

    List<String> liste = initListOfString();
    //liste.stream().map((s)->s.toUpperCase()).forEach(System.out::println);
    liste.stream().map(String::toUpperCase).forEach(System.out::println);

    //.stream() for sequential operations , .parallelStream() for parallel operations
    //source of stream can be:
    //Stream.of(val1,val2,val3...) , Stream.of(array) and list.stream().
    //or final Stream<String> splitOf = Stream.of("A,B,C".split(", "));
```

}

Autres exemples :

```
String s = IntStream.iterate(1, i -> i * 2) //generation d'une sequence a priori infinie 1, 2, 4, ...
    .limit(10) //limitation a 10 iterations
    .mapToObj(String :: valueOf) //transformation de chaque élément en String
    .collect(Collectors.joining(" ; ")); //collector rassemblant tout en une seule grande chaine
    //via des concatenations (en boucle) avec le separateur/jointeur spécifié
System.out.println(s);           //affiche 1 ; 2 ; 4 ; 8 ; 16 ; 32 ; 64 ; 128 ; 256 ; 512
```

```
List<Product> listProd = ProductUtil.initSampleProductList();
Optional<Product> ofp=listProd.stream().filter(p -> p.getPrice() >= 200).findFirst();
System.out.println("premier produit trouve avec prix >= 200 : " + ofp.orElse(null));

boolean auMoinsUnProduitExistantQuiCommenceParPrinter =
    listProd.stream().anyMatch(p -> p.getLabel().startsWith("printer"));
if(auMoinsUnProduitExistantQuiCommenceParPrinter)
    System.out.println("au moins un des produits existants commence par printer");
```

```
double prixMaxi =listProd.stream()
    .map(p -> p.getPrice())
    .reduce(Double::max).orElse(0.0);
System.out.println("prixMaxi="+prixMaxi);
```

```
List<String> liste1 = Arrays.asList("rouge", "vert", "bleu");
List<String> liste2 = Arrays.asList("jaune", "orange", "bleu");
List<String> listeCouleurs = Stream.concat(liste1.stream(), liste2.stream())
    .distinct()
    .collect(Collectors.toList());
System.out.println("liste des couleurs (sans doublon)="+listeCouleurs);
```

→ liste des couleurs (sans doublon)=[rouge, vert, bleu, jaune, orange]

```
List<Point> points = Arrays.asList(new Point(1, 2), new Point(2, 4));
System.out.println("liste de points =" + points);
points.stream().forEach(p -> p.translate(10, 5)); //forEach() = operation terminale (void)
    // p.translate() : void/ne retourne rien et modifie p
System.out.println("Apres translation(10,5), liste de points =" + points);
```

liste de points =[java.awt.Point[x=1, y=2], java.awt.Point[x=2, y=4]]
 Apres translation(10,5), liste de points =[java.awt.Point[x=11,y=7], java.awt.Point[x=12,y=9]]

XI - MultiThreading et asynchronisme

1. ForkJoin

1.1. basic fork/join

```
public class EssaiBasicForkJoin {
    public static void main(String[] args) {
        System.out.println("debut - main");
        Thread t = new Thread(new MyRunnableCode());
        t.start(); // basic sort of fork()
        System.out.println(("suite - main / avant join"));
        try {
            t.join(); //attente de la fin de l'exécution du thread
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("fin main - apres join");
    }
}
```

-->

```
debut - main
suite - main / avant join
Thread-0
fin main - apres join
```

1.2. fork/join (depuis java 1.7)

Le framework **fork / join** en Java (depuis le jdk 1.7) est idéal pour **un problème qui peut être divisé en parties plus petites et résolu en parallèle**.

Les étapes fondamentales d'un problème fork / join sont les suivantes:

- **Diviser le problème en plusieurs morceaux**
- **Résoudre chacune des pièces en parallèle**
- **Combinez chacune des sous-solutions en une solution globale**

Une [ForkJoinTask](#) est l'interface qui définit un tel problème. On s'attend généralement à ce que vous sous-classiez l'une de ses implémentations abstraites (généralement la [RecursiveTask](#)) plutôt que d'implémenter l'interface directement.

Détail technique :

Le *ForkJoinPool* est le coeur du framework. Il s'agit d'une implémentation de [ExecutorService](#) qui gère les threads de travail et nous fournit des outils pour obtenir des informations sur l'état et les performances du pool de threads.

Les threads de travail ne peuvent exécuter qu'une tâche à la fois, mais *ForkJoinPool* ne crée pas de thread séparé pour chaque sous-tâche. Au lieu de cela, chaque thread du pool a sa propre file d'attente à deux extrémités (double ended *queue/deque*) qui stocke les tâches.

Cette architecture est essentielle pour équilibrer la charge de travail du thread à l'aide de l'algorithme "work-stealing" .

1.3. fork/join appliqué sur un quickSort en mode "multi-processeurs"

MyQuickSortAlgo.java (version sans fork/join)

```
package tp.langage.thread;
public class MyQuickSortAlgo {

    static void echanger(double[] tableau ,int indice1 ,int indice2){
        double temp = tableau[indice1];
        tableau[indice1] = tableau[indice2];
        tableau[indice2] = temp;
    }

    static int partition(double[] tableau,int deb,int fin){
        int indicePivot=deb; //au sens indice initial qui va évoluer
        double valeurPivot=tableau[deb];//valeur du pivot (= arbitrairement valeur en première position du tableau)
        //via une future permutation , cette valeur sera à une future autre position

        for(int i=deb+1;i<=fin;i++){
            if (tableau[i]<valeurPivot){
                indicePivot++; //nouvelle valeur pour le futur indice du pivot (qui peut encore évoluer selon boucle en cours)
                echanger(tableau,indicePivot,i); //pour placer à la "future gauche" de l'indice provisoire du pivot
                //tous les éléments plus petits que le pivot
            }
        }
        echanger(tableau,deb,indicePivot); //permutation pour que la valeur du pivot soit rangée à sa place (précédemment calculée)
        //et pour qu'un des éléments plus petits soit placé au début (à gauche )

        return indicePivot;
    }

    //version ordinaire (sans optimisation multi-proc):
    static void tri_rapide(double[] tableau,int deb,int fin){
        if(deb<fin){
            //partitionner le tableau en 2 parties partiellement ré-arrangées .
            //d'un coté tous les éléments plus petits que le pivot , de l'autre coté tous les éléments plus grands:
            int positionPivot=partition(tableau,deb,fin);
            tri_rapide(tableau,deb,positionPivot-1); //trier le sous tableau des plus petits éléments que le pivot
            tri_rapide(tableau,positionPivot+1,fin); //trier le sous tableau des plus grands éléments que le pivot
        }
    }

    static void quick_sort(double[] tableau){
        tri_rapide(tableau, 0, tableau.length - 1 );
    }
}
```

MyQuickSortMultiProc.java (avec fork/join)

```
package tp.langage.thread;

import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

//ForkJoin (RecursiveAction) / divide & conquer since jdk 1.7
public class MyQuickSortMultiProc extends RecursiveAction {

    private static final long serialVersionUID = 1L;
    private static final int FORK_JOIN_MIN_SIZE=1024;

    private double[] tab;
    private int start,end;

    public MyQuickSortMultiProc(double[] tableau,int deb,int fin){
        this.tab = tableau;
        this.start = deb;
        this.end = fin;
    }

    static void quick_sort_multiProc(double[] tableau){
        MyQuickSortMultiProc myQuickSortMultiProc=
            new MyQuickSortMultiProc(tableau, 0, tableau.length - 1);
        ForkJoinPool threadPool = new ForkJoinPool();
        threadPool.invoke(myQuickSortMultiProc);
    }

    @Override //RecursiveAction
    protected void compute() {
        // pas de paramètre et donc les tab et indices
        // doivent être renseignés en tant qu'attributs + constructeurs
        MyQuickSortMultiProc sousTriGaucheViaForkJoin=null;
        MyQuickSortMultiProc sousTriDroitViaForkJoin = null;
        //System.out.println("MyQuickSortMultiProc.compute() executé par "+Thread.currentThread().getName() );

        if(start<end){
            //partitionner le tableau en 2 parties partiellement ré-arrangées .
            //d'un coté tous les éléments plus petits que le pivot , de l'autre coté tous les éléments plus grands:
            int positionPivot=MyQuickSortAlgo.partition(tab,start,end);

            //NB: étant donné que la version forkJoin ajoute une complexité au niveau du code
            // (instance à créer , thread à gérer) , cette version ne sera activée/utilisée
            //que pour trier des sous tableaux dont la taille minimum est supérieure à
            //FORK_JOIN_MIN_SIZE=1024

            if(positionPivot - start > FORK_JOIN_MIN_SIZE){
                sousTriGaucheViaForkJoin= new MyQuickSortMultiProc(tab, start, positionPivot-1);
                sousTriGaucheViaForkJoin.fork(); //déléguer le tri du sous tableau des plus petits éléments que le pivot
            } else MyQuickSortAlgo.tri_rapide(tab, start, positionPivot-1);
        }
    }
}
```



```

if(end - positionPivot > FORK_JOIN_MIN_SIZE){
    sousTriDroitViaForkJoin= new MyQuickSortMultiProc(tab, positionPivot+1,end);
    sousTriDroitViaForkJoin.fork(); //déléguer le tri du tableau des plus grands éléments que le pivot
} else MyQuickSortAlgo.tri_rapide(tab, positionPivot+1, end);

if(sousTriGaucheViaForkJoin!=null) sousTriGaucheViaForkJoin.join(); //attendre
if(sousTriDroitViaForkJoin!=null) sousTriDroitViaForkJoin.join(); //attendre

//NB: il existe invokeAll(recursiveAction1 , recursiveAction2) qui declenche en // .fork() et .join()
}
}
}

```

TestMultiProcesseurs.java (tests avec rapidités comparées)

```

package tp.langage.thread;

public class TestMultiProcesseurs {

    public static void main(String[] args) {

        System.out.println("nb processors:" + Runtime.getRuntime().availableProcessors());

        double[] t1 = produce_init_tab();
        double[] copyOfT1 = t1.clone();
        display_tab(t1); //display_tab(copyOfT1);
        System.out.println("tri ordinaire (quick-sort) ");
        test_tri(t1);
        System.out.println("tri (quick-sort) optimisé pour machine multi-processeurs ");
        test_tri_multiProc(copyOfT1);
    }

    static double[] produce_init_tab() {
        //double[] t = { 5,2,1,9,3,4,12,8,16,6 };
        //double[] t = { 26,7,5,2,1,9,3,4,34,12,8,16,6,78,10,89,33,23,90,123,72,3,48 };
        //final int taille=10;
        final int taille=1024*1024*8;
        double[] t = new double[taille];
        for(int i=0;i<taille;i++){
            t[i]=Math.random()*taille;
        }
    }
}

```

```

    return t;
}

static void display_tab(double[] tab){
    if(tab.length <= 30) {
        for(double x : tab)
            System.out.print(x + " ");
        System.out.print("\n");
    } else{ System.out.println("tableau de taille = " + tab.length);
    }
}

static void test_tri(double[] tab){
    long td = System.nanoTime();
    MyQuickSortAlgo.quick_sort(tab);
    long tf = System.nanoTime();    display_tab(tab);
    System.out.println("## " + (tf-td)/ 1000000 + " ms");
}

static void test_tri_multiProc(double[] tab){
    long td = System.nanoTime();
    MyQuickSortMultiProc.quick_sort_multiProc(tab);
    long tf = System.nanoTime();    display_tab(tab);
    System.out.println("** " + (tf-td) / 1000000 + " ms");
}
}

```

Résultats (avec i7):

```

nb processors:4
tableau de taille = 8388608
## tri ordinaire (quick-sort)
tableau de taille = 8388608
## 872 ms
** tri (quick-sort) optimisé pour machine multi-processeurs
tableau de taille = 8388608

```

**** 538 ms**

Autre exemple de fork/join (calcul de moyenne ou d'écartType) .

SequentialComputing.java

```
package tp.thread.sam;
//interface d'une référence de fonction pour calcul ordinaire de somme ou moyenne ou ...
public interface SequentialComputing {
    //start et end sont les indices sur la plage du tableau à manipuler .
    //arg = null ou éventuel argument nécessaire à un calcul (ex: arg=moyenne pour calcul de variance)
    double basicCompute(double[] numbers, int start, int end, Double arg); //sum or average or ....
}
```

ResultAggregate.java

```
package tp.thread.sam;
//interface d'une référence de fonction pour recombinaison 2 sous sommes ou 2 sous moyennes
public interface ResultAggregate {
    Double composeTotalRes(double res_tab1, int taille_tab1, double res_tab2, int taille_tab2);
}
```

MyStatsAlgo.java

```
package tp.thread;

import tp.thread.sam.ResultAggregate;
import tp.thread.sam.SequentialComputing;

//calcul de somme , moyenne ou variance sur grands tableaux
public class MyStatsAlgo {
    public static final int DECOMP_MIN_SIZE=1024*50;

    public static final Double composeMoyenneTotale(double moyenne_tab1, int taille_tab1,
                                                    double moyenne_tab2, int taille_tab2) {
        return (moyenne_tab1*taille_tab1 + moyenne_tab2*taille_tab2) / (taille_tab1 + taille_tab2);
    }

    //versions ordinaires (sans decomposition en 2 sous parties):

    static Double moyenne_ordinaire_subPart(double[] tableau, int deb, int fin, Double notUsedArg){
        Double somme = 0.0;
        for(int i=deb; i<=fin ;i++) {
            somme += tableau[i];
        }
        int sizeSubPart = (fin - deb)+1;
    }
```

```

        return somme / sizeSubPart;
    }

    static Double variance_ordinaire_subPart(double[] tableau,int deb,int fin,Double moyenne){
        Double varianceFoisN = 0.0;
        for(int i=deb; i<=fin ;i++) {
            varianceFoisN += (tableau[i] - moyenne) * (tableau[i] - moyenne);
        }
        int sizeSubPart = (fin - deb)+1;
        return varianceFoisN/sizeSubPart;
    }

//versions avec decomposition en 2 sous parties:

    static double moyenne_decomp_subPart(double[] tableau,int deb,int fin){
        return compute_decomp_subPart(tableau,deb,fin,null,
            MyStatsAlgo::moyenne_ordinaire_subPart,MyStatsAlgo::composeMoyenneTotale);
    }

    static double variance_decomp_subPart(double[] tableau,int deb,int fin,double moyenne){
        return compute_decomp_subPart(tableau,deb,fin,moyenne,
            MyStatsAlgo::variance_ordinaire_subPart,MyStatsAlgo::composeMoyenneTotale);
    }

    static double compute_decomp_subPart(double[] tableau,int deb,int fin,Double arg,
        SequentialComputing seqComputing , ResultAggregate resAggregate){
        double resSp;
        int sizeSubPart = (fin - deb)+1;
        if(sizeSubPart >= DECOMP_MIN_SIZE){
            int indiceMilieu = deb + (sizeSubPart / 2);
            double resCalculSousPartie1 = compute_decomp_subPart(tableau,deb,indiceMilieu-1,arg,seqComputing,resAggregate);
            double resCalculSousPartie2 = compute_decomp_subPart(tableau,indiceMilieu,fin,arg,seqComputing,resAggregate);
            resSp = resAggregate.composeTotalRes(resCalculSousPartie1 , (indiceMilieu - deb) ,
                resCalculSousPartie2 , (fin - indiceMilieu +1) );
        }
        else
            resSp = seqComputing.basicCompute(tableau,deb,fin,arg);
        return resSp;
    }

//fonctions de niveau principal (appels simples , niveau global):
    static double moyenne_ordinaire(double[] tableau){
        return moyenne_ordinaire_subPart(tableau, 0, tableau.length - 1 , null );
    }

    static double ecartType_ordinaire(double[] tableau){
        double moyenne = moyenne_ordinaire_subPart(tableau, 0, tableau.length - 1 , null );
        double variance = variance_ordinaire_subPart(tableau, 0, tableau.length - 1 , moyenne );
        return Math.sqrt(variance);
    }

    static double moyenne_decomp(double[] tableau){
        return moyenne_decomp_subPart(tableau, 0, tableau.length - 1 );
    }

    static double ecartType_decomp(double[] tableau){

```

```

double moyenne = moyenne_decomp_subPart(tableau, 0, tableau.length - 1 );
double variance = variance_decomp_subPart(tableau, 0, tableau.length - 1 , moyenne);
    return Math.sqrt(variance);
}
}

```

MyStatsMultiProc.java

```

package tp.thread;

import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

import tp.thread.sam.ResultAggregate;
import tp.thread.sam.SequentialComputing;

public class MyStatsMultiProc extends RecursiveTask<Double> {
    private static final int FORK_JOIN_MIN_SIZE=1024*50;
        // THRESHOLD/SEUIL à éventuellement ajuster selon complexité du calcul
    private double[] tab;
    private int start,end;
    private SequentialComputing seqComputing; //référence de fonction pour calcul ordinaire de somme ou moyenne ou ...
    private ResultAggregate resAggregate; //référence de fonction pour recombinaison 2 sous sommes ou 2 sous moyennes
    private Double arg; //null ou éventuel argument nécessaire à un calcul (ex: arg=moyenne pour calcul de variance)

    public MyStatsMultiProc(double[] tableau,int deb,int fin , Double arg,
        SequentialComputing seqComputing , ResultAggregate resAggregate){
        this.tab = tableau; this.start = deb; this.end = fin; this.arg = arg;
        this.seqComputing = seqComputing; this.resAggregate = resAggregate;
    }

    static double moyenne_multiProc(double[] tableau){
        MyStatsMultiProc myStatsMultiProc= new MyStatsMultiProc(tableau, 0, tableau.length - 1,
            null,MyStatsAlgo::moyenne_ordinaire_subPart,MyStatsAlgo::composeMoyenneTotale);
        ForkJoinPool threadPool = new ForkJoinPool();
        return threadPool.invoke(myStatsMultiProc);
    }

    static double ecartType_multiProc(double[] tableau){
        double moyenne = moyenne_multiProc(tableau);
        MyStatsMultiProc myStatsMultiProc= new MyStatsMultiProc(tableau, 0, tableau.length - 1,
            moyenne,MyStatsAlgo::variance_ordinaire_subPart,MyStatsAlgo::composeMoyenneTotale);
        ForkJoinPool threadPool = new ForkJoinPool();
        double variance = threadPool.invoke(myStatsMultiProc);
        return Math.sqrt(variance);
    }

    @Override
    protected Double compute() {
        Double res =0.0;
        //System.out.println("MyStatsMultiProc.compute() exécuté par "+Thread.currentThread().getName() );

        //NB: étant donné que la version forkJoin ajoute une complexité au niveau du code
        // (instance à créer , thread à gérer) , cette version ne sera activée/utilisée
        //que pour traiter des sous tableaux dont la taille minimum est supérieure à FORK_JOIN_MIN_SIZE

        int sizeSubPart = (end - start)+1;
    }
}

```

```

if(sizeSubPart >= FORK_JOIN_MIN_SIZE) {
    int indiceMilieu = this.start + (sizeSubPart / 2);
    // pas de parametre dans .compute() et donc les tab et indices doivent être renseignés
    // en tant qu'attributs + constructeurs
    MyStatsMultiProc sousCalculGaucheViaForkJoin=
        new MyStatsMultiProc(tab,start,indiceMilieu-1,arg,seqComputing,resAggregate);
    MyStatsMultiProc sousCalculDroitViaForkJoin=
        new MyStatsMultiProc(tab,indiceMilieu,end,arg,seqComputing,resAggregate);

    sousCalculGaucheViaForkJoin.fork(); //déléguer (via potentiel autre thread)

    //sous solution A (.compute() ) :
    //Double resCalculSousPartie2 = sousCalculDroitViaForkJoin.compute(); //faire soit même (via meme thread)

    //sous solution B (.fork/join ) :
    sousCalculDroitViaForkJoin.fork(); //déléguer (via potentiel autre thread)
    Double resCalculSousPartie2 = sousCalculDroitViaForkJoin.join(); //attendre

    Double resCalculSousPartie1 = sousCalculGaucheViaForkJoin.join(); //attendre

    res = resAggregate.composeTotalRes(resCalculSousPartie1 , (indiceMilieu - start) ,
                                        resCalculSousPartie2 , (end - indiceMilieu +1) );
} else {
    res = seqComputing.basicCompute(this.tab,this.start,this.end,this.arg);
}
return res;
}
    
```

Exemple de performances comparées (sur un petit i7) :

```

nb processors:4
tableau de taille = 134217728
## moyenne ordinaire (sans decomposition)
##MOYENNE=500.0102357180581
## 601 ms
$$ moyenne avec decomposition
$$MOYENNE=500.01023571807764
$$ 174 ms
** moyenne avec decomposition optimise pour machine multi-processeurs (fork/join)
**MOYENNE=500.01023571807764
** 64 ms
    
```

2. CompletableFuture , streams asynchrones

CompletableFuture<T> disponible depuis le *jdk 1.8* est une version améliorée de **Future<T>** pour **enchaîner certains traitements asynchrones** et qui comporte *quelques similitudes avec les "callback" et "Promise.then()" de javascript* .

```
public class CompletableFuture<T>
extends Object
implements Future<T>, CompletionStage<T>
```

dans le package **java.util.concurrent** .

Dans les grandes lignes , la **principale valeur ajoutée de CompletableFuture<T>** vis à vis de **Future<T>** réside dans l'**implémentation de l'interface CompletionStage<T>** de manière à **bien resynchroniser / ordonner différentes tâches asynchrones qui seront par nature exécutées en différé** (dans le futur) .

Autrement dit , un bloc d'instructions de ce type :

```
CompletableFuture.supplyAsync(traitementAsynchrone1 )
    .thenApply( traitementAsynchrone2 )
    .thenApply(traitementAsynchrone3)
    .thenAccept( traitementAsynchroneFinal );
```

correspond à enregistrement de tâches asynchrones à effectuer en différé dès que possible .

Le future résultat du *traitementAsynchrone1* constituera l'entrée de la tâche *traitementAsynchrone2* qui ne pourra alors être démarrée que lorsque la tâche *traitementAsynchrone1* sera terminée et ainsi de suite via cet enchaînement de `.thenApply()` `.then...` `()`

Rappel :

- l'interface (uni-)fonctionnelle **Supplier<T>** comporte l'unique méthode **T .get()** et correspond à un producteur ou *fournisseur de valeur de Type T* .
- l'interface (uni-)fonctionnelle **Function<T,R>** comporte la méthode **R .apply(T t)** et correspond à une *fonction appliquée à un unique paramètre de type T et retournant R* .
- l'interface (uni-)fonctionnelle **Consumer<T>** comporte la méthode **void .accept(T t)** et correspond à un *consommateur d'élément de type T (sans valeur produite)* .

Principales méthodes de CompletableFuture<T> et CompletionStage<T> :

NB1 : l'abréviation **CF<T>** est à comprendre comme CompletableFuture<T>

NB2 : Sans précision de l'executor (souvent en tant que dernier paramètre facultatif des méthodes surchargées) , la classe CompletableFuture lance en interne les tâches asynchrones via **ForkJoinPool.commonPool()** par défaut .

NB3 : Au sein de ce tableau , les méthodes en *italiques* seront à considérées comme "**static**" .
et <T,R> est généralement à interpréter comme < ? super T , ? extends R>

CF<Void> runAsync (Runnable r)	retourne un CF<T> correspondant à la future exécution asynchrone de la tâche r (Runnable)
CF<U> <i>supplyAsync</i> (Supplier<U> s)	retourne un CF<U> correspondant à la future exécution asynchrone de la tâche s (fournissant une valeur de type U)
CF<R> <i>thenApply</i> (Function<T,R> f)	Souvent en milieu d'enchaînement, après la fin du CF<T> précédent (vu comme préfixe (this)) , retourne un CF<R> correspondant à la future exécution asynchrone de la tâche fonctionnelle f
CF<R> thenCompose (Function<T,CF<R>> f)	Un peu comme thenApply() mais en précisant une fonction qui (immédiatement, sans attente) construit elle même une instance de CF <R> qui sera consommée dans la suite de l'enchaînement
CF<T> exceptionally (Function<Throwable ex,T> f)	Souvent en milieu d'enchaînement, transforme une exception potentielle en élément de type T pour le bon déroulement de la suite des enchaînements, retransmet la valeur inchangée si pas d'exception .
CF<R> handle (BiFunction<T, Throwable,R> f)	Combine le comportement de thenApply() et exceptionally() via une tâche asynchrone prenant 2 argument en entrée : value et exception .
CF<Void> <i>thenAccept</i> (Consumer<T> c)	En terminaison, après la fin du CF<T> précédent (vu comme préfixe (this)) , retourne un CF<Void> correspondant à la future exécution asynchrone de la tâche terminale c (Consumer avec argument en entrée)
CF<Void> thenRun (Runnable r)	Souvent en terminaison, après la fin du CF<T> précédent (vu comme préfixe (this)) , retourne un CF<Void> correspondant à la future exécution asynchrone de la tâche terminale r (Runnable sans argument en entrée)
boolean complete (T value) et boolean completeExceptionally (Throwable ex)	si tâche pas terminée , fixe le résultat de cette tâche qui sera ultérieurement récupérée par .get().

Remarque : si cf.*thenApplyAsync*(...); à la place de cf.*thenApply*(...); alors tâche quelquefois exécutée par encore un autre thread.

2.1. Premiers exemples de CompletableFuture

EssaiAsyncJava8.java

```
package tp.langage.thread;

import java.util.concurrent.CompletableFuture; import java.util.function.Supplier;

public class EssaiAsyncJava8 {
    public static void main(String[] args) {
        System.out.println("debut main / interpreted by " + Thread.currentThread().getName());

        //initialisation asynchrone:
        //CompletableFuture<Void> cf = CompletableFuture.runAsync(aRunnableObject); //with no return result !!!
        CompletableFuture<Double> completableFuture1 =
            /*CompletableFuture.supplyAsync( new Supplier<Double>(){
                public Double get(){ LongTask.simulateLongTask("long computing - p1", 2000);
                    return 2.0; }
            });*/
        CompletableFuture.supplyAsync( ()-> { LongTask.simulateLongTask("long ...- p1", 2000);
            /*throw new RuntimeException("exceptionXY");*/ return 2.0; } );

        //En cas d'exception en asynchrone/tâche de fond:
        CompletableFuture<Double> safecompletableFuture1 =
            completableFuture1.exceptionally(ex -> { System.out.println("problem: " +
                ex.getMessage()); return 0.0; } );
        /*CompletableFuture<Double> safecompletableFuture1 =
            completableFuture1.handle((resOk,ex) -> {
                if(resOk!=null) return resOk;
                else { System.out.println("problem: " + ex.getMessage()); return 0.0; } } );*/

        System.out.println("suite main A / interpreted by " + Thread.currentThread().getName());

        // continuations asynchrones (avec Function<T1,T2>):
        CompletableFuture<Double> completableFuture2 =
            safecompletableFuture1.thenApply((x) -> { LongTask.simulateLongTask("long computing -
                p2", 2000); return x*x; } );

        CompletableFuture<String> completableFuture3 =
            completableFuture2.thenApply((x) -> { LongTask.simulateLongTask("long computing - p3",
                2000); return String.valueOf(x); } );

        System.out.println("suite main B / interpreted by " + Thread.currentThread().getName());
        //fin/terminaison asynchrone:
        completableFuture3.thenAccept((x) -> System.out.println(x) );
    }
}
```

```
//completableFuture.thenRun()->System.out.println("ok");//with no input !!!

LongTask.simulateLongTask("pause pour éviter arrêt complet du programme " +
                           " avant la fin des taches de fond" , 8000);
System.out.println("fin main / interpreted by " + Thread.currentThread().getName());
}
}
```

Résultats :

```
debut main / interpreted by main
suite main A / interpreted by main
>>(begin)long computing - p1 / by ForkJoinPool.commonPool-worker-3
suite main B / interpreted by main
>>(begin)pause pour éviter arrêt complet du programme avant la fin des taches de fond / by main
<<(end)long computing - p1 / by ForkJoinPool.commonPool-worker-3
>>(begin)long computing - p2 / by ForkJoinPool.commonPool-worker-3
<<(end)long computing - p2 / by ForkJoinPool.commonPool-worker-3
>>(begin)long computing - p3 / by ForkJoinPool.commonPool-worker-3
<<(end)long computing - p3 / by ForkJoinPool.commonPool-worker-3
4.0
<<(end)pause pour éviter arrêt complet du programme avant la fin des taches de fond / by main
fin main / interpreted by main
```

Variation syntaxique (générant le même résultat) :

EssaiAsyncJava8V2.java

```
package tp.langage.thread;

import java.util.concurrent.CompletableFuture;

public class EssaiAsyncJava8V2 {

    public static Double extractInitValue(){
        LongTask.simulateLongTask("long computing - p1" , 2000);
        /*throw new RuntimeException("exceptionXY");*/ return 2.0;
    }

    public static Double auCarre(Double x){
        LongTask.simulateLongTask("long computing - p2" , 2000); return x*x;
    }

    public static String convertAsString(Double x){
        LongTask.simulateLongTask("long computing - p3" , 2000); return String.valueOf(x);
    }
}
```

```

public static void displayString(String s){
    System.out.println(s) ;
}

public static void main(String[] args) {
    System.out.println("debut main / interpreted by " + Thread.currentThread().getName());

    CompletableFuture.supplyAsync(EssaiAsyncJava8V2::extractInitValue )
        .exceptionally(ex -> { System.out.println("problem: " +
            ex.getMessage()); return 0.0; } )
        .thenApply(EssaiAsyncJava8V2::auCarre )
        .thenApply(EssaiAsyncJava8V2::convertAsString)
        .thenAccept(EssaiAsyncJava8V2::displayString );

    System.out.println("suite main / interpreted by " + Thread.currentThread().getName());
    LongTask.simulateLongTask("pause pour eviter arrêt complet du programme" +
        " avant la fin des taches de fond" , 8000);
    System.out.println("fin main / interpreted by " + Thread.currentThread().getName());
}
}

```

Autre variation syntaxique avec this et lambda expressions :

EssaiAsyncJava8V2Bis.java

```

package tp.langage.thread;
import java.util.concurrent.CompletableFuture;
public class EssaiAsyncJava8V2Bis {
    public static void simuLong(String number,long nbMs) {
        LongTask.simulateLongTask("long computing - p"+number , nbMs);
    }
    private double initialXValue;
    private String result;

    public void noStaticMethodWithLambda() {
        this.initialXValue=2;
        //code am"lirable avec synchronized(this){ this... }
        CompletableFuture.supplyAsync(()->{ simuLong("p1",2000);
            return this.initialXValue; } )
            .thenApply( (x)->{ simuLong("p2",2000); return x*x; } )
            .thenApply( (x)->{ simuLong("p3",2000); return String.valueOf(x); } )
            .thenAccept( (s)->{System.out.println(s); this.result=s});
        LongTask.simulateLongTask("pause pour eviter arrêt complet du programme"
            + " avant la fin des taches de fond" , 8000);
    }
}

```

```

        System.out.println("result="+this.result);
    }

    public static void main(String[] args) {
        EssaiAsyncJava8V2Bis thisApp = new EssaiAsyncJava8V2Bis();
        thisApp.noStaticMethodWithLambda();
    }
}

```

result=4

2.2. Combinaisons avec CompletableFuture

Exemple partiel : *EssaiAsyncJava8V3.java*

```

package tp.langage.thread;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionStage;

public class EssaiAsyncJava8V3 {

    //plein de code commun avec EssaiAsyncJava8V2 (pas répété ici)

    public static Double plus(Double x, Double y){
        LongTask.simulateLongTask("long computing - plus" , 2000); return x+y;
    }

    public static CompletionStage<Double> cfInitVal() {
        return CompletableFuture.supplyAsync(EssaiAsyncJava8V3::extractInitValue );
    }

    //méthode static pour pour then.compose(...) :
    public static CompletionStage<String> cfAsString(Double x){
        CompletableFuture<Double> cfDouble = new CompletableFuture<Double>();
        cfDouble.complete(x);
        System.out.println("**** tout le debut de cfAsString sans attente ****");
        return cfDouble.thenApplyAsync(EssaiAsyncJava8V3::convertAsString);
    }

    public static void main(String[] args) {
        System.out.println("debut main de EssaiAsyncJava8V3");
        //thenCompose register another future to apply/compose to thisFuture (without waiting)
        // to produce a new future :
        cfInitVal().thenCompose(EssaiAsyncJava8V3::cfAsString)
            .thenAccept(EssaiAsyncJava8V3::displayString );

        CompletableFuture<Double> cfD1 = new CompletableFuture<Double>();
    }
}

```

```

cfD1.complete(3.0); //COMPLETE BY MAIN --> NEED .thenApplyAsync() to be continued by other thread

CompletableFuture<Double> cfD2 = new CompletableFuture<Double>();
cfD2.complete(2.0); //COMPLETE BY MAIN --> NEED .thenApplyAsync() to be continued by other thread
System.out.println("suite du main de EssaiAsyncJava8V3");
CompletableFuture<Double> cfD3 = cfD1.thenApplyAsync(EssaiAsyncJava8V3::auCarre);//3*3=9
CompletableFuture<Double> cfD4 = cfD2.thenApplyAsync(EssaiAsyncJava8V3::auCarre);//2*2=4

//thenCombine apply a biFunction from 2 futures (this & other) to produce a new future

CompletableFuture<Double> cfD5 =
    cfD3.thenCombine(cfD4, EssaiAsyncJava8V3::plus);//9+4=13
cfD5.thenAccept((x)->System.out.println(x));

CompletableFuture<Double> cfD6 = cfD1.thenApplyAsync(EssaiAsyncJava8V3::auCarre);//3*3=9
CompletableFuture<Double> cfD7 = cfD2.thenApplyAsync(EssaiAsyncJava8V3::auCarre);//2*2=4
cfD6.thenAcceptBoth(cfD7, (x,y) -> System.out.println(x*y));//9*4 = 36

LongTask.simulateLongTask("pause pour éviter arrêt complet du programme "
    + "avant la fin des taches de fond", 8000);
System.out.println("fin main de EssaiAsyncJava8V3 / interpreted by " + Thread.currentThread().getName());
}
    
```

Autres combinaisons :

Variantes proches de thenCombine :

.thenAcceptBoth(otherFuture, biConsumer) --> action terminale (sans valeur calculée ni retournée) avec valeurs en entrées lorsque les 2 futures sont terminés

.runAfterBoth(otherFuture, runnable) --> action sans valeur en entrée lorsque les 2 futures sont terminés

Variantes proches de thenAcceptBoth/runAfterBoth :

.thenAcceptEither(otherFuture, consumer) --> action avec valeur en entrées lorsque le premier des 2 futures est terminé

.runAfterEither(otherFuture, runnable) --> action sans valeur en entrée lorsque le premier des 2 futures est terminé

Variante proche de thenAcceptEither :

thenApplyEither(otherFuture , function) --> génère un nouveau future (avec valeur) pour poursuivre un enchaînement avec thenApply() ou autre.

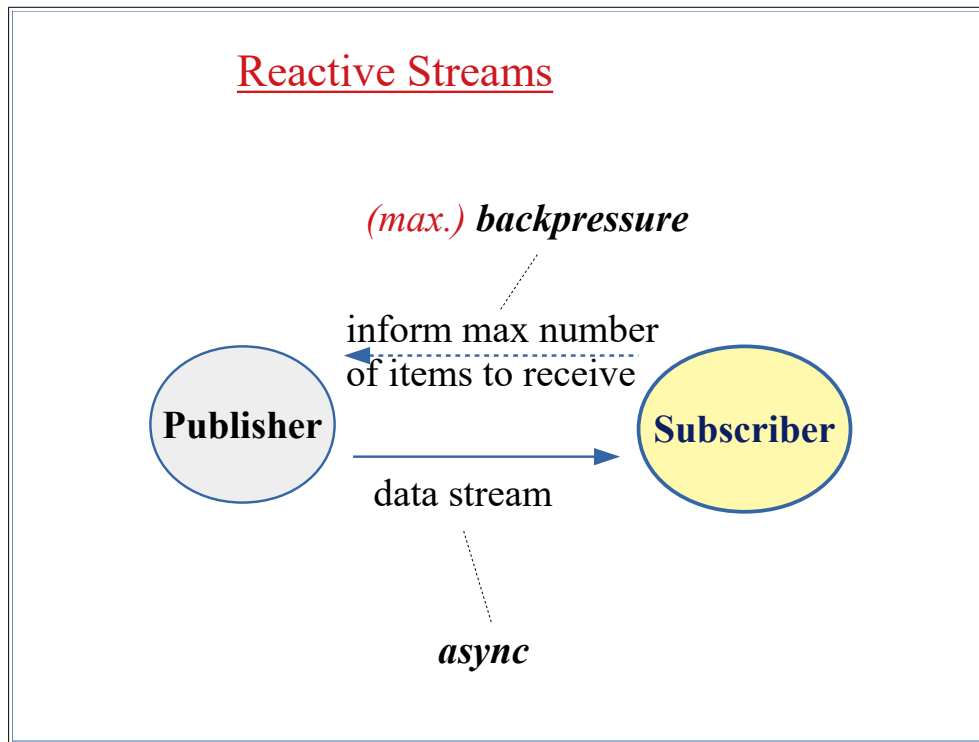
Variantes avec nombres d'arguments variables:

.allOf(...) pour attendre la fin de n(3 ou plus) futures

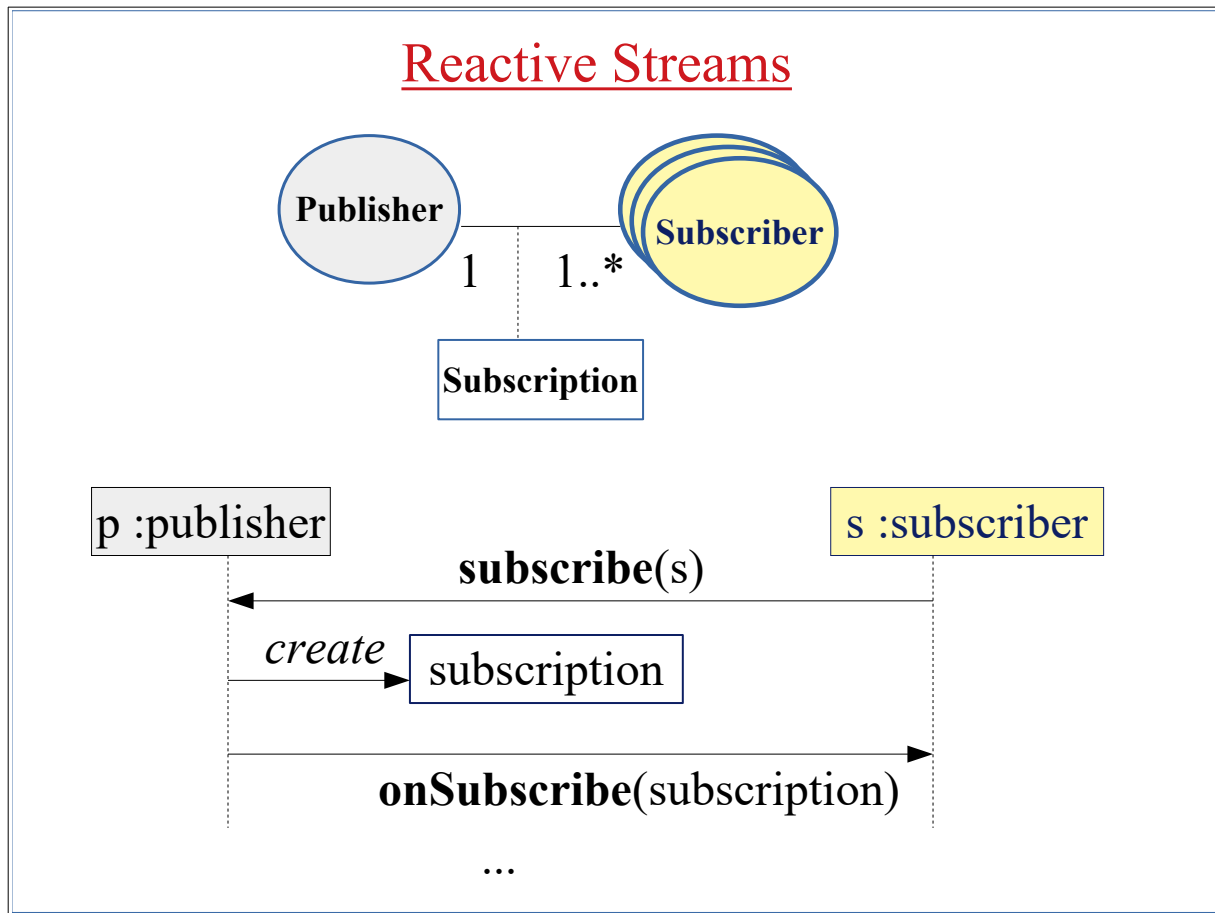
`.anyOf(...)` pour attendre la fin du plus rapide parmi n

3. Reactive Streams (depuis java 9)

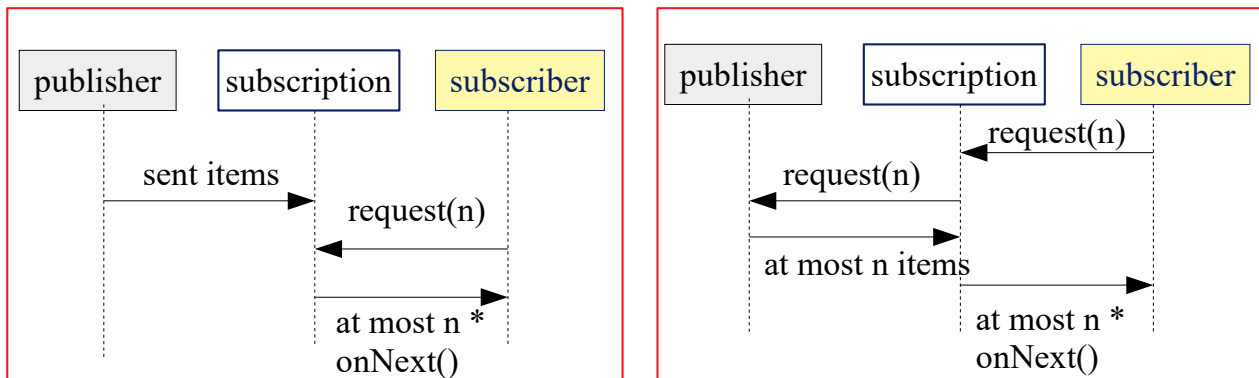
3.1. Concepts des "reactive streams"



Reactive Streams



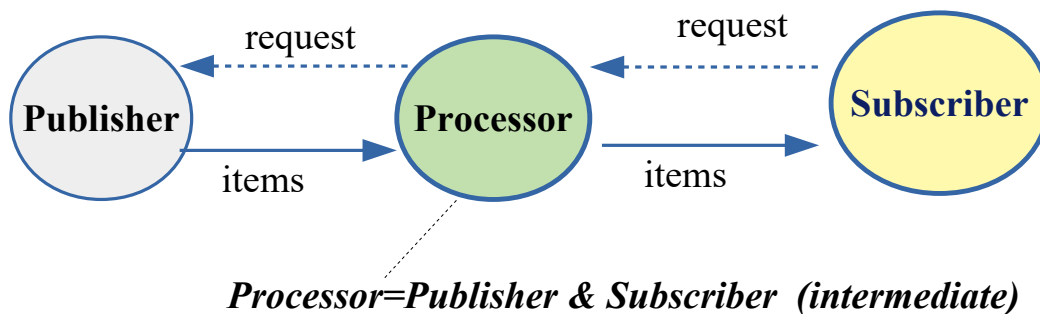
Reactive Streams



queue approach/scenario

Or

generate onDemand scenario



3.2. Abstractions standards du langage java (Flow)

La pseudo classe **Flow** du package **java.util.concurrent** ne fait que regrouper un ensemble d'interfaces complémentaires :

```
public final class Flow {
    @FunctionalInterface
    public static interface Publisher<T> {
        public void subscribe(Subscriber<? super T> subscriber);
    }
    public static interface Subscriber<T> {
        public void onSubscribe(Subscription subscription); //notification of subscription
        public void onNext(T item); //data/item reception/notification
        public void onError(Throwable throwable); //exception reception/notification
        public void onComplete(); //end of stream notification (no more item to receive)
    }
}
```



```

public static interface Subscription {
    public void request(long n); //ask to receive at most n more items , may be call several times
    public void cancel();
}

public static interface Processor<T,R> extends Subscriber<T>, Publisher<R> {}

```

--> utilisation avec implémentations de type

*class **MySubscriber**<T> implements **Flow.Subscriber**<T> { ... } , etc*

3.3. Mise en oeuvre élémentaire via **SubmissionPublisher**

NB : La classe `java.util.concurrent.SubmissionPublisher<T>` est une implémentation prédéfinie simple de `Flow.Publisher<T>` qui comporte une méthode `.submit()` à généralement appeler plusieurs fois pour envoyer des données aux abonnés (*subscribers*) .

Exemple :

MySimplePrintSubscriber.java

```

package tp.langage.flow;

import java.util.concurrent.Flow;
import java.util.concurrent.Flow.Subscription;

public class MySimplePrintSubscriber implements Flow.Subscriber<Object> {
    private Subscription subscription;
    private String subscriberName;
    private int nbTotalReceived = 0;
    private int nbTotalRequest = 0;
    private static final int NB_OTHER_ITEMS=4;

    public MySimplePrintSubscriber() {
        this.subscriberName="mySimplePrintSubscriber";
    }
    public MySimplePrintSubscriber(String subscriberName){
        this.subscriberName=subscriberName;
    }

    private void requestSomeItems() {
        subscription.request(NB_OTHER_ITEMS);
        this.nbTotalRequest+=NB_OTHER_ITEMS;
    }

    @Override
    public void onSubscribe(Subscription subscription) {

```

```

    this.subscription = subscription;
    System.out.println(this.subscriberName + ">> Received subscription notification");
    requestSomeItems();
}

@Override
public void onNext(Object item) {
    this.nbTotalReceived++;
    System.out.println(this.subscriberName + ">> Received item: " + item.toString() + " thread:"
        + Thread.currentThread().getName() + " nbTotalRequest="
        + this.nbTotalRequest + " nbTotalReceived="+this.nbTotalReceived);
    if(nbTotalReceived==nbTotalRequest) {
        requestSomeItems();
    }
}

@Override
public void onError(Throwable error) {
    System.out.println(this.subscriberName + ">> Error occurred: " + error.getMessage());
}

@Override
public void onComplete() {
    System.out.println(this.subscriberName + ">> complete");
}
}

```

MySubmissionPublisherApp.java

```

package tp.langage.flow;

import java.util.concurrent.SubmissionPublisher;

public class MySubmissionPublisherApp {

    public static void pause(long nbMs) {
        try { Thread.sleep(nbMs); } catch (InterruptedException e) { e.printStackTrace(); }
    }

    public static void main(String[] args) {
        sansProcesseur();
    }

    public static void sansProcesseur() {
        SubmissionPublisher<Object> publisher = new SubmissionPublisher<>();
        publisher.subscribe(new MySimplePrintSubscriber("s1"));
        System.out.println("Submitting first items...");
        for (int i = 0; i < 10; i++) {
            publisher.submit(i); //onNext() may be called on subscriber(s) if request(...)
            if(i==5) {
                pause(1000);
            }
        }
    }
}

```

```

        publisher.subscribe(new MySimplePrintSubscriber("s2"));
        System.out.println("Submitting others items...");
    }
}
pause(1000);
publisher.close(); //onComplete() will be called on subscriber(s)
}
}

```

Résultats :

```

Submitting first items...
s1>> Received subscription notification thread:ForkJoinPool.commonPool-worker-3
** debut de pause de 1000 ms , thread:main
s1>> Received item: 0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=1
s1>> Received item: 1 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=2
s1>> Received item: 2 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=3
s1>> Received item: 3 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=4
s1>> Received item: 4 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=5
s1>> Received item: 5 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=6
** fin de pause de 1000 ms , thread:main
Submitting others items...
s2>> Received subscription notification thread:ForkJoinPool.commonPool-worker-3
** debut de pause de 1000 ms , thread:main
s2>> Received item: 6 thread:ForkJoinPool.commonPool-worker-5 nbTotalRequest=4 nbTotalReceived=1
s1>> Received item: 6 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=7
s2>> Received item: 7 thread:ForkJoinPool.commonPool-worker-5 nbTotalRequest=4 nbTotalReceived=2
s1>> Received item: 7 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=8
s2>> Received item: 8 thread:ForkJoinPool.commonPool-worker-5 nbTotalRequest=4 nbTotalReceived=3
s1>> Received item: 8 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=12 nbTotalReceived=9
s2>> Received item: 9 thread:ForkJoinPool.commonPool-worker-5 nbTotalRequest=4 nbTotalReceived=4
s1>> Received item: 9 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=12 nbTotalReceived=10
** fin de pause de 1000 ms , thread:main
s2>> complete thread:ForkJoinPool.commonPool-worker-5
s1>> complete thread:ForkJoinPool.commonPool-worker-3

```

AuCarreProcessor.java

```

package tp.langage.flow;
import java.util.concurrent.Flow.Subscriber; import java.util.concurrent.Flow.Subscription;
import java.util.concurrent.SubmissionPublisher;

public class AuCarreProcessor extends SubmissionPublisher<Object>
    implements Subscriber<Object> {
    private Subscription subscription;

    @Override
    public void onSubscribe(Subscription subscription) {
        this.subscription = subscription;
        subscription.request(1);
    }

    @Override
    public void onNext(Object item) {
        double val = Double.parseDouble(item.toString());
        submit(String.valueOf(val*val));
        subscription.request(1);
    }
}

```

```

}

@Override
public void onError(Throwable error) {
    error.printStackTrace();
    closeExceptionally(error);
}

@Override
public void onComplete() {
    System.out.println("AuCarreProcessor completed");
    close();
}
}

```

MySubmissionPublisherApp.java (V2)

```

package tp.langage.flow;

import java.util.concurrent.SubmissionPublisher;

public class MySubmissionPublisherApp {

    //.... pause() et sansProcessor()

    public static void main(String[] args) {
        //sansProcesseur();
        avecProcesseur();
    }

    public static void avecProcesseur() {
        //publisher <--> processor <--> subscriber
        AuCarreProcessor processor = new AuCarreProcessor();
        processor.subscribe(new MySimplePrintSubscriber("s1"));
        SubmissionPublisher<Object> publisher = new SubmissionPublisher<>();
        publisher.subscribe(processor);
        System.out.println("Submitting items...");
        for (int i = 0; i < 10; i++) {
            publisher.submit(i); //onNext() may be called on subscriber(s) if request(...)
        }
        pause(1000);
        publisher.close(); //onComplete() will be called on subscriber(s)
    }
}

```

Résultats :

```

Submitting items...
s1>> Received subscription notification thread:ForkJoinPool.commonPool-worker-3
** debut de pause de 1000 ms , thread:main
s1>> Received item: 0.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=1
s1>> Received item: 1.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=2
s1>> Received item: 4.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=3
s1>> Received item: 9.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=4 nbTotalReceived=4
s1>> Received item: 16.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=5
s1>> Received item: 25.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=6

```

```
s1>> Received item: 36.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=7
s1>> Received item: 49.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=8 nbTotalReceived=8
s1>> Received item: 64.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=12 nbTotalReceived=9
s1>> Received item: 81.0 thread:ForkJoinPool.commonPool-worker-3 nbTotalRequest=12 nbTotalReceived=10
** fin de pause de 1000 ms , thread:main
AuCarreProcessor completed
s1>> complete thread:ForkJoinPool.commonPool-worker-5
```

3.4. "Reactive Streams" avec RxJava

RxJava (*proche de RxJs*) est une des implémentations proches des "Reactive Streams" de java>=9 .

RxJava existe depuis longtemps et fait partie de la famille des technologies créées par Netflix . Il existe des versions 1.x et 2.x et 3.x .

RxJava comporte plein de similitude avec la version javascript "RxJs" dont l'abstraction "**Observable**" .

RxJava est souvent utilisé au sein de développement Android pour effectuer des appels Http en asynchrone (un peu comme RxJs au sein d'une application "Angular") .

Certaines versions de RxJava peuvent être utilisées avec le jdk 1.6 .

La version 3.x nécessite au moins java 1.8 .

URL officielle de "RxJava" : <https://github.com/ReactiveX/RxJava>

3.5. "ReactiveStreams" avec Reactor

Reactor est une autre implémentation proche des "Reactive Streams" de java>=9 .

La classe reactor.adapter.JdkFlowAdapter comporte des méthodes .publisherToFlowPublisher() et .flowPublisherToFlux() permettant de convertir des **Flux** de **Reactor** en Flow java9 et vice versa .

Le projet "Reactor" est en partie géré par la communauté "Spring" et est intégré dans le framework "Spring 5" avec les "WebFlux" .

Reactor est un projet plus récent que RxJava . Il nécessite au minimum le jdk 1.8 (comme Spring5)

URL officielle de Reactor : <https://projectreactor.io>

XII - Fondamentaux réseau et sockets

1. Rappels fondamentaux (réseaux)

1.1. Couches réseaux

couches réseaux	fonctionnalités / responsabilités / caractéristiques
7 – Application	logique de traitement spécifique à une application
6 - Présentation	couche logicielle prenant en charge le format des données véhiculées (transformations éventuelles , entêtes avec longueurs , ...) ceci est nécessaire pour faire communiquer des machines différentes (Unix, Win32, 64bits ,) et pour interfacer des programmes écrits dans des langages différents (C , java , ...)
5 - Session	prise en charge des session utilisateurs (connexions, déconnexions,)
4 - Transport	couche logicielle permettant de s'assurer que les paquets soient bien transportés de bout en bout (pas de pertes , bon ordre ,)
3 - Réseau	couche logicielle (avec adressage logique) permettant d'acheminer des paquets en traversant si besoin des routeurs et de multiples réseaux physiques
2 - Liaison	couche logicielle pilotant un (sous)-réseau physique : ex: Ethernet , Token-Ring,
1 - Physique	matériel réseau = carte réseau , cables , ... avec adresse réseau physique / mac

1.2. TCP/IP

Adresse IP = adresse logique (attribuée par configuration/administration) et indépendante de la machine .

=====> permet de traverser les routeurs

==> consulter un site internet de type "*wikipédia*" ou bien "*comment-ça-marche*" pour obtenir des détails sur la **structure d'une adresse IP** (ex: 192.56.23.78) .

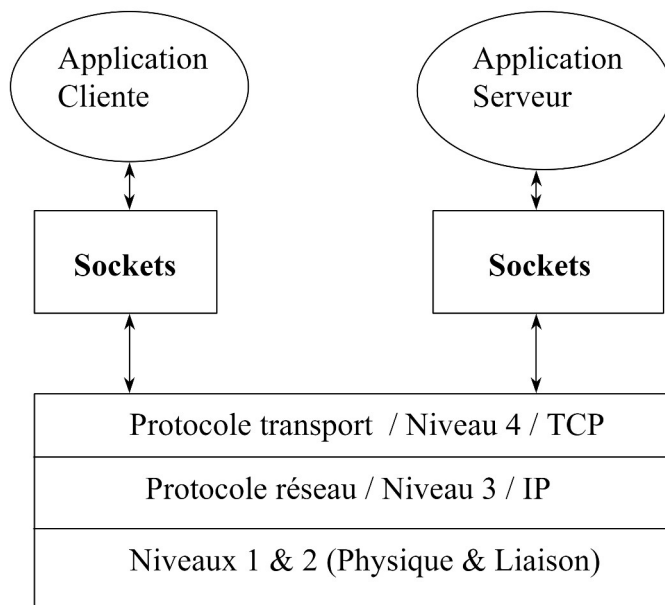
TCP = Transport Control Protocol ==> permet de s'assurer que de bout en bout (quelque soit les routages intermédiaires) les paquets d'un grand messages ne soient ni perdus, ni dupliqués , ni dans le désordre.

2. Api des sockets originelle (langage c)

Les **sockets** constituent une **API très standard (simple et d'assez bas niveau)** provenant du monde **UNIX pour établir des communications réseaux (transferts de paquets d'octets)**.

A l'origine (dans le monde UNIX) les sockets avaient été introduites pour effectuer des transferts d'informations à travers le réseau tout en employant le formalisme "Fichier" (lecture/écriture ...).

Les sockets s'appuient sur le modèle classique "Client / Serveur" :



L'API des sockets se situe juste au dessus des couches "réseau" et "transport" (3 et 4). Il faut donc gérer la représentation des informations que l'on envoie sur le réseau.

2.1. Notion d'adresse internet :

Pour identifier le service distant avec lequel il faut entrer en communication, l'API des sockets utilise la notion d'adresse internet. Il s'agit d'une structure de donnée qui regroupe les informations suivantes:

- Adresse IP de l'ordinateur (exemple: 128.127.12.10)
- N° de port du service (exemple: 9999)
- Protocole utilisé (UDP, TCP, ...)

NB: Certains services ont des n° de port réservé (ex: http = 80, ftp = 21, telnet = 23, ...). Il faut donc utiliser des n° de port élevé pour ne pas entrer en conflit avec ceux qui sont prédéfinis (voir fichier [WinNt/system32/drivers]/etc/services).

2.2. représentation (format/encodage) réseau

Les machines connectées au réseau peuvent être de différentes natures (PC , station UNIX ,). Sur ces différentes machines la représentation interne des nombres entiers peut éventuellement être différente (octets de poids forts avant ou après ceux de poids faibles).

Les adresses réseaux qui sont des nombres entiers doivent absolument être manipulées sans ambiguïté pour que la communication puisse s'établir correctement.

Pour régler ce problème, l'API des sockets comporte un ensemble de fonctions qui servent à convertir des nombres de la représentation réseau vers la représentation machine (et inversement).

Fonction	Action
u_long htonl(u_long hostlong)	host to network long (32 bits)
u_short htons(u_short hostshort)	host to network short (16 bits)
u_long ntohl(u_long netlong)	network to host long
u_short ntohs(u_short nettshort)	network to host short

D'autre part, la fonction **inet_addr()** permet de convertir en un nombre entier une chaîne de caractères contenant une adresse internet (notation pointée ex: "128.126.200.45").

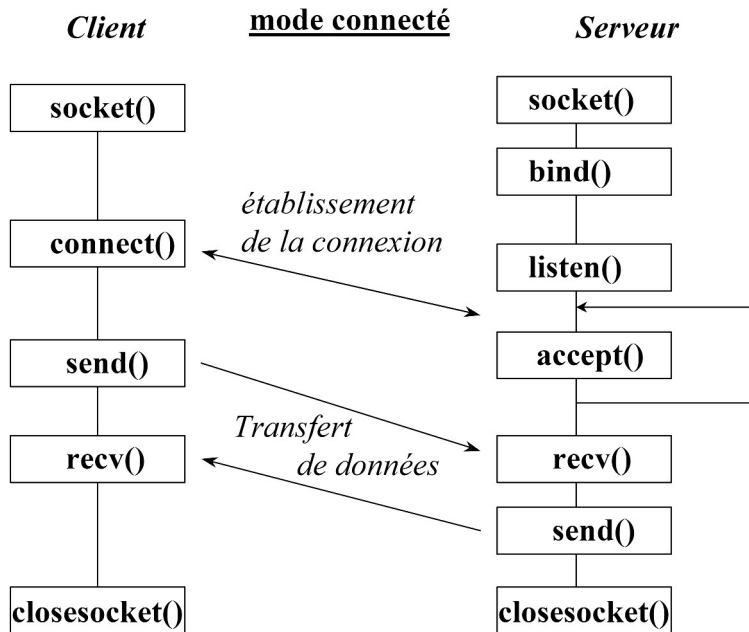
Inversement, la fonction **inet_ntoa()** permet de convertir l'adresse IP en une chaîne ascii.

L' API des sockets comporte un ensemble de fonctions **get...by...** qui permettent de récupérer des informations telles que l'adresse IP ou le nom réseau d'une machine.

Ces informations sont en partie récupérées dans */etc/hosts* ou *%systemroot%/system32/drivers/etc/hosts* et *.../etc/services* et *protocole DNS* , ...

Fonction	Action
gethostbyaddr()	Donne le nom en fonction de l'adresse réseau
gethostname()	Donne le nom de la machine locale
gethostbyname()	Donne l'adresse en fonction du nom
getprotobyname()	Donne le numéro du protocole en fonction d'un nom
getprotobynumber()	Donne le nom d'un protocole en fonction de son numéro.
getservbyname()	Donne le port en fonction d'un nom de service
getservbyport()	Donne le nom du service en fonction du port

2.3. Mode connecté (SOCK_STREAM):



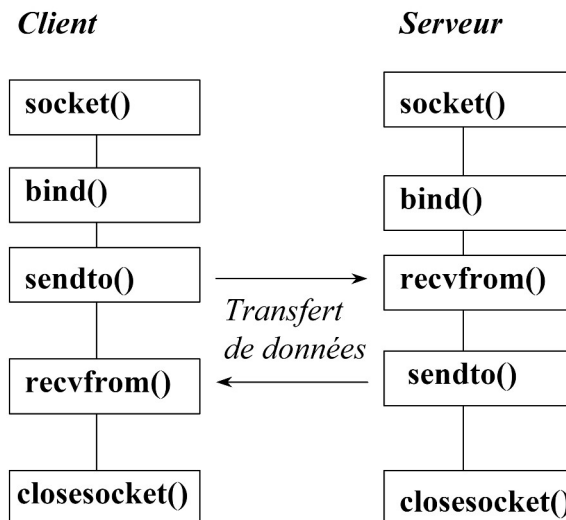
Le mode connecté **SOCK_STREAM** permet d'établir des communications fiables , bidirectionnelles entre deux entités connectées . Utilisation de TCP.

Les sockets de type SOCK_STREAM gèrent des flux d'octets en full-duplex.

Les protocoles de communication utilisés en mode connecté permettent de s'assurer qu'il n'y a aucune perte ni aucune duplication de données. Si au bout d'un moment, les données n'ont toujours pas pu être transmises, la connexion est brisée --> message d'erreur de type Timeout

2.4. Mode non connecté (SOCK_DGRAM)

mode datagramme (non connecté)



Un datagramme, est un petit bloc de données de taille fixe qui est envoyé vers un autre élément du réseau sans s'y être préalablement connecté. Ce mécanisme n'est pas très fiable (On utilise généralement UDP) . Les performances sont par contre assez radicales .

Le mode non connecté **SOCK_DGRAM** permet d'envoyer (et de recevoir) des datagrammes vers un correspondant quelconque en utilisant les fonctions **sendto** et **recvfrom**.

3. Vue d'ensemble sur les sockets java

Les classes java **Socket** et **ServerSocket** du package *java.net* fonctionnent en mode **TCP** avec une logique de connexion à établir avant d'entamer des communications bidirectionnelles fiables pendant un certains temps.

Les classes **DatagramSocket** et **DatagramPacket** du package *java.net* fonctionnent en mode **UDP** avec une logique d'envoi/réception spontané sans connexion pré-établie .

NB: étant donné que l'api des sockets est une api de très bas niveau , il faudra veiller à ce que les applications qui communiquent entre elles se mettent d'accord sur :

- le format des messages véhiculés (en binaire ou en mode texte, taille fixe ou taille variable, ...).
- le protocole d'échange prévu (simple envoi , envoi/réponse unique , cycle d'envois/réponses, ...)

4. Socket du côté Serveur

Principaux constructeurs de `ServerSocket` :

<code>ServerSocket()</code>	Default port number = 0 $\Rightarrow$ dynamically allocated (dans plage des ports éphémères)
<code>ServerSocket(port)</code>	Avec numéro de port spécifié
<code>ServerSocket(port,backlog)</code>	En précisant en plus le paramètre backlog = nombre maximum de requête de connexion en attentes dans la file (valeur par défaut =50)

Comportement essentiel :

Chaque **nouvelle demande de connexion** de la part d'un nouveau client est **acceptée** par la méthode `.accept()` qui **retourne une nouvelle socket spécifiquement prévue pour communiquer avec ce client** .

Lorsque le serveur a besoin de gérer simultanément plusieurs clients, cette nouvelle socket liée au client est généralement exploitée par un nouveau thread exécutant le code d'une tâche de dialogue/communication spécifique aux fonctionnalités du serveur .

```
import java.net.ServerSocket;  import java.net.Socket;

public class MySocketServer {
    static int port =9632; //par défaut
    public static void main(String[] args) {
        try {
            ServerSocket socketServeur = new ServerSocket(port);
            System.out.println("Lancement du serveur MySocketServer port="+port);
            while (true) {
                Socket socketClient = socketServeur.accept();
                Thread t = new Thread(new MySocketTask(socketClient));
                t.start();
            }
        } catch (Exception e) { e.printStackTrace();    }
    }
}
```

et

```
public class MySocketTask implements Runnable {
    private Socket socketClient;
    public MySocketTask(Socket socketClient){ this.socketClient=socketClient;  }
    @Override
    public void run() {
        try { InputStream in = socketClient.getInputStream();
            OutputStream out = socketClient.getOutputStream(); ... } catch(...){...}
        }
    }
}
```

5. Socket du coté client

```
import java.io.*; import java.net.UnknownHostException;
import java.net.InetAddress;    import java.net.Socket;
```

```
public class MySocketClient {
    static int port = 9632;
    ...
    public static void main(String[] args) {
        String host = (args.length>0)?args[0]:"localhost";
        InetAddress serveur = null;
        try {
            serveur = InetAddress.getByName(host);
        } catch (UnknownHostException e) {
            throw new RuntimeException(e);
        }

        try (Socket socket = new Socket(serveur, port)){
            send_xyz_message(socket);
        } catch (Exception e) {
            e.printStackTrace();
        } //try with auto_closeable resource=socket .
    }
}
```

et

```
public static void send_xyz_message(Socket socket){
    try {
        InputStream in = socket.getInputStream();
        OutputStream out = socket.getOutputStream();
        //send request to server:
        byte[] requestData= .... ;
        out.write(requestData);

        //waiting & retrieve response:
        //expected size of main part of response : 72 bytes or other
        byte[] mainFirstPartOfResponseData = in.readNBytes(72);
        ....
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

6. DatagramSocket en mode non connecté/UDP

6.1. Envoi d'une requête depuis un client :

```
// Initialise un objet "adresse IP" en fonction d'un nom de machine:
InetAddress address = InetAddress.getByName(hostName);

// Création du paquet à envoyer , on précise ici le n° de port du destinataire:
DatagramPacket paquet =
    new DatagramPacket (buffer, buffer.length ,address, numPortDest);

// Création d'une nouvelle socket en mode non connecté:
//NB : num port alloué dynamiquement (ex : 58564 ou autre)
DatagramSocket clientDynamicDatagramSocket = new DatagramSocket();

// Envoi du paquet:
clientDynamicDatagramSocket.send(paquet);

Si réception d'une réponse prévue :
clientDynamicDatagramSocket.setSoTimeout(30000);
clientDynamicDatagramSocket.send(paquetEnvoi);
clientDynamicDatagramSocket.receive(paquetReception);
....
```

6.2. Réception d'une requête depuis un serveur

```
DatagramSocket serverDatagramSocket= new DatagramSocket(port);

DatagramPacket paquet = new DatagramPacket(buffer, buffer.length);
serverDatagramSocket.receive(paquet); // reception (remplissage du paquet)

// conversion byte[] → String
String s=new String(buffer,0,paquet.getLength());

// Affichage de l'émetteur et du message reçu :
System.out.println("Received from " +
    packet.getAddress().getHostName() + ":" +
    packet.getPort() + ": " + s);
}

Si retour d'une réponse prévue :
DatagramPacket paquetEnvoi = new DatagramPacket(responseData,
    responseData.length, paquetReception.getAddress(), paquetReception.getPort());
serverDatagramSocket.send(paquetEnvoi);
```

7. Transfert de données binaires

7.1. Transfert de chaîne de caractères au format UTF8 sous forme de tableau d'octets

```
//transform little string (<=64 char) as utf8 bytes array of fixed size=64
public static byte[] utf8Buffer64FromLittleString(String s){
    byte[] bytes = s.getBytes(StandardCharsets.UTF_8);
    //System.out.println("initial length:" + bytes.length);
    byte[] expanded = Arrays.copyOf(bytes, 64); //fill last chars by \0
    //System.out.println("expanded length:" + expanded.length);
    return expanded;
}
```

```
//transform utf8 bytes array of fixed size=64 or ... to little String
public static String stringFromUtf8Buffer(byte[] bytes){
    String s = new String(bytes, StandardCharsets.UTF_8);
    //System.out.println("s="+s);
    int firstNullPos = s.indexOf('\0');
    s=s.substring(0,firstNullPos); //remove null chars at end of string
    //System.out.println("s="+s);
    return s;
}
```

Ce genre de fonctions utilitaires peuvent servir à transformer une chaîne java en tableau d'octets de longueur fixe de manière à dialoguer avec une autre application qui attends des transferts de données de longueurs bien déterminées (ex : 64 octets) .

Si besoin de transformer un integer i en bytes[] :

```
byte[] iData = ByteBuffer.allocate(Integer.BYTES).putInt(i).array(); // Integer.BYTES = 4
```

et dans le sens inverse :

```
ByteBuffer buffer = ByteBuffer.allocate(Integer.BYTES);
for(int i=0;i<4;i++)
    buffer.put(iData[i]);
buffer.rewind();
i = buffer.getInt();
```

Si besoin de transformer un tableau de double en bytes[] :

```
public static byte[] bytesFromDoubleArray(double array[]){  
    int n= array.length;  
    ByteBuffer buff = ByteBuffer.allocate(8*n);  
    for(int i=0;i<n;i++) {  
        buff.putDouble(i*8, array[i]);  
    }  
    return buff.array();  
}
```

et dans le sens inverse : .put() , .rewind() , .getDouble() , ...

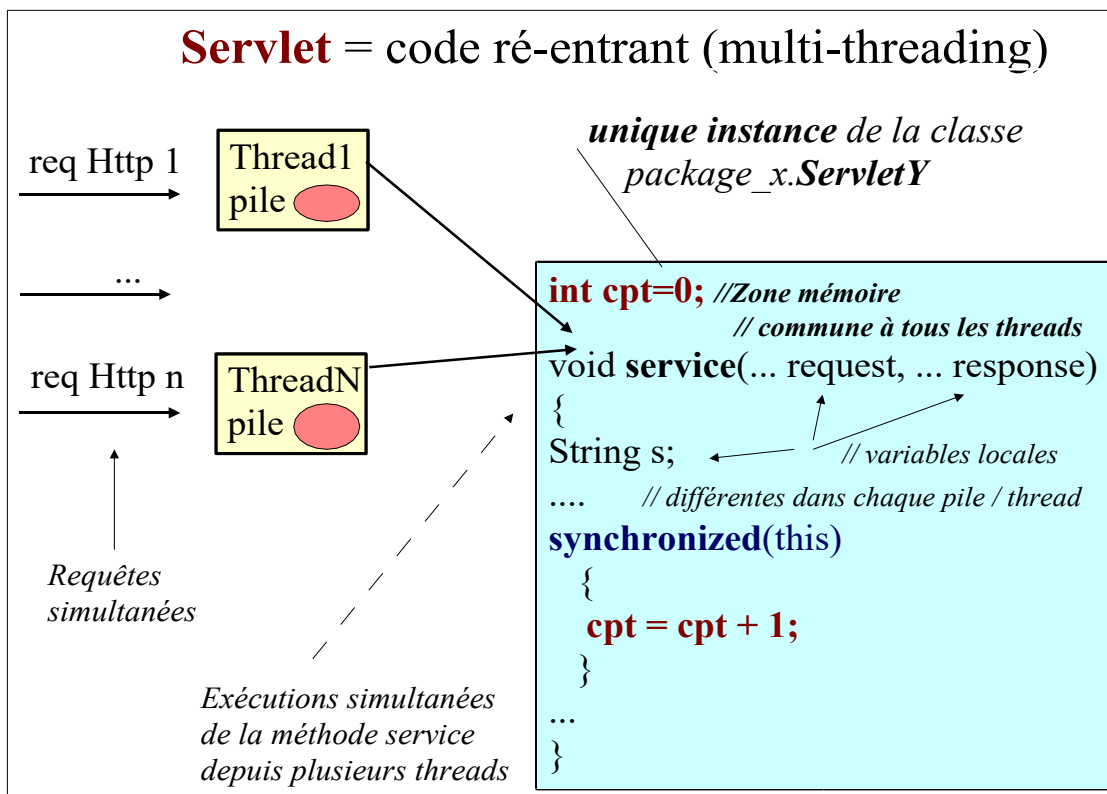
XIII - Application réseau (patterns , ...)

1. Patterns de communication

1.1. Client / serveur en mode "multi-threads"

L'application serveur démarre un nouveau thread pour traiter séparément chaque nouvelle requête entrante. Si trop de requêtes arrivent en même temps certaines d'entre elles sont mise en attente et on a généralement un nombre maximum de threads qui peuvent fonctionner en parallèle .

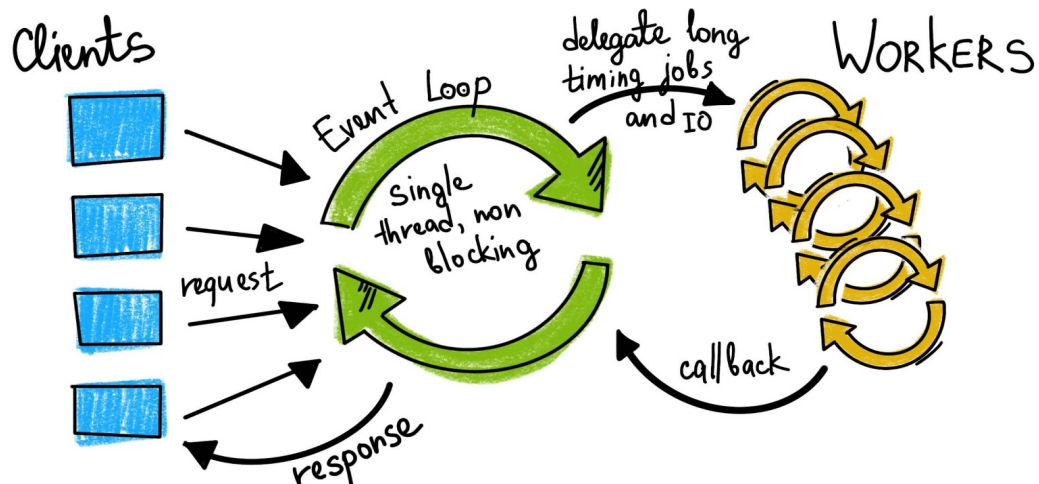
Exemple concret : "Servlet java"



NB: Les **threads virtuels** disponibles à partir de java 21 peuvent quelquefois être utiles pour gérer un très grand nombre de threads (dont certains sont en sommeil) sans trop perdre en performance .

1.2. Client / serveur en mode "REPL et callback asynchrones"

Single Threaded Event Loop Model



Exemple concret : "nodeJs" (le langage javascript moderne avec callback , Promise et async/await est assez approprié pour ce style de code) .

Exemple de mise en œuvre java : partie facultative "Netty/Reactor/WebClient" de Spring .

Java n'est pas très approprié pour ce style de code .

1.3. Peer-to-peer

Le modèle "**peer-to-peer**" (pair à pair) est un modèle de communication décentralisé où deux appareils ou plus communiquent directement entre eux sans avoir besoin d'un serveur central. Dans la communication peer-to-peer, chaque pair peut agir à la fois comme un client et un serveur, ce qui lui permet d'envoyer et de recevoir des données.

Les sockets en mode TCP/connecté ou bien UDP/Datagram non connecté sont tout à fait appropriées pour mettre en œuvre le modèle "peer-to-peer" en java .

Exemples :

<https://github.com/WindyFeg/Chat-application-P2P-java-socket>

https://swpatterns.com/codesample/peer-to-peer_java/

...

1.4. Remote Procedure Call

Exemple :

- RMI (Remote Method Invocation)
- RMI-over-IIOP (variante basée sur IIOP de Corba) et utilisée par EJB-Remote
- ...

Limitations : souvent spécifiques à un langage (ex : java)

1.5. Web-Services

Intérêts :

- interopérabilité (plusieurs langages : java , js , python, c/c++ , php , ...) et plusieurs O.S. (linux, windows, ...)
- standardisation

Exemple :

- Web-Services SOAP (en XML)
- Web-Services REST (HTTP et souvent JSON)
- ...

1.6. Echanges de messages asynchrones

Exemples :

- Api JMS (Java Message Service) avec ActiveMQ , RabbitMq , WebSphereMQ, ...
- Kafka
- ...

2. Gestion des erreurs réseaux

- Traitement approprié des exceptions
- Logs
- Reprise sur erreur si possible

3. Sécurité réseau

- DMZ et pare-feu
- Pièges et bonnes pratiques

4. Performance et optimisation (réseau)

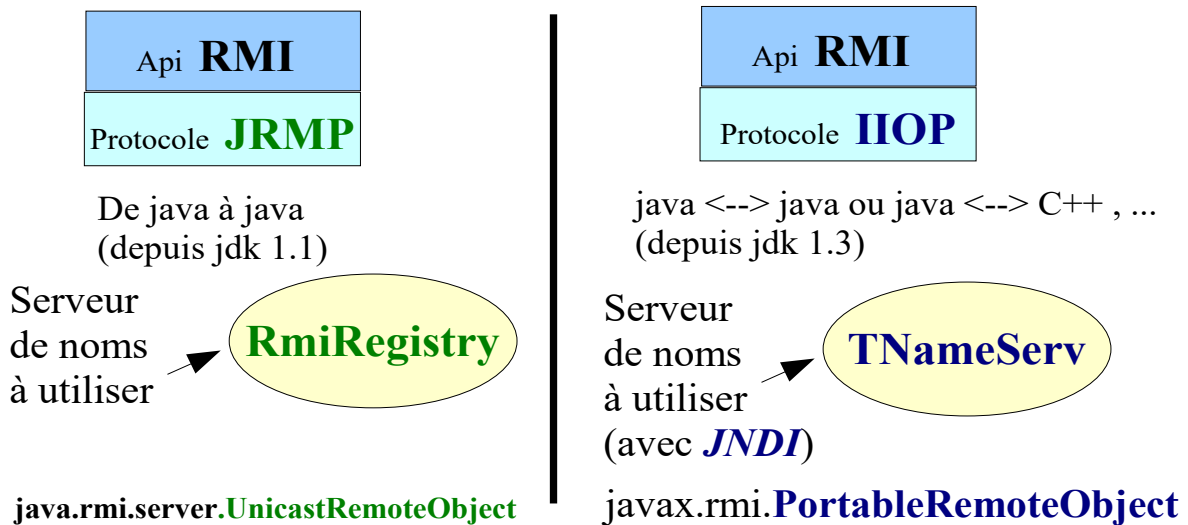
- Latences et goulets
- ...

XIV - Remote Procedure Call en java (RMI)

1. Présentation api RMI (Remote Method Invocation)

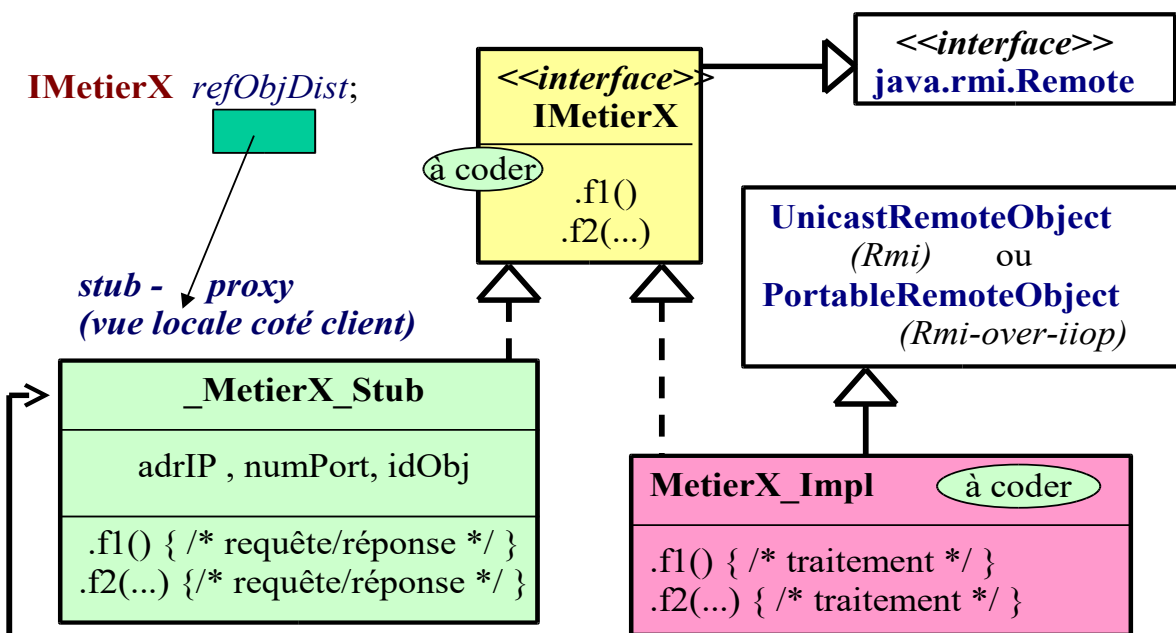
RMI (Remote Method Invocation)

R.M.I. peut s'appuyer sur les protocoles **IIOP** (lié à CORBA) ou **JRMP** (Java Remote Method Protocol).



1.1. Structure du code java (RMI)

R.M.I. – Structure du code java



généré automatiquement via **rmic** et téléchargé vers le client si besoin

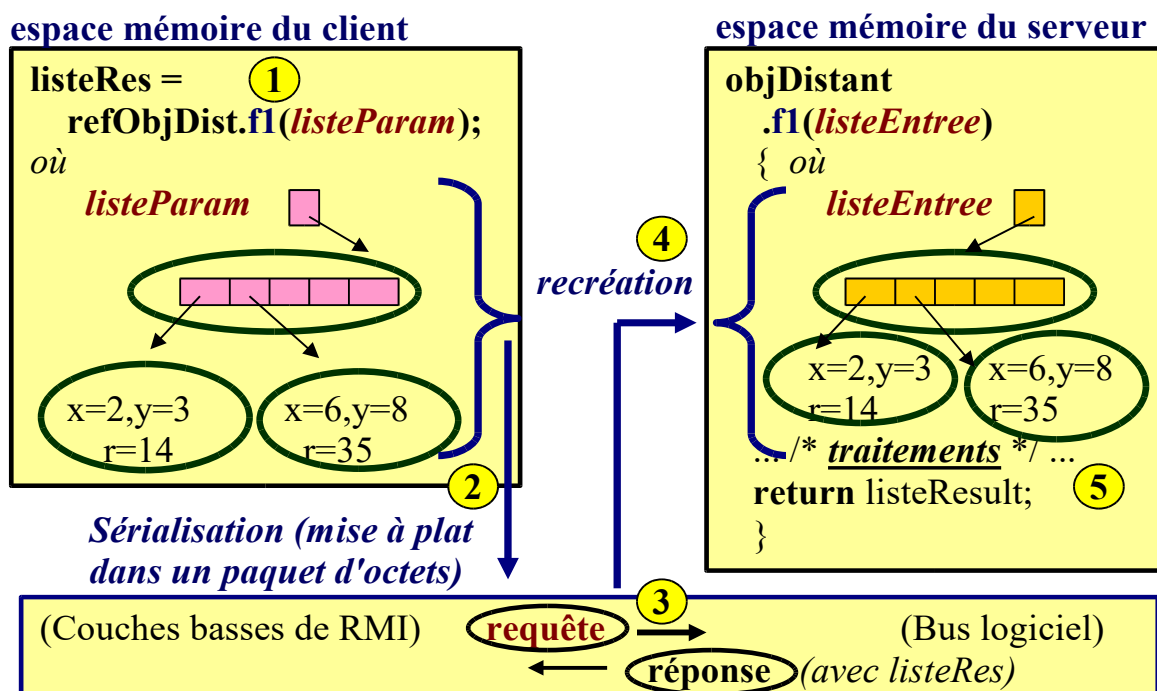
1.2. Code type de l'interface de l'objet distant (RMI):

```
package mes_interfaces;

public interface HoraireInfo extends java.rmi.Remote
{
    public String methodeA(int x,String obj) throws java.rmi.RemoteException;
    public String getChHeure() throws java.rmi.RemoteException;
}
```

1.3. Passage d'objet en paramètre d'un appel distant RMI:

Spécificités de R.M.I. - sérialisation



Très Important:

Lorsque l'on passe en paramètre un objet java local à une fonction distante, celui-ci est entièrement recopié au niveau du serveur.

Il en va de même pour les objets (ou collections d'objets) en "valeur de retour".

Cette copie automatique fait intervenir les mécanismes de **sérialisation**. Ainsi si l'objet local (en paramètre) comporte des références vers d'autres (sous-)objets locaux, c'est alors toute une grappe d'objets "java" qui est ainsi automatiquement recopiée dans l'espace mémoire du serveur.

2. Code type d'une api rmi (interface)

À placer idéalement dans un projet maven à part car besoin ultérieurement d'une dépendance à la fois du côté client et du côté serveur d'implémentation .

```
package tp.finance.api;
import java.rmi.*;
public interface Converter extends Remote {
    public double euroToFranc(double e) throws RemoteException;
    public double francToEuro(double f) throws RemoteException;
    public String getAuteur() throws RemoteException;
}
```

et

```
public interface SerieStat extends Remote {
    public Stat buildSerieStat(Serie serie) throws RemoteException;
}
```

avec

<pre>public class Serie implements java.io.Serializable { private String label; private List<Double> values ; ...} </pre>	<pre>public class Stat implements java.io.Serializable { private String label; private int taille; private double moyenne; //... } </pre>
--	--

3. Code type de l'application serveur:

```
package tp.serveur.impl;
import java.net.UnknownHostException;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import tp.finance.api.Converter;

public class ConverterImpl extends UnicastRemoteObject implements Converter {
    private static final long serialVersionUID = 1L;

    public ConverterImpl() throws RemoteException {
        super(); // pour exportation
    }
    public double euroToFranc(double e) throws RemoteException {
        return (e*6.55957);
    }
    ...
}
```

```

package tp.serveur.app;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

import tp.serveur.impl.MinibankImpl;

/* VERSION1 - RMI-over-JRMP sans JNDI */

// Application serveur RMI
public class AppServMiniBank {

    public static void main(String[] args) {
        try {
            Registry rmiRegistry = LocateRegistry.createRegistry(1099);

            System.out.println("demarrage du serveur RMI <<AppServMiniBank>> ...");
            System.out.println("... creation et exportation de la factory <<MiniBank>> ");
            MiniBankImpl miniBank = new MiniBankImpl();
            String chNom="MiniBank";
            System.out.println("... enregistrement sous le nom "+chNom);
            //Naming.rebind(chNom,miniBank);
            rmiRegistry.rebind(chNom,miniBank);
            System.out.println("... serveur initialise en attente des requetes");
        }
        catch(Exception ex)
        {
            ex.printStackTrace();
        }
    }
}

```

Petite variante avec JNDI :

```

...
java.util.Properties props = System.getProperties();
if(props.get("java.naming.factory.initial")==null)

props.put("java.naming.factory.initial",
    "com.sun.jndi.rmi.registry.RegistryContextFactory");

if(props.get("java.naming.provider.url")==null)
    props.put("java.naming.provider.url","rmi://localhost:1099");

InitialContext ctx = new InitialContext() ;
ctx.rebind(chNom,miniBank);
...

```

4. Code type de l'application cliente (Rmi-JRMP)

```

package tp.client;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
...

public class MiniBankSimpleClient {

public static void main(String[] args) {
    try {
        String chHostname = "localhost";
        if(args.length>0){
            chHostname = args[0];
        }
        System.out.println("demarrage de appli MiniBankSimpleClient ...");

        Registry rmiRegistry = LocateRegistry.getRegistry(chHostname, 1099);
        MiniBankRemoteFactory miniBank =
            (MiniBankRemoteFactory) rmiRegistry.lookup("MiniBank");
        System.out.println("connexion a la factory <<MiniBank>> effectuee .");

        SimuEmprunt simuEmprunt = (SimuEmprunt) miniBank.remoteSimulateurEmprunt();
        System.out.println("reference obtenue vers un objet distant de type SimuEmprunt.");

        double mensualite = simuEmprunt.calculMensualite(100000,2,20*12);
        System.out.println("100000 E sur 20ans avec un taux de 2%\n
            necessite des mensualites de " + mensualite);

    } catch (Exception ex) {
        ex.printStackTrace();
    }
}
}

```

Petite variante via JNDI :

```

... /* pas directement besoin de LocateRegistry dans version JNDI */
java.util.Properties props = System.getProperties();
if(props.get("java.naming.factory.initial")==null)
    props.put("java.naming.factory.initial",
        "com.sun.jndi.rmi.registry.RegistryContextFactory");
if(props.get("java.naming.provider.url")==null)
    props.put("java.naming.provider.url","rmi://" + chHostname + ":1099");
InitialContext ctx = new InitialContext() ;
MiniBankRemoteFactory miniBank = (MiniBankRemoteFactory) ctx.lookup("MiniBank");

```


NB : au sein des très anciennes versions de java , LocateRegistry.getRegistry n'existait pas encore et il fallait établir la connexion de manière complexe via :

```
import java.rmi.Naming;
import java.rmi.RMISecurityManager;
...
if(System.getSecurityManager()==null){
    System.setSecurityManager(new RMISecurityManager());
}

String chUrl = "rmi://" + chHostname + "/MiniBank";
RemoteMiniBank miniBank = (RemoteMiniBank) Naming.lookup(chUrl);
```

et
java -Djava.security.policy=for_rmi.policy cli.ClientRmi

```
avec for_rmi.policy
grant {
    permission java.net.SocketPermission "*", "connect, resolve";
    permission java.util.PropertyPermission "*", "read, write";
    permission java.io.FilePermission "<<ALL FILES>>", "read";
};
```

Il fallait en outre lancer explicitement de serveur *rmiregistry* avec les bonnes options de sécurité et préciser la valeur de la propriété système *java.rmi.server.codebase* correspondant à l'endroit depuis lequel on pourra télécharger les classes "stub" nécessaires à RMI. Cette URL (en file:// , ftp:// ou http://) est stockée dans le registre Rmi lors de l'enregistrement (bind).

4.1. Structure "maven" multi-modules

```
<modules>
  <module>cliRmi</module>
  <module>servRmi-api</module>
  <module>servRmi-impl</module>
</modules>
```

Dépendance commune au client et au serveur :

```
<dependency>
  <groupId>tp</groupId>
  <artifactId>servRmi-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>
</dependency>
```

LancerServeurRMI.bat (point clef)

```
set M2_REPOSITORY=C:\Users\d2fde\.m2\repository
set CP="%M2_REPOSITORY%/tp/servRmi-api/0.0.1-SNAPSHOT/servRmi-api-0.0.1-SNAPSHOT.jar;%M2_REPOSITORY%/tp/servRmi-impl/0.0.1-SNAPSHOT/servRmi-impl-0.0.1-SNAPSHOT.jar"
java -cp %CP% tp.serveur.app.AppServMiniBank
```

```
pause
```

LancerClientRMI.bat (exemple)

```
set M2_REPOSITORY=C:\Users\d2fde\.m2\repository
set CP="%M2_REPOSITORY%\tp\servRmi-api\0.0.1-SNAPSHOT\servRmi-api-0.0.1-SNAPSHOT.jar;%M2_REPOSITORY%\tp\cliRmi\0.0.1-SNAPSHOT\cliRmi-0.0.1-SNAPSHOT.jar"
set REMOTE_HOST=localhost
REM set REMOTE_HOST=192.168.0.200
java -cp %CP% tp.client.MiniBankSimpleClient %REMOTE_HOST%
pause
```

4.2. Fonctionnalités possibles avancées

- Callback
- ...
- Sockets personnalisées et SSL

Il est possible d'ajouter des fonctionnalités de cryptage et/ou d'authentification via des "SocketFactory" qui vont créer des sockets spéciales (prise en compte de SSL) .

Ces usines à sockets peuvent être spécifiées en tant que paramètres de l'une des versions du constructeur de **UnicastRemoteObject** (super classe pour s'exporter).

5. RMI over IIOP (comme "EJB Remote")

RMI peut être considéré comme une couche haute pouvant s'appuyer sur JRMP (Java Remote Method Protocol) ou bien sur IIOP (Internet Inter-ORB Protocol).

- **JRMP** est le protocole par défaut dédié à RMI;
- **IIOP** est le protocole standard pour faire communiquer les objets CORBA.

RMI over IIOP permet à des objets Java distants de communiquer avec des objets CORBA qui peuvent éventuellement être écrits en C++ .

La principale particularité liée à "RMI over IIOP" consiste à ce que la classe d'implémentation (coté serveur) hérite de `javax.rmi.PortableRemoteObject` à la place de `java.rmi.server.UnicastRemoteObject`.

D'autre part le serveur de nom (hiérarchisé) à utiliser est **tnameserv.exe** (implémentant CosNaming de CORBA).

Les spécifications de **J2EE** préconisent quelquefois l'utilisation de **RMI-over-IIOP** pour des raisons d'ouverture: les EJB sont ainsi accessibles depuis un programme C++ ou autre.

NB : En règle générale , RMI-over-IIOP est indirectement mis en œuvre via des "EJB Remote" qui sont "pris en charge par un serveur d'application" de type JBoss ou Websphere .

Mise en œuvre de RMI-over-IIOP:**rmic -iiop** → stub / squelette version RMI-over-IIOP

(possibilité d'implémentation duale JRMP / IIOP) .

rmic -idl → pour éventuel client C++ ou autre.Le client et le serveur peuvent accéder au service de nom via **JNDI**:

```
java -Djava.naming.factory.initial=com.sun.jndi.cosnaming.CNCtxFactory
      -Djava.naming.provider.url=iiop://servnom:900      ClientRmiOverIIOP
```

où **servnom** correspond à la machine sur laquelle est lancé tnameserv.exe .**5.1. Fragment de code du client:**

```
import javax.rmi.PortableRemoteObject;
import javax.naming.InitialContext;

...
InitialContext ctx = new InitialContext();
Object obj = ctx.lookup("/NomObjet");
HelloInterface myobj = (HelloInterface)
    PortableRemoteObject.narrow(obj,HelloInterface.class);
...
```

5.2. Fragment de code du serveur:

```
import java.rmi.*;
import javax.naming.*;

HelloServer obj = new HelloServer() ; // classe héritant de PortableRemoteObject

/* Fixer des valeurs par défaut pour ce qui concerne les propriétés d'accès au serveur de nom
tnameserv.exe */
java.util.Properties props = System.getProperties();
if(props.get("java.naming.factory.initial")==null)
    props.put("java.naming.factory.initial","com.sun.jndi.cosnaming.CNCtxFactory");
if(props.get("java.naming.provider.url")==null)
    props.put("java.naming.provider.url","iiop://localhost:900");

InitialContext ctx = new InitialContext();
ctx.rebind("/NomObjet",obj);...
```

XV - Mapping Objet-Relationnel (ORM) et JPA

1. Problématique "O.R.M."

1.1. Objectif & contraintes:

L'objectif principal d'une technologie de mapping objet/Relationnel est d'établir une correspondance relativement transparente et efficace entre :

un ensemble d'objets en mémoire

et

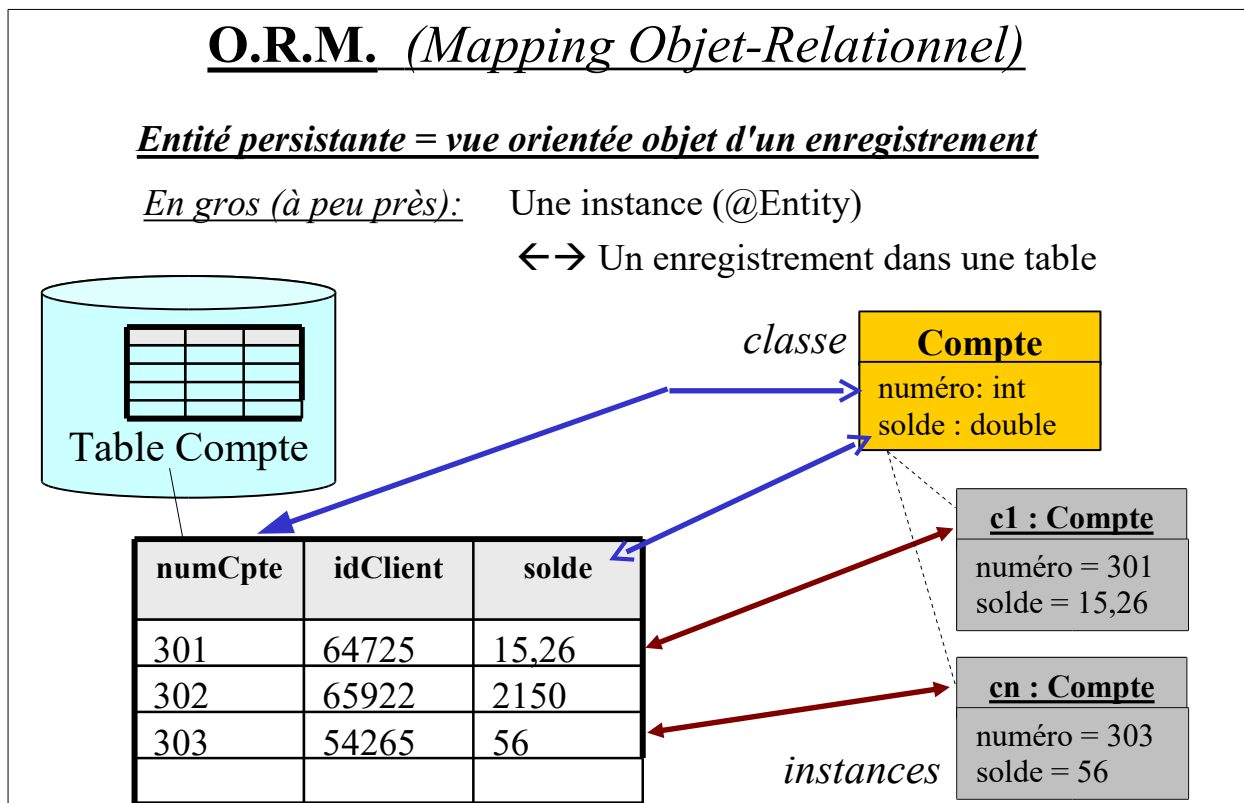
un ensemble d'enregistrements d'une base relationnelle.

O.R.M. (Mapping Objet-Relationnel)

Entité persistante = vue orientée objet d'un enregistrement

En gros (à peu près): Une instance (@Entity)

↔ Un enregistrement dans une table



Une telle technologie doit permettre à un programme orienté objet de ne voir que des objets dont certains sont des **objets persistants**. *Le code SQL est en très grande partie caché* car les objets persistants sont automatiquement pris en charge par la technologie "O.R.M."

Autrement dit, le code SQL n'est plus (ou très peu) dans le code "java" mais est généré automatiquement à partir d'une configuration de mapping (fichiers XML, annotations, ...).

Contraintes : pour être **exploitable** , une **technologie "O.R.M."** se doit d'être :

- **simple** (à configurer et à utiliser) et **intuitive**
- **portable** (possibilité de l'intégrer facilement dans différents serveurs d'application)
- **efficace** (bonnes performances, fiable , ...)
- capable de gérer les **transactions** ou bien de participer à une transaction déjà initiée
- ...

1.2. Éléments techniques devant être bien gérés

Au delà des simples associations élémentaires du type

"1 enregistrement d'une table <---> 1 instance d'une classe" ,

une **technologie "O.R.M." sophistiquée** doit savoir gérer certaines **correspondances évoluées** :

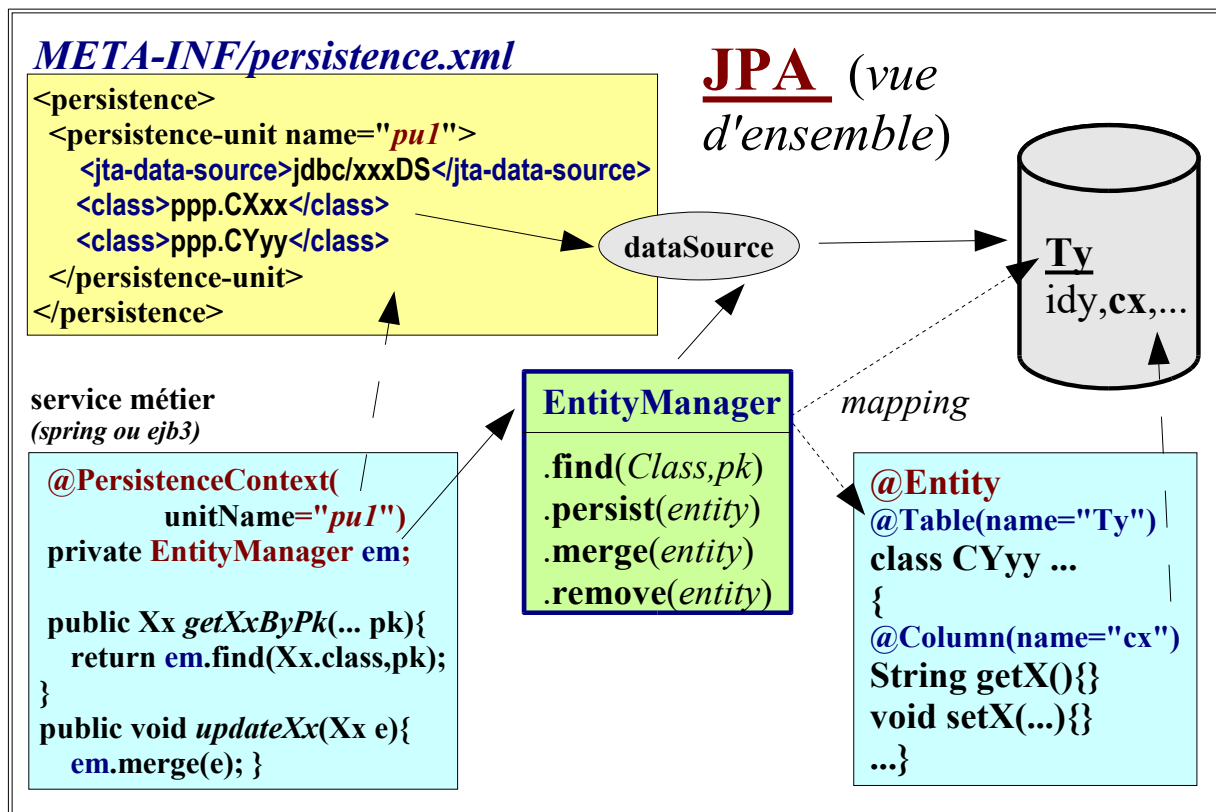
- **Associations "1-1" , "1-n" , "n-n"** (jointures entre tables <---> relations entres objets)
- **Objets "valeur"** (sous parties d'enregistrements , tables secondaires , ..)
- **Généralisation/Héritage/Polymorphisme** (<---> schéma relationnel ?)
- ...

Une technologie "O.R.M." doit sur ces différents points être **flexible et efficace** .

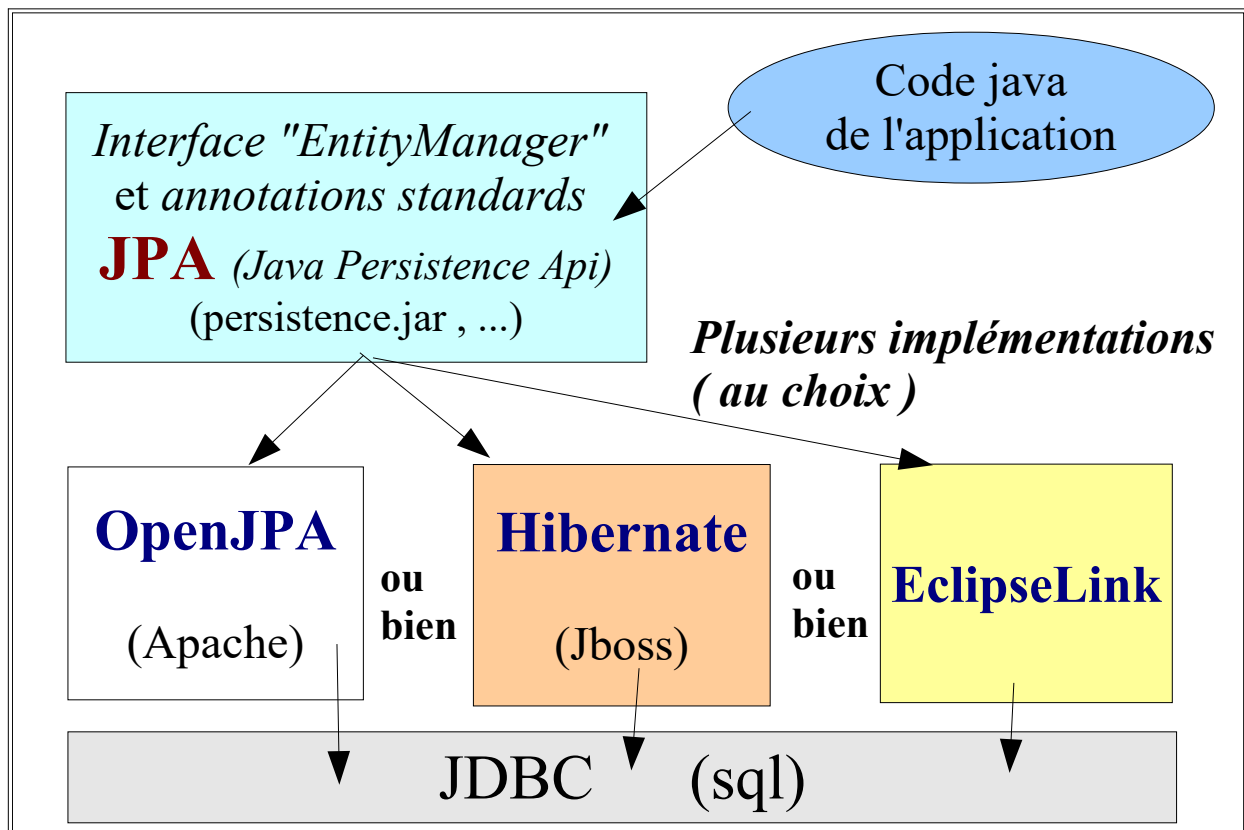
2. JPA (Java Persistence Api)

JPA (Java Persistence Api) et @Entity

- **API O.R.M. officielle de Java EE**
- **Configuration** basée sur des **annotations** (*avec configuration xml possible : orm.xml*).
- Utilisable dans les contextes suivants:
 - dans **module d'EJB3**
 - dans **module Spring**
 - de façon **autonome** (java sans serveur)
- Plusieurs **implémentations** internes disponibles (*OpenJPA , Hibernate* depuis version 3.2 ,)



2.1. Plusieurs implémentations disponibles pour JPA



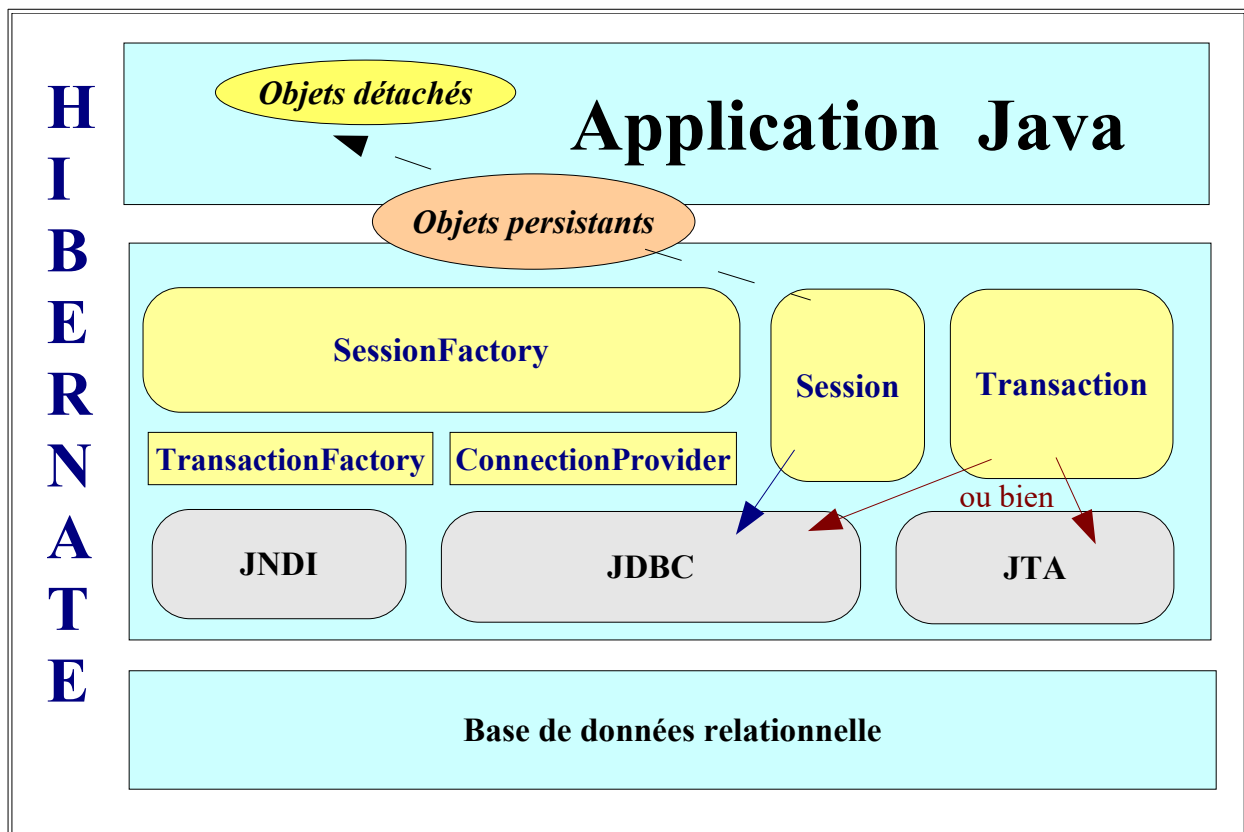
Selon le contexte, l'implémentation de JPA sera libre ou bien quasi imposée par le serveur d'applications (ex : JBoss Application Serveur utilise en interne Hibernate pour implémenter JPA).

XVI - Architecture d'Hibernate (historique, évolution)

1. Hibernate: ancienne architecture & configuration

1.1. Architecture historique d'hibernate (sans JPA)

NB : Vers environ 2002, 2003, 2004 , les premières versions d'hibernate avaient mises au point avant JPA et étaient structurées de cette façon :



NB: Les transactions (commit/rollback) sont supervisées à partir de l'objet abstrait "Transaction" d'Hibernate . Celui-ci peut soit s'appuyer sur des transactions simples JDBC (sur une seule base de données) ou bien sur des transactions complexes "JTA" gérées par le serveur d'application hôte (ex: WebSphere, Weblogic, Jboss, Geronimo,).

D'autre part, l'utilisation exacte de l'objet "Transaction" d'hibernate est assez variable en fonction du style de programmation utilisé (explicite/implicite , déclaratif , ...).

L'objet "Session" d'hibernate est de loin le plus important : il correspond à une sorte de "D.A.O." générique permettant de déclencher les opérations habituelles de persistance (CRUD : Create, Retreive, Update, Delete). L'objet "session" d'hibernate est soit "ouvert" ou "fermé" et ceci a une très grande incidence sur la connexion JDBC/SQL sous-jacente et sur l'état des objets (persistants ou détachés) .

1.2. Principaux objets de l'API Hibernate (sans JPA) :

SessionFactory	Un objet de ce type par base de données. cet objet représente un cache de tous les mappings "objet/relationnel" liés à une certaine base de données . Les traitements associés à cet objet sont ré-entrants (multi-threading possible) . Cet objet sert à fabriquer des objets "Session".
Session	Un objet de ce type représente une session utilisateur et est lié à un seul thread. Une session englobe un certain nombre d' objets persistants dont les valeurs sont mappées avec le contenu de certaines tables relationnelles. Chaque objet session incorpore une connexion JDBC et sert à récupérer ou fabriquer des objets de type "Transaction"
Transaction	Un objet de l'api hibernate représentant une transaction que l'on peut explicitement délimiter (begin , commit/rollback) . cet objet se base sur des transactions sous-jacentes de type JDBC, JTA ou
Objets persistants	Objets java (JavaBean / POJO) comportant des valeurs persistantes et des traitements "métier" . Tant qu'ils sont liés à une session, ces objets ont des valeurs synchronisées/synchronisables avec la base de données. Dès que la session est fermée, ces objets deviennent détachés et peuvent être directement réutilisés pour tout transfert ou affichage.
...	

1.3. Anciens fichiers de configuration (selon variantes)

<i>Utilisation d'Hibernate</i>	<i>Configuration générale (base de données ,)</i>	<i>Configuration fine du mapping objet relationnel</i>
Hibernate seul	hibernate.cfg.xml	xxx.hbm.xml , yyy.hbm.xml
Hibernate intégré dans Spring	Partie de springConf.xml	xxx.hbm.xml, yyy.hbm.xml
Hibernate intégré dans JPA (lui même intégré dans EJB3 ou Spring ou ...)	META-INF/persistence.xml (+ ou)	annotations (@Entity ,@Table , @Column,) dans le code java des entités

Structure ordinaire du code de l'application allant autour d'Hibernate:

DAO (ou Test) avec Hibernate seul	Ordres O.R.M. via " session " + gestion explicite des transactions + gestion explicite de la session
DAO ou Service avec " Spring+Hibernate "	Ordres O.R.M. via hibernateTemplate (+ try/catch)
Test ou DAO avec JPA (sans EJB3 ni Spring)	Ordres O.R.M. + gestion explicite des transactions + gestion explicite "entityManager"
Service avec JPA (intégré dans Spring ou EJB3)	Ordres O.R.M. via " entityManager " (+try/catch)

2. Hibernate en tant qu'implémentation de JPA>=2

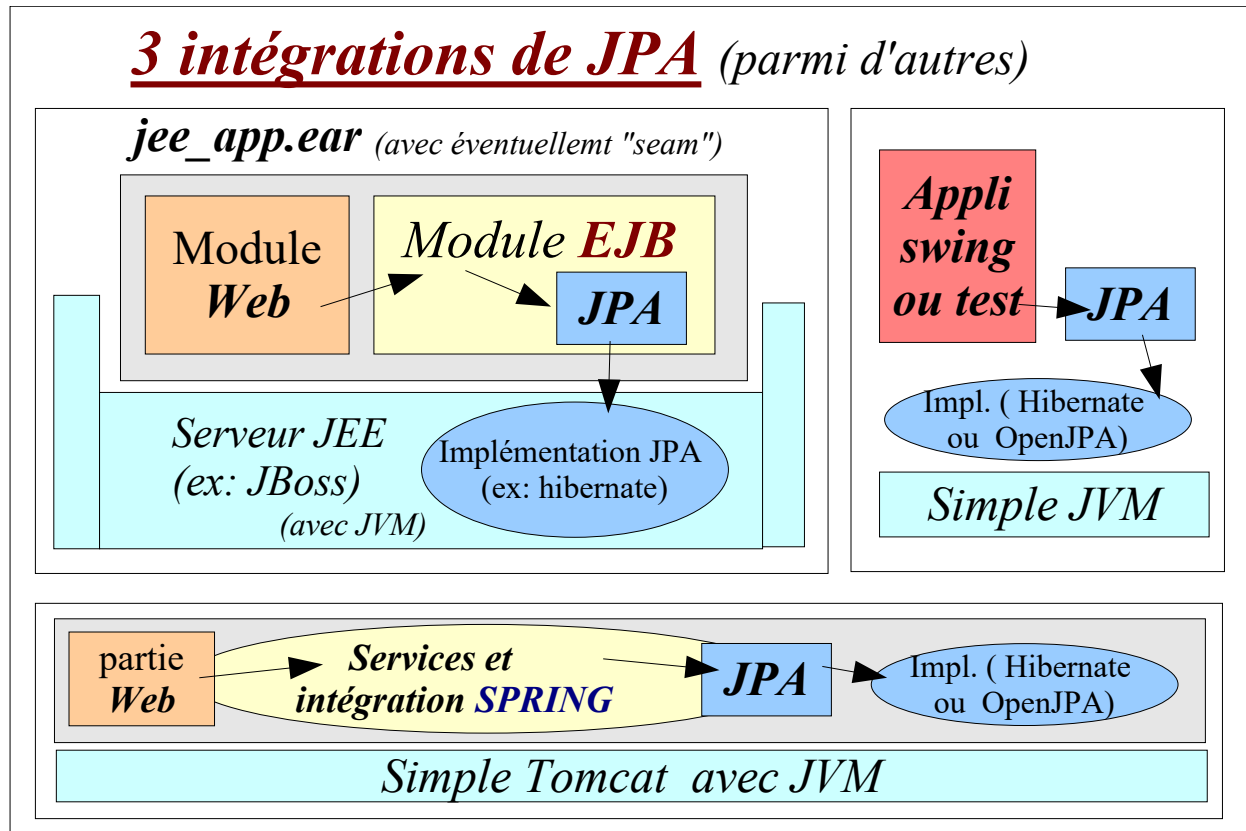
- Avant 2005/2006 , Hibernate était souvent utilisé seul ou avec spring et JPA n'existait pas encore .
A cette époque les annotations (@...) n'existaient pas encore et tout se configurait en xml (application.xml de spring + hibernate.cfg.xml + xxx.hbm.xml)
- **Dès 2005/2006 et EJB3/JPA1, Hibernate a été restructuré pour devenir une implémentation de JPA 1 .**
Les projets de cette époque sont très progressivement passé d'une configuration purement xml à une configuration "xml + annotations) : META-INF/persistence.xml + configuration spring xml + @Entity , @ManyToOne, ...
JPA 1.0 étant fonctionnellement moins riche que Hibernate , certains projets des années 2007, 2008 utilisaient encore à cette époque certaines parties d'Hibernate en direct (sans systématiquement se restreindre au standard JPA) .
- **A partir de 2009,2010 et l'avènement de JPA 2, presque toutes les fonctionnalités d'Hibernate sont devenues accessibles via JPA .** Il n'y a maintenant plus vraiment d'intérêt à utiliser Hibernate en direct sans passer par le standard JPA .
Les versions JPA 2.1 de 2011 , JPA 2.2 de 2017 et JPA 3.0 de 2020 sont venues conforter cet état de fait .

-
- Bien qu'utilisable "toute seule" (sans spring ni EJB et via une gestion explicite des transactions), **la technologie JPA/Hibernate est, au sein des vrais projets d'entreprises, quasi systématiquement utilisée de manière intégrée à la technologie EJB 3.x ou bien Spring de manière à bénéficier de la gestion automatique et implicite des transactions (selon le standard Java EE) .**
 - A partir de 2015 environ, la configuration du framework spring a petit à petit évolué d'une configuration "XML + annotations" à une configuration "javaConfig + annotations + .properties/.yaml) et **la plupart des projets modernes sont basés maintenant sur une intégration de JPA/Hibernate dans SpringBoot + SpringDataJpa .**

XVII - JPA : architecture & configuration

1. Présentation de JPA (Java Persistence Api)

L'api JPA peut être utilisée de façon très variable (dans EJB>=3 , dans Spring ou seul)



Une intégration "EJB>=3" ou JakartaEE s'appuie essentiellement sur ce qui est offert par le serveur d'applications (Jboss, WebSphere, ...). Par exemple JBoss comporte déjà en lui toutes les bibliothèques (.jar) correspondant à l'API JPA (et son implémentation basée sur Hibernate).

Une intégration "Spring" s'appuie par contre généralement sur des bibliothèques choisies (Hibernate par défaut ou OpenJPA ou) et qui sont souvent livrées et déployées avec l'application .

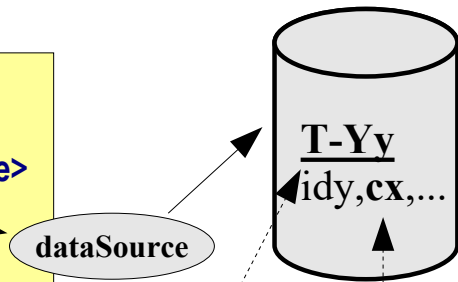
Une classe de Test "JUnit" exécutée avec une simple JVM peut très bien utiliser directement l'API JPA (avec ou sans Spring) en s'appuyant sur une des implémentations disponibles (Hibernate ou OpenJPA ou ...).

2. Unité de Persistance (META-INF/persistence.xml)

Unité de persistance (*PersistenceUnit*)

META-INF/persistence.xml

```
<persistence>
  <persistence-unit name="pu1">
    <jta-data-source>jdbc/xxxDS</jta-data-source>
    <class>ppp.CXxx</class>
    <class>ppp.CYyy</class>
  </persistence-unit>
</persistence>
```



DAO (ejb ou spring ou ...)

```
@PersistenceContext(unitName="pu1")
private EntityManager manager;
```

```
@Entity
@Table(name="T-Yy")
class CYyy ...
{
  @Column(name="cx")
  String getX(); void setX(...);
  ...}
CYyy
```

META-INF/persistence.xml (exemple de l'époque EJB3)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence" version="2.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
    http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" >
  <persistence-unit name="OrderManagement">
    <description>....</description>
    <jta-data-source>java:/MyOrderDS</jta-data-source>
    <!-- <mapping-file>orm.xml</mapping-file> -->
    <class>com.widgets.Order</class>      <class>com.widgets.Customer</class>
    <properties>
      <!-- <property name="hibernate.hbm2ddl.auto" value="create-drop"/> -->
      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

`<persistence-unit>` comporte un attribut optionnel *"transaction-type"* dont les valeurs possibles sont *"JTA"* ou *"RESOURCE_LOCAL"*.

La valeur par défaut est *"JTA"* dans un env. EE (ex : EJB3) et *"RESOURCE_LOCAL"* dans un env. SE.

La valeur de `<jta-data-source>` ou `<non-jta-data-source>` correspond au *nom JNDI global de la source de données SQL* (pool de connexions permettant d'atteindre une base de données précise).

[Rappel : le serveur Jboss A.S. ajoute le préfixe "java:/" sur les noms JNDI des "dataSources"]

L'ensemble des classes (avec annotations `@Entity`) devant être prises en compte et prises en charge par le contexte de persistance se définit via une (ou plusieurs et complémentaires) indication(s) suivante(s):

- Une liste explicite de classes (balises `<class>`)
- un ou plusieurs fichier(s) xml de mapping "objet/relationnel" (balises `<mapping-file>`)
- ...

NB: Dans le cas particulier d'un module EJB3 ou Spring moderne, les classes comportant des annotations `@Entity` seront automatiquement détectées et il n'est donc pas absolument nécessaire de les déclarées dans le fichier `META-INF/persistence.xml`

NB: un fichier xml de type **orm.xml** (rare et facultatif) constitue soit une alternative vis à vis d'une configuration basée sur des annotations Java5 , soit un mécanisme de redéfinition.

Dans le cas où JPA est implémenté via la sous couche Hibernate , on peut éventuellement fixer la propriété **hibernate.hbm2ddl.auto** à l'une des valeurs suivantes :

- * **create** [re-crée les tables au démarrage de l'application]
- * **create-drop** [créer les tables au démarrage de l'application et les supprime à l'arrêt]
- * **update** [mise à jour des tables sans suppression]
- * **none** (ou le tout en commentaire)

====> Les "create table" sont générés à partir des informations `@Table` , `@Column` ,

====> Cette fonctionnalité peut être pratique en mode développement mais est très dangereuse en mode production !!!

Variante de `META-INF/persistence.xml` pour intégration dans Spring (sans **springBoot**) :

```
<persistence version="2.2" xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
<persistence-unit name="myPersistenceUnit"
  transaction-type="RESOURCE_LOCAL">
<provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <!-- ... <class>entity.persistence.jpa.XxxYyy</class> -->
<properties>
  <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect" />
  <!-- <property name="hibernate.hbm2ddl.auto" value="update" /> -->
  <property name="javax.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
  <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/mydb" />
  <property name="javax.persistence.jdbc.user" value="root" />
  <property name="javax.persistence.jdbc.password" value="root" />
</properties>
</persistence-unit>
</persistence>
```

NB: `<provider>...HibernatePersistenceProvider</provider>` permet de préciser la technologie choisie pour implémenter JPA (Hibernate ou eclipseLink ou OpenJpa) .

2.1. Evolution de la configuration JPA

Durant les années 2005-2015, la technologie JPA 1 (puis JPA2) avait été principalement utilisée de manière intégrée à des applications "JEE/EJB3" ou bien "Spring-framework".

- Dans le cadre (originel) d'une **application "JEE/EJB3"**, JPA est configuré via le fichier standard **META-INF/persistence.xml** et au sein de ce fichier la balise **<jta-data-source></jta-data-source>** référence un nom logique de "dataSource" (objet de configuration spécifique au serveur qui indique comment de connecter à une base relationnelle).
La configuration de ce dataSource est spécifique à un type et à une version du serveur JavaEE (ex : WebSphere-AS 7 ou Jboss-wildfly 16 ou WebLogic ou Tomee ou ...).
Par exemple, le serveur **Jboss-Wildfly_ou_EAP** configure un "dataSource" dans le fichier **standalone.xml**.
- Dans le cadre d'une application Spring (Spring-framework sans spring-boot), la configuration JPA était **partiellement** stockée dans le fichier standard **META-INF/persistence.xml** et avec un complément de configuration dans un fichier de configuration XML de Spring (ex : **application.xml**) comportant la configuration "au format spring" d'un dataSource (pour se connecter à une base de données)

A partir de 2015, le framework "Spring" a beaucoup évolué (*configuration simplifiée sans xml, démarrage sans serveur, ...*) et la plupart des applications modernes sont maintenant basées sur **Spring-Boot**.

"**spring-boot-jpa**" se configure maintenant directement dans le fichier **application.properties** (ou **.yml**) et dans ce contexte le fichier **META-INF/persistence.xml** n'est plus analysé/pris en compte.

Adaptation spring-boot :

application.properties

```
server.servlet.context-path=/appliSpringJpa
server.port=8080
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/mydb?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

# si l'option spring.jpa.hibernate.ddl-auto=create est activée, toutes les
# tables nécessaires seront re-crées automatiquement à chaque démarrage
# à vide et en fonction de la structure des classes java (@Entity)
spring.jpa.hibernate.ddl-auto=create

#enable spring-data (generated dao implementation classes)
#spring.data.jpa.repositories.enabled=true
```

NB : une propriété JPA/Hibernate est configurée via

`<property name="javax.persistence.xyz" value="xyz-value" />` dans le fichier standard *META-INF/persistence.xml*

et via `spring.jpa.properties.javax.persistence.xyz=xyz-value` dans *application.properties* de *Spring-boot*

Options secondaires (disponibles en version 2.2 de JPA)

#tables créés automatiquement au démarrage et fichier sql déclenché automatiquement :
`...javax.persistence.schema-generation.database.action=drop-and-create`
`...javax.persistence.sql-load-script-source=META-INF/data.sql`

#fichiers sql générés (pour consultation) mais pas déclenchés:
`...javax.persistence.schema-generation.create-source=metadata`
`...javax.persistence.schema-generation.scripts.action=drop-and-create`
`...javax.persistence.schema-generation.scripts.create-target=src/main/script/create.sql`
`...javax.persistence.schema-generation.scripts.drop-target=src/main/script/drop.sql`

3. JPA3 et JakartaEE

```
import javax.persistence.*;
import jakarta.persistence.*;
```

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd"
  version="3.2">
...
```

```
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.2.0</version>
</dependency>

<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>7.1.7.Final</version>
</dependency>
```

```
@GeneratedValue(strategy = GenerationType.UUID)
private UUID id;
```

`GenerationType.UUID` est disponible (sans extension) depuis jpa 3

4. Configuration JPA/Hibernate sans spring ni jakartaEE

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>7.3.1.Final</version>
</dependency>
```

NB : les versions récentes d'hibernate-core comportent en elle toute la partie "JPA"

Au moins un driver JDBC nécessaire:

h2	Mysql (ou mariaDB)	Postgresql
<pre><dependency> <groupId>com.h2database</groupId> <artifactId>h2</artifactId> <version>2.4.240</version> </dependency></pre>	<pre><dependency> <groupId>com.mysql</groupId> <artifactId>mysql-connector-j</artifactId> <version>9.6.0</version> </dependency></pre>	<pre><dependency> <groupId>org.postgresql</groupId> <artifactId>postgresql</artifactId> <version>42.7.10</version> </dependency></pre>

Fichier de configuration **META-INF/persistence.xml** (à généralement placer dans src/main/resources)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd"
  version="3.2">
  <persistence-unit name="myapp" transaction-type="RESOURCE_LOCAL">
    <description>configuration JPA , base de donnees , ...</description>
    <!-- on precise ici la technologie choisie pour coder/implementer JPA Hibernate (ou OpenJpa ou EclipseLink) -->
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <!-- indiquer liste des classes avec @Entity (facultatif avec Spring ou autre) -->
    <class>tp.core.persistence.entity.Compte</class>
    <class>tp.core.persistence.entity.Client</class>
    <!-- configurer d'une facon ou d'une autre une connexion a la base de donnees -->
    <!-- aujourd'hui via des proprietes specifiques Hibernate -->
    <properties>
      <!-- <property name="hibernate.hbm2ddl.auto" value="create"/> -->
      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect" />
      <!-- <property name="hibernate.show_sql" value="true"/> -->
      <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
      <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/db?createDatabaseIfNotExist=true" />
      <property name="jakarta.persistence.jdbc.user" value="root" />
      <property name="jakarta.persistence.jdbc.password" value="root" />
    </properties>
  </persistence-unit>
</persistence>
```


Exemple de classe Java permettant d'utiliser JPA/Hibernate sans framework additionnel (sans spring ni jarkartaEE) après l'avoir initialiser en fonction de META-INF/persistence.xml.

MyJpaUtil.java

```
package tp.core.persistence.jpa;

import jakarta.persistence.*;      import java.util.Properties;
import java.util.function.Consumer; import java.util.function.Function;

public class MyJpaUtil {
    private static EntityManagerFactory emf=null;
    private static boolean hbm2ddlAuto=false; //or true with "create"

    private static ThreadLocal<EntityTransaction> transactionThreadLocal = new ThreadLocal<>();
    private static ThreadLocal<EntityManager> entityManagerThreadLocal = new ThreadLocal<>();

    public static EntityManagerFactory entityManagerFactory(){
        if(emf==null) emf=initEmf();
        return emf;
    }

    public static void setHbm2ddlAuto(boolean hbm2ddlAuto){ MyJpaUtil.hbm2ddlAuto=hbm2ddlAuto; }
    public static boolean getHbm2ddlAuto(){ return MyJpaUtil.hbm2ddlAuto; }

    public static EntityManagerFactory initEmf(){
        EntityManagerFactory emf =null;
        Properties props = new Properties();
        props.setProperty("hibernate.hbm2ddl.auto", hbm2ddlAuto?"create":"none");
        /*emf = Persistence.createEntityManagerFactory("myapp");*/
        emf = Persistence.createEntityManagerFactory("myapp",props);
        //NB: myapp= name du persistent-unit configuré dans META-INF/persistence.xml
        return emf;
    }

    public static EntityManager entityManager(){
        EntityManager existingEntityManager = entityManagerThreadLocal.get();
        if(existingEntityManager!=null)
            return existingEntityManager;
        /*else*/
        return entityManagerFactory().createEntityManager();
    }

    public static void beginGlobalJpaTransaction(){
        EntityManager entityManager = entityManager();
        entityManagerThreadLocal.set(entityManager);
        EntityTransaction globalTransaction = entityManager.getTransaction();
        transactionThreadLocal.set(globalTransaction);
        globalTransaction.begin();
    }

    public static void commitGlobalJpaTransaction(){
        EntityTransaction globalTransaction = transactionThreadLocal.get();
        transactionThreadLocal.remove();
        globalTransaction.commit();
        EntityManager entityManager = entityManagerThreadLocal.get();
        entityManagerThreadLocal.remove();
        entityManager.close();
    }
}
```



```

public static void rollbackGlobalJpaTransaction() {
    EntityTransaction globalTransaction = transactionThreadLocal.get();
    transactionThreadLocal.remove();
    if (globalTransaction.isActive()) {
        globalTransaction.rollback();
    }
    EntityManager entityManager = entityManagerThreadLocal.get();
    entityManagerThreadLocal.remove();
    entityManager.close();
}

public static void doInTransaction(Consumer<EntityManager> work) {
    execOrDoInTransaction(null, work); //no return value
}

public static Object execInTransaction(Function<EntityManager, Object> function) {
    return execOrDoInTransaction(function, null); //with return value (to be casted)
}

public static Object execOrDoInTransaction(Function<EntityManager, Object> function, Consumer<EntityManager> work) {
    Object result = null;
    EntityTransaction localTransaction = null;
    EntityManager entityManager = entityManager();
    EntityTransaction existingGlobalTransaction = transactionThreadLocal.get();
    if (existingGlobalTransaction == null) {
        localTransaction = entityManager.getTransaction();
    }
    try {
        if (existingGlobalTransaction == null) {
            localTransaction.begin();
        }
        if (function != null) {
            result = function.apply(entityManager); //with return value
        }
        if (work != null) {
            work.accept(entityManager); //no return value
        }
        if (localTransaction != null) {
            localTransaction.commit();
        }
    }
    catch (Exception e) {
        if (localTransaction != null && localTransaction.isActive()) {
            localTransaction.rollback();
        }
        throw e;
    }
    finally {
        if (localTransaction != null) {
            entityManager.close();
        }
    }
    return result;
}

```

NB: une grosse partie de ce code (très technique) n'est ici que pour gérer proprement les transactions JPA/Hibernate sans se faire aider par Spring ni EJB/jakartaEE (au sein d'un projet spring ou springBoot @Transactional fait tout ça automatiquement) .

5. Configuration du mapping JPA via annotations

Le mapping de JPA se configure généralement en insérant quelques annotations JAVA dans le code JAVA des entités.

5.1. Exemple d'entité persistante (@Entity)

```
@Entity
@Table(name = "T_CUSTOMER")
@Getter @Setter @NoArgsConstructor
public class Customer implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)//auto_incr in database
private Long number;

@Column(name = "customer_name")
private String name;
private Address address;

@OneToMany(...)
private List<Order> orders = new ArrayList<Order>();

@ManyToMany(...)
private List<PhoneNumber> phones = new ArrayList<PhoneNumber>();

// Business method to add a phone number to the customer
public void addPhone(PhoneNumber phone) {
    this.getPhones().add(phone);
    // Update the phone entity instance to refer to this customer
    phone.addCustomer(this);
}
}
```

Dans l'exemple précédent :

- l'annotation **@Entity** permet de marquer la classe Customer comme une classe d'objets persistants
- l'annotation **@Table(name="T_CUSTOMER")** permet d'associer la classe java "Customer" à la table relationnelle "T_CUSTOMER" .
- l'annotation **@Column(name="customer_name")** permet d'associer l'attribut "name" de la classe "Customer" à la colonne "customer_name" de la table relationnelle.
- l'annotation **@Id** permet de marquer le champ "number" comme **identifiant de l'entité (et comme clef primaire de la table)**.
- les autres annotations (**@OneToMany** , ...) seront approfondies dans un chapitre ultérieur.

NB: Lorsqu'une annotation (@Column ou @Table ou ...) n'est pas présente , les noms sont censés coïncider (sachant que les minuscules et majuscules ont quelquefois de l'importance du côté SQL/relationnel selon le système d'exploitation hôte (ex: linux)).

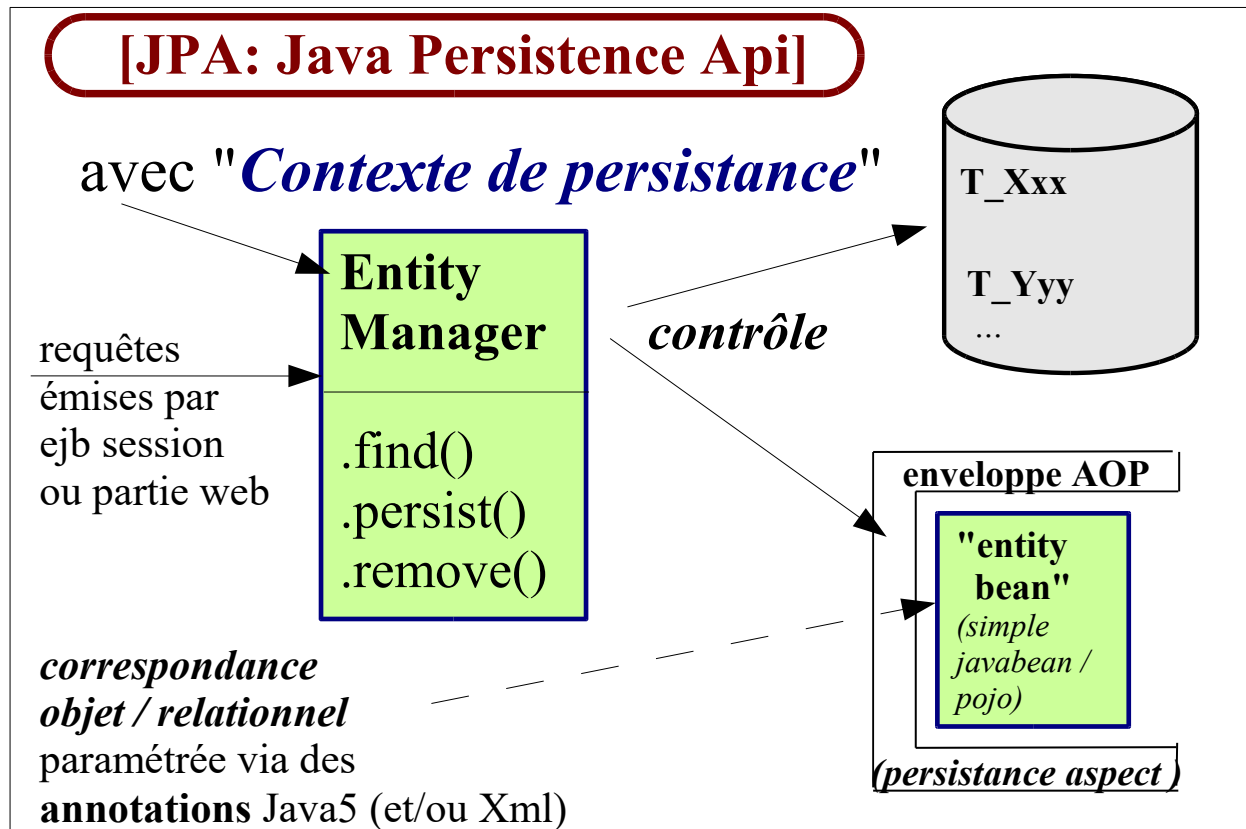
Bonnes pratiques : CamelCase en java et snake_case en base (noms des tables et des colonnes).

Remarques importantes:

- Les annotations paramétrant le mapping objet/relationnel (@Id , @Column, @OneToMany, ...) peuvent soit être placées au dessus de l'attribut privé soit être placées au dessus de la méthode en "get".
[au dessus des "private" --> plus lisible car dans le haut de la classe]
[au dessus des "get" --> plus clair et/ou plus fonctionnel en cas d'héritage]
Attention: il faut absolument garder un style homogène (si quelques annotations au dessus des "privates" et quelques annotations au dessus des "getXxx()" --> ça ne fonctionne pas !!!
sauf si annotation spéciale @Access de JPA 2 avec *AccessType.FIELD* ou *PROPERTY*)
- Les annotations paramétrant des injections de dépendances (@Resource , @PersistenceContext , ...) peuvent soit être placées au dessus de l'attribut privé soit être placées au dessus de la méthode en "set".
- Même si une propriété (avec private +get/set) n'est marquée par aucune annotation , elle sera tout de même prise en compte lors du mapping objet-relationnel.
- Pour éventuellement (si nécessaire) désactiver le mapping d'une propriété d'une classe persistante, il faut utiliser l'annotation "@Transient" (au dessus du "private" ou du "getter" de la propriété).
- @GeneratedValue(strategy = GenerationType.IDENTITY) est le paramétrage le plus courant (à côté de @Id) pour que la clef primaire auto-incrémentée par la base de données puisse bien remonter en mémoire dans l'objet persistant java .
- @Enumerated(EnumType.STRING) permet de stocker la valeur d'une énumération sous forme de chaîne de caractères dans une colonne d'une table relationnelle.
- @Temporal(TemporalType.DATE ou TemporalType.TIME ou TemporalType.TIMESTAMP) permet de préciser la valeur significative d'une date java (java.util.Date) .
Cette ancienne annotation est aujourd'hui considérée comme obsolète/deprecated au sens où il est préférable d'utiliser **LocalDate** ou **LocalTime** ou **LocalDateTime** à la place de la très ancienne classe java.util.Date .
- ...

XVIII - EntityManager et entités persistantes

1. Entity Manager et son contexte de persistance



Un objet persistant (@Entity avec toutes ces annotations) doit être pris en charge par un **gestionnaire d'objets persistants** (implicitement associé à un "*contexte de persistance*") de façon à ce que la liaison avec la base de données soit bien gérée.

Ci après, figure l'interface prédéfinie "**EntityManager**".

Celle-ci comporte tout un tas de **méthodes fondamentales** permettant de **déclencher/contrôler des opérations classiques de persistance** (*recherches, mises à jour, suppressions, ...*).

```
package javax.persistence;

/*Interface used to interact with the persistence context. */
public interface EntityManager {

    public void persist(Object entity); /* Make an instance managed and persistent. */

    public <T> T merge(T entity); /* Merge the state of the given entity
                                   into the current persistence context. */
    public void remove(Object entity); /* Remove the entity instance. */

    public <T> T find(Class<T> entityClass, Object primaryKey); /* Find by primary key. */
    public <T> T getReference(Class<T> entityClass, Object primaryKey);
                                   /* Get an instance, whose state may be lazily fetched. */
}
```

```

public void flush(); /* Synchronize the persistence context to the underlying database. */
public void setFlushMode(FlushModeType flushMode);
public FlushModeType getFlushMode();
public void lock(Object entity, LockModeType lockMode);
    /* Set the lock mode for an entity object contained in the persistence context. */

public void refresh(Object entity); /* Refresh the state of the instance from the database */

public void clear(); /* Clear the persistence context */
public boolean contains(Object entity);
    /* Check if the instance belongs to the current persistence context */
public Query createQuery(String qlString);
    /* Create an instance of Query for executing a JPQL statement */
public Query createNamedQuery(String name); /* in JPQL or native SQL */
public Query createNativeQuery(String sqlString);
public Query createNativeQuery(String sqlString, Class resultClass);
public Query createNativeQuery(String sqlString, String resultSetMapping);
public void joinTransaction(); /* join JTA active transaction */
public Object getDelegate(); /* implementation specific underlying provider */

public void close(); /* Close an application-managed EntityManager */
public boolean isOpen();
public EntityTransaction getTransaction();
}

```

Un **contexte de persistance** (associé à *EntityManager*) peut:

- soit être **créé et géré par le conteneur** (ex: EJB , éventuellement WEB)
puis injecté via l'annotation "**@PersistenceContext**" ou bien récupéré via un lookup JNDI .
- soit être explicitement créé via **createEntityManager()** de **EntityManagerFactory**.
Son cycle de vie est alors à gérer (==> **.close()** à explicitement appeler)

2. Transaction JPA

L'interface "EntityManager" d'un **contexte de persistance** comporte une méthode **getTransaction()** retournant de façon abstraite un objet permettant de contrôler (totalement ou partiellement) la transaction en cours:

```
public interface EntityManager {

    public void begin(); /* Start a resource transaction. */
    public void commit(); /* Commit the current transaction */
    public void rollback(); /* Roll back the current transaction. */

    public void setRollbackOnly(); /* Mark the current transaction so that the only possible
    outcome of the transaction is for the transaction to be rolled back. */
    public boolean getRollbackOnly();

    public boolean isActive();
}
```

Remarque importante:

Ces transactions JPA peuvent en interne s'appuyer sur:

- *des transactions simples et locales JDBC*
ou bien
- *des transactions complexes JTA (distribuées)*

Selon la configuration de **META-INF/persistence.xml** et la configuration du serveur d'application hôte (Jboss, WebSphere, Tomcat ,).

D'autre part, en fonction du contexte d'intégration de JPA, les transactions seront explicitement ou implicitement gérées d'une des façons suivantes:

- **gestion déclarative** (et automatique/implicite) des **transactions** au sein des **EJB3**
- **gestion déclarative** (et automatique/implicite) des **transactions** au sein de **Spring**
- **gestion explicitement programmée des transactions** (sans framework d'intégration)

Exemple (avec gestion explicite):

```
try {
    entityManager.getTransaction().begin();
    Adresse nouvelleAdresse = new Adresse("rue elle", "80000", "Amiens");
    Client nouveauClient = new Client();
    nouveauClient.setNom("nom du nouveau client");
    nouveauClient.setAdressePrincipale(nouvelleAdresse);
    this.entityManager.persist(nouveauClient);
    num_cli = nouveauClient.getIdClient();
    System.out.println("id du nouveau client: " + num_cli);
    entityManager.getTransaction().commit();
}
catch (Exception e) {
    this.entityManager.getTransaction().rollback();
    e.printStackTrace();
}
```

3. Différents états - objet potentiellement persistant

Etats objets	Caractéristiques
transient (proche état détaché)	Nouvel objet créé en mémoire, pris en charge par la JVM Java mais par encore contrôlé par une session Hibernate ou bien l'entityManager de JPA (le mapping objet/relationnel n'a jamais été activé). un tel objet n'a quelquefois par encore de clef primaire (elle sera souvent attribuée plus tard lors d'un appel à entityManager.persist() ou session.save())
persistant	objet actuellement sous le contrôle d'une session Hibernate ou de l'entityManager de JPA (<i>avant session.close() ou entityManager.close()</i>). Le mapping objet/relationnel est alors actif et une synchronisation (mémoire ==> base de données) est alors automatiquement déclenchée suite à une mise à jour (changement d'une valeur d'un attribut).
détaché	objet qui n'est plus sous le contrôle d'une session Hibernate ou de l'entityManager de JPA (<i>après session.close() ou entityManager.close()</i>) . Les valeurs en mémoire sont conservées mais ne seront plus mises à jour (mapping objet/relationnel désactivé). Un objet détaché pourra éventuellement être ré-attaché à une session Hibernate (lors d'un update ou save_or_update() sur une session hibernate ou lors d'un appel à merge() sur l'entityManager de JPA) et ses éventuelles nouvelles valeurs pourront alors être synchronisées dans la base de données.

4. Cycle de vie d'un objet JPA/Hibernate

transient ==> **persistant** (==> ...)

(via **new**) (via **session.save()** ou **entityManager.persist()**)
insert into

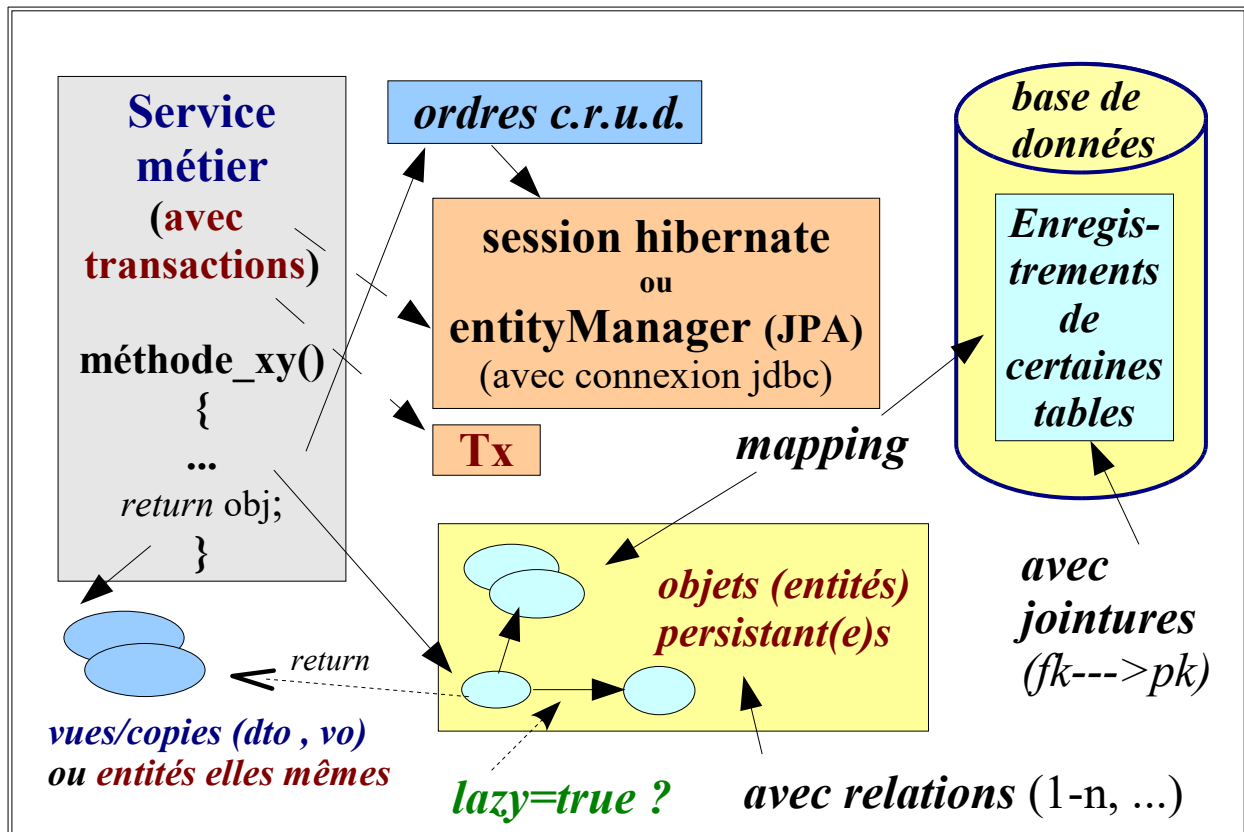
ou bien (après hibernation prolongée):

persistant ==> **détaché** (==> **persistant**)
(via **query**) (via session.close() ou entityManager.close()) (via **session.update()** , ...
ou **entityManager.merge()**)
select *update set ...*

5. Synchronisation automatique dans l'état persistant

Synchronisation automatique dans l'état persistant (.update() ou .persist() non obligatoire), mais .save() ou .merge() obligatoire pour un objet détaché.

6. objet persistant et architecture n-tiers



Remarques importantes:

- La technologie O.R.M. JPA ou hibernate sert essentiellement à remonter (et gérer) en mémoire un ensemble d'objets Java en tous points synchronisés avec les enregistrements d'une base de données relationnelle.
- Pour obtenir de bonnes performances (*lazy=true*) sans pour autant trop compliquer le code de l'application, il faut considérer les objets "entités persistantes" comme une structure d'ensemble orientée objet virtuellement liée à l'ensemble des données de la base (sachant que les mécanismes internes de JPA/Hibernate remontent les données en mémoire qu'en fonction des accès réellement effectués sur la structure objet [*lazy=true*]).
- La couche métier appelante (généralement développée avec Spring ou des EJB) doit souvent retourner des valeurs vers la couche présentation (IHM). Ces valeurs sont soit des références directes sur les entités persistantes elles mêmes ou bien des copies partielles sous formes de vues métiers (objets sérialisables / D.T.O. / V.O.).
- Lorsque JPA/Hibernate est intégré dans Spring ou les EJB, les transactions sont automatiquement gérées par le conteneur et le code est alors significativement simplifié.

7. Principales méthodes JPA / EntityManager et Query

Principales méthodes de l'objet **EntityManager**:

méthodes	traitements
.find (class,pk)	Recherche en base et retourne un objet ayant les classe java et clef primaire indiquées
.createQuery (req_jpql)	Créer une requête JPQL et retourne cet objet
.refresh (obj)	Met à jour les valeurs d'un objet en mémoire en fonction de celles actuellement présentes dans la base de données (reload) .
.persist (obj_détaché) <i>ou bien assez inutilement</i> .persist (obj_déjà_persistant)	<u>Rend persistant un objet détaché</u> : Sauvegarde les valeurs d'un nouvel objet dans la base de données (insert into ...) et retourne une exception de type EntityExistsException si l'entité existe déjà . La clef primaire est quelquefois automatiquement calculée lors de cette opération
.merge (obj_détaché) <i>ou bien assez inutilement</i> .merge (obj_déjà_dans_contexte_persistence)	Sauvegarde ou bien met à jour (update ...) les valeurs d'un objet dans la base de données. Cette méthode s'appelle .merge() car l'entité passé en argument peut quelquefois être utilisée pour remplacer les valeurs d'une ancienne version (avec la même clef primaire) déjà présente dans le contexte de persistance.
.remove (obj)	Supprime l'objet (delete ... from where dans la base de données)
.flush() déclenché indirectement via .commit()	Synchronise l'état de la base de données à partir des nouvelles valeurs des entités persistantes qui ont été modifiées en mémoire.

manager.**persist()** de **JPA** correspond à peu près au session.**save()** de **Hibernate**

manager.**find()** de **JPA** correspond à peu près au session.**get()** de **Hibernate**

manager.**remove()** de **JPA** correspond à peu près au session.**delete()** de **Hibernate**

manager.**merge()** de **JPA** correspond à peu près au session.**save_or_update()** de **Hibernate**

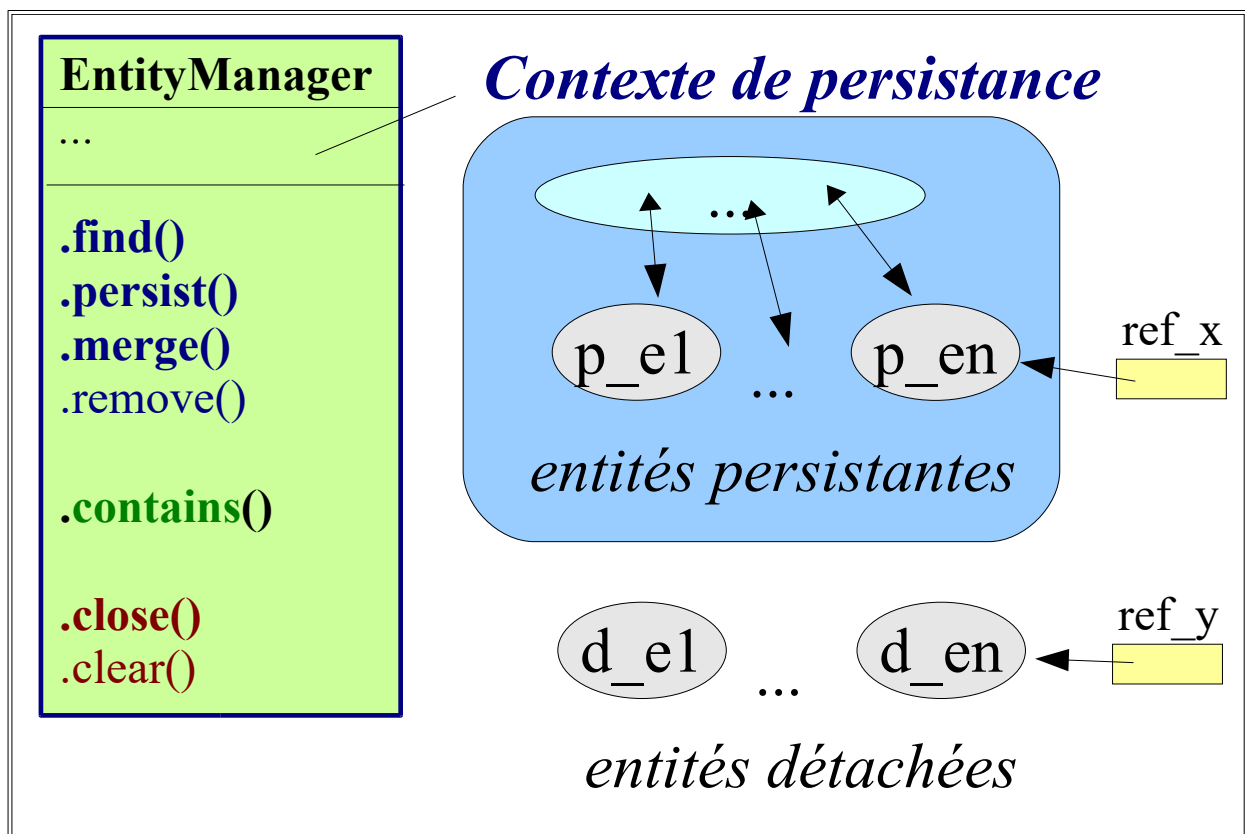
NB:

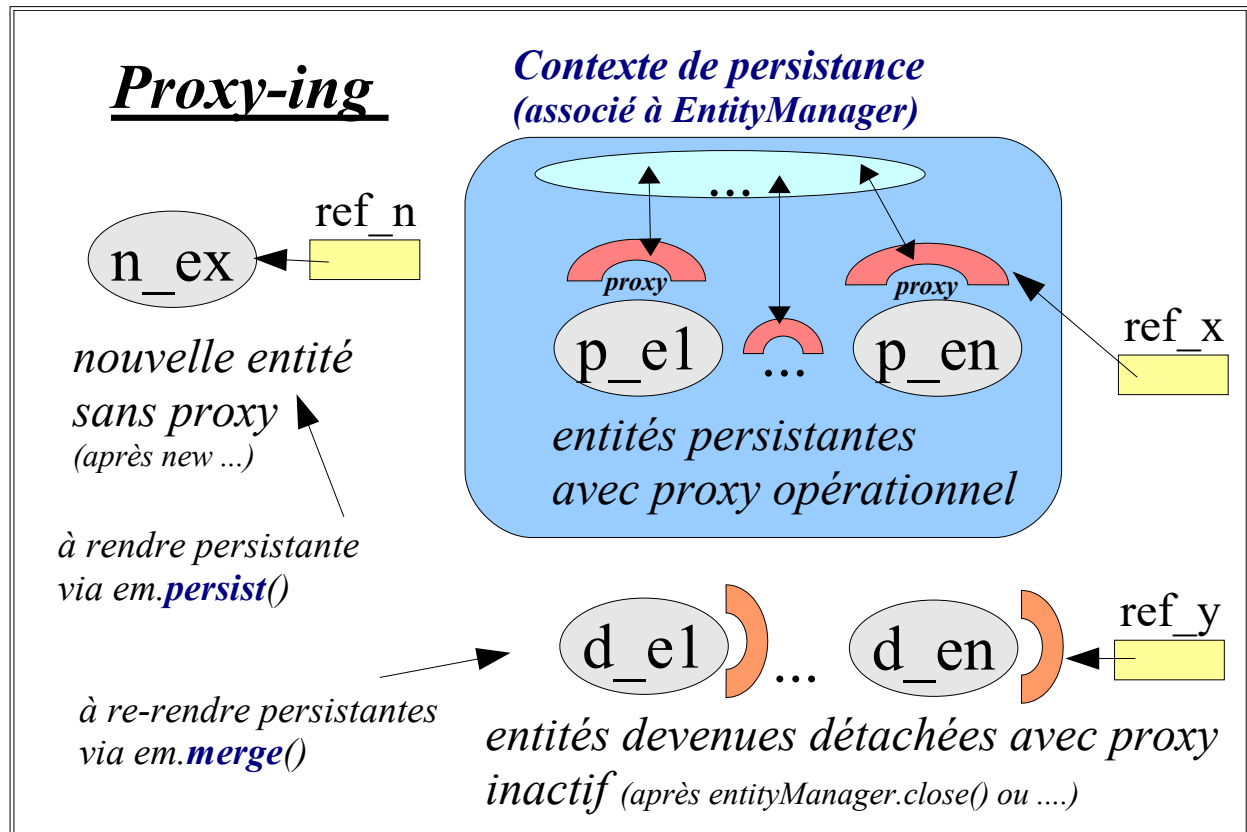
entityManager.getTransaction().commit() déclenche automatiquement **entityManager.flush()**

.clear() vide le contexte de persistance [tout passe à l'état détaché] sans effectuer de **.flush()**

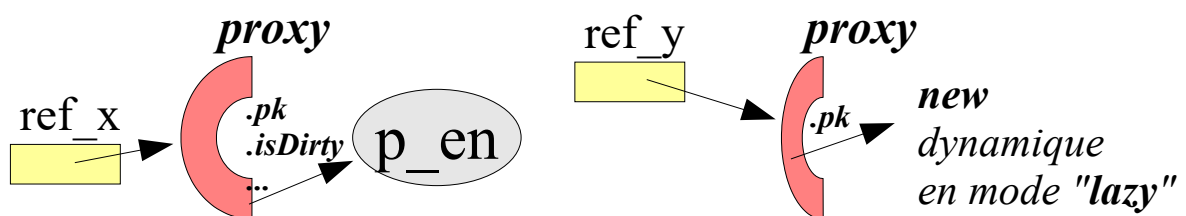
Principales méthodes de l'objet *Query*:

méthodes	traitements
<code>.setParameter(paramName,paramValue)</code>	met à jour un paramètre de la requête jpql
<code>.getSingleResult()</code>	recupère l'unique objet d'une requête simple.
<code>.getResultList()</code>	retourne sous forme d'objet " <i>java.util.List</i> " la liste des objets retournés en résultat de la requête JPQL.
<code>.executeUpdate()</code>	exécute une requête autre qu'un select
<code>.setMaxResults()</code>	fixe le nombre maxi d'éléments à récupérer

8. Contexte de persistance et proxy-ing



Proxy-ing et persistance



Le "**proxy**" (généré par cglib ou autre) sert entre autres à

- **marquer les entités comme devant être mises à jour** en base (*isDirty=true*) lors du prochain "**.flush()**"
- remonter les entités en mémoire de façon différée (**lazy**).

```
ref_persistent = entityManager.merge(ref_détaché);
ref_proxy_sans_entité = entityManager.getReference(class,pk);
```

XIX - Langage de requêtes JPQL

1.1. Présentation générale de HQL et JPQL

JPQL signifie *JPA Query Language*

HQL ou HbQL signifie *Hibernate Query Language*

Ces 2 langages correspondent à des *minis langages dérivés de SQL* et qui permettent d'**exprimer des requêtes** dans le cadre un **mapping objet/relationnel** géré par un **contexte de persistance** JPA ou par une **session Hibernate**.

1.2. Principaux éléments de syntaxe de JPQL et HQL

Le langage JPQL (ou HQL) n'est pas sensible à la casse pour sa partie SQL ("select" est équivalent à "SELECT") mais est sensible à la casse pour sa partie JAVA (la classe "Cat" n'est pas la même de "CAT" et la propriété c.weight n'est pas la même que c.WEIGHT).

Pour récupérer toutes les instances persistantes d'un certain type (c'est à dire tous les enregistrements de la table relationnelle associée) , une simple requête du genre "***from Cat***" (ou bien "***from ppp.Cat***" en précisant optionnellement le package java) est suffisante .

NB: La requête "***from Cat***" retourne non seulement des instances de la classe "***Cat***" mais également des instances des *éventuelles sous classes* (ex: "***DomesticCat***") . Le comportement de JPQL/HQL est bien en accord avec le "***polymorphisme***" du monde objet java.

De façon à pouvoir faire référence à une **instance** (du type précisé par from) dans une autre partie de la requête (ex: where) , on a généralement besoin d'un **alias** que l'on introduit par le mot clef optionnel "***as***" :

"from Cat ***as c***" <==> "from Cat c"

"select c from Cat as c where c.weight > 8"

NB: la partie where est exprimée en utilisant la syntaxe ***nom_alias_instance.nom_propriété*** et non pas ***nom_alias_table.nom_colonne*** .

Autrement dit , la syntaxe de JPQL/HQL est centrée sur le monde objet (java) et pas sur le monde relationnel .

1.3. Quelques exemples de requêtes JPQL/HQL:

Rappel : syntaxe générale du **SELECT** du langage **SQL** :

```
SELECT fieldlist      FROM tablenames      [WHERE searchcondition]
      [GROUP BY fieldlist [HAVING searchconditions] ] [ ORDER BY fieldlist ]
```

Exemples JPQL/HQL:

```
SELECT cat.name FROM DomesticCat AS cat WHERE cat.name LIKE 'fri%'
```

```
SELECT avg(cat.weight), sum(cat.weight), max(cat.weight),
count(cat) FROM Cat cat ==> retourne une liste avec un seul
élément (ligne) de type Object[]
```

```
FROM Cat cat WHERE cat.mate.name is not null
```

```
SELECT c FROM Customer c JOIN c.orders o WHERE c.status = 1
(où c.orders est paramétré par @OneToMany ou @ManyToMany ...)
```

```
... where o.country IN ('UK', 'US', 'France')
```

```
... where town LIKE 'P%' où % signifie sous chaîne quelconque de 0 à n caractères
```

```
... where p.age BETWEEN 15 and 19
```

```
SELECT DISTINCT auth FROM Author auth
```

WHERE EXISTS

```
(SELECT spouseAuth FROM Author spouseAuth WHERE spouseAuth = auth.spouse)
```

```
SELECT auth FROM Author auth WHERE auth.salary >= ALL(SELECT a.salary FROM Author a
WHERE a.magazine = auth.magazine)
```

```
SELECT mag FROM Magazine mag WHERE (SELECT COUNT(art) FROM mag.articles art) > 10
```

```
SELECT pub FROM Publisher pub JOIN pub.magazines mag ORDER BY o.revenue, o.name
```

1.4. Lancement d'une requête JPQL (JPA)

Exemples:

```
import javax.persistence.TypedQuery;
...
public List<Client> getAllClient()
{
    TypedQuery<Client> query =
        entityManager.createQuery("Select c from Client as c", Client.class);
    return query.getResultList();
} ...
```

```
public Categorie getCategorieprincipale() {
    Categorie cat=null;
    Query q = entityManager.createQuery("select c from Categorie as c "
                                     + "where c.parentId is null");
    cat = (Categorie) q.getSingleResult();
    return cat;
}

public List<Element> getElementByDesignation(String designation) {
    Query q = entityManager.createQuery("select e from Element as e "
                                     + "where e.designation=:design");
    q.setParameter("design", designation);      return q.getResultList();
}
```

Remarque importante:

Les API "Hibernate" et "JPA 1" sont nées à l'époque de la transition entre les jdk <=1.4 et les jdk >= 1.5 (lorsqu'il n'existait pas encore de "generic" : List au lieu de List<T>).

La méthode createQuery de l'interface EntityManager est surchargée.

Il existe une version qui renvoie un "TypedQuery" et qui accepte un second paramètre de type Class permettant de préciser le type de retour précis ce qui évite un warning.

D'autre part, les méthodes createQuery() et setParameter() ont été prévues pour s'enchaîner et l'on peut donc écrire :

```
return
    entityManager.createQuery("select c from Client as c where c.age >=:ageMini", Client.class)
        .setParameter("ageMini",18)
        .getResultList();
```

1.5. Lancement d'une requête native avec JPA

```
Query query = entityManager.createNativeQuery(
    "SELECT NAME, SURNAME, AGE FROM PERSON");
List list = query.getResultList();
```

résultat ==>

une liste de tableaux (Object[]) avec NAME à l'index 0, SURNAME à l'index 1 et AGE à l'index 2

Via une version surchargée de createNativeQuery ayant un second paramètre on peut quelquefois préciser le type de retour souhaité (si compatible).

1.6. NamedQuery placée dans une classe d'entité persistante

```
@Entity
@NamedQuery(name="Country.findAll", query="SELECT c FROM Country c")
public class Country {
    ...
}
```

ou bien

```
@Entity
@NamedQueries({
    @NamedQuery(name="Country.findAll",
        query="SELECT c FROM Country c"),
    @NamedQuery(name="Country.findByName",
        query="SELECT c FROM Country c WHERE c.name = :name"),
})
public class Country {
    ...
}
```

et appel via createNamedQuery("named_of_query", ResultType.class) :

```
TypedQuery<Country> query =
    entityManager.createNamedQuery("Country.findAll", Country.class);

List<Country> results = query.getResultList();
```

Intérêt de "NamedQuery" par rapport aux requêtes ordinaires ?

---> c'est à la fois une **affaire de style** et une **affaire de performance**.

- Dans le cas où un service métier n'utilise aucun DAO en arrière plan et utilise l'entityManager en direct, les "NamedQuery" ont l'avantage de bien séparer détails de persistance avec logique métier.
- Etant placées au dessus des classes "@Entity" chargées en mémoire et analysées dès le démarrage de l'application, les requêtes de type @NamedQuery peuvent ainsi être **préparées** par le SGBDR et s'exécutent donc très vite.

1.7. Eventuel DAO générique

Dans la plupart des projets java , on utilise un DAO générique (dont on hérite) pour ne pas répéter trop de fois le même code .

Dans le cadre d'un projet basé sur le framework additionnel **Spring**, les DAOs génériques sont déjà codés au sein de la partie "**SpringDataJpa**" (interface prédéfinie **JpaRepository<E,ID>**) .

Dans le cadre d'un projet n'utilisant pas le framework Spring , on peut éventuellement se coder son propre DAO générique en s'inspirant du code suivant (qui pourrait encore être simplifié dans le contexte d'un projet jakartaEE apportant **@Transactional**) :

```
package tp.core.persistence.generic;
import java.util.List;
/*
DAO = Data Access Object with CRUD methods (Create,Retreive,Update,Delete)
GenericDAO<E,ID> where E = entity type , ID = primaryKey type
=====
usage:
public interface XyzDAO extends GenericDAO<XyzEntity,String_or_Long> { ...}
*/

public interface GenericDAO<E,ID> {
    public List<E> findAll();
    public E findById(ID id);
    public E saveNew(E entity);
    public void update(E entity);
    public E saveOrUpdate(E entity,ID id);
    public void deleteById(ID id);
}
```

```
package tp.core.persistence.generic;
import tp.core.persistence.jpa.MyJpaUtil;
import java.util.List;

public class GenericDaoJpa<E,ID> implements GenericDAO<E,ID> {
    private Class<E> entityClass;
    public GenericDaoJpa( Class<E> entityClass) { this.entityClass = entityClass; }

    public List<E> findAll() {
        List<E> entityList = null;
        entityList = (List<E>) MyJpaUtil.execInTransaction(entityManager ->
            entityManager.createQuery("SELECT e FROM " + entityClass.getSimpleName()
                + " e",entityClass).getResultList()
        );
    }
}
```



```
        return entityList;
    }

    public E findById(ID id) {
        E entity = null;
        entity = (E) MyJpaUtil.execInTransaction(entityManager -> {
            entityManager.find(entityClass,id)
        });
        return entity;
    }

    public E saveNew(E entity) {
        MyJpaUtil.doinTransaction(entityManager -> {
            entityManager.persist(entity);
        });
        return entity;
    }

    public void update(E entity) {
        MyJpaUtil.doinTransaction(entityManager -> {
            entityManager.merge(entity);
        });
    }

    public E saveOrUpdate(E entity, ID id) {
        if(id==null) {
            return saveNew(entity);
        }
        else {
            E exististingEntity = findById(id);
            if(exististingEntity==null)
                return saveNew(entity);
            else {
                update(entity);
                return entity;
            }
        }
    }

    public void deleteById(ID id) {
        MyJpaUtil.doinTransaction(entityManager -> {
            E entity=entityManager.find(entityClass,id);
            entityManager.remove(entity);
        });
    }
}
```

Simplification potentielle du code (dans un projet où @Transactional est disponible soit en version Spring soit en version jakartaEE) :

<pre>public void update(E entity) { MyJpaUtil.doinTransaction(entityManager -> { entityManager.merge(entity); }); }</pre>	<pre>@Transactional public void update(E entity) { entityManager.merge(entity); }</pre>
---	--

Exemple d'utilisation par héritage :

```
public interface XYZDAO extends GenericDAO<XYZEntity,String_or_Long> {
    public XYZ findByXxx(String xxx)
    ...
}
```

```
public class XYZDaoJpa extends GenericDaoJpa<XYZEntity,String_or_Long> implements XYZDAO{
    public XYZDaoJpa() {
        super(XYZEntity.class);
    }
    ...
    public XYZ findByXxx(String xxx) {
        XYZ entity = null;
        entity = (XYZ) MyJpaUtil.executeInTransaction(entityManager ->
            entityManager.createQuery("SELECT e FROM XYZ e WHERE e.xxx = :xxx",
                XYZ.class).setParameter("xxx",xxx).getSingleResultOrNull()
        );
        return entity;
    }
}
```

Exemple de classe de test :

```
package tp.core.persistence.dao;

import lombok.extern.slf4j.Slf4j;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import tp.core.persistence.jpa.MyJpaUtil;
import tp.core.persistence.entity.XYZ;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

@Slf4j
```

```
public class TestXyzDAO {
    private static XyzDAO xyzDAO;

    @BeforeAll
    public static void initDao(){
        MyJpaUtil.setHbm2ddlAuto(true);
        xyzDAO=new XyzDaoJpa();
    }

    @Test
    public void testCrudXyz(){
        String pk="id1";
        xyzDAO.saveNew(new Xyz("Xyz" , pk , ..., 2.02 )); //insertion
        Xyz eRelu = xyzDAO.findById(pk); //relecture pour vérification
        log.info("eRelu="+eRelu);
        assertEquals("Xyz",eRelu.getName());
        assertEquals(2.02,eRelu.getPrice(),0.00001);

        eRelu.setName("Xyz2"); eRelu.setPrice(4.04);
        xyzDAO.update(eRelu); //mise à jour
        Xyz eRelu2 = xyzDAO.findById(pk); //relecture pour vérification
        log.info("eRelu2="+eRelu2);
        assertEquals("Xyz2",eRelu2.getName());
        assertEquals(4.04,eRelu2.getPrice(),0.00001);

        xyzDAO.deleteById(pk); //suppression
        Xyz eRelu3 = xyzDAO.findById(pk); //relecture pour vérification
        assertNull(eRelu3);
    }
}
```

1.8. Protection vis à vis des injections SQL

Avec JPA pour bien se protéger contre les injections SQL il faut utiliser à fond `.setParameter()` et ne surtout pas trop abuser des **concaténations** dans la constitution des requêtes.

Mauvais code (dangereux) :

```
public List<Customer> findByEmail(String customerEmail) {
    return entityManager.createQuery("SELECT c FROM Customer c WHERE c.email = '"
        + customerEmail + "'", Customer.class).getResultList() ;
}
```

Bon code :

```
public List<Customer> findByEmail(String customerEmail) {
    return entityManager.createQuery("SELECT c FROM Customer c WHERE c.email =:email"
        , Customer.class).setParameter("email",customerEmail).getResultList() ;
}
```

Test d'injection SQL :

//valeur potentiellement saisie au clavier dans un formulaire par un "hacker" :

```
String customerEmail = "toto@xyz.com OR c.email='admin@xyz.com' " ;
List<Customer> customers = customerDAO.findByEmail (customerEmail);
```

```
customerEmail = "toto@xyz.com OR 1=1" ;
List<Customer> customers2 = customerDAO.findByEmail (customerEmail);
```

Mauvais comportement avec concaténation :

ça retourne trop de clients (toto et admin ou bien toto et tous les autres)

Bon comportement avec setParameter() : soit exception, soit liste vide .

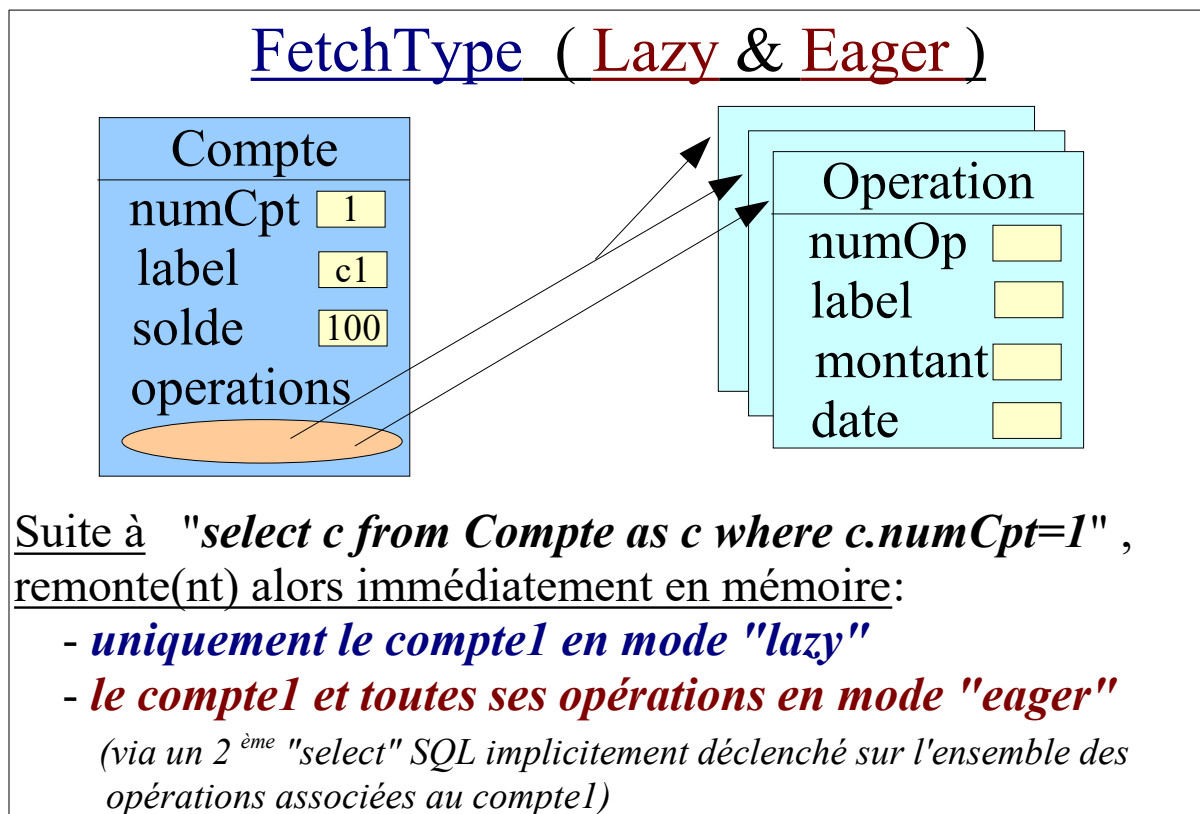
XX - O.R.M. JPA (généralités)

1. Vue d'ensemble sur les entités, valeurs et relations

1.1. Entités et objets valeurs (embedded)

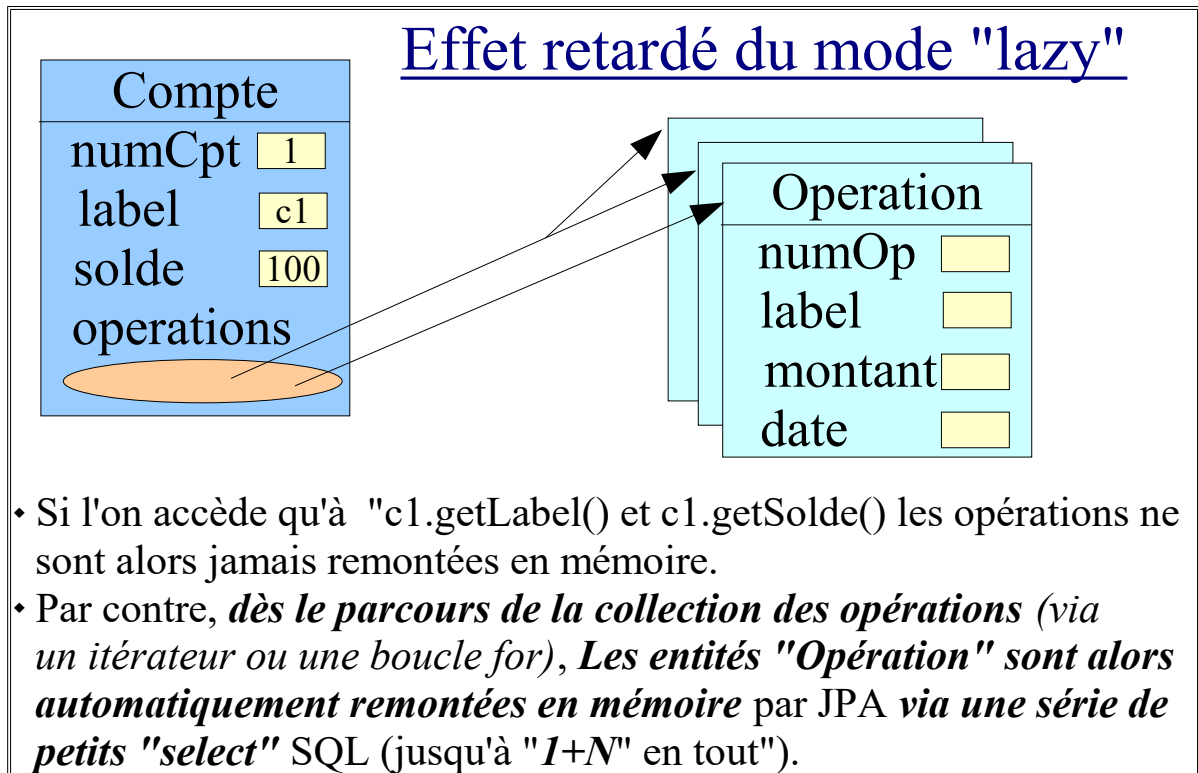
- Une **entité** (@Entity) doit absolument comporter une **clef primaire** (@Id)
- Un sous objet imbriqué (@Embedded) n'a pas de clef primaire et doit absolument être référencé par une entité pour exister.

1.2. FetchType (lazy & eager)



L'attribut optionnel **lazy="true"** d'hibernate (ou bien **fetchType=Lazy** de JPA) permet de demander une **initialisation tardive** des membres d'une collection (les valeurs des éléments de la collection ne seront recherchées en base que lorsqu'elles seront consultées via par exemple un itérateur java). Ceci permet d'obtenir dans certains cas de meilleurs performances . Il faut cependant veiller à récupérer les valeurs de la collection avant le commit et la fermeture de la session (sinon ==> c'est trop tard : **LazyInitialisationException**).

Il est fortement conseillé d'affecter explicitement la valeur "lazy" ou "eager" au paramètre **fetchType** (très important sur le plan des performances).



Exemple JPA:

---> fetch=FetchType.LAZY or fetch=FetchType.EAGER;

```
@OneToMany( fetch=FetchType.LAZY , mappedBy="compte" )
private List<Operation> dernieres_operations ;
```

NB : Si le mode "LAZY" est configuré dans une annotation JPA (ou dans un fichier hbm.xml), il est tout de même possible d'*expliquer ponctuellement une demande de chargement "tout d'un seul coup" via une requête JPQL* du type "select **distinct** c from Compte c **inner join fetch** c.operations inner join c.proprietaires cli where cli.numero = :numCli"

1.3. Operations en cascade

Lorsqu'une opération (delete , update , save, ...) est effectuée sur une entité persistante, elle est alors automatiquement répercutée sur les sous objets de type "valeurs/embbded" .

Lorsque par contre on a affaire à une relation (1-1 ou 1-n) de type "entité-entité" , l'opération (save, update , delete) effectuée sur l'entité principale n'est pas automatiquement répercutée sur les entités liées. On peut cependant demander explicitement à JPA (ou hibernate) de répercuter une opération de mise à jour de l'entité principale vers une ou plusieurs entité(s) liée(s) via l'attribut **cascade** d'une relation .

Syntaxe JPA (dans les annotations):

```
@OneToMany(cascade=CascadeType.ALL , ....)
```

ou bien avec ensemble de cascades combinées:

```
cascade = {CascadeType.PERSIST, CascadeType.MERGE}, ...
```

NB: depuis JPA2 , la cascade "all" + "*delete-orphan*" permet en plus de supprimer automatiquement en base des entités "enfants" qui seraient détachées de leurs parents (suite un une suppression de référence(s) dans une collection par exemple) .

NB: ceci fonctionne même si la mention "cascade" n'est pas précisée dans le schéma relationnel de la base de données.

2. Identité d'une entité (clef primaire)

2.1. Générateurs de clefs primaires

Une **clef primaire** (*ex*: numéro de facture) est assez souvent **générée automatiquement** via un **algorithme** basé sur la notion de **compteur**.

Exemple (JPA) :

```
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
@Column(name="numero")
private Long num ;
```

NB :

- **@GeneratedValue(strategy=GenerationType.IDENTITY)** pour dire que la valeur de num (colonne numero) sera générée par **auto-incrémentation** par la base de donnée lors d'un insert into sql (déclenché via entityManager.persist())
- **@GeneratedValue** permet aussi de bien remonter en mémoire dans l'attribut num la valeur auto-incrémentée par la base
- La stratégie **IDENTITY** est une manière d'auto-incrémenter qui est compatible avec les versions récentes de MySql/MariaDB , Postgres , H2, Oracle , ...

D'autres stratégies existent (*ex* : **SEQUENCE** pour certaines bases telles que Oracle) .

2.2. Eventuelle clef primaire composée

Avec JPA:

Si une clef primaire est composée, il faut alors utiliser les annotations

@EmbeddedId (pour annoter la propriété lié au sous objet "clef primaire")

et

@Embeddable (pour annoter une *classe de clef primaire devant être sérialisable , publique , vu comme un "JavaBean" et devant coder equals() et hashCode()).*

Exemple de classe de clef primaire composée:

@Embeddable

```
public class CustomerPK implements java.io.Serializable
{
    private long id;
    private String name;

    public CustomerPK() { }

    public CustomerPK(long id, String name) {
        this.id = id;    this.name = name;
    }

    public long getId() { return id; }
    public void setId(long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int hashCode() {
        return (int) id + name.hashCode();
    }

    public boolean equals(Object obj)
    {
        if (obj == this) return true;
        if (!(obj instanceof CustomerPK)) return false;
        if (obj == null) return false;
        CustomerPK pk = (CustomerPK) obj;
        return pk.id == id && pk.name.equals(name);
    }
}
```

@Entity

```
public class Customer implements java.io.Serializable
{
    CustomerPK pk;
    ...
    public Customer(){ }

    @EmbeddedId
    public CustomerPK getPk() {
        return pk;
    }

    public void setPk(CustomerPK pk){
        this.pk = pk;
    }
}
```



```
... }
```

3. Propriétés d'une colonne

```
@Column(uptdatable = false, name = "flight_name", nullable = false, length = 50)
```

L'annotation `@Column` se transpose en sous balise `<column>` des fichiers de mapping hibernate (.hbm.xml).

4. Relations (1-1, n-1, 1-n et n-n)

<i>Annotations JPA</i>	<i>Significations</i>
@OneToOne	1-1 (fk [unique] ---> pk) [<i><many-to-one unique='true'> de hibernate</i>]
@OneToMany	1-n (collection avec clef [pk <----fk=clef])
@ManyToOne	n-1 (fk ----> pk)
@ManyToMany	n-n (fk1 ----> (k1,k2) ---> pk2)

NB: les annotations `@OneToOne`, `@OneToMany` et `@ManyToMany` peuvent éventuellement comporter un attribut `"mappedBy"` dont la valeur correspond à la *propriété qui sert à établir une correspondance inverse (et secondaire)* au sein d'une **relation bidirectionnelle**.

NB: les mises à jour des relations effectuées uniquement du côté secondaire d'une relation bidirectionnelle ne seront pas automatiquement sauvegardées (tant que le côté principal de la relation n'aura pas été explicitement réajusté).

L'annotation `@JoinColumn` permet d'indiquer (via `name="..."`) le *nom de la colonne correspondant à la clef étrangère* dans la base de données.

exemple:

```
@OneToOne(cascade = {CascadeType.ALL})
@JoinColumn(name = "ADDRESS_ID")
public Address getAddress()
{
    return address;
}

public void setAddress(Address address)
{
    this.address = address;
}
```

4.1. Eventuelle classe abstraite (pour héritage)

```
...
import jakarta.persistence.MappedSuperclass;
...
@MappedSuperclass
public class BasicEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String label;
    ...
}
```

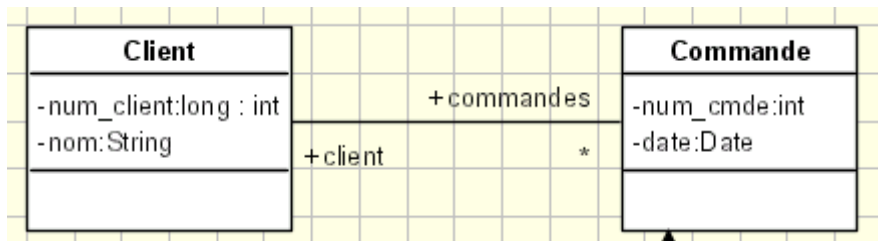
```
@Entity
@Table(name="product_with_details")
public class ProductWithDetails extends BasicEntity {
    ...
}
```

NB : grâce à *@MappedSuperclass* placé sur la classe BasicEntity, les sous classes récupèrent bien certains paramétrages "jpa" associés aux éléments hérités (ex : id avec *@Id* et *@GeneratedValue*) .

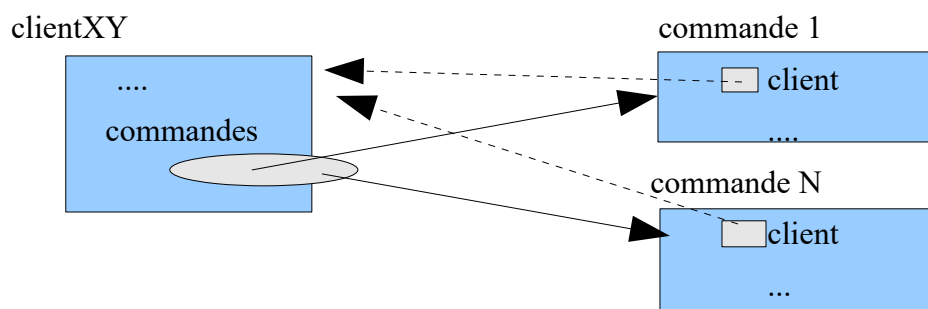
XXI - O.R.M. JPA – détails (1-n , 1-1 , n-n, ...)

1.1. Relations 1-n et n-1 (entité-entités)

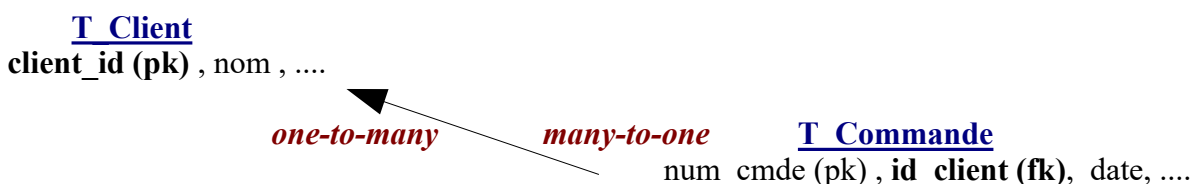
UML:



Java:



Base de données:



Annotations JPA:

```

@Entity
@Table(name = "T_Commande")
public class Commande
{
    @ManyToOne
    @JoinColumn(name = "id_client") //fk (nom de colonne dans la table)
    private Client client;
    ...
    public Client getClient() { return client; }
    public void setClient(Client client) { this.client = client; }
}
  
```

```

@Entity
@Table(name = "T_Client")
public class Client
{
    //NB : "client" (valeur de mappedBy) = nom (java) de la relation inverse
    @OneToMany(fetch=FetchType.LAZY, mappedBy="client")
    private List<Commande> commandes;
    ...
}
  
```

```

public List<Commande> getCommandes() { return commandes; }
public void setCommandes(List<Commande> cmdes) { this.commandes = cmdes; }

public void addCommande(Commande c) {
    if (commandes == null) commandes = new ArrayList<Commande>();
    commandes.add(c); }
}

```

Rappels importants (pour JPA):

- La valeur de l'attribut "**mappedBy**" de l'annotation **@OneToMany** correspond à la **propriété (dans l'autre classe java) qui sert à établir une correspondance inverse** au sein d'une **relation bidirectionnelle**.
- L'annotation **@JoinColumn** permet d'indiquer (via name="...") le nom de la colonne correspondant à la clef étrangère dans la base de données.

1.2. Relations 1-n (avec JoinTable) évolutives vers du n-n

Bien qu'une relation 1-n (symétriquement n-1) soit souvent basée sur une jointure simple avec une clef étrangère en base, il est tout de même possible de mettre en place une relation 1-n avec une table de jointure en base. Déjà structurée en base comme une relation n-n, une telle relation 1-n a le mérite d'être facilement évolutive vers du n-n.

Exemple :

```

@Entity
@Table(name="Compte")
public class Compte {
    ...
    @ManyToOne //evolutif vers @ManyToMany
    @JoinTable(name="ClientCompte",
               joinColumns={@JoinColumn(name="num_compte")},
               inverseJoinColumns={@JoinColumn(name="num_client")})
    private Client client;
}

```

```

@Entity
@Table(name="Client")
public class Client extends Personne {
    ...
    @OneToMany(mappedBy="client")
    private List<Compte> comptes;
    ...
}

```

avec

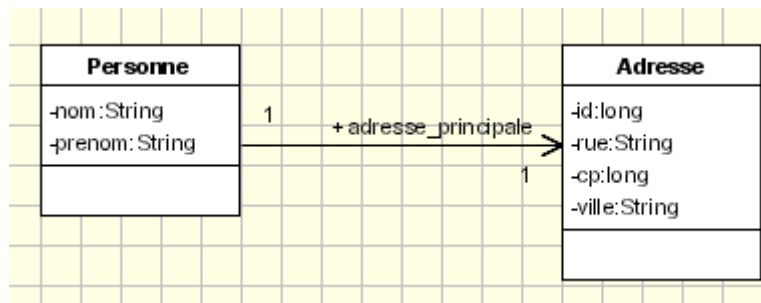
```

create table ClientCompte( num_client integer, num_compte integer,
                          primary key(num_client,num_compte));
ALTER TABLE ClientCompte ADD CONSTRAINT client_avec_compte_valide
    FOREIGN KEY (num_compte) REFERENCES Compte(numero);
ALTER TABLE ClientCompte ADD CONSTRAINT compte_avec_client_valide
    FOREIGN KEY (num_client) REFERENCES Client(num_client);

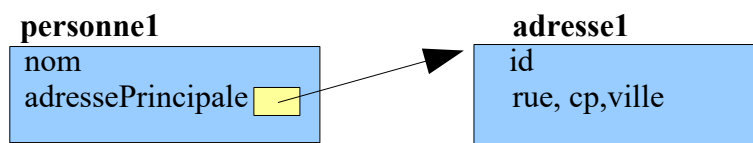
```

1.3. Relation 1-1 via clef étrangère (évolutif vers n-1)

UML:



Java:



====> exemple: `personne1.getAdressePrincipale().getVille();`

Base de données:

T Personne

id (pk) , nom , prenom , , id_adr (fk)

@OneToOne (JPA)

T Adresse

id (pk) , rue , cp , ville

1-1 ici évolutif vers n-1 (exactement structuré et paramétré comme n-1 et @ManyToOne) .
Lorsque c'est possible une contrainte **unique=true** sur la colonne id_adr (fk) peut indiquer une intention de 1-1 plutôt que n-1 .

JPA:

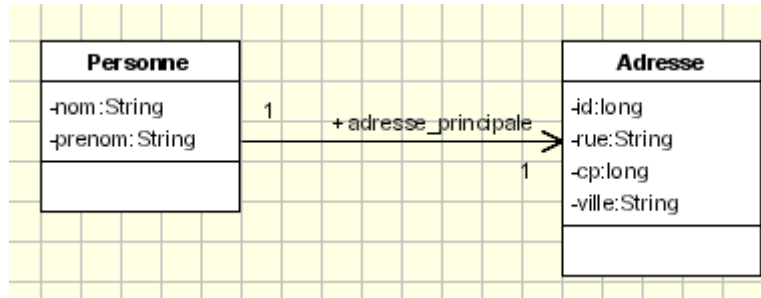
```

@Entity
public class Personne
{
    @OneToOne()
    @JoinColumn(name = "id_adr", unique=true) //fk

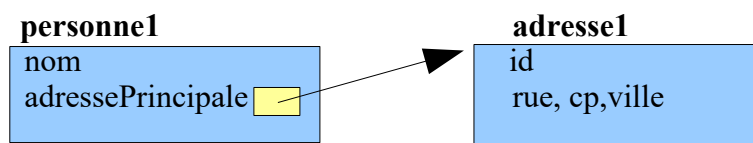
    private Adresse adressePrincipale;
    ...
    public Adresse getAdressePrincipale() {
        return adressePrincipale;
    }
    public void setAdressePrincipale(Adresse adressePrincipale) {
        this.adressePrincipale = adressePrincipale;
    }
}
  
```

1.4. Relation 1-1 via Table secondaire et correspondance pk

UML:

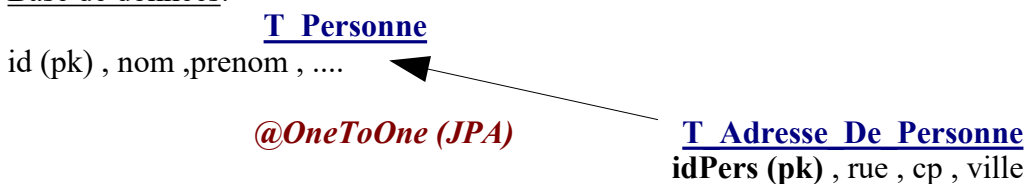


Java:



====> exemple: `personne1.getAdressePrincipale().getVille();`

Base de données:



Les valeurs de idPers (clef primaire) de la table secondaire "T_Adresse_De_Personne" doivent absolument correspondre à celle de "id" (clef primaire) de la table principale "T_Personne". Cette correspondance rigide entre les clefs primaires des tables principale et secondaire n'est pas évolutif vers du n-1 . c'est du 1-1 pour toujours .

JPA:

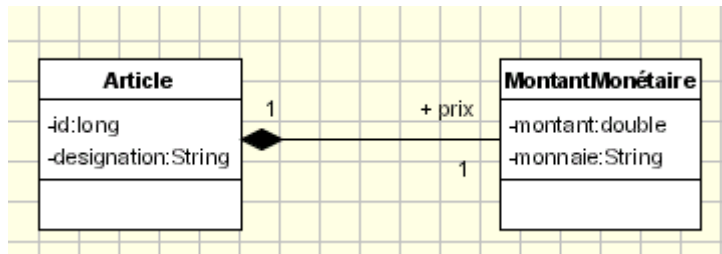
<pre> @Entity @Table(name="T_Personne") public class Personne { @Id @GeneratedValue(strategy=....IDENTITY) private Long id ; @OneToOne(optional=true , mappedBy="personne", cascade = CascadeType.ALL) private AdresseDePersonne adressePrincipale; ... } </pre>	<pre> @Entity @Table(name="T_Adresse_De_Personne") public class AdresseDePersonne { @Id private Long idPers; private String rue ; private String cp; private String ville; @OneToOne(optional=false) // @PrimaryKeyJoinColumn @MapsId //mieux @JoinColumn(name="idPers") private Personne personne; } </pre>
--	---

NB : @MapsId (de JPA2) est mieux que @PrimaryKeyJoinColumn (JPA 1)

1.5. Sous objets incorporable au sein d'une entité

Appelés objets "*Valeur composite*" ou "*Component*" dans la terminologie *Hibernate* , et appelés objets "*Embeddable*" dans la terminologie *JPA*, d'éventuels sous objets composés (non partageables et sans clef primaire) peuvent être utilisés pour structurer une entité.

UML:



En base de données:

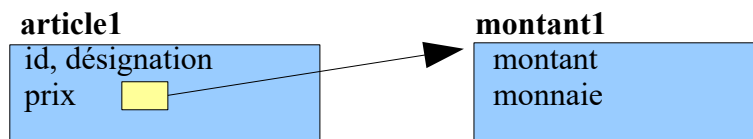
T Article

id (pk) , designation , *montant* , *monnaie* ,

==> mapping relationnel associé (par défaut):

- pas de table relationnelle séparée pour les sous objets "valeurs"
- liste des propriétés du composant directement incluse dans la table de l'objet contenant .

En Java:



==> `xxx.getPrix().getMontant();` et `xxx.getPrix().getMonnaie();`

NB: La classe java du composant imbriqué n'a pas d' ID (pas de clef primaire) . Elle respecte néanmoins les conventions "**JavaBean**" pour les éléments internes (ici montant et monnaie) .

JPA:

Une classe d'objet incorporable doit normalement être annotée via "**@Embeddable**" .

Une propriété d'une entité qui référence un objet imbriqué doit être annotée via **@Embedded** .

Les spécifications JPA se limitent à un seul niveau d'imbrication.

exemple:

@Embeddable

```

public class MontantMonetaire
{
    private double montant; // + get/set
    private String monnaie; // + get/set
    public MontantMonetaire() { }

    public MontantMonetaire(double montant, String monnaie) {
        this.montant = montant;    this.monnaie = monnaie;
    }
}
  
```

```
}
```

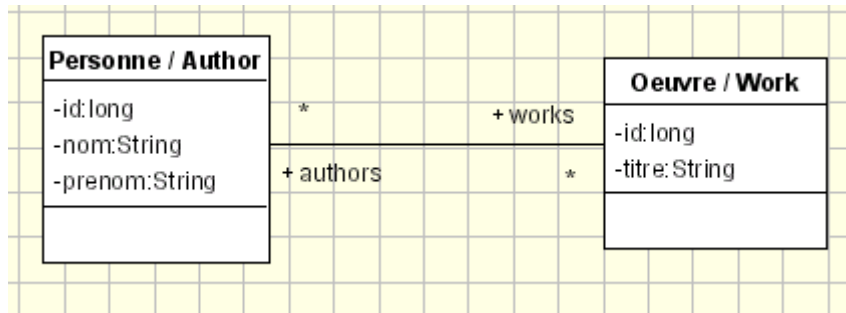
```
@Entity
@Table(name = "T_Article")
public class Article
    private long id; // avec get/set et @Id

    @Embedded
    /* @AttributeOverrides({
        @AttributeOverride(name = "montant", column = @Column(name = "montant")),
        @AttributeOverride(name = "monnaie", column = @Column(name = "monnaie"))
    }) */
    private MontantMonetaire prix;
    ...
    public MontantMonetaire getPrix() { return prix; }
    public void setPrix( MontantMonetaire prix) { this.prix = prix; }
}
```

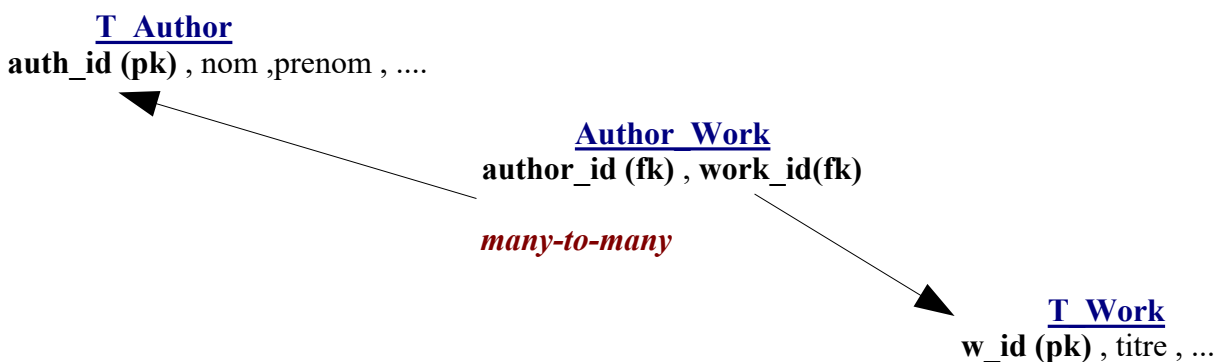
NB : `@AttributeOverride` n'est réellement utile que si les noms des colonnes sont différentes des noms des propriétés java de l'objet incorporé .

1.6. Relation n-n ordinaire

UML:



Mapping relationnel ==> *Table intermédiaire*



Java:

--> des collections dans les 2 sens
avec un sens principal et un sens secondaire (avec mappedBy).

JPA:

du côté "principal" de la relation (coté où les mises à jour seront "persistées"):

```
@Entity
public class Work
{
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "Author_Work",
        joinColumns = {@JoinColumn(name = "work_id")},
        inverseJoinColumns = {@JoinColumn(name = "author_id")})
    private List<Author> authors;
    ...
    public List<Author> getAuthors() {
        return authors;
    }
    public void setAuthors(List<Author> authors) { this.authors = authors; }
    ..
}
```

NB: l'attribut **joinColumns** (de @JoinTable) correspond à la **clef étrangère pointant vers l'entité courante** et l'attribut **inverseJoinColumns** correspond à la **clef étrangère pointant vers les éléments de l'autre côté de la relation (ceux qui seront rangés dans la collection paramétrée)**.

et du coté inverse/secondaire:

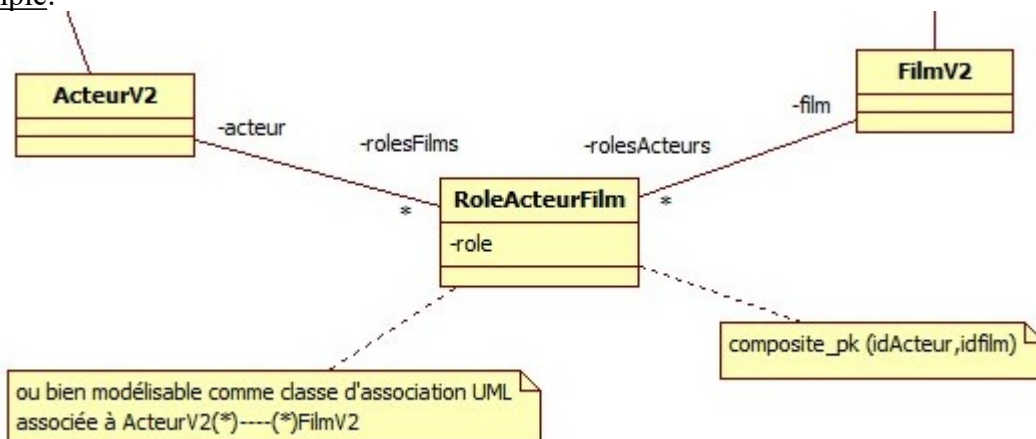
```
@Entity
public class Author
{
    @ManyToMany(mappedBy="authors")
    private List<Work> works;
    ...
    public List<Work> getWorks() {
        return works;
    }
    public void setWorks(List<Work> works) {    this.works = works;    }
}
```

1.7. Relation n-n avec détails d'association

Remarque importante sur les relations (n-n):

Lorsque la table de jointure comporte des informations supplémentaires autres que les clefs étrangères, on est alors obligé de décomposer la relation n-n en deux relations (1-n + n-1) et les éléments de la table de jointure sont alors vus comme des entités ayant une clef primaire (souvent composée par les 2 clefs étrangères).

Exemple:



```
CREATE TABLE ActeurFilmV2
( idActeur integer,
  idFilm integer ,
  role VARCHAR(64) ,
  primary key(idActeur,idFilm));

ALTER TABLE ActeurFilmV2 ADD CONSTRAINT avec_acteur_valideV2
    FOREIGN KEY (idActeur) REFERENCES Acteur(idActeur);
ALTER TABLE ActeurFilmV2 ADD CONSTRAINT avec_film_valideV2
    FOREIGN KEY (idFilm) REFERENCES Film(idFilm);
```

```

@Entity
@Table(name="Acteur")
public class ActeurV2 {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private long idActeur ;

    private String nom ;

    @OneToMany(mappedBy="acteur")
    private List<RoleActeurFilm> rolesFilms;

    ...
}

```

```

@Entity
@Table(name="ActeurFilmV2")
public class RoleActeurFilm {

    @EmbeddedId
    private RoleActeurFilmCompositePk pk ;

    private String role;

    @ManyToOne
    @JoinColumn(name="idActeur")
    @MapsId("idActeur") //pk.idActeur
    private ActeurV2 acteur;

    @ManyToOne
    @JoinColumn(name="idFilm")
    @MapsId("idFilm") //pk.idFilm
    private FilmV2 film;

    public RoleActeurFilm() {
        super(); pk = new RoleActeurFilmCompositePk();
    }

    public RoleActeurFilm(String role, ActeurV2 acteur, FilmV2 film) {
        super();
        this.role = role; this.acteur = acteur; this.film = film;
        pk=new RoleActeurFilmCompositePk(acteur.getIdActeur(),
                                           film.getIdFilm());
    }

    //+get/set , ...
}

```

NB: Avec JPA 1.0 , @MapsId() n'existait pas et @JoinColumn(name="id_...", insertable=false, updatable=false) était indispensable pour que JPA puisse gérer un double mapping où les colonnes "idFilm" et "idActeur" sont déjà précisées au sein de la clef primaire (composite pk).

Depuis JPA 2, l'annotation MapsId("id...") permet de préciser la sous

partie de la classe de clef primaire composite java qui sera associée à la clef étrangère (du @ManyToOne / @JoinColumn) .
Un futur appel à `roleActeurFilm.setActeur( acteur )` déclenchera alors automatiquement une mise à jour de **`pk.idActeur`** .

```
@Embeddable
public class RoleActeurFilmCompositePk implements Serializable {
    private static final long serialVersionUID = 1L;

    private long idActeur;
    private long idFilm;

    public RoleActeurFilmCompositePk() {
        super();
    }

    public RoleActeurFilmCompositePk(long idActeur, long idFilm) {
        super();
        this.idActeur = idActeur;
        this.idFilm = idFilm;
    }

    //+get/set , equals et hashCode (generated by eclipse)
}
```

```
@Entity
@Table(name="Film")
public class FilmV2 {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private long idFilm;
    private String producteur;
    private String titre;

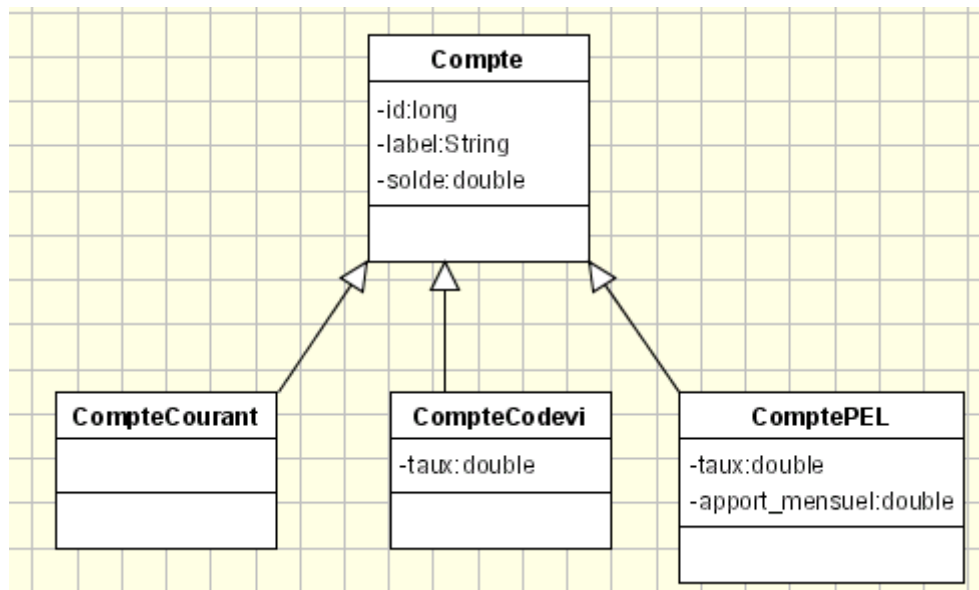
    @Column(name="dateSortie")
    @Temporal (DATE) //partie significative = DATE et pas DateTime
    public Date date;

    @OneToMany(mappedBy="film")
    private List<RoleActeurFilm> rolesActeurs ;

    ...
}
```

2. Relations d'héritage & polymorphisme

UML:



2.1. stratégie "une seule table par hiérarchie de classes"

Stratégie "une table par hiérarchie de classes"
(avec colonne discriminante)

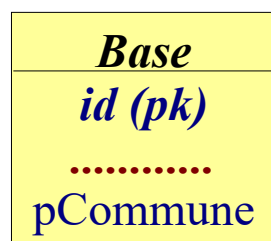


Table "Base_et_dérivés"

<i>id</i>	<i>typeDérivé</i> (discriminateur)	<i>pC.</i>	<i>p1a</i>	<i>p1b</i>	<i>p2</i>	<i>p3</i>
1	D1	xx	aa	bb	Null	Null
2	D2	yy	Null	Null	22	Null
3	D3	zz	Null	Null	Null	33

Colonne discriminante (*typeDérivé*)

Null (N/A)

Une seule grande table permet d'héberger les instances de toute une hiérarchie de classe.

Pour distinguer les instances des différentes sous classes , on utilise une propriété discriminante (à telle valeur correspond telle sous classe).

Cette stratégie (relativement simple) est assez pratique et adaptée dans le cas où il y a peu de différences structurelles entre les sous classes .

Contrainte : les colonnes liées aux attributs des sous classes ne peuvent pas avoir la contrainte "NOT NULL" . (**Pour une instance de la sous classe D1 , les colonnes inutilisées de la sous classe D2 doivent avoir la valeur NULL**) .

Pour la classe de base (JPA):

```
@Entity
@Table(name="Compte")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "typeCompte",
                    discriminatorType = DiscriminatorType.STRING)
/* @DiscriminatorValue("CompteCourant") //éventuellement en tant que valeur par défaut
// et si pas de sous classe CompteCourant mais utilisation directe du parent "Compte" */
public class Compte
{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private long id;
    private String label;
    private double solde;

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getLabel() { return label; }
    public void setLabel(String label) { this.label = label; }

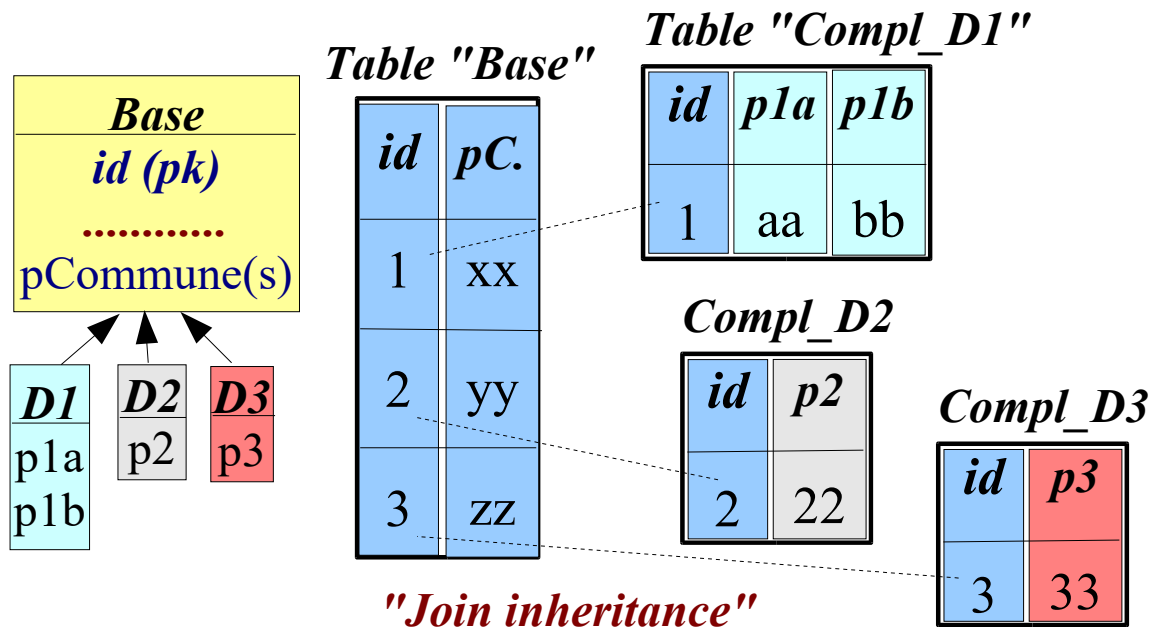
    public double getSolde() { return solde; }
    public void setSolde(double solde) { this.solde = solde; }
}
```

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorValue("PEL")
public class ComptePEL extends Compte
{
    private double tauxInteret; //+get/set
    private double apportMensuel; //+get/set
    ...
}
```

2.2. Stratégie "Join_inheritance"

Stratégie "table principale

+ une table (de compléments) par classe fille"



```
@Entity
@Table(name="Compte")
@Inheritance(strategy=InheritanceType.JOINED )
public class Compte
{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private long id;
    private String label; // +get/set
    private double solde; // +get/set

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    ...
}
```

NB : Bien que "pas indispensable" , une éventuelle colonne discriminante pourrait éventuellement être ajoutée si l'implémentation de JPA en tient compte du point de vue des performances.

```
@Entity
@Table(name="ComptePEL")
@Inheritance(strategy=InheritanceType.JOINED )
public class ComptePEL extends Compte
{
    private double tauxInteret; //+get/set
    private double apportMensuel; //+get/set

    ...
}
```

2.3. Stratégie (assez rare) "une table par classe"

Avec tout un tas de restrictions (voir documentation de référence si besoin)

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Compte
{
    @Id
    @GeneratedValue(strategy=GenerationType.TABLE)
    ...
    public int getId()    {    return id;    }

    public void setId(int id)  {    this.id = id;    }

    ...
}
```

```
@Entity
public class ComptePEL extends Compte
{
    private double tauxInteret; // +get/set
    ...
}
```

2.4. Polymorphisme

Une requête JPQL/HQL exprimée avec un type correspondant à une classe parente retournera (par défaut / sans restriction explicite) **toutes les instances de toute une hiérarchie de classes (classe mère + classe fille)**. Le type exact de chacune des instances retournées sera précis (bien que compatible avec le type de la sur-classe) ==> **polymorphisme complet** (java/mémoire + au niveau O.R.M.).

Autrement dit :

select c from Compte as c retournera tous les types de Compte (courant , codevi , pel).

"select c from ComptePEL as c" ne retournera que des comptes de types "PEL" .

XXII - JPA : aspects divers et avancés

1. Quelques spécificités de JPA

1.1. Une entité répartie dans 2 tables (principale, secondaire)

NB: avec JPA, il est également possible d'associer quelques propriétés d'une entité à une table secondaire :

```
@Entity
@Table(name = "CUSTOMER")
@SecondaryTable(name = "CUSTOMER_ADDRESS")
public class Customer implements java.io.Serializable
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    Long id;

    @Column(name = "STREET", table = "CUSTOMER_ADDRESS")
    private String street ;

    @Column(name = "CITY", table = "CUSTOMER_ADDRESS")
    private String city ;
    ...
    public String getStreet() { return street; }
    public void setStreet(String street) { this.street = street; }

    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
}
```

==> dans ce cas (clef primaire table_secondeire = clef_primaire_table_principale) .

Possibilité de choisir/spécifier le nom de la colonne clef primaire de la table secondaire :

```
@SecondaryTable(name = "CUSTOMER_ADDRESS",
    pkJoinColumns = @PrimaryKeyJoinColumn(name = "customer_id"))
```

@SecondaryTables existe au cas où on aurait besoin de plusieurs tables secondaires.

1.2. Éléments requis sur une classe d'entité (JPA):

- vraie classe (pas interface ni enum) non finale (pas de mot clef "final" , on doit pouvoir en

hériter).

- un constructeur par défaut (sans argument) obligatoire et devant être public ou protégé (d'autres constructeurs sont possibles mais non indispensables).
- annotation "@Entity" ou bien <entity ...> dans un descripteur Xml.
- éventuelle implémentation de *java.io.Serializable* si besoin d'un transfert distant à l'état détaché.

NB:

- Des héritages sont possibles (entre "entité" et "non entité" et vice versa) et le polymorphisme peut s'appliquer sans problème.
- L'état d'une entité est liée à l'ensemble des valeurs de ses attributs (devant être privés ou protégés) .
- L'accès aux propriétés d'une entité doit s'effectuer via des "getter/setter" (conventions JavaBean).

La persistance (mapping ORM) s'applique sur toutes les propriétés non "transientes" d'une entité. Sachant qu'une éventuelle propriété non persistante doit être marquée via "private transient ..." et par l'annotation "@Transient" .

1.3. Verrous (optimistes et pessimistes)

Il existe deux modèles de verrouillage pour les données d'une base de données :

- le verrouillage optimiste, qui verrouille l'enregistrement uniquement lors de la validation des changements dans la base de données.
- le verrouillage pessimiste, qui verrouille l'enregistrement au moment de son édition.

NB : en mode optimiste, si la mise à jour de l'enregistrement aboutit pour un utilisateur, les autres utilisateurs qui tentent de valider leurs mises à jour concurrentes sont informés de l'existence d'un conflit et devront ré-essayer d'effectuer une mise à jour cohérente si possible.

1.4. Verrouillage optimiste

D'après la spécification officielle JPA 2.1 , une entité est automatiquement prête à être gérée en mode optimiste par le "provider_JPA" (ex:hibernate) si elle comporte une propriété qui est mappé avec @Version .

Mode de fonctionnement (en mode optimiste) :

- en début de transaction, le numéro de version est lu et mémorisé en mémoire dans l'entité
- au moment où la transaction souhaite finaliser sa mise à jour, le numéro de version est

relu/revérifié dans la base de données.

- Si ce numéro de version a changé entre temps (autre mise à jour concurrente plus rapide), une exception de type ***OptimisticLockException*** est levée et on peut quelquefois retenter une mise à jour cohérente (selon le contexte si c'est possible ou pas).
- Sinon, en cas de commit réussi en mode "update", le numéro de version est alors automatiquement incrémenté.

@Entity

```
public class Student {  
  
    @Id  
    private Long id;  
  
    private String name;  
  
    private String lastName;  
  
    @Version  
    private Integer version;  
  
    // +get/set  
}
```

Éléments imposés sur numéro de version : doit être numérique, sur la table principale et unique pour une entité donnée.

2. EntityGraph (depuis JPA 2.1)

Un "entityGraph" correspond à un nouveau paramétrage de *JPA ≥ 2.1* permettant de préciser au moment de l'exécution d'une requête quels sont les attributs d'une entité qui doivent être immédiatement chargés de la base vers la mémoire (comme si on avait placé le paramétrage *FetchType.EAGER* au niveau de cet attribut).

En gros, via les entityGraph, plus de EAGER dans le code, moins besoin du mot clef FETCH dans les requêtes JPQL et quelquefois la possibilité de déclencher des requêtes à plusieurs niveaux de profondeur.

2.1. Paramétrage via @NamedEntityGraph

```
@Entity
@NamedEntityGraph(name = "entity-graph-customer-accounts",
    attributeNodes = {
        @NamedAttributeNode(value="accounts"),
        @NamedAttributeNode(value="address"),
    }
)
class Customer {
    ....
    @OneToMany(fetch = FetchType.LAZY, mappedBy="customer")
    private List<Account> accounts ;
    ...
    @OneToOne(mappedBy="customer")
    private Address address ;
    ...
}
```

Depuis java 8, les annotations de JPA sont répétables. L'imbrication de @NamedEntityGraph au sein de @NamedEntityGraphs n'est utile que pour java ≤ 7.

NB : il est également possible de paramétrer un entityGraph par code en partant de l'entityManager.

```
EntityGraph<Customer> entityGraph = entityManager.createEntityGraph(Customer.class);
entityGraph.addAttributeNodes("accounts");
entityGraph.addAttributeNodes("address");
```

2.2. Utilisation d'un entityGraph

propriété	comportement
javax.persistence.fetchgraph	eager pour attributs de l'entityGraph et lazy pour les autres
javax.persistence.loadgraph	eager pour attributs de l'entityGraph et "par défaut" pour les autres

via entityManager.find() :

```
EntityGraph entityGraph = entityManager.getEntityGraph("entity-graph-customer-accounts");
Map<String, Object> properties = new HashMap<>();
properties.put("javax.persistence.fetchgraph", entityGraph); //jakarta.persistence.fetchgraph
return entityManager.find(Customer.class, idCustomer, properties);
```

via requête JPQL compatible :

```
EntityGraph entityGraph = entityManager.getEntityGraph("entity-graph-customer-accounts");
return entityManager.createQuery(
    "SELECT c FROM Customer c WHERE c.id = ?1", Customer.class)
    .setParameter(1, idCustomer)
    .setHint("javax.persistence.fetchgraph", entityGraph)
    .getSingleResult();
```

via complexe api criteria (pour les puristes du typage fort) :

```
EntityGraph entityGraph = entityManager.getEntityGraph("entity-graph-customer-accounts");
CriteriaBuilder criteriaBuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> criteriaQuery = criteriaBuilder.createQuery(Customer.class);
Root<Customer> root = criteriaQuery.from(Customer.class);
criteriaQuery.where(criteriaBuilder.equal(root.<Long>get("id"), idCustomer));
TypedQuery<Customer> typedQuery = entityManager.createQuery(criteriaQuery);
typedQuery.setHint("javax.persistence.loadgraph", entityGraph); //ou jakarta.persistence...
return typedQuery.getSingleResult();
```

2.3. Avec "subgraph" pour plusieurs niveaux de profondeur

```
@NamedEntityGraph(name = "entity-graph-customer-accounts-operations",
    attributeNodes = {
        @NamedAttributeNode(value="accounts", subgraph = "account-subgraph"),
        @NamedAttributeNode(value="address")
    },
    subgraphs = {
        @NamedSubgraph( name = "account-subgraph",
            attributeNodes = {
                @NamedAttributeNode("operations")
            }
        )
    }
})
```

customer → *address*

customer → *accounts* → *operations*

2.4. Précautions et limitations

JPA/Hibernate ne gère pour l'instant pas très bien le "DISTINCT" dans les requêtes automatiquement générées et déclenchées (surtout dans le cadre de relations "many-to-many").

3. Cycle de persistance (pour @Entity) et annotations/callbacks associées pour "Listener"

<i>Annotations (callback)</i>	<i>déclenchement (étape du cycle de persistance)</i>
@PrePersist	juste avant passage à l'état persistant
@PostPersist	juste après passage à l'état persistant (clef primaire souvent affectée)
@PreRemove	juste avant suppression
@PostRemove	juste après suppression et destruction
@PreUpdate	juste avant mise à jour
@PostUpdate	juste après mise à jour
@PostLoad	juste après chargement en mémoire (suite à une recherche)

```

@Entity
@Table(name = "CUSTOMER")
@EntityListeners(CustomerCallbackListener.class)
public class Customer implements java.io.Serializable
{
    ...
}

```

```

public class CustomerCallbackListener
{
    @PrePersist
    public void doPrePersist(Customer customer) {
        System.out.println("doPrePersist: About to create Customer: " + ...);
    }

    @PostPersist public void doPostPersist(Customer customer) {
        System.out.println("doPostPersist: Created Customer: " + ...);
    }

    @PreRemove public void doPreRemove(Customer customer) {
        System.out.println("doPreRemove: About to delete Customer: " + ...);
    }

    @PostRemove public void doPostRemove(Customer customer) {
        System.out.println("doPostRemove: Deleted Customer: " + ...);
    }

    @PreUpdate public void doPreUpdate(Customer customer) {
        System.out.println("doPreUpdate: About to update Customer: " + ...);
    }

    @PostUpdate public void doPostUpdate(Customer customer) {
        System.out.println("doPostUpdate: Updated Customer: " + ...);
    }

    @PostLoad public void doPostLoad(Customer customer) {
        System.out.println("doPostLoad: Loaded Customer: " + ...);
    }
}

```

4. Annotations diverses de JPA

4.1. @OrderBy sur collections (ex : @OneToMany)

```
@OrderBy("name ASC")
List<Foo> fooList;
```

4.2. CLOB & BLOB

L'annotation **@Lob** permet de spécifier que la valeur d'une propriété correspondra à un encodage (texte ou binaire d'un objet potentiellement de grande taille variable)

Deux variantes:

- **CLOB** – **Character Large Object** will store large text data (ex : JSON STRING)
- **BLOB** – **Binary Large Object** is for storing binary data like images, audio, or video

```
@Entity
public class Report {
    @Id
    @GeneratedValue
    private long id;

    @Lob
    @Column(name = "CONTENT")
    private String reportContent;

    @Lob
    @Column(name = "IMAGE")
    private byte[] reportImage;
    .....
}
```

4.3. @Basic

@Basic est une annotation facultative que l'on peut placer sur une propriété dont le type est basique (String, Integer, ...).

Implicite avec les attributs par défaut `fetch=FetchType.EAGER` et `optional = true`), **@Basic** qui influe sur la partie java (contrairement à **@Column** qui paramètre le côté "colonne en base de données") est essentiel utile (en mode explicite) pour fixer les valeurs de l'attribut `optional` à `false` ou bien de l'attribut `fetch` à `FetchType.LAZY` dans certains cas rares.

L'utilisation conjointe de **@Basic(fetch=FetchType.LAZY)** et **@Lob** peut être quelquefois intéressante.

Ceci dit, ça dépend de l'implémentation (provider "JPA").
Paramétrage nécessaire pour hibernate :

```
<plugin>
<groupId>org.hibernate.orm.tooling</groupId>
<artifactId>hibernate-enhance-maven-plugin</artifactId>
<version>${hibernate.version}</version>
<executions>
  <execution>
    <configuration>
      <failOnError>true</failOnError>
      <enableLazyInitialization>true</enableLazyInitialization>
    </configuration>
    <goals>
      <goal>enhance</goal>
    </goals>
  </execution>
</executions>
</plugin>
```


5. Variantes pour le mapping JPA

Il peut quelquefois être intéressant de faire coexister différentes variantes d'un mapping JPA de manière à :

- soit obtenir des détails sur un niveau limité de données
- soit obtenir que l'essentiel d'une grande arborescence de données

Une piste envisageable consiste à partir d'une classe un peu abstraite et par héritage , décliner toute une série de variantes .

Exemple :

```
//no @Entity , no @Table here but only on subclasses
@MappedSuperclass //NB: @MappedSuperclass is important, abstract may be omitted
public abstract class AbstractCustomer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstname;
    private String lastname;
    ...
}
```

```
@Entity
@Table(name="customer")
public class CustomerWithAddress extends AbstractCustomer{
    @OneToOne(optional=true,
                mappedBy="customer",
                cascade = CascadeType.PERSIST,fetch = FetchType.LAZY)
    private Address address;

    public CustomerWithAddress() {super();
    }

    public CustomerWithAddress(Long id, String firstname, String lastname) {
        super(id, firstname, lastname);
    }
}
```

```
@Entity
@Table(name="customer")
public class CustomerWithAccounts extends AbstractCustomer{
    ....
    ....
}
```

6. Précautions à prendre pour ne pas trop détériorer les performances

6.1. No "lazy" on "one-to-one"

Dans le cas d'une relation `@OneToOne`, la suggestion `FetchType.LAZY` n'est pour l'instant pas bien prise en compte par JPA/Hibernate (pour des raisons techniques) aussi bien en mode "JPQL" qu'en mode "nativeQuery". On pourra éventuellement envisager des variantes de mapping (avec ou sans détails en mode "one-to-one") pour contourner cette limitation.

6.2. Ajout d'index sur certains champs importants :

NB: via `@Id` on précise la clef primaire qui est automatiquement indexée (rien à faire en plus de ce côté là).

On peut néanmoins demander à indexer (en base de données) des champs/colonnes supplémentaires de manière à optimiser les performances.

Exemple :

```
@Table(name="customer" ,
        indexes = @Index(name="idx_email" , columnList = "email" , unique = true))
```

En mode `"hibernate.hbm2ddl.auto"="create"`, les index sont automatiquement créés en base de données .

NB :

- on peut placer plusieurs index complémentaires
- le paramétrage `columnList` est essentiel par contre la spécification du nom de l'index est facultative et `unique` vaut par défaut `false` .

6.3. Ajout d'index sur relations fréquemment utilisées :

```
@Entity
public class Book {
    // ...

    @ManyToOne
    @JoinColumn(name = "author_id")
    @org.hibernate.annotations.Index(name = "author_id_index") // Adding an index
    private Author author;
```

```
// ...  
}
```

6.4. Batch fetching

@BatchSize on class level means that the specified batch size for the entity will be applied for all @*ToOne lazy associations with that entity.

```
@Entity  
@BatchSize(size = 10) // Set the batch size for batch fetching  
public class Order {  
    // ... in order to retrieve order-items 10 by 10 (lazy , @OneToMany / mappedBy ).  
}
```

6.5. Eventuelle gestion des caches via Spring

La gestion des caches de second niveau d'hibernate est assez délicate car "au cas par cas" en fonction de la version de hibernate et en fonction de la technologie tierce-partie utilisée (ex : Ehcache ou Caffeine).

En s'appuyant sur les fonctionnalités du framework hôte (ex : "Spring") , on peut simplifier la gestion des caches de second niveau .

Exemple :

```
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    @Cacheable("products")
    public Product getProductById(Long productId) {
        return databaseService.fetchProductById(productId);
    }

    @CacheEvict(value = "products", key = "#productId")
    public void updateProduct(Long productId, Product updatedProduct) {
        databaseService.updateProduct(productId, updatedProduct);
    }
}
```

6.6. Pagination si possible

Pageable pageable = PageRequest.of(pageNumber, pageSize, Sort.by(sortBy).descending());

à utiliser au sein de Spring-Data-Jpa sur de grandes quantités de données à parcourir ...

6.7. Transaction en mode read-only pour lectures seules

@Transactional(readOnly = true)

It's good practice to define the @Transactional(readOnly = true) annotation at the class level and only override it for read-write methods.

Other good idea : separated classes for query and updates (CQRS pattern) .

6.8. Eventuelles projections directes sur DTO/VO ou PartialEntities depuis requêtes JPQL

Vocabulaire :

- On parle généralement en terme d'**agrégation** lorsque l'on peut potentiellement récupérer des valeurs dans différentes entités pour les regrouper au sein d'un même objet.
- On parle généralement en terme de **projection** lorsque l'on ne récupère que les parties qui nous intéressent au sein d'un objet approprié.

Exemple1 :

```
SELECT new com.acme.example.CustomerDetails(c.id, c.status, o.count)
FROM Customer c JOIN c.orders o WHERE o.count > 100
```

Exemple2 :

```
public BasicClient getBasicClientWithoutAddress(long id) {
    final String jpqlQuery="SELECT new tp.appliJpa.entity.BasicClient(c.id,c.prenom,c.nom) "
        +" FROM ClientAvecAdresse c WHERE c.id=?1";

    /* ou bien
    final String jpqlQuery="SELECT new tp.appliJpa.entity.ClientAvecAdresse(c.id,c.prenom,c.nom)
    FROM ClientAvecAdresse c WHERE c.id=?1"; */
    return entityManager.createQuery(jpqlQuery, BasicClient.class)
        .setParameter(1, id).getSingleResult();
}
```

Avec des versions récentes de jpa/hibernate , ça peut également fonctionner avec des "records" .

6.9. NativeQuery avec @SqlResultSetMapping et constructeur

```
@SqlResultSetMapping(
    name = "BasicClientMapping",
    classes = @ConstructorResult( targetClass = BasicClient.class,
        columns = {
            @ColumnResult(name = "id", type = Long.class),
            @ColumnResult(name = "nom"),
            @ColumnResult(name = "prenom")}))
/* @MappedSuperclass */
public class BasicClient {
    /* @Id @GeneratedValue(strategy = GenerationType.IDENTITY) */
    private Long id;
    private String prenom;
    private String nom;
    ...
    public BasicClient(Long id, String prenom, String nom) {
        super(); this.id = id;  this.prenom = prenom;  this.nom = nom;
    }
}
```

}

```

public BasicClient getBasicClientWithoutAddress(long id) {
    final String nativeQuery="SELECT c.id,c.prenom,c.nom FROM Client c WHERE c.id=?1";
    return (BasicClient) entityManager.createNativeQuery(nativeQuery, "BasicClientMapping")
        .setParameter(1, id).getSingleResult();
}

```

7. Map et apports de JPA 2.0

7.1. Détails sur persistance d'une énumération

@Enumerated(*EnumType.STRING*)

ou bien

@Enumerated(*EnumType.ORDINAL*) //by default

à placer au dessus d'une propriété dont le type est une énumération java.

7.2. Tables d'associations (map)

Des relations 1-n (et n-n) peuvent soit être matérialisées comme des collections , soit être codées comme des tables d'associations ("map") .

Une table d'associations ("map") nécessite absolument une clef (à préciser) .

La propriété d'un objet de la "Map" qui sera associée à cette clef peut être renseignée via l'annotation **@MapKey**(name="...") .

Si l'indication "@MapKey" n'est pas présente , la clef primaire (id) est alors utilisée comme clef par défaut.

JPA 1 ne supportait que des "map" de sous objets "Entity" .

JPA 2 est maintenant capable de gérer en plus des "map" d'éléments simples ("basic").

Etant donné qu'un élément simple n'a pas obligatoirement d'identifiant (pk) , la clef peut être renseignée par l'annotation @MapKeyColumn .

Exemple (JPA 1 ou 2):

```

...
import java.util.Map;
import javax.persistence.*;

@Entity
@Table(name="Qcm")
public class Qcm {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private long idQcm;
}

```

```
private String titre;  
  
@OneToMany(mappedBy="qcm")  
@MapKey(name="numQuestion")  
private Map<Long,Question> questions;  
  
//+get/set  
}
```

7.3. Collection d'objets "embedded" (sans id):

Depuis la version 2 de JPA, il est possible de définir (via `@ElementCollection` et `@CollectionTable`) des collections de sous objets imbriqués (sans id). Cependant ceci ne fonctionne qu'à un seul niveau (pas de sous-sous objets (sans id) dans des sous-objets (sans id)).

En UML, ceci se modélise généralement comme une composition (agrégation forte / losange noir) avec une multiplicité "*" ou "1..*" .

La sémantique associée induit des cascades automatiques !!!

Exemple:

Qcm avec Entités "Question" ayant les caractéristiques suivantes:

- clef primaire composé de (idQcm , numQuestion)
- collection de sous-objets "Réponse" imbriqués (sans id)

Les Questions sont stockés dans une table Question.

Les sous objets réponses seront stockés dans une table "Reponse" ayant une clef primaire composée (idQcm, numQuestion, numReponse) . Cette composition est basée sur

- une sous partie (idQcm , numQuestion) correspondante à la clef primaire le l'entité parente
Cette sous partie "fk" de liaison avec le parent sera précisée via l'attribut "*joinColumns*" de l'annotation `@CollectionTable`
- une sous partie complémentaire quelconque (numReponse ou numLigne ou index)

@Embeddable

```
public class QuestionPK implements Serializable{

    @Column(name="idOfQcm")
    private long idQcm;

    @Column(name="numQuestion")
    private int num;

    ...
}
```

@Embeddable

```
public class Reponse {
    // NB: dans cette classe , on ne fait pas apparaître les propriétés (idQcm,numQuestion) pourtant
    // présentes dans la table "Reponse" , les mécanismes JPA s'en chargeront automatiquement !!!
    private char numReponse;
    private String texte;
    private boolean ok;
    // + get/set
}
```



```

@Entity
public class Question /* de qcm */{
    @Id
    private QuestionPK pk;

    @Column(insertable=false,updatable=false) // pour éviter conflit avec partie "num" ok pk
    private int numQuestion;

    @ManyToOne
    @JoinColumn(name="idOfQcm",insertable=false,updatable=false)
    private Qcm qcm;

    private String texte;

    @ElementCollection()
    @CollectionTable(name="Reponse", joinColumns={
        @JoinColumn(name="idOfQcm"),
        @JoinColumn(name="numOfQuestion")})
    private List<Reponse> reponses;
    // + get/set
}

```

Structure de la base de données:

```

CREATE TABLE Qcm(idQcm integer auto_increment PRIMARY KEY, titre VARCHAR(128));
INSERT INTO Qcm Values (1,'qcm1');  INSERT INTO Qcm Values (2,'qcm2');

CREATE TABLE Question ( idOfQcm integer,  numQuestion integer,  texte VARCHAR(64),
                        primary key (idOfQcm,numQuestion));
ALTER TABLE Question ADD CONSTRAINT question_avec_qcm_valide
    FOREIGN KEY (idOfQcm) REFERENCES Qcm(idQcm);
INSERT INTO Question Values (1,2,"question2-qcm1"); INSERT INTO Question Values (1,1,"question1-qcm1");
INSERT INTO Question Values (2,1,"question1-qcm2"); INSERT INTO Question Values (2,2,"question2-qcm2");

CREATE TABLE Reponse ( idOfQcm integer,  numOfQuestion integer,  numReponse CHAR,
                        texte VARCHAR(64), ok boolean, primary key (idOfQcm,numOfQuestion,numReponse));
ALTER TABLE Reponse ADD CONSTRAINT reponse_avec_question_valide
    FOREIGN KEY (idOfQcm,numOfQuestion) REFERENCES Question(idOfQcm,numQuestion);

INSERT INTO Reponse Values (1,1,'A',"repA_question1-qcm1",false);
INSERT INTO Reponse Values (1,1,'B',"repB_question1-qcm1",true);
INSERT INTO Reponse Values (1,2,'A',"repA_question2-qcm1",true);
...

```

7.4. Quelques autres apports de JPA 2

JPA 1 était simplement capable de gérer une relation 1-n (vers des entités) en la définissant (via `mappedBy="..."`) comme la relation inverse d'une relation n-1.

Autrement dit, la spécification de la colonne clef étrangère devait absolument être effectuée du côté "n-1" (via `@JoinColumn` à côté de `@ManyToOne`) et le lien devait donc être :

- soit unidirectionnel (que du côté n-1 / `@ManyToOne`)
- soit bidirectionnel

Depuis JPA 2, il est maintenant possible d'établir un lien 1-n (unidirectionnel sans lien inverse n-1) en spécifiant la clef étrangère du côté 1-n (`@JoinColumn` proche de `@OneToMany`).

Bien que possible, cette nouvelle fonctionnalité ne devrait idéalement être utilisée que pour établir un lien unidirectionnel 1-n entre une entité d'un package et des entités d'un autre package métier/fonctionnel.

Entre deux types d'entités d'un même package les liaisons bidirectionnelles ne posent aucun problème.

8. Api "Criteria" de JPA 2

8.1. Apports et inconvénients de l'API "Criteria"

L'api "Criteria" (qui est apparue avec la version 2 de JPA) correspond à une syntaxe alternative à JPQL pour exprimer et déclencher des requêtes retournant des entités persistantes "JPA" .

Contrairement à JPQL , l'api des criteria ne s'appuie plus sur un dérivé de SQL mais s'appuie sur **une série d'expression de critères de recherche qui sont entièrement construits via des méthodes "java"** .

L'api "Criteria" est plus "orientée objet" et plus "fortement typée" que JPQL.

Le **principal avantage** réside dans la **détection de certaines erreurs dès la phase de compilation**.

A l'inverse, la plupart des erreurs dans l'expression d'une requête JPQL ne sont en général détectées que lors de l'exécution des tests unitaires.

Du côté négatif , l'API "Criteria" de JPA n'est syntaxiquement pas très simple (voir franchement complexe) et il faut un certain temps pour s'y habituer (le code est moins "lisible" que JPQL au premier abord) .

8.2. Exemple basique (pour la syntaxe et le déclenchement)

```
public List<Devise> getAllDevise() {
    /*return this.entityManager.createQuery(
        "select d from Devise d",Devise.class).getResultList();*/

    //construction:
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Devise> criteriaQuery = cb.createQuery(Devise.class);

    //préparation (expression des critères de recherche)
    Root<Devise> deviseRoot = criteriaQuery.from(Devise.class);
    criteriaQuery.select(deviseRoot);

    //déclenchement (assez semblable) à celui d'une requête JPQL :
    return this.entityManager.createQuery(criteriaQuery).getResultList();
}
```

XXIII - Exemple avec prédicat pour partie ".where"

```
public Devise getDeviseByName(final String deviseName) {
    /* return this.entityManager.createQuery(
        "select d from Devise d where d.monnaie = :deviseName", Devise.class)
        .setParameter("deviseName", deviseName)
        .getSingleResult(); */
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Devise> criteriaQuery = cb.createQuery(Devise.class);

    Root<Devise> deviseRoot = criteriaQuery.from(Devise.class);

    Predicate pEqMonnaie = cb.equal(deviseRoot.get("monnaie") , deviseName);

    criteriaQuery.select(deviseRoot);
    criteriaQuery.where(pEqMonnaie);

    return this.entityManager.createQuery(criteriaQuery).getSingleResult();
}
```

1.1. Exemple relativement simple avec jointure:

```
public List<Compte> findComptesByNumCli(long numCli) {
    /*
    return this.getEntityManager().createQuery("SELECT cpt FROM
    Client cli JOIN cli.comptes cpt WHERE cli.numero= :numCli",Compte.class)
    .setParameter("numCli", numCli)
    .getResultList();
    */
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Compte> criteriaQuery = cb.createQuery(Compte.class);

    Root<Client> clientRoot = criteriaQuery.from(Client.class);

    Predicate pEqNumCli = cb.equal(clientRoot.get("numero") , numCli);
    //Predicate pEqNumCli = cb.equal(clientRoot.get(Client_.numero) , numCli);

    Join<Client, Compte> joinComptesOfClient = clientRoot.join("comptes");
    //Join<Client, Compte> joinComptesOfClient = clientRoot.join(Client_.comptes);

    criteriaQuery.select(joinComptesOfClient);
    criteriaQuery.where(pEqNumCli);

    return this.entityManager.createQuery(criteriaQuery).getResultList();
}
```

NB: L'exemple précédent n'est pas "fortement typé" et certaines erreurs ("types incompatibles , "propriété n'existant pas ou mal orthographiée") peuvent passer inaperçues lors de la phase de compilation.

1.2. Classes "modèles" pour des expressions rigoureuses des critères de recherche :

```
...
//Predicate pEqNumCli = cb.equal(clientRoot.get("numero") , numCli);
Predicate pEqNumCli = cb.equal(clientRoot.get(Client_.numero) , numCli);

//Join<Client, Compte> joinComptesOfClient = clientRoot.join("comptes");
Join<Client, Compte> joinComptesOfClient = clientRoot.join(Client_.comptes);
....
```

Au sein de cette écriture beaucoup plus rigoureuse ,

`Client_.numero` sera interprété comme une propriété `".numero"` qui existe bien sur une entité de type `"Compte"` et qui de plus est de type `long` .

Ces informations sont issue d'une classe java "modèle" dont le nom se termine (par convention) par un caractère "underscore" et dont le début coïncide avec la classe de l'entité persistante décrite.

(La classe modèle `"Client_"` décrit la structure de la classe `"Client"` préfixée par `@Entity`).

Une classe modèle doit absolument être placée dans le même package java que la classe d'entité persistante à décrire. On pourra cependant paramétrer maven et/ou l'IDE eclipse de façon à placer les classes "modèles" dans un autre répertoire car celles-ci sont générées automatiquement.

Exemple de classe "modèle"

```
package tp.myapp.minibank.core.entity;

import javax.annotation.Generated;
import javax.persistence.metamodel.ListAttribute;
import javax.persistence.metamodel.SingularAttribute;
import javax.persistence.metamodel.StaticMetamodel;

@Generated(value = "org.hibernate.jpamodelgen.JPAMetaModelEntityProcessor")
@StaticMetamodel(Client.class)
public abstract class Client_ {

    public static volatile SingularAttribute<Client, String> password;
    public static volatile SingularAttribute<Client, Long> numero;
    public static volatile ListAttribute<Client, Compte> comptes;
    public static volatile SingularAttribute<Client, Adresse> adresse;
    public static volatile SingularAttribute<Client, String> telephone;
    public static volatile SingularAttribute<Client, String> nom;
    public static volatile SingularAttribute<Client, String> prenom;
    public static volatile SingularAttribute<Client, String> email;
}
```

NB : La classe modèle `_Client` a été générée via un plugin maven `"hibernate-jpamodelgen"` (ou autre) qui peut se déclencher au moment de la compilation des classes ordinaires `"Client"` , ...

.../...

```

<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-jpamodelgen</artifactId>
  <version>4.3.1.Final</version>
  <scope>provided</scope>
</dependency>

...

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.3</version>
  <configuration>
    <source>1.7</source>
    <target>1.7</target>
    <compilerArguments>
      <processor>
        org.hibernate.jpamodelgen.JPAMetaModelEntityProcessor
      </processor>
    </compilerArguments>
  </configuration>
</plugin>

```

NB : il existe une autre façon de déclencher `org.hibernate.jpamodelgen` (plus paramétrable) .

Les classes modèles "XXX_" et les packages java sont par défaut générés dans le répertoire "**target/generated-sources/annotations**"

Il faut ensuite paramétrer le projet eclipse pour qu'il voit les classes générées :

- 1) activer le menu contextuel "**properties / compiler / annotation-processing**" depuis le projet eclipse "...-ejb"
- 2) **enable specific** / "**target/generated-sources/annotations**"
- 3) ajouter une utilisation de XXX_.yyy dans le code du DAO

XXIV - Cache niveau 2 Hibernate (JPA/SpringBoot)

1. Caches de niveau 1 et 2 (Jpa/Hibernate)

- Le **cache** de premier **niveau** est lié à la session **Hibernate** (et au **EntityManager** de JPA), et les objets qui sont **cachés** ne sont visibles que pour une seule transaction. Quand la session (ou **EntityManager**) est fermé(e), le **cache** n'a plus d'existence. Ce **cache** ne peut pas être désactivé.
- Le **cache** de **second niveau** est quant à lui lié à l'ensemble de la session factory d'**Hibernate** (ou **EntityManagerFactory** de JPA)

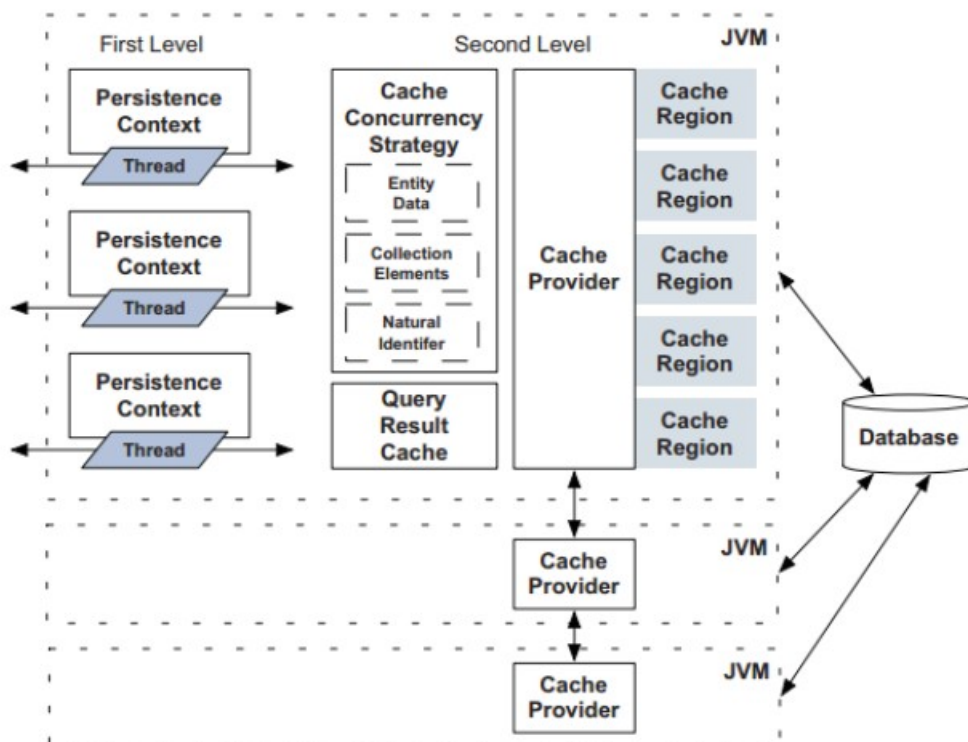
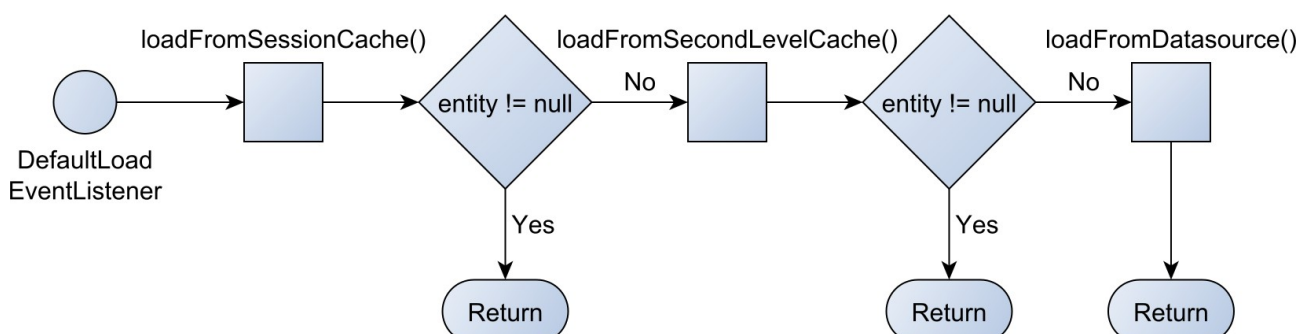


Figure 1: Hibernate's two levels cache (source: Manning)



2. Cache de niveau 2 Hibernate

2.1. Utilité éventuelle

“The cache is usually useful only for read mostly classes”

Data that changes rarely.

Non-critical data (for example, content-management data).

Data that's local to the application and not modified by other applications.

Bad candidates include:

Data that is updated often.

Financial data, where decisions must be based on the latest update.

Data that is shared with and/or written by other applications.

2.2. Principe

Le cache de second niveau de Hibernate (associé à l'ensemble d'un SessionFactory / EntityManagerFactory) n'est par défaut pas activé .

Une fois activé (via `hibernate.cache.use_second_level_cache=true`) , il faut le paramétrer .

2.3. Cache Concurrency Strategy

- **READ_ONLY**: Used only for entities that never change (exception is thrown if an attempt to update such an entity is made). It is very simple and performant. Very suitable for some static reference data that don't change
- **NONSTRICT_READ_WRITE**: Cache is updated after a transaction that changed the affected data has been committed. Thus, strong consistency is not guaranteed and there is a small time window in which stale data may be obtained from cache. *This kind of strategy is suitable for use cases that can tolerate eventual consistency*
- **READ_WRITE**: This strategy guarantees strong consistency which it achieves by using 'soft' locks: When a cached entity is updated, a soft lock is stored in the cache for that entity as well, which is released after the transaction is committed. All concurrent transactions that access soft-locked entries will fetch the corresponding data directly from database
- **TRANSACTIONAL**: Cache changes are done in distributed **XA** transactions. A change in a cached entity is either committed or rolled back in both database and cache in the same XA transaction

2.4. Providers de caches (à choisir)

Implementation	read-only	nonstrict-read-write	read-write	transactional
EH Cache	Yes	Yes	Yes	No
OS Cache	Yes	Yes	Yes	No
Swarm Cache	Yes	Yes	No	No
JBoss Cache	No	No	No	Yes

Exemple:

spring.jpa.hibernate.cache.region.factory_class=org.hibernate.cache.jcache.JCacheRegionFactory
 spring.jpa.**hibernate.javax.cache.provider**=org.ehcache.jsr107.*EhcacheCachingProvider*

2.5. Préciser quelles sont les classes qui pourront être utilisées par le cache de 2eme niveau

Sur certaines classes d'entités persistantes sulement :

```
import javax.persistence.Cacheable;
import javax.persistence.Entity;
import org.hibernate.annotations.CacheConcurrencyStrategy;

@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
@Entity
public class Ville {
  ....
}
```

Quelques liens pour approfondir si besoin le sujet :

<https://www.baeldung.com/hibernate-second-level-cache>

<https://www.boraji.com/hibernate-5-jcache-ehcache-3-configuration-example>

....

2.6. Cache de niveau 2 Hibernate avec ehcache

Dans **pom.xml**

```
...
    <dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-jcache</artifactId>
        <version>7.3.1.Final</version>
    </dependency>

    <!-- dependance complémentaire pour cache second niveau -->
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-jcache</artifactId>
        <version>7.3.1.Final</version>
    </dependency>

    <!-- Ehcache 3.x -->
    <dependency>
        <groupId>org.ehcache</groupId>
        <artifactId>ehcache</artifactId>
        <version>3.12.0</version>
        <classifier>jakarta</classifier>
        <exclusions>
            <exclusion>
                <groupId>org.glassfish.jaxb</groupId>
                <artifactId>jaxb-runtime</artifactId>
            </exclusion>
        </exclusions>
    </dependency>

    <dependency>
        <groupId>org.glassfish.jaxb</groupId>
        <artifactId>jaxb-runtime</artifactId>
        <version>[3,3.1)</version>
    </dependency>
```

Au sein de **META-INF/persistence.xml**

```
...
<property name="hibernate.cache.use_second_level_cache" value="true" />
<property name="hibernate.cache.region.factory_class" value="org.hibernate.cache.jcache.internal.JCacheRegionFactory" />
<property name="hibernate.javax.cache.uri" value="ehcache.xml" />
<property name="hibernate.cache.use_query_cache" value="true" />
<property name="hibernate.javax.cache.provider" value="org.ehcache.jsr107.EhcacheCachingProvider" />
...
```

Exemple de fichier **ehcache.xml** (dans src/main/resources) :

```
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://www.ehcache.org/v3"
  xmlns:jsr107="http://www.ehcache.org/v3/jsr107"
  xsi:schemaLocation="
    http://www.ehcache.org/v3 http://www.ehcache.org/schema/ehcache-core-3.12.xsd
    http://www.ehcache.org/v3/jsr107 http://www.ehcache.org/schema/ehcache-107-ext-3.0.xsd">

  <cache-template name="entities">
    <expiry>
      <ttl unit="seconds">5</ttl>
    </expiry>
    <resources>
      <heap unit="entries">1000</heap>
      <offheap unit="MB">10</offheap>
    </resources>
  </cache-template>

  <cache
    alias="tp.core.persistence.entity.StockEntity"
    uses-template="entities">
  </cache>
</config>
```

Test de l'usage du cache :

```
public class TestCache {
  private static XyzDAO xyzDAO;
  private static EntityManagerFactory emf;

  @BeforeAll
  public static void initDao() {
    MyJpaUtil.setHbm2ddlAuto(true);
    emf=MyJpaUtil.entityManagerFactory();
    xyzDAO=new XyzDaoJpa();
  }

  @Test
  public void testSaveAndFindWithCache() {
    String pk="id1";
    xyzDAO.saveNew(new StockEntity("Sxx", pk, "FR12367890", 3.03 ));
    long t1 = System.currentTimeMillis();
    Xyz sRelu = xyzDAO.findById(pk);
    long t2 = System.currentTimeMillis();
    Cache cache = emf.getCache(); //2nd Level cache (shared by all users/threads/sessions/entityManager)
    boolean wasCached = cache.contains(Xyz.class, pk);
    log.info("findById execution time (with or without cache)="+(t2-t1));
    //ex: 12ms with cache , 35ms without cache
    log.info("sRelu="+sRelu);
    log.info("wasCached="+wasCached);
    assertEquals("Sxx",sRelu.getName());
    assertTrue(wasCached);
  }
}
```

2.7. Cache de niveau 2 Hibernate (avec ehcache et SpringBoot)

Dans **pom.xml**

```
...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<!-- avec dependances indirectes : jakarta-persistence-api (jpa)
      et hibernate-core -->

<!-- dependance complémentaire pour cache second niveau -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-jcache</artifactId>
  <!-- <version>5.6.5.Final</version> -->
</dependency>

<!-- Ehcache 3.x -->
<dependency>
  <groupId>org.ehcache</groupId>
  <artifactId>ehcache</artifactId>
  <!-- <version>3.9.9</version> -->
</dependency>
```

Dans **application.properties** (ou bien META-INF/persistence.xml et)

```
....
#logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.cache=DEBUG
logging.level.org.ehcache=DEBUG
...
spring.jpa.hibernate.cache.use_second_level_cache=true
spring.jpa.hibernate.cache.region.factory_class=org.hibernate.cache.jcache.JCacheRegionFactory
spring.jpa.hibernate.javax.cache.provider=org.ehcache.jsr107.EhcacheCachingProvider
```

Exemple de logs :

```
2022-05-18 18:33:31.793 INFO 3052 --- [      main] org.ehcache.core.EhcacheManager      : Cache
'com.m2i.tp.appliSpringJpa.entity.Employe' created in EhcacheManager.

2022-05-18 18:33:31.803 DEBUG 3052 --- [      main] o.h.c.s.s.DomainDataRegionTemplate  : Generating entity cache access
[read-write] : NavigableRole[com.m2i.tp.appliSpringJpa.entity.Employe]

...

2022-05-18 18:33:34.674 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Caching data from load
[region='com.m2i.tp.appliSpringJpa.entity.Employe' (AccessType[read-write])] : key[com.m2i.tp.appliSpringJpa.entity.Employe#1]
-> value[CacheEntry(com.m2i.tp.appliSpringJpa.entity.Employe)]
```

```
2022-05-18 18:33:34.708 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Getting cached data from
region ['com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] by key
[com.m2i.tp.appliSpringJpa.entity.Employee#1]

2022-05-18 18:33:34.709 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Checking readability of read-
write cache item [timestamp='6770244053622784', version='null'] : txTimestamp='6770244053839872'

2022-05-18 18:33:34.709 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Cache hit : region =
'com.m2i.tp.appliSpringJpa.entity.Employee', key = 'com.m2i.tp.appliSpringJpa.entity.Employee#1'

...

2022-05-18 18:33:34.715 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Getting cached data from
region ['com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] by key
[com.m2i.tp.appliSpringJpa.entity.Employee#1]

2022-05-18 18:33:34.715 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Checking readability of read-
write cache item [timestamp='6770244053622784', version='null'] : txTimestamp='6770244053860353'

2022-05-18 18:33:34.715 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Cache hit : region =
'com.m2i.tp.appliSpringJpa.entity.Employee', key = 'com.m2i.tp.appliSpringJpa.entity.Employee#1'

2022-05-18 18:33:34.723 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Locking cache item
[region='com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] : 'com.m2i.tp.appliSpringJpa.entity.Employee#1'
(timeout=6770244299665408, version=null)

2022-05-18 18:33:34.738 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Getting cached data from
region ['com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] by key
[com.m2i.tp.appliSpringJpa.entity.Employee#1]

2022-05-18 18:33:34.739 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Checking readability of read-
write cache item [timestamp='6770244053954560', version='null'] : txTimestamp='6770244053966849'

2022-05-18 18:33:34.739 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Cache hit : region =
'com.m2i.tp.appliSpringJpa.entity.Employee', key = 'com.m2i.tp.appliSpringJpa.entity.Employee#1'

...

2022-05-18 18:33:34.741 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Getting cached data from
region ['com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] by key
[com.m2i.tp.appliSpringJpa.entity.Employee#1]

2022-05-18 18:33:34.742 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Checking readability of read-
write cache item [timestamp='6770244053954560', version='null'] : txTimestamp='6770244053979137'

2022-05-18 18:33:34.742 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Cache hit : region =
'com.m2i.tp.appliSpringJpa.entity.Employee', key = 'com.m2i.tp.appliSpringJpa.entity.Employee#1'

2022-05-18 18:33:34.747 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Locking cache item
[region='com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] : 'com.m2i.tp.appliSpringJpa.entity.Employee#1'
(timeout=6770244299763712, version=null)

2022-05-18 18:33:34.751 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Skipping #remove call in read-
write access to maintain SoftLock : com.m2i.tp.appliSpringJpa.entity.Employee#1

2022-05-18 18:33:34.754 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Unlocking cache item
[region='com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] : com.m2i.tp.appliSpringJpa.entity.Employee#1

2022-05-18 18:33:34.757 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Getting cached data from
region ['com.m2i.tp.appliSpringJpa.entity.Employee' (AccessType[read-write])] by key
[com.m2i.tp.appliSpringJpa.entity.Employee#1]

2022-05-18 18:33:34.757 DEBUG 3052 --- [      main] o.h.c.s.support.AbstractReadWriteAccess : Cache hit, but item is
unreadable/invalid : region = 'com.m2i.tp.appliSpringJpa.entity.Employee', key = 'com.m2i.tp.appliSpringJpa.entity.Employee#1'

2022-05-18 18:33:34.774 DEBUG 3052 --- [ionShutdownHook] o.e.c.i.s.DefaultStatisticsService : Cache removed
com.m2i.tp.appliSpringJpa.entity.Employee

2022-05-18 18:33:34.775 INFO 3052 --- [ionShutdownHook] org.ehcache.core.EhcacheManager : Cache
'com.m2i.tp.appliSpringJpa.entity.Employee' removed from EhcacheManager.

2022-05-18 18:33:34.778 DEBUG 3052 --- [ionShutdownHook] org.ehcache.core.spi.ServiceLocator : Stopping 18 Services...

2022-05-18 18:33:34.779 DEBUG 3052 --- [ionShutdownHook] o.e.c.i.s.DefaultStatisticsService : Stopping service

2022-05-18 18:33:34.780 DEBUG 3052 --- [ionShutdownHook] org.ehcache.core.spi.ServiceLocator : All Services successfully
stopped, 18 Services in 2ms

2022-05-18 18:33:34.780 DEBUG 3052 --- [ionShutdownHook] org.ehcache.core.EhcacheManager : Close successful.
```

XXV - Logs , Tests unitaires , JUnit, ...

1. TDD (Test Driven Development)

De manière itérative et incrémentale :

- 1) Eventuelle conception/modélisation UML → structure de données et interfaces java + début d'implémentation .
- 2) Ecriture des tests (et premiers lancements normalement en échec)
- 3) Ecriture progressive des implémentations
- 4) Relancement des tests unitaires qui doivent normalement finir par réussir

AAA/3A (Arrange, Act, Assert)

- **Arrange:** Set up the test environment.
- **Act:** Execute the code to test.
- **Assert:** Verify the results.

Notion de conception émergente (en développement agile)

Au lieu de faire une grande conception d'ensemble dès le début (ex : en UML) on effectue une conception de l'architecture logicielle au fur et à mesure du développement des différentes fonctionnalités .

Fonctionnalité_A → code A1 et architecture pour A

+ Fonctionnalité_B → code B1 + code A2 remanié au sein d'une architecture remaniée A+B

+ Fonctionnalité_C → code C1 + code A3/B2 remanié dans architecture remaniée A+B+C

etc/etc

2. Tests unitaires avec JUnit 5

2.1. Présentation de JUnit 3,4,5

JUnit est un *framework* simple permettant d'effectuer des **tests** (unitaires , de non régression, ...) au cours d'un développement java . [Projet Open source ---> <http://junit.sourceforge.net/> , <http://junit.org>] . *JUnit est intégré au sein des IDE Eclipse et IntelliJ.*

JUnit existe en versions 3 , 4 et 5 avec des différences significatives d'une version à l'autre .

2.2. Présentation des anciennes versions 3 et 4

La très ancienne version 3 de JUnit n'utilisait pas d'annotations . Tout était basé sur un héritage (TestCase) et sur des conventions de nom sur les méthodes (setUp() , tearDown() , testXy()) . Depuis environ 2006/2007 , cette historique version 3 est devenue petit à petit obsolète (utilisée seulement dans les anciens projets).

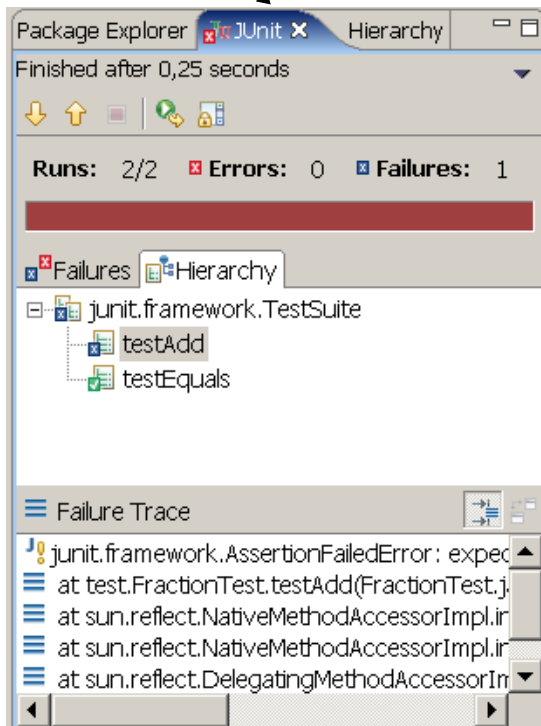
La version 4 a été massivement utilisée dans la majorité des projets java de 2007 à 2019 environ . La version 4 était basée sur le package org.junit (Test, Assert, ...) et les principales annotations

étaient `@Test` , `@Before` , `@After` , `@BeforeClass` , `@AfterClass` .

2.3. Lancement des tests unitaires

Lancement des tests unitaires

Depuis l'IDE eclipse:
Run as ... / JUnit test



TestSuite en JUnit3
et lancement après une sélection
de **package** en JUnit4 pour lancer
d'un coup toute une série de tests.

Comptabilisations :

Error(s) : exceptions java
non rattrapées.

Failure(s) : assertions
non vérifiées.

VERT si aucune erreur.

Depuis "maven" :

coder des classes nommées "**Test**Xy"
ou "XY**Test**" dans **src/test/java**
et lancement via **mvn test** ou autre.

2.4. Présentation de JUnit 5 ("jupiter")

La **version 5 de JUnit** a été entièrement restructurée et s'appuie sur certaines nouvelles fonctionnalités apportées par la **version 8 du langage java** (lambda expressions,) .

Principales différences entre JUnit4 et JUnit5 :

JUnit 4	JUnit 5
package org.junit	package org.junit.jupiter.api
Assert.assertTrue() , Assert.assertEquals(...)	Assertions .assertTrue() , Assertions .assertEquals(...)
@Before , @After	@BeforeEach , @AfterEach
@BeforeClass , @AfterClass (avec static)	@BeforeAll , @AfterAll (avec static)
@Test(timeout="20")	@Timeout() ou bien Assertions.assertTimeout(Duration.ofMillis(200), () -> {}) ;

@Ignore	@Disabled
@Rule	@ExtendWith
@Category	@Tag
...	...

2.5. Configuration maven pour JUnit 5 ("jupiter")

```

...
<properties>
  <junit.jupiter.version>5.4.2</junit.jupiter.version>
</properties>
<dependencies>
<dependency>
  <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter-api</artifactId>
  <version>${junit.jupiter.version}</version> <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter-engine</artifactId>
  <version>${junit.jupiter.version}</version> <scope>test</scope>
</dependency>
</dependencies>
<build> <plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.1.2</version> <!-- Need at least 2.22.0 to support JUnit 5 -->
  </plugin>
</plugins> </build> ...

```

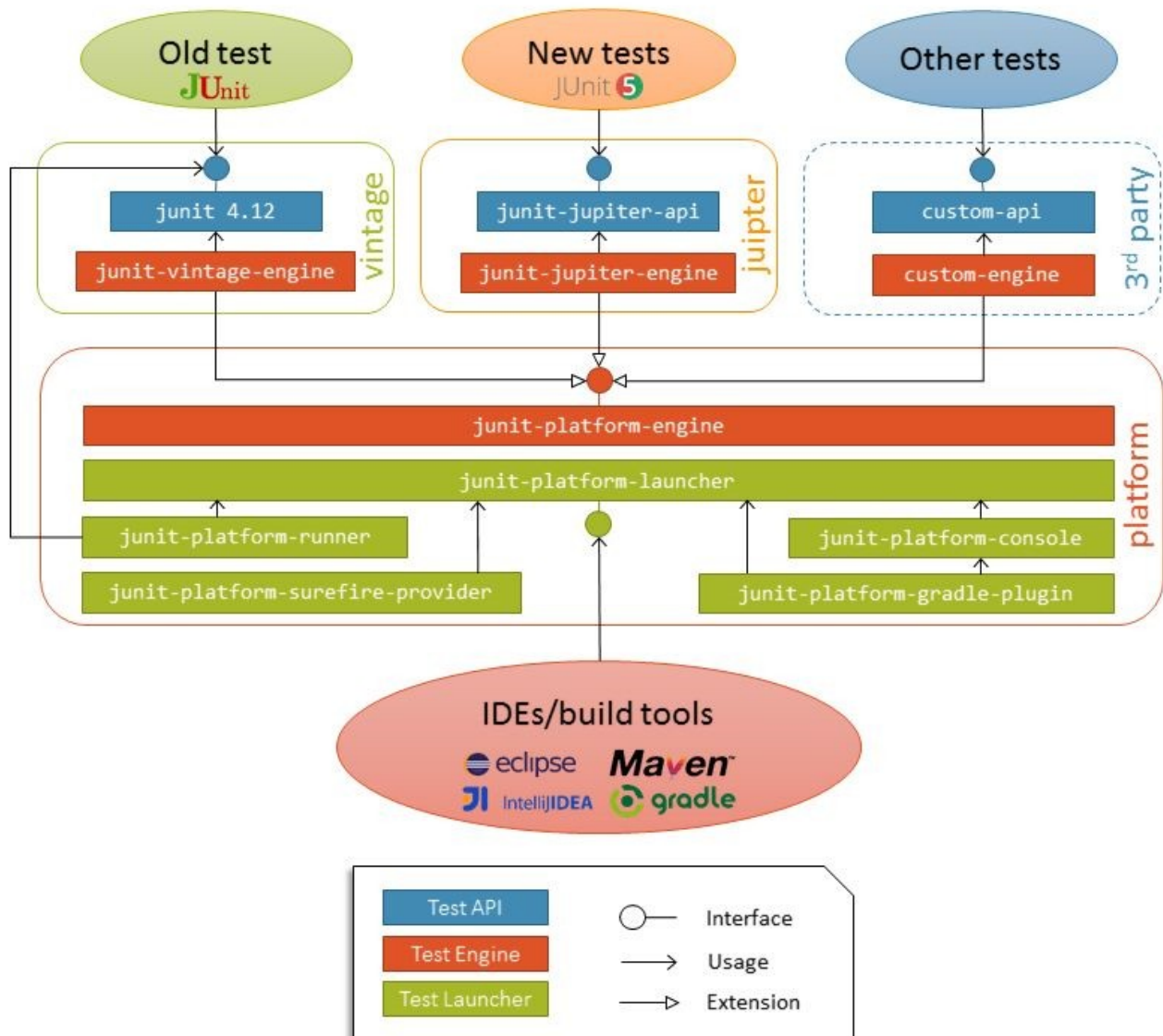
Eventuels compléments :

junit-jupiter-params *support des tests paramétrés avec JUnit Jupiter.*

junit-vintage-engine *pour interpréter et exécuter des anciens tests codés en JUnit 3 ou 4*

junit-platform-.... *pour intégration et lancement (console , maven , gradle ,)*

2.6. Structure de JUnit5



2.7. Test Unitaire avec JUnit 5 (@Test, @BeforeEach, @AfterEach)

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
...
public class TestCalculatrice {
    private static Logger logger = LoggerFactory.getLogger(TestCalculatrice.class);
    private CalculatriceEx calculatriceEx; //à tester

    @BeforeEach //@BeforeEach in junit5 , @Before in junit4
    public void initCalculatriceEx() {
        this.calculatriceEx = new CalculatriceEx();
    }

    @Test
    public void testSum() {
        calculatriceEx.pushVal(1.1);
        calculatriceEx.pushVal(2.2);
        calculatriceEx.pushVal(3.3);
        double somme = calculatriceEx.sum();
        logger.trace("somme(1.1, 2.2, 3.3)="+somme);
        //Assert.assert...() avec jUnit 4 et Assertions.assert...() avec junit5/jupiter
        Assertions.assertEquals( 6.6 , somme, 0.00000001);
    }

    @Test
    public void testAverage() {
        calculatriceEx.pushVal(1.1);
        calculatriceEx.pushVal(1.5);
        double moyenne = calculatriceEx.average();
        logger.trace("moyenne(1.1, 1.5)="+moyenne);
        Assertions.assertEquals( 1.3 , moyenne, 0.00000001);
    }
}
```

NB: Comme avec JUnit 4, par défaut JUnit 5 crée une nouvelle instance de la classe de test pour exécuter chaque méthode de test. (*@TestInstance(Lifecycle.PER_METHOD)* par défaut)

```
Assertions.assertEquals(expectedValue, effectiveValue , delta ) ;
Assertions.assertTrue(booleanExpression) ;   Assertions.notNull(...) ...
```

NB :

```
import static org.junit.jupiter.api.Assertions.assertTrue;
```

permet d'écrire **assertTrue** (res==5) au lieu de *Assertions.assertTrue*(res==5) et ça peut aider à masquer la différence de préfixe entre JUnit 4 et JUnit5 .

2.8. Test Unitaire JUnit5 avec static (@BeforeAll , @BeforeAll)

```
import org.junit.jupiter.api.BeforeAll;
...
public class TestCalculatrice {
    private static SimpleCalculatrice calculatrice; //à tester

    @BeforeAll //@BeforeAll in jUnit5 , @BeforeClass in jUnit4
    public static void initCalculatrice() {
        calculatrice = new SimpleCalculatrice();
    }

    @Test
    public void testAddition() {
        double resAdd=calculatrice.addition(5.5, 6.6);
        logger.trace("addition(5.5,6.6)="+resAdd);
        Assertions.assertTrue(resAdd>= (12.1 - 0.00000001) &&
                               resAdd <= (12.1 + 0.00000001));
    }

    @Test
    public void testMultiplication() {
        double resMult=calculatrice.multiplication(2.0, 3.3);
        logger.trace("multiplication(2.0, 3.3)="+resMult);
        Assertions.assertEquals( 6.6 , resMult, 0.00000001);
    }
}
```

Variante (sans static) :

```
import org.junit.jupiter.api.* ; ...
import org.junit.jupiter.api.TestInstance.Lifecycle;

@TestInstance(Lifecycle.PER_CLASS)
//a single reused and reinitialized instance of this test class for all test methods (ex:TestPerClassInstance@aeab9a1)
public class TestPerClassInstance {
    private static Logger logger = LoggerFactory.getLogger(TestPerClassInstance.class);

    @BeforeAll
    public void initBeforeAll() {
        logger.trace("TestPerMethodInstance.initBeforeAll() called (no static way) on this="+this );
    }

    @Test
    public void test1() {
        logger.trace("TestPerClassInstance.test1() called on this="+this );
    }
}
```

NB : En plaçant **@TestInstance(Lifecycle.PER_CLASS)** au dessus d'une classe de test Junit5 (imbriquée ou pas) , on a comme comportement le fait qu'une seule instance de la classe de test sera utilisée pour exécuter successivement toutes les méthodes de tests de la classe de test (contrairement au comportement par défaut PER_METHOD) .

2.9. Désactivation d'un test

@Ignore en V4 (org.junit.Ignore) , @Disabled en V5(org.junit.jupiter.api.Disabled)

WrongTests.java

```
package tp.converter;
import static org.junit.jupiter.api.Assertions.assertTrue;
import org.junit.jupiter.api.Disabled; import org.junit.jupiter.api.Test;

public class WrongTests {
    @Test
    @Disabled //test désactivé/ignoré
    public void testWithError() {
        int a=3; int b=0;
        assertTrue(a/b==0);//div by zero exception/error !!!
    }

    @Test
    @Disabled //test désactivé/ignoré
    public void testWithBadAssertion() {
        assertTrue(2+2==5); //2+2==4 !!!! not 5 !!!
    }

    @Test
    public void goodTrivialTest() { assertTrue(2+2==4);
    }
}
@Disabled("test temporairement désactivé")
sur classe ou méthode de test
```

2.10. Spécificités de JUnit5

```
@DisplayName("Ma classe de test JUnit5 que j'aime")
public class MyTest {
    @Test
    @DisplayName("Mon cas de test Xy qui va bien")
    void Testxy() { // ... }
}
```

NB: Les valeurs des @DisplayName seront affichées au sein des rapports d'exécution des tests

NB: contrairement à JUnit4, les **méthodes** préfixées par @BeforeEach , @Test , **n'ont plus absolument besoin d'être public** (protected ou bien *la visibilité implicite par défaut de niveau package suffit*) .

```
Iterable<Integer> attendu = new ArrayList<>(Arrays.asList(1, 2, 3));
Iterable<Integer> actuel = new ArrayList<>(Arrays.asList(1, 2)); //manque 3
Assertions.assertIterableEquals(attendu, actuel); //echec
```

vérifie que les collections ont mêmes tailles et mêmes valeurs.

@RepeatedTest(value = 3) est une variante de @Test permettant de lancer n fois un même test

2.11. Test avec timeout

```
@Test()
@Test(timeout = 200 , unit = TimeUnit.MILLISECONDS)
public void testWithTimeout() {
    try {
        Thread.sleep(100); //ok
        //Thread.sleep(300); //not_ok
    } catch (InterruptedException e) { System.err.println(e.getMessage()); }
}
```

ou bien

```
@Test()
public void testWithTimeoutV2() {
    Assertions.assertTimeout(Duration.ofMillis(200),
        () -> { try {
            Thread.sleep(100); //ok
            //Thread.sleep(300); //not_ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    });
}
```

2.12. Assertions avec lambdas :

AssertAll() avec lambdas :

```
Assertions.assertAll("wrong Dimension",
    () -> Assertions.assertTrue(elt.getWidth() == 400, "wrong width"),
    () -> Assertions.assertTrue(elt.getHeight() == 500, "wrong height"));
```

vérifie que chacune des lambdas est ok (sans exception) .

AssertThrows avec lambda :

```
@Test
void verifierException() {
    String valeur = "123a";
    NumberFormatException nfe =
        Assertions.assertThrows(NumberFormatException.class,
            () -> { Integer.valueOf(valeur); }
        );
}
```

vérifie qu'une exception est bien levée (suite à erreur volontaire) et du bon type .

AssertTimeout avec lambda :

```

@Test
void verifierTimeout() {
    Assertions.assertTimeout(Duration.ofMillis(200),
        () -> { * traitement potentiellement trop lent * return "...";
        });
}

```

2.13. Suppositions (pour exécuter ou pas un test selon le contexte) :

```
assumeTrue( System.getenv("OS").startsWith("Windows") );
```

//la suite du test ne sera exécutée que si os courant est Windows

//si le test n'est pas exécuté --> même comportement que si test vide --> ok/réussi/vert

//variante avec lambda exécutée qui si supposition ok :

```

assumingThat(System.getenv("OS").startsWith("Windows"), () -> {
    assertTrue(new File("C:/Windows").exists(), "Repertoire Windows inexistant");
});

```

2.14. Tests imbriqués de JUnit5

```

public class MyTest {

    @BeforeEach
    void mainInit() {
        System.out.println("BeforeEach / first level");
    }

    @Nested
    class MonTestImbrique {
        @BeforeEach
        void subInit() {
            System.out.println("BeforeEach imbrique");
            valeur = 5;
        }

        @Test
        void simpleTestImbrique() {
            System.out.println("SimpleTest imbrique valeur=" + valeur);
            Assertions.assertEquals(5, valeur);
        }
    }
}

```

```
}
}
```

NB : On peut éventuellement utiliser le mode `PER_CLASS` au niveau de la classe comportant des sous classes annotées avec `@Nested` pour contrôler le nombre d'appels effectués (au niveau des méthodes préfixées par `@BeforeAll` et `@AfterAll`).

2.15. Tests paramétrés via source/série de valeurs

Après avoir ajouté la dépendance nécessaire *junit-jupiter-params* , on peut écrire des méthodes de tests avec un paramètre qui sera alimenté avec une source/série de valeurs .

Un test paramétré sera ainsi lancé plusieurs fois (avec des arguments différents pour obtenir une certaine variété)

@ParameterizedTest

```
@ValueSource(ints = { 1, 2, 3 })
void testParametreAvecValueSource(int valeur) {
    assertEquals(valeur + valeur, valeur * 2);
}
```

`@ValueSource(ints = { 1, 2, 3 })` ou `@ValueSource(strings = { "un", "deux" })` ou ...

import java.util.stream.Stream;

```
@ParameterizedTest
@MethodSource("fournirDonneesParametres")
void testTraiterSelonMethodSource(String element) {
    assertTrue(element.startsWith("elt"));
}

static Stream<String> fournirDonneesParametres() {
    return Stream.of("elt1", "elt2");
}
```

```

public enum Direction { NORD , SUD, EST , OUEST }
@ParameterizedTest
    @EnumSource(TestParameterizedWithSources.Direction.class)
    void testTraiterSelonEnumValues(Direction direction) {
        assertTrue(direction.toString().length()>=3);
        logger.trace("testTraiterSelonEnumValues direction="+direction);
    }

```

```

@ParameterizedTest
    @CsvSource(delimiter = ';',
        value= { "1;1;un" ,
                  "2;4;quatre" ,
                  "3;9;neuf"
                }
    )
    void testTraiterSelonCsvSource(int n,int expectedCarre,String comment) {
        assertTrue(n*n == expectedCarre);
        logger.trace("testTraiterSelonCsvSource n="+n
            + " expectedCarre="+expectedCarre
            + " comment="+comment);
    }

```

```

@ParameterizedTest
    @CsvFileSource( resources= "/carres.csv", delimiter = ';', numLinesToSkip = 1)
    //files= "carres.csv" found at eclipse project root
    //resources= "/carres.csv" found in src/test/resources
    void testTraiterSelonCsvFileSource(int n,int expectedCarre,String comment) {
        assertTrue(n*n == expectedCarre);
        logger.trace("testTraiterSelonCsvFileSource n="+n
            + " expectedCarre="+expectedCarre
            + " comment="+comment);
    }

```

src/test/resources/carres.csv

```

n;carre;comment
1;1;un
2;4;quatre
3;9;neuf

```


2.16. Tests dynamiques / @TestFactory

```
...
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.ArrayList;
import java.util.Collection;
import org.junit.jupiter.api.DynamicTest;
import org.junit.jupiter.api.TestFactory;

class MonTestSimple {

    @TestFactory
    Collection<DynamicTest> dynamicTestsWithCollection() {
        Collection<DynamicTest> myDynamicTests = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
            int val = i;
            myDynamicTests.add(DynamicTest.dynamicTest("Ajout " + val + "+" + val,
                () -> assertEquals(val * 2, val + val));
        }
        return myDynamicTests;
    }
}
```

--> à priori intéressant que si couplé avec une introspection dynamique (framework de tests).

2.17. Suite de Tests (JUnit 5)

Durant la période de transition JUnit4 vers JUnit5 , une suite de Tests pouvait être codée de cette manière :

```
...
import org.junit.platform.runner.JUnitPlatform;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.runner.RunWith;

//NB: JUnitPlatform nécessite org.junit.platform:junit-platform-runner:1.9.3 dans pom.xml
// @RunWith() nécessite org.junit.vintage:junit-vintage-engine:5.9.3 dans pom.xml

@RunWith(JUnitPlatform.class)
@SelectClasses{
    TestCalculsSimples.class,
    TestEmptySerie.class,
    TestSerie.class
}
public class CalculsSuiteV1 {
}
```

Au sein d'un projet récent (prise en charge au sein d'un IDE récent) , une suite de tests moderne junit5 sera plutôt codée de cette manière :

```
...
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

//@RunWith(JUnitPlatform.class) will be deprecated , use @Suite instead
//but @Suite not already supported by some old IDE (ex: ok with eclipse 2023-03)

@Suite
@SelectClasses({
    TestCalculsSimples.class,
    TestEmptySerie.class,
    TestSerie.class
})
public class CalculsSuite {
}
```

```
@Suite
@SelectPackages({ "tp.calculs", "tp.converter" })
public class AllTestsSuite {
}
```

2.18. Suite de Tests avec Tags

@Tag("nomDeTag") de JUnit5 peut être placé sur une classe ou bien une méthode de test

Ces noms de tags (préalablement appelés catégories en Junit4) permettront d'effectuer ultérieurement des filtrages sur les tests à lancer ou pas .

Il est éventuellement possible de placer plusieurs tags complémentaires au dessus d'un même test (ex : **@Tag("prod") @Tag("dev")**) .

Exemple :

```
...
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;

public class TestWithCategoryTag {
    private static Logger logger = LoggerFactory.getLogger(TestWithCategoryTag.class);

    @Test
    @Tag("SlowTests")
    public void testA() {
        logger.trace("slow testA");
    }
}
```

```

@Test
@Tag("FastTests")
public void testB() {
    logger.trace("fast testB");
}

@Test
@Tag("FastTests") @Tag("ImportantTests")
public void testC() {
    logger.trace("important fast testC");
}

@Test
@Tag("SlowTests") @Tag("SecondaryTests")
public void testD() {
    logger.trace("secondary slow testD");
}
}

```

```

...
import org.junit.platform.suite.api.ExcludeTags;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
//@IncludeTags({"FastTests"})
@ExcludeTags({"SlowTests", "SecondaryTests"})
@SelectClasses({
    TestWithCategoryTag.class ,
    TestWithAssumptions.class
})
public class FastTestsSuite {
}

```

NB.:

- par défaut , @SelectPackages(...) sélectionne tous les sous-packages également .
- On peut éventuellement ajouter @IncludePackages(...) ou bien @ExcludePackages(...) pour filtrer les sous-packages à sélectionner.
- On peut également ajouter @IncludeClassNamePatterns({ "^.*Simple\$" }) pour filtrer les noms des classes de Tests à lancer .
- Possibilité de filter les category/tags au sein de la config maven:

```

<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <properties>
      <includeTags>FastTests</includeTags>
    </properties>
  </configuration>
</plugin>

```

2.19. Extensions JUnit5

L'ancien concept de règle (@Rule) JUnit4 a été remplacé par la notion d'extension (plus clair) en JUnit5 .

Exemple d'extension personnalisée :

```
package tp.extensions;
import org.junit.jupiter.api.extension.AfterEachCallback;
import org.junit.jupiter.api.extension.BeforeEachCallback;
import org.junit.jupiter.api.extension.ExtensionContext;
import org.junit.jupiter.api.extension.ExtensionContext.Namespace;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;

//NB1: just one instance of this class
//(don't use attributes but context.getStore(Namespace.GLOBAL or SPECIFIC_NAMESPACE))
//NB2: JUnit5 Extensions seems to be rather slow (may be aop/weaver) ...
public class MyPerfCustomExtension implements AfterEachCallback, BeforeEachCallback {
    private static Logger logger = LoggerFactory.getLogger(MyPerfCustomExtension.class);
    private static final Namespace NAMESPACE =
        Namespace.create("tp", "extensions", "MyPerfCustomExtension");

    @Override
    public void beforeEach(ExtensionContext context) throws Exception {
        long startTime = System.nanoTime();
        logger.info("MyPerfCustomExtension, before execution of "
            + context.getDisplayName()
            + " at startTime=" + startTime );
        Namespace methodNamespace =
            NAMESPACE.append(context.getDisplayName());
        context.getStore(methodNamespace).put("startTime", startTime);
    }

    @Override
    public void afterEach(ExtensionContext context) throws Exception {
        long endTime = System.nanoTime();
        Namespace methodNamespace =
            NAMESPACE.append(context.getDisplayName());
        long startTime = (Long) context.getStore(methodNamespace).get("startTime");
        logger.info("MyPerfCustomExtension, after execution of "
            + context.getDisplayName()
            + " execDuration (ms) without good precision = "
            + ((double)(endTime - startTime)/1000000)
            + " startTime=" + startTime + " endTime=" + endTime );
    }
}
```

Utilisation d'une extension JUnit5 :

```

package tp.converter.other;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;
import tp.extensions.MyPerfCustomExtension;

@ExtendWith(MyPerfCustomExtension.class)
public class TestWithExtension {
    private static Logger logger = LoggerFactory.getLogger(TestWithExtension.class);

    @Test
    public void fastTest() {
        try {
            Thread.sleep(20); //ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    }

    @Test
    public void slowTest() {
        try {
            Thread.sleep(60); //ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    }
}

```

Résultats :

MyPerfCustomExtension, before execution of slowTest() at startTime=510085534119300

*MyPerfCustomExtension, after execution of slowTest() execDuration (ms) without good precision = **92.514**
 startTime=510085534119300 endTime=510085626633300*

MyPerfCustomExtension, before execution of fastTest() at startTime=510085636236500

*MyPerfCustomExtension, after execution of fastTest() execDuration (ms) without good precision = **35.5131**
 startTime=510085636236500 endTime=510085671749600*

Possibilité d'utiliser conjointement plusieurs extensions complémentaires :

```

@ExtendWith({ MyPerfCustomExtension.class , MyBasicLogExtension.class })
public class TestWithExtension {
    ...
}

```

Principales extensions prédéfinies de JUnit5

Extensions	Caractéristiques
TempDirectory et @TempDir	Destruction automatique (en fin de tests) des fichiers et répertoires temporaires créés durant le test. NB : @TempDir peut être placé sur un attribut (static ou pas) d'une classe ou bien peut être placé sur un paramètre d'entrée d'une méthode de test
...	

Exemples :

```
import org.junit.jupiter.api.io.TempDir;
...
@TempDir
File tempDir;

@Test
void testWithTempDir() throws IOException {
    assertTrue(this.tempDir.isDirectory(), "Should be a directory ");

    File f1 = new File(tempDir, "f1.txt");
    List<String> lines = Arrays.asList("aaa", "bbb", "ccc");

    Files.write(f1.toPath(), lines);

    assertAll(
        () -> assertTrue(Files.exists(f1.toPath()), "File should exist"),
        () -> assertLinesMatch(lines, Files.readAllLines(f1.toPath())));
}
```

2.20. Test(s) dans interface (contrat d'interface)Code à tester :

```
public interface IDecorate {
    //returned String must contains inputString
    //and may be decorated by any prefix or any suffix or both
    String decorate(String inputString);
}
```

```
public class DecorateWithPrefix implements IDecorate{
    @Override
```

```

    public String decorate(String inputString) {
        return ">>>" + inputString;
    }
}
//-----
public class DecorateWithSuffix implements IDecorate {
    @Override
    public String decorate(String inputString) {
        return inputString + "<<<";
    }
}

```

```

package tp.other;
import static org.assertj.core.api.Assertions.assertThat;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;

public interface IDecorateContract {

    static final Logger logger = LoggerFactory.getLogger(IDecorateContract.class);
    IDecorate create();

    @Test
    default void postcondition() {
        String parameter = "abc"; //or random string
        String res = create().decorate(parameter);
        logger.trace("res="+res);
        assertThat(res).isNotBlank()
            .contains(parameter);
    }
}

```

```

...
public class TestDecorateWithSuffix implements IDecorateContract {
    @Override
    public IDecorate create() {
        return new DecorateWithSuffix();
    }
    //AVEC DECLenchement AUTOMATIQUE
    //du @Test postcondition() defini dans l'interface IDecorateContract
}

```

... TRACE IDecorateContract:20 - res=>>>abc

XXVI - Profiling JVM, gestion mémoire , ...

1. JMX (Java Management Extensions)

JMX est une architecture et une api se voulant standard et visant à **structurer l'administration de diverses ressources** (périphériques, modules logiciels, ...).

1.1. Architecture JMX

JMX est structuré en **3 niveaux**:

1. **Instrumentation** (ressources à administrer : MBean = Managed Bean)
2. **Agent** (intermédiaire permettant de contrôler , fédérer et exposer d'un point de vue administratif un ensemble de ressources)
3. **Service** (console d'administration, ... et éventuels services annexes, ...)

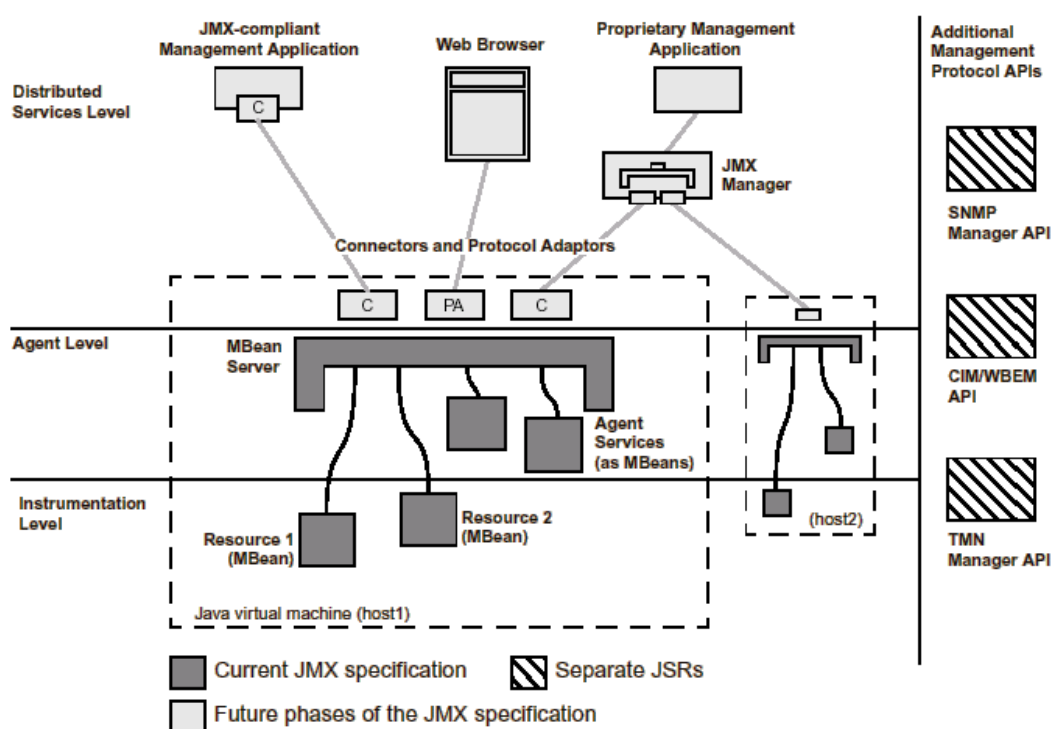


FIGURE 1-1 Relationship Between the Components of the JMX Architecture

1.2. Composants Mbean de JMX

Un composant de type "MBean" doit pouvoir être géré comme un composant générique et doit donc externaliser les points suivants:

- Attributs accessibles via des noms (propriétés).
- Opérations (fonctions) que l'on peut invoquer.
- Notifications (événements) qui peuvent être émis.
- Constructeur de la classe du composant.

La norme **JMX** distingue 4 grands types de MBeans:

<i>type de MBean</i>	<i>caractéristiques</i>
Standard MBeans	basé sur les conventions "JavaBean" et l'introspection de java
Dynamic MBeans	avec mécanismes dynamiques pour s'exposer (via l'interface javax.management.DynamicMBean) qu'au moment de l'exécution.
Open MBeans	extension du modèle "dynamic" pour garantir une interopérabilité.
Models MBeans	extension du modèle "dynamic" avec comportement par défaut.

La gestion des événements se fait classiquement en java via les éléments suivants (du package *javax.management*):

- **Notification** (événements)
- **NotificationListener** (interface pour réception des événements)
- **NotificationBroadcaster** (interface pour envoi d'événements)
- **MBeanServer.addNotificationListener(...)** , ...

1.3. Lien entre JMX et la métrologie

Beaucoup de serveurs d'applications JAVA EE utilisent en interne l'architecture JMX pour :

- contrôler les éléments internes du serveur (arrêts/démarrages , réglages)
- récupérer des mesures internes (utilisation des ressources , compteurs, ...)

1.4. Exemple élémentaire de Mbean JMX

<pre>public interface BasicMBean { int getNbProcessors(); long getTotalUsedMemory(); public int getCounter(); public void setCounter(int c); void resetMyCounter(); }</pre>	<pre>public class Basic implements BasicMBean { private int mycounter=0; public int getCounter() { return mycounter; } public void setCounter(int c) { mycounter=c; } public void resetMyCounter() { mycounter=0; } public int getNbProcessors() { return Runtime.getRuntime().availableProcessors(); } public long getTotalUsedMemory() { return Runtime.getRuntime().totalMemory(); } }</pre>
---	---

.../...

```

package tp;

import javax.management.*;
import java.lang.management.ManagementFactory;

public class BasicJmxApp {

    public static void main(String[] args) {
        try {
            //NB: valid object name = domain:keys where keys is xx=valxxx,yy=valyy
            ObjectName myBasicMetricsMBeanObjectName =
                new ObjectName("tp.basic_jmx:type=basic,name=my_basic_metrics");
            MBeanServer server = ManagementFactory.getPlatformMBeanServer();
            server.registerMBean(new Basic(), myBasicMetricsMBeanObjectName);
        } catch (MalformedObjectNameException | InstanceAlreadyExistsException |
            MBeanRegistrationException | NotCompliantMBeanException e) {
            e.printStackTrace();
            /* par exemple si méthode d'accès aux mesures ne commence pas par get alors
            javax.management.NotCompliantMBeanException:
            MBean class tp.MyBasicMetricsMBeanImpl does not implement DynamicMBean,
            and neither follows the Standard MBean conventions */
        }
        System.out.println("BasicJmxApp");
        for(int i=10;i>=0;i--){ System.out.println(i);
            try {
                Thread.sleep(10000);//10s
            } catch (InterruptedException e) { throw new RuntimeException(e); }
        }
    }
}

```

BasicJmxApp

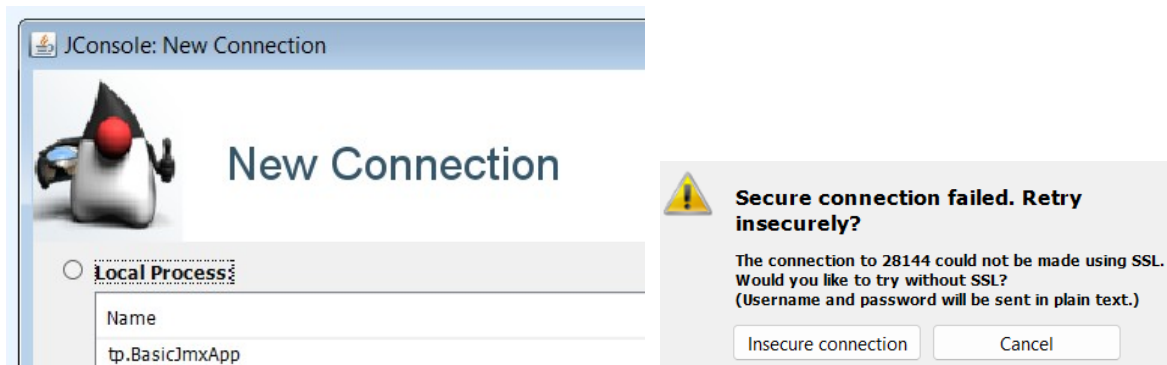
10

9

8

...

jconsole



.../...

Java Monitoring & Management Console - pid: 44168 tp.BasicJmxApp

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

JMImplementation
com.sun.management
java.lang
java.nio
java.util.logging
jdk.management
jdk.management.jfr
tp.basic_jmx
 basic
 my_basic_metrics
 Attributes
 Counter
 TotalUsedMemory
 NbProcessors
 Operations
 resetMyCounter

Attribute value

Name	Value
TotalUsedMemory	255852544

Refresh

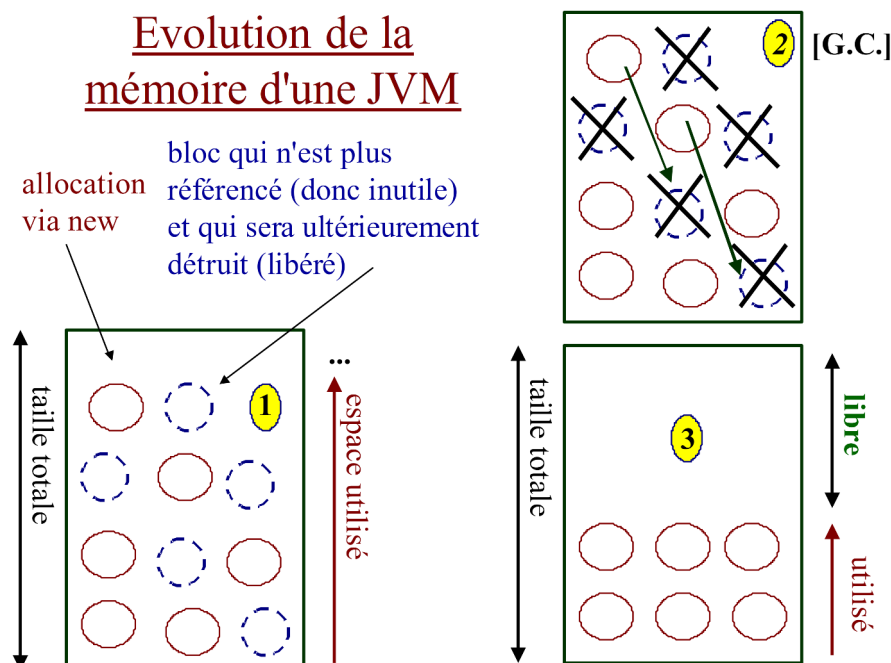
MBeanAttributeInfo

Name	Value
Attribute:	
Name	TotalUsedMemory
Description	Attribute exposed for management
Readable	true
Writable	false
Is	false
Type	long

2. Présentation de HotSpot JVM

La plate forme **Java** intègre en standard (dans JavaSE) une **JVM classique et relativement bien optimisée** appelée "**HotSpot**".

3. Vue extrêmement simplifiée du "Garbage Collector"



3.1. Evolution du garbage collector de java

1) toutes premières versions de java :

serial_GC (même pas optimisé pour le multiThread)

2) **Parallel/ParallelOld GC** (au alentour de java 4,5,6,7)

avec premières optimisations (certaines exécutions en //)

3) **Concurrent Mark Sweep (CMS) GC**

aux alentours de java 8 , devenu obsolète en java 13 , 14 .

L'implémentation CMS (Concurrent Mark Sweep) utilise plusieurs threads simultanés pour analyser le Heap (« mark ») et pour chercher les objets inutilisés à nettoyer (« sweep »). Conçu pour les applications qui préfèrent des pauses plus courtes et peuvent se permettre de partager les ressources du processeur pour le nettoyage pendant que l'application est en cours d'exécution. Les applications qui utilisent ce type de GC réagissent en moyenne plus lentement, mais n'arrêtent pas de le faire pendant la phase de nettoyage.

4) **Garbage First (G1) GC**

le standard d'aujourd'hui

G1 (Garbage First) est conçu pour les applications qui s'exécutent sur des machines multiprocesseurs avec un grand espace mémoire. Une fois le Heap divisé en sections de tailles similaires, chacune avec une plage contiguë de mémoire virtuelle, il exécute la phase de « mark » simultanément et, dès que terminé, sait quelles sections sont pour la plupart vides. G1 se concentre d'abord sur ces zones, produisant généralement une quantité importante d'espace libre. Il remplace le CMS car il est plus efficace en termes de performances.

5) en mode « expérimentations » : **ZGC** , **EpsilonGC**, **Shenandoah** ,...

=====

Selon la version de la jvm , on peut choisir entre plusieurs implémentations du Garbage Collector (avec G1 par défaut en général) :

- Serial Garbage Collector **-XX:+UseSerialGC**
- Parallel Garbage Collector **-XX:+UseParallelGC**
- CMS Garbage Collector **-XX:+UseConcMarkSweepGC**
- G1 Garbage Collector **-XX:+UseG1GC**

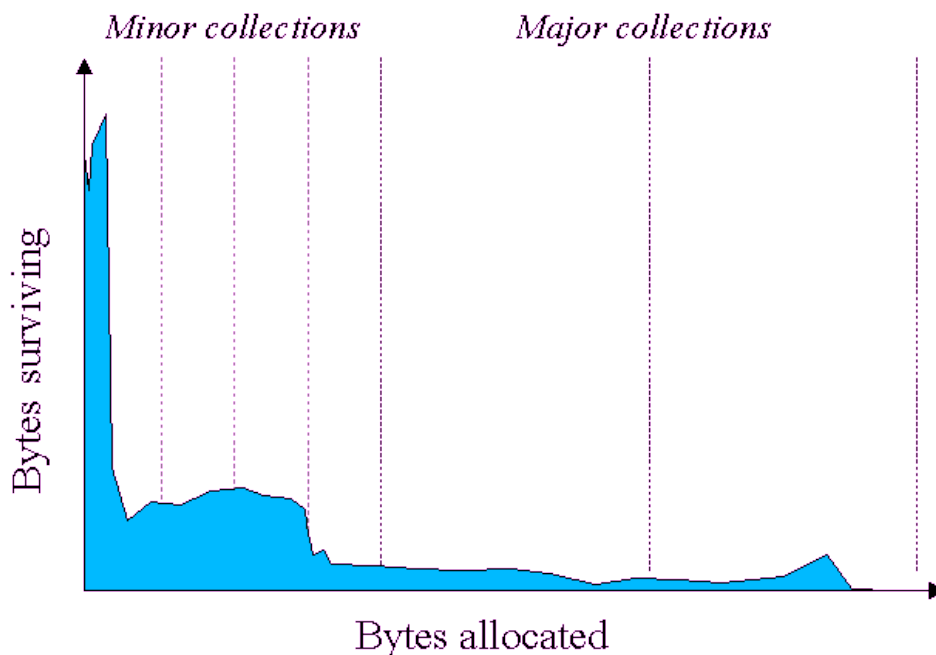
4. Générations

La machine virtuelle Hospot distingue des générations de mémoires.

Certains objet de java (ex: itérateur) ne sont généralement utilisés qu'au sein d'une boucle et peuvent être très rapidement détruits .

A l'inverse certains autres objets ont une durée de vie beaucoup plus longue au sein de l'application .

Le diagramme ci-dessous montre un exemple (assez classique) de répartition de la mémoire allouée du programme en fonction de la durée de vie des objets :



Le pic (sur la partie gauche) du diagramme ci-dessus montre que beaucoup d'objets java ne sont utilisés (référéncés) que très brièvement et peuvent donc être rapidement détruits.

- ⇒ En rangeant , les "jeunes" et "vieux" objets dans des zone mémoires séparées , la JVM HotSpot est ainsi capable de plus rapidement scanner ce qui doit être détruit .

Une **génération** est une **zone mémoire** qui gère des **collections d'objets de même age**.

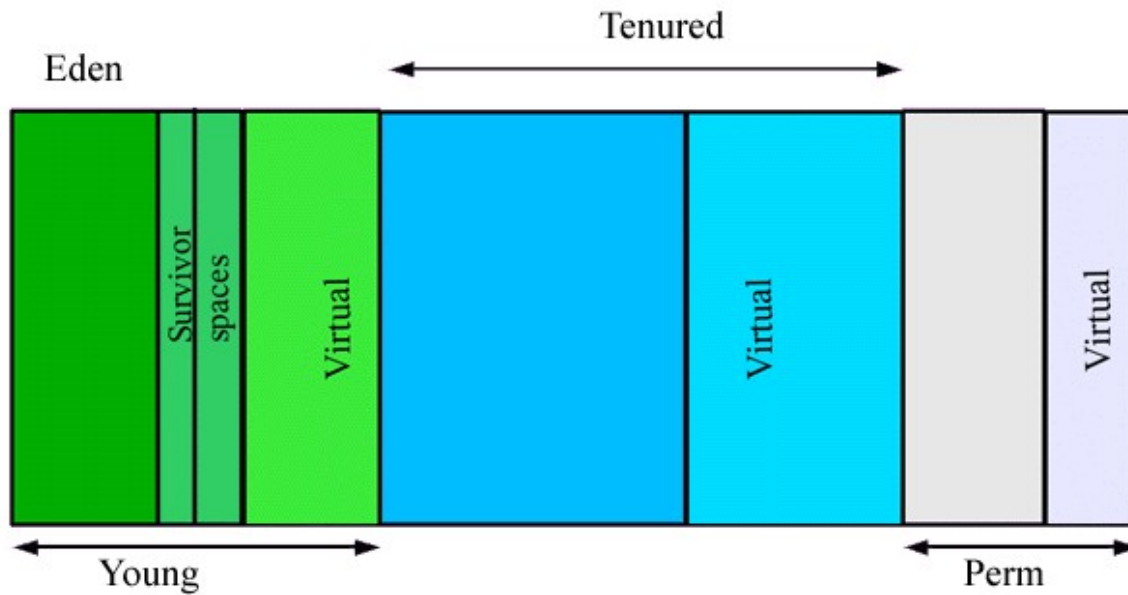
Un ramassage mineur (**partial GC**) à lieu sur chaque jeune génération lorsque celle-ci est pleine.

Les objets "survivant" sont alors rangés dans la génération suivante.

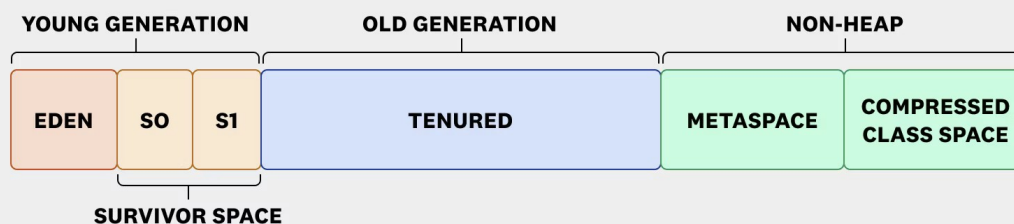
Un ramassage majeur (**full GC**) qui est généralement beaucoup plus long à lieu lorsqu'une ancienne génération de mémoire arrive à saturation.

De façon à ne pas parler en terme de "mortalité infantile des objets" (assez négatif) , le vocabulaire utilisé par HotSpot est le suivant:

Eden = **Génération la plus jeune** . Tous les objets "java" naissent dans l'Eden , qu'ils soient voués à une courte ou longue existence, **ils iront tous au paradis** ,



Generational heap layout

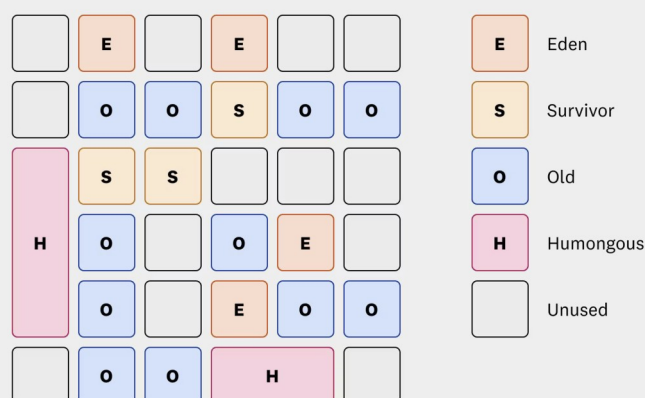


NB: Les **zones virtuelles** correspondent à de la **mémoire virtuellement réservée** (à tel ou tel usage) **mais encore physiquement allouée** (sous forme de pages mémoires "mapées" en RAM ou en SWAP) .

3 Grandes générations:

Young (Eden +Survivor)	Jeunes (<i>Partial GC</i>) , Ramassage rapide de type " copying " ==> les objets survivants sont recopiés ailleurs (dans un espace libre réservé à cet effet)
Tenured	Anciens (<i>Full GC</i>) , Ramassage plus lent de type " Mark-compact " ==> re-compactage de la mémoire sur place
Perm	Permanent ==> pour la mémoire utilisée par la JVM elle même

4.1. Spécificités du G1

G1 garbage collector region layout

Le ramasse-miettes G1 (Garbage-First) est un ramasse-miettes générationnel sophistiqué conçu pour améliorer la gestion du tas et offrir des temps de pause prévisibles. Contrairement aux collecteurs traditionnels qui divisent la mémoire en grandes régions contiguës, G1 partitionne le tas Java en plusieurs régions de taille fixe, chacune pouvant appartenir aux espaces Eden, Survivor, Old ou Humongous. Cette conception basée sur les régions offre une plus grande flexibilité dans la gestion de la mémoire et permet une collecte incrémentielle de parties du tas, en particulier de l'ancienne génération

5. Indicateurs de performances et mesures

5.1. indicateurs de performances

throughput	rythme de traitement , temps passé à effectuer des traitements applicatifs effectifs (le serveur répond , renvoie des réponses aux clients)
pause	temps ou l'application semble ne pas répondre (phases de GC, ...)
footprint	poids en mémoire d'un processus (pages de code, de données , caches, ...)
promptness	durée qui sépare la fin d'un objet et la disponibilité de la mémoire libérée.

⇒ Les paramètres d'optimisation sont souvent antagonistes . On recherche souvent un bon compromis .

5.2. mesures

- La charge mémoire (**footprint**) se mesure généralement avec les programmes utilitaires du système d'exploitation (pmap ,vmstat, Gestionnaire de tâches , ...).
- Le rythme effectif de traitements (**throughput**) se mesure avec un utilitaire de stress et de mesure de montée en charge tel que JMeter ou LoadRunner .
- Les pauses (qui peuvent être mises en lumière par un utilisateur impatient) se mesurent via les indications fournies par la JVM elle même:

java -verbose:gc MyApp

L'option moderne **-Xlog:gc*** (anciennement **-XX:+PrintGCDetails**) permet d'obtenir plus de détails

exemple sur ancienne version de java:

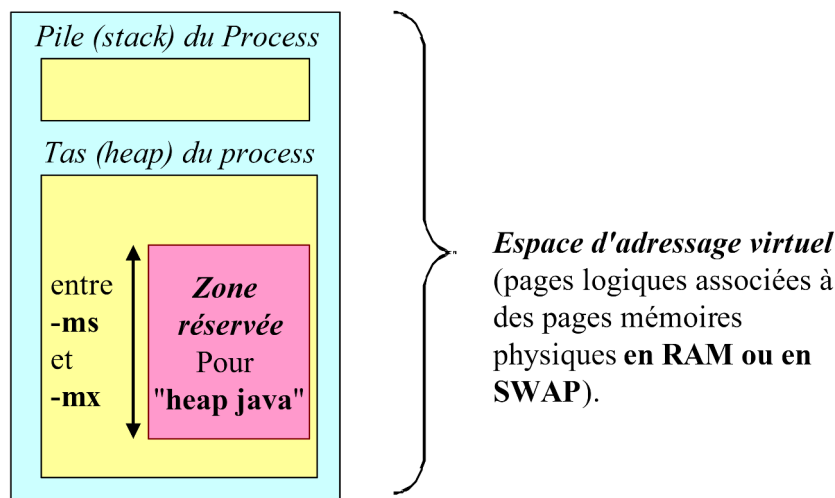
⇒ [type_GC taille_occupée_avant -> taille_occupée_après (espace total *sans perm*) , temps_ramassage]

```
[GC 325407K->83000K(776768K), 0.2300771 secs]
[GC 325816K->83372K(776768K), 0.2454258 secs]
[Full GC 267628K->83769K(776768K), 1.8479984 secs]
```

6. Dimensionnement du tas global (total java heap)

"Heap java" dans "Heap Process"

Processus associé à une JVM



L'espace total alloué au tas de java (**total heap size**) est par défaut automatiquement ajusté par la machine virtuelle en fonction de la charge mémoire effective de l'application.

Cette **taille totale** peut évoluer entre des valeurs **mini** et **maxi** qui sont déterminées par défaut (environ 64M en général pour le maxi) ou bien fixées via les paramètres suivants:

-Xms	taille minimale (initiale) du heap java
-Xmx	taille maximale du heap java (si cette valeur est trop petite ==> memory full)

Exemple: **java -Xms128m -Xmx256m package.MainClass**
java -Xms1g -Xmx2g package.MainClass

NB:

- Dans tous les cas, la valeur de **-ms** doit **au moins être égale à la moitié de -mx** pour que la taille totale ne fluctue pas énormément et pour que cette taille totale puisse rapidement atteindre son "niveau de croisière" (élevé quand le serveur bosse bien).
- On peut éventuellement fixer la valeur de **-ms** à l'identique de celle de **-mx** si l'on souhaite que la JVM réserve dès le départ une grande zone "totale" que l'on est quasiment sûr d'utiliser (grosse activité prévue sur le serveur). ==> Ceci permet d'obtenir des mesures stables et assez déterministes .

7. Paramétrage (tuning) du comportement recherché

<i>Maximum pause time goal</i> -XX:MaxGCPauseMillis=<nnn>	nombre de ms maximum recherché pour l'exécution d'un ramassage (GC) . Ce maximum toléré sera comparé à la somme de la moyenne et de l'écart type et le GC fera de son mieux pour atteindre cet objectif. Plus la valeur est faible et plus le GC sera obligé de se déclencher fréquemment. Compromis à faire entre latence et déclenchements nombreux Cette valeur doit être assez faible pour satisfaire les utilisateurs impatientes (ex : 200)

-XX:ParallelGCThreads=8 et **-XX:ConcGCThreads=4** sont des paramètres (ici à 8 et 4) qui correspondent au nombre de thread qui pourront être employés pour des GC mineurs (en //) ou majeurs

-XX:G1HeapRegionSize=8m permet de définir la taille de chaque Région G1 À 8 Mo. Une taille de région plus petite entraîne généralement des pauses GC plus fréquentes mais plus courtes, tandis qu'une taille de région plus grande réduit la fréquence des GC mais peut augmenter le temps de pause

Il existent plein d'autres paramétrages pointus.

Voici un exemple de combinaison :

-Xms4G -Xmx4G

-XX :+UseG1GC

-XX :MaxGCPauseMillis=200

-XX :ParallelGCThreads=8 -XX :ConcGCThreads=4

-XX :G1HeapRegionSize=8m

-XX :InitiatingHeapOccupancyPercent=45

-XX :G1MixedGCCountTarget=8 -XX :G1MixedGCLiveThresholdPercent=85

-XX :G1HeapWastePercent=5

-XX :+UseStringDeduplication

-Xlog :gc*:file=/var/logs/gc.log :time,uptime,filecount=5,filesize=10M

7.1. Monitoring JVM (depuis java 5)

Le **jdk** est livré en standard avec certains utilitaires de surveillance :
Ces programmes (placés dans le sous répertoire **bin**)

jps (Jvm Process Status)	Liste les JVM en cours d'exécution sur la machine locale
jstat (Jvm Stat)	Affiche des statistiques sur une JVM
jstatd (jstat daemon)	Agent (daemon) à lancer en tâche de fond et servant d'intermédiaire pour obtenir les stats à distance via RMI
jconsole	console <i>JMX</i> dédiée à java (management & monitoring)

jps [-v]

⇒ liste les JVM et leurs *<vmid>* (PID)

jstat -options

⇒ liste les options possibles (ex: -gc , -class , ...)

jstat -gc <vmid> 5s 6

⇒ affiche 6 fois (toutes les 5s) des stats sur le GC de la machine virtuelle identifiée par tel ID .

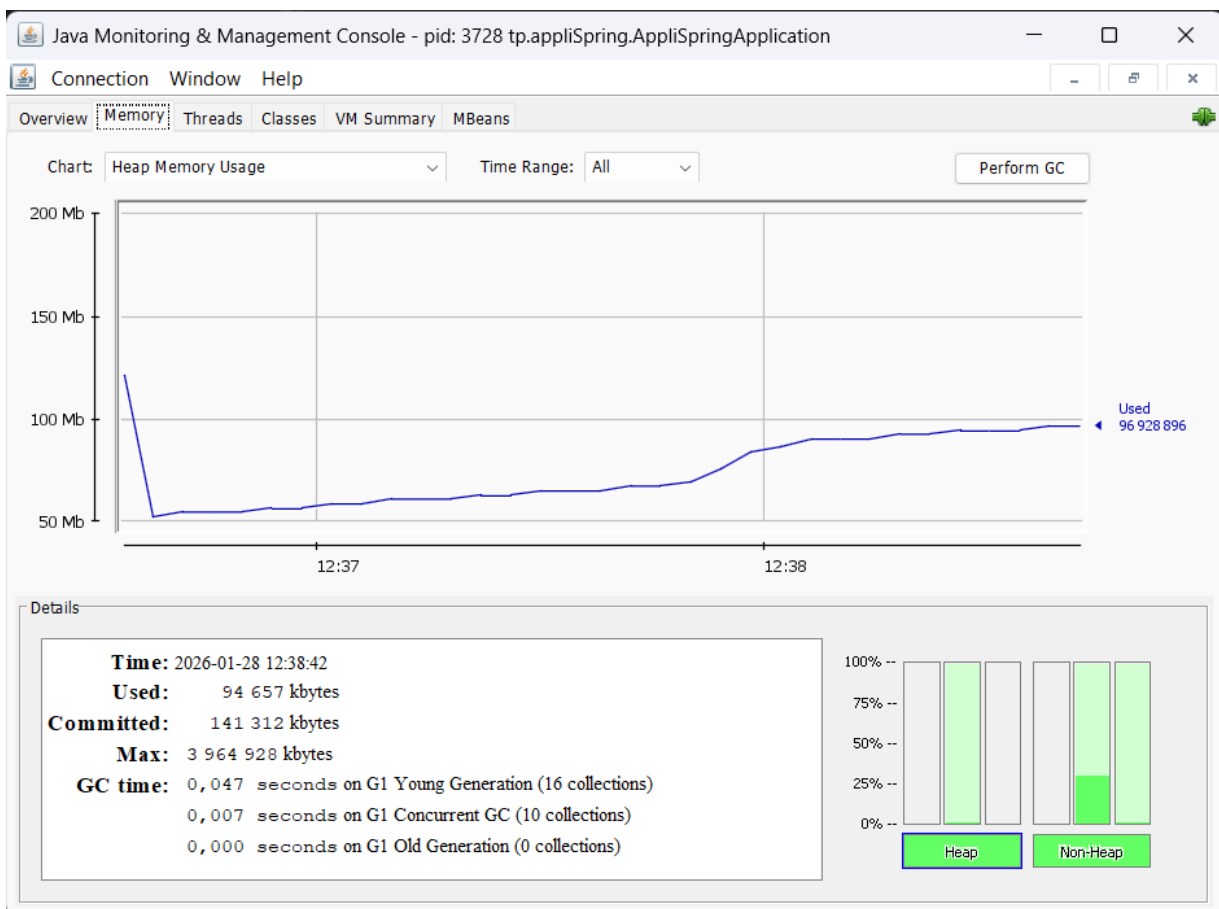
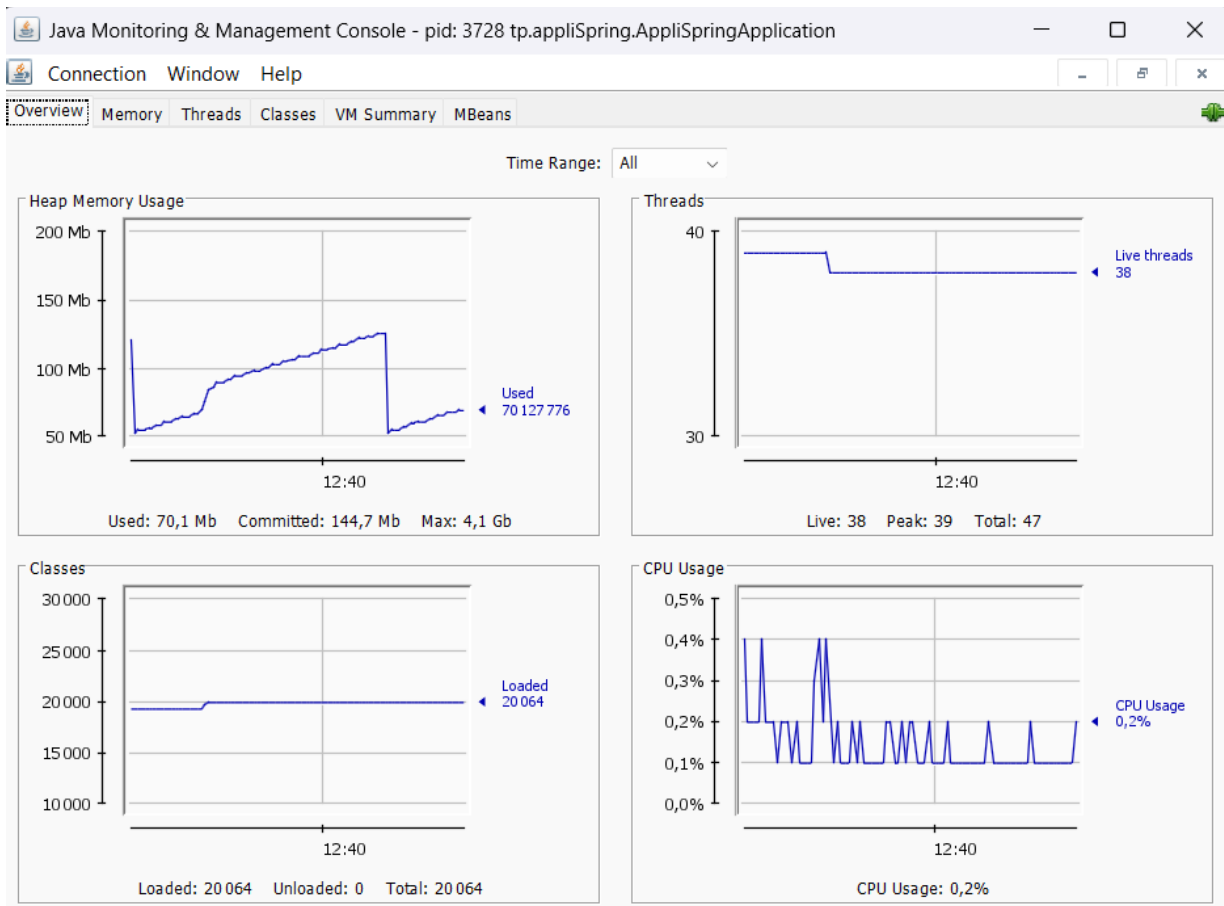
jconsole [*<vmid>*]

⇒ démarre la console graphique jconsole en lui demandant de se connecter à l'application ayant l' ID spécifié .

NB: **jconsole** ne fonctionne qu'avec des programme JAVA managés via JMX (lancés avec l'option **-Dcom.sun.management.jmxremote**)

.../...

NB : si on lance **jconsole** sans préciser de vmid , une boîte de dialogue est alors automatiquement proposée pour pouvoir choisir l'application java à surveiller .



Java Monitoring & Management Console - pid: 3728 tp.appliSpring.AppliSpringApplication

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

Virtual Machine: Java HotSpot(TM) 64-Bit Server VM version 21.0.9+7-LTS-338
Vendor: Oracle Corporation
Name: 3728@D2F2026

Process CPU time: 20,562 seconds
JIT compiler: HotSpot 64-Bit Tiered Compilers
Total compile time: 1,982 seconds

Live threads: 38
Peak: 39
Daemon threads: 32
Total threads started: 47

Current classes loaded: 20 061
Total classes loaded: 20 061
Total classes unloaded: 0

Current heap size: 115 137 kbytes
Maximum heap size: 3 964 928 kbytes
Committed memory: 141 312 kbytes
Pending finalization: 0 objects

Garbage collector: Name = 'G1 Young Generation', Collections = 16, Total time spent = 0,047 seconds
Garbage collector: Name = 'G1 Concurrent GC', Collections = 10, Total time spent = 0,007 seconds
Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0,000 seconds

Operating System: Windows 11 10.0
Architecture: amd64
Number of processors: 16
Committed virtual memory: 408 108 kbytes

Total physical memory: 15 854 444 kbytes
Free physical memory: 1 821 940 kbytes
Total swap space: 25 781 140 kbytes
Free swap space: 771 832 kbytes

8. Processus d'optimisation (tuning)

Cycle

- Analyser
- Coder/paramétrer
- Mesurer pour vérifier

avec itérations à prévoir pour amélioration progressive.

Principaux objectifs de la phase d'intégration/optimisation :

- amélioration des temps de réponse
- amélioration de la consommation mémoire-mémoire
- amélioration éventuelle du temps de démarrage
- éventuelle détection et correction des fuites de mémoire

9. Outils pour l'optimisation des exécutions java

Outils ou réglages	Fonctionnalités
Options de logs sur la JVM (ex : <i>-Xlog:gc ...</i>)	Pratique en production (mode texte)

jconsole	Console JMX pour surveiller une JVM , pratique en phase de développement/intégration
JMeter (open source)	Outil de simulation de charge très paramétrable : on envoie des requêtes en rafale et on mesure les temps de réponses et le nombres de réponses fabriquées et renvoyées.
SoapUi (opensource) ou ReadyApi (payant)	Pour requêter des WebServices « soap » ou « rest »
JProfiler (avec licence nécessaire ou période d'évaluation)	Un peu comme jconsole mais plus sophistiqué
Java VisualVM	Outils graphique open source s'appuyant en interne sur les outils de base (jstat ,...), autrefois packagé dans le jdk8 , maintenant à télécharger séparément . Quelques mesures pointues (ex : virtual thread , dump de heap pour analyse , ...) et intégration possible dans VisualStudioCode
IntelliJ Profiler	Extension de la version ultime de l'IDE IntelliJ
Eclipse MAT	Memory Analyser (analyse de heap-dump pour détecter des fuites de mémoire)

10. Essentiel de Java VisualVM

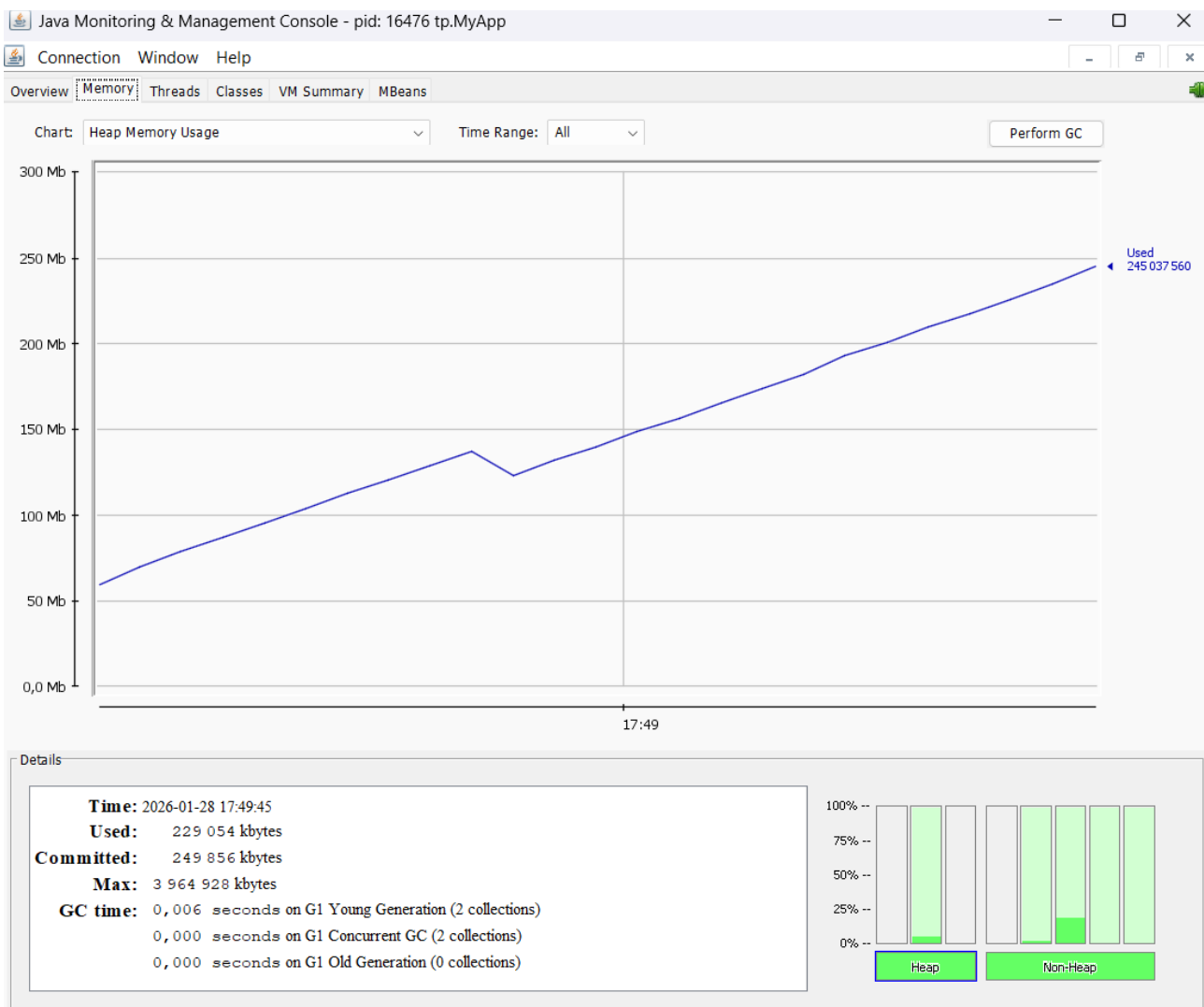
NB: VisualVM en version 22 ne fonctionne pas encore avec le JDK25 et si l'on souhaite le lancer avec java21 , ça peut se faire à peu près de cette manière :

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
visualvm.exe --jdkhome "%JAVA_HOME%"
```

ou bien

```
export JAVA_HOME=C:\Program Files\Java\jdk-21
visualvm.exe --jdkhome "${JAVA_HOME}"
```

11. Détection d'une fuite de mémoire



Une augmentation assez régulière et sans quasi redescente importante de la mémoire consommée est souvent un signe de fuite de mémoire potentielle.

Au sein d'une application ordinaire une fuite de mémoire s'observe sur une très grande période de temps (ex : plusieurs jours) si seulement quelques octets sont régulièrement ajouté/répandus au niveau de la fuite.

Exemple typique de code java provoquant une fuite de mémoire :

```
public class FuiteMemoire {

    // Static list
    private static List<Object> staticList = new ArrayList<>();

    public static void addObject(Object obj) {
        staticList.add(obj);
    }
}
```

```

public static void memoryLeakExample() {
    for (int i = 0; i < 1000; i++) {
        Object obj = new byte[1024 * 1024];

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        System.out.println("add in memory leak ...");
        addObject(obj); // Add the object to the static list
    }
}

```

Principales causes de fuites de mémoire :

- référence inédite (accumulation transitive, ...)
- références circulaires
- mauvaise gestion des ressources (éléments mal fermés, mal libérés).
Exemple : oubli de try/catch/finally avec close ou de try with resources
- Utilisation abusive de variables statiques
- utilisation de librairies tierces (bugs , problèmes ci-dessus de manière indirecte)

Symptômes classiques d'une fuite de mémoire :

- performances qui se dégradent
- consommation mémoire qui augmente constamment
- activité intense du « garbage collector »
- Exception OutOfMemoryError

Quelques approches préventives :

- raisonner en tant que cycle de vie (sur les objets de l'application) : à quel moment ils sont créés , utilisés, libérés ,
- éventuelles utilisations de références faibles pour des caches élémentaires
- éventuelles tâches de nettoyage à déclencher régulièrement

Outils pour détecter une fuite de mémoire :

- 1) visualiser le problème (avec jconsole, VisualVM ou des logs)
- 2) utiliser un outils permettant d'effectuer un « heap dump » puis de l'analyser pour identifier les endroits suspects (ex : Eclipse Memory Analyser (MAT)).

Pour générer un « heap dump » (photographie de la mémoire utilisée à un instant t), on peut s'appuyer sur VisualVM ou un autre outil graphique de ce genre et on peut aussi lancer des lignes de commandes s'appuyant directement sur le jdk :

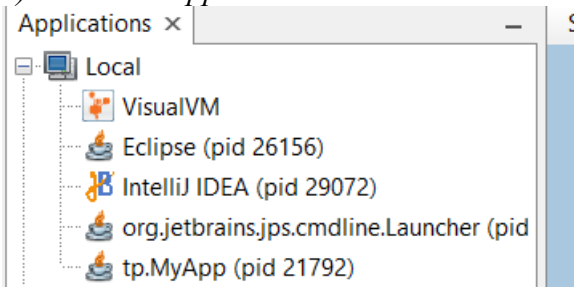
```
jmap -dump:[live],format=b,file=<file-path> <pid>
jmap -dump:live,format=b,file=/tmp/dump.hprof 12587
jmap -dump:live,format=b,file=c:/temp/dump1.hprof 12587
```

ou bien

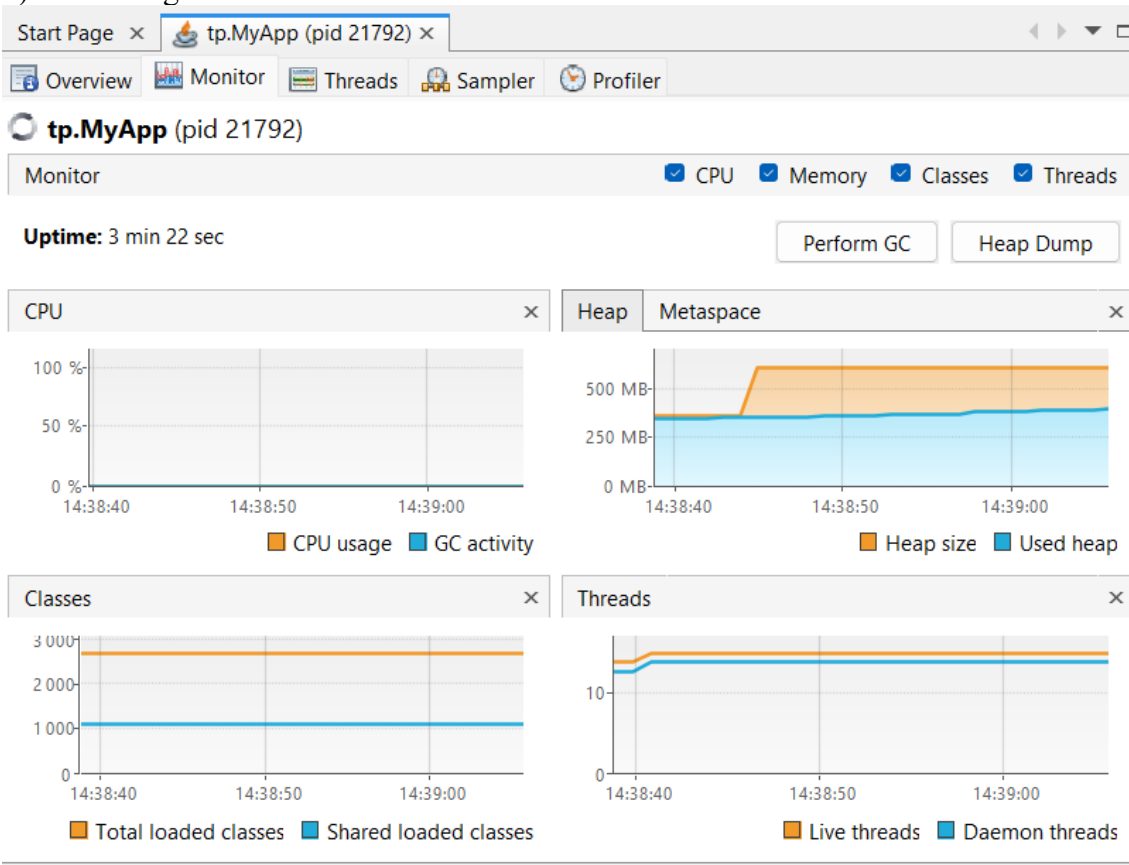
```
jcmd <pid> GC.heap_dump <file-path>
jcmd 12587 GC.heap_dump /tmp/dump.hprof
jcmd 12587 GC.heap_dump c:/temp/dump2.hprof
```

Exemple d'utilisation de VisualVM :

1) choix de l'application à surveiller :



2) monitoring



3) bouton « Heap Dump »

Start Page x tp.MyApp (pid 21792) x

Overview Monitor Threads Sampler Profiler heapdump 14:39:56 x

tp.MyApp (pid 21792)

Heap Dump

Summary

Heap		Environment	
Size:	530 096 096 B	System	Windows 11 (10.0)
Classes:	2 924	Architecture:	amd64 64bit
Instances:	94 124	Java Home:	\Prog\ eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre
Classloaders:	36	Java Version:	21.0.9 2025-10-21 LTS
GC Roots:	2 759	Java Name:	OpenJDK 64-Bit Server VM (21.0.9+10-LTS, mixed mode, sharing)
Objects Pending for Finalization:	0	Java Vendor:	Eclipse Adoptium
		JVM Uptime:	4 min 12 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]		Classes by Size of Instances [view all]	
byte[]	19 129 (20,3 %)	byte[]	264 185 672 B (49,8 %)
java.lang.String	17 856 (19 %)	int[]	263 207 640 B (49,7 %)
java.lang.Object[]	4 413 (4,7 %)	java.lang.String	428 544 B (0,1 %)
java.util.concurrent.ConcurrentHashMap\$Node	4 375 (4,6 %)	java.lang.Object[]	237 872 B (0 %)
java.util.HashMap\$Node	3 452 (3,7 %)	java.lang.reflect.Method	179 344 B (0 %)

Instances by Size [view all]		Dominators by Retained Size [view all]	
byte[] #1 : 1 048 576 items	1 048 592 B (0,2 %)	Retained sizes must be computed first:	
byte[] #2 : 1 048 576 items	1 048 592 B (0,2 %)	Compute Retained Sizes	
byte[] #3 : 1 048 576 items	1 048 592 B (0,2 %)		
byte[] #5 : 1 048 576 items	1 048 592 B (0,2 %)		
byte[] #4 : 1 048 576 items	1 048 592 B (0,2 %)		

En développant « instance by size » (ou)

et en analysant les références on peut éventuellement déjà voir certaines choses :

Name

byte[] #1 : 1 048 576 items

<items>

<references>

[1] in java.lang.Object[] #4350 : 366 items

elementData in java.util.ArrayList #460 : 250 elements

static staticList in class tp.FuiteMemoire : FuiteMemoire

[1] in java.lang.Object[] #4063 : 10 items

tp.MyApp (pid 21792)

[heapdump] 14:39:56

save as ... **heapdump-1770039596155.hprof** (ou [c:/temp/dump1.hprof](#) ou .bin ou .dump)

Installation de Eclipse Memory Analyzer :

Help/Install new Software , ...

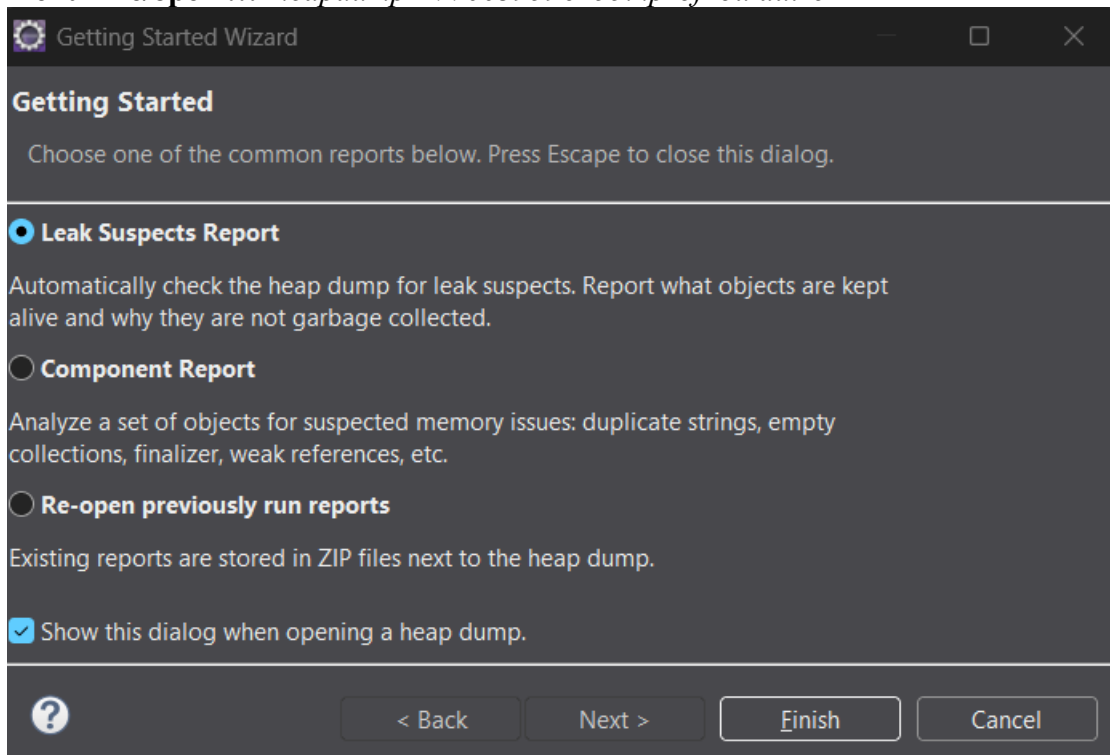
Performance, Profiling and Tracing Tools

GCov Integration	8.21.0.202512022002
GDB Tracepoint Analysis	11.2.0.202512032044
GProf Integration	8.21.0.202512022002
LTTng Kernel Analysis	11.2.0.202512032044
LTTng Userspace Analysis	11.2.0.202512032044
Memory Analyzer	1.16.1.202501091339

puis redémarrage

Utilisation d'eclipse Memory Analyzer :

new perspective « Memory Analysis »

menu File/open ... *heapdump-1770039596155.hprof* ou autre

The classloader/component **"jdk.internal.loader.ClassLoaders\$AppClassLoader @ 0x712439668"** occupies **262 157 600 (98,45 %)** bytes. The memory is accumulated in one instance of **"java.lang.Object[]"**, loaded by **"<system class loader>"**, which occupies **262 149 480 (98,45 %)** bytes.

Keywords

jdk.internal.loader.ClassLoaders\$AppClassLoader
java.lang.Object[]

▼ **Shortest Paths To the Accumulation Point** 📄

Class Name	Shallow Heap	Retained Heap
java.lang.Object[366] @ 0x72e000000	1 480	262 149 480
elementData java.util.ArrayList @ 0x712559518	24	262 149 504
staticList class tp.FuiteMemoire @ 0x712400108	8	262 149 600
[1] java.lang.Object[10] @ 0x712579618	56	262 149 656
elementData java.util.ArrayList @ 0x712579600	24	262 149 680
classes jdk.internal.loader.ClassLoaders\$AppClassLoader @ 0x712439668 JNI Global	96	262 157 600

Attention : les fichiers « .hprof » (dump du heap) sont potentiellement très volumineux (ex : 600Mo ou 1Go) . Il faut donc penser à les supprimer une fois qu'ils ont été bien analysés .

XXVII - Annexe – Tests avancés (JaCoCo, Mockito,...)

1. Analyse de la couverture des tests

Toutes les parties importantes du code ont bien été testées ?

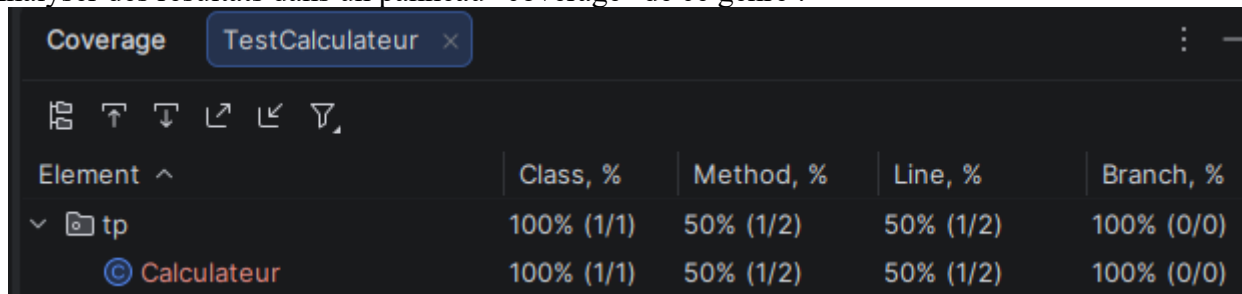
Rien n'a été oublié (*packages, classes, méthodes, branches de git, ...*) ?

1.1. Couverture des tests dans IntelliJ

Installer si nécessaire (depuis Settings/plugins) le plugin "Code coverage for java".

Lancer un ou plusieurs tests via "Run With Coverage".

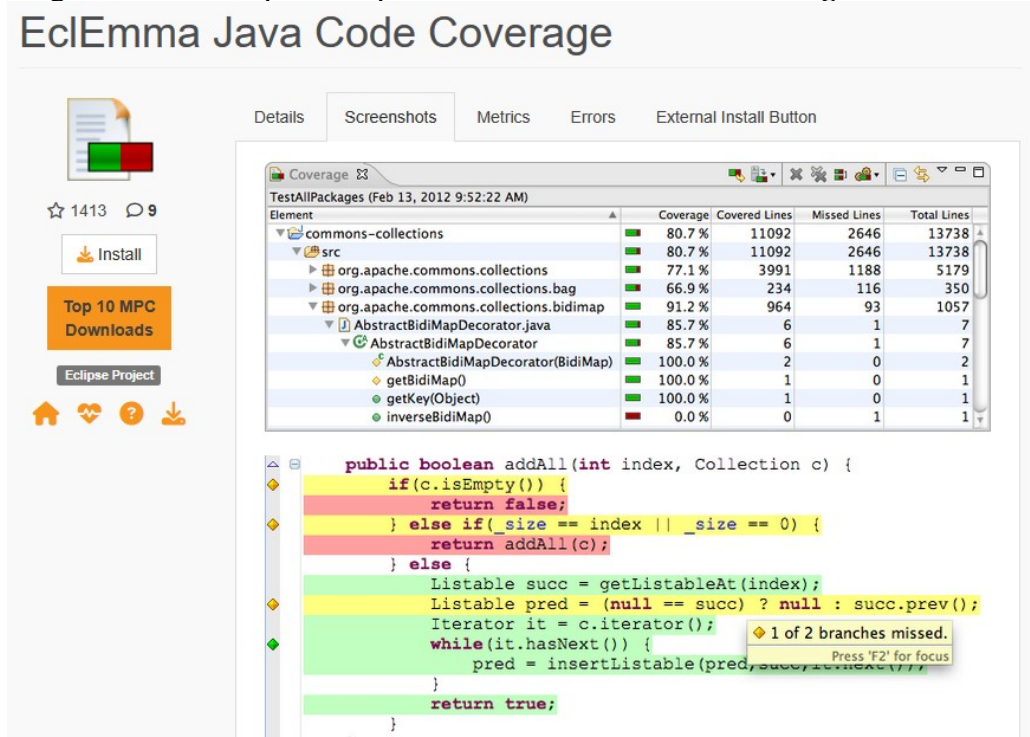
Analyser des résultats dans un panneau "coverage" de ce genre :



Element ^	Class, %	Method, %	Line, %	Branch, %
tp	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)
Calculateur	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)

1.2. Couverture des tests dans eclipse

Plugin de référence pour eclipse : *EclEmma Java Code Coverage*



EclEmma Java Code Coverage

Details Screenshots Metrics Errors External Install Button

Element	Coverage	Covered Lines	Missed Lines	Total Lines
TestAllPackages (Feb 13, 2012 9:52:22 AM)				
commons-collections	80.7 %	11092	2646	13738
src	80.7 %	11092	2646	13738
org.apache.commons.collections	77.1 %	3991	1188	5179
org.apache.commons.collections.bag	66.9 %	234	116	350
org.apache.commons.collections.bidimap	91.2 %	964	93	1057
AbstractBiDiMapDecorator.java	85.7 %	6	1	7
AbstractBiDiMapDecorator	85.7 %	6	1	7
AbstractBiDiMapDecorator(BiDiMap)	100.0 %	2	0	2
getBiDiMap()	100.0 %	1	0	1
getKey(Object)	100.0 %	1	0	1
inverseBiDiMap()	0.0 %	0	1	1

```

public boolean addAll(int index, Collection c) {
    if(c.isEmpty()) {
        return false;
    } else if(_size == index || _size == 0) {
        return addAll(c);
    } else {
        Listable succ = getListableAt(index);
        Listable pred = (null == succ) ? null : succ.prev();
        Iterator it = c.iterator();
        while(it.hasNext()) {
            pred = insertListable(pred, succ, it.next());
        }
        return true;
    }
}

```

1 of 2 branches missed. Press 'F2' for focus

1.3. Configuration maven pour le plugin "jaCoCo"

pom.xml

```
...
<build> <plugins>  <plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.14</version>
    <executions>
        <execution>
            <goals> <goal>prepare-agent</goal> </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals> <goal>report</goal> </goals>
        </execution>
    </executions>
</plugin></plugins></build></project>
```

1.4. Analyse des rapports de couverture des tests de jaCoco

après avoir lancer *mvn test* ou *mvn package* ou autre ,
 ouvrir **target/site/jacoco/index.html**
 pour visualiser les rapports de couverture des tests

with_jacoco

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
tp		63 %		n/a	1	3	1	3	1	3	0	1
Total	4 of 11	63 %	0 of 0	n/a	1	3	1	3	1	3	0	1

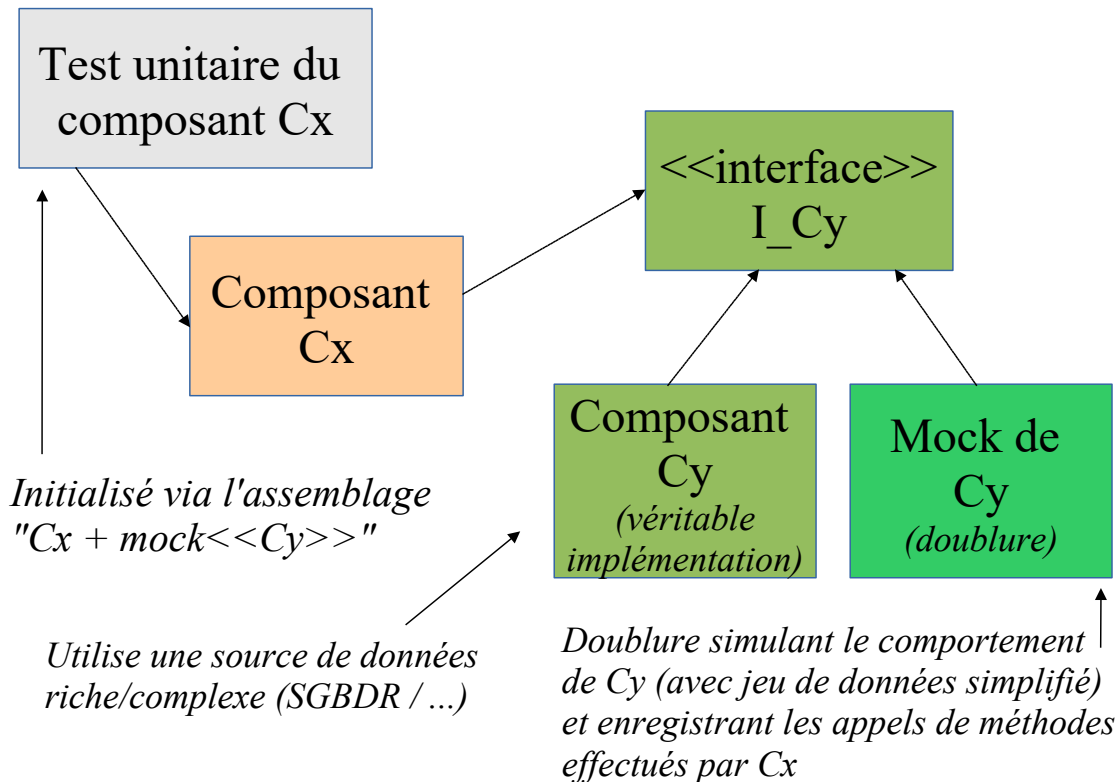
with_jacoco > tp > Calculateur

Calculateur

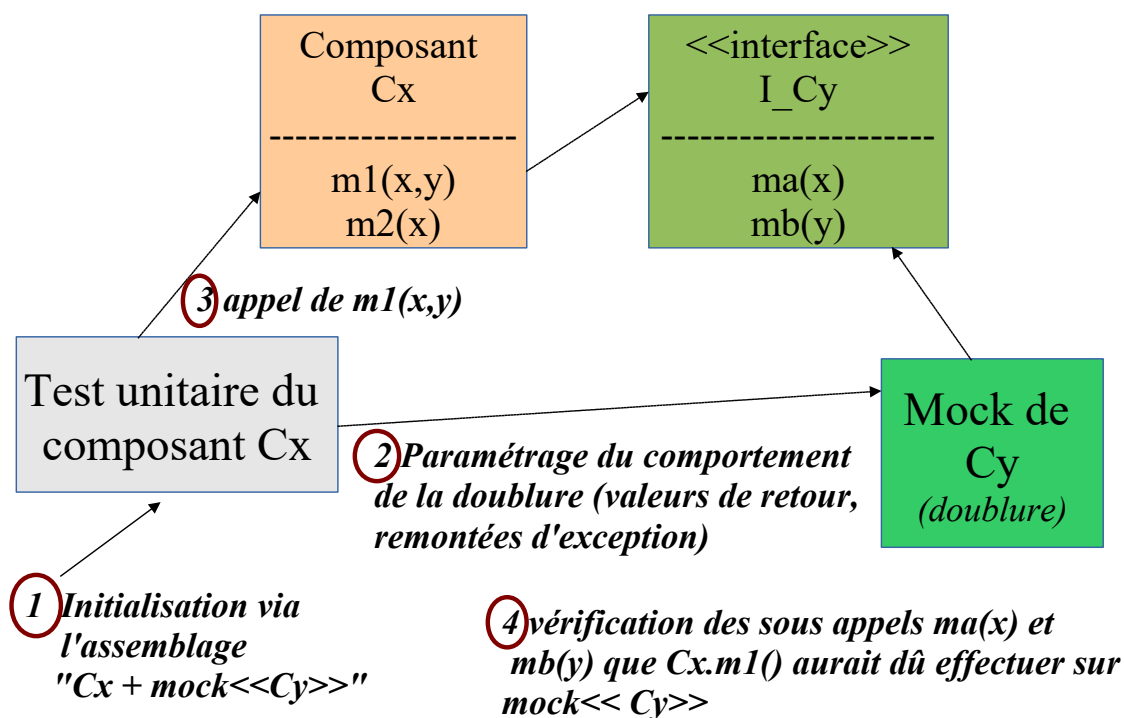
Element	Missed Instructions	Cov.
mult(double, double)		0 %
add(double, double)		100 %
Calculateur()		100 %
Total	4 of 11	63 %

2. Positionnement et intérêts des "mocks"

Positionnement des "mocks"



intérêts des "mocks"



3. Mockito

3.1. Présentation de Mockito

Mockito est une technologie java assez populaire pour mettre en œuvre des "Mocks" (simulacres).

3.2. Dépendance maven nécessaire

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <!-- <artifactId>mockito-junit-jupiter</artifactId> -->
  <version>5.10.0</version>
</dependency>
```

3.3. Initialisation du Mock au niveau d'un test Junit:

Solution1 (par annotation "**@Mock**" interprétée du fait de **@RunWith(MockitoJUnitRunner.class)**) :

```
import org.junit.runner.RunWith;
import org.mockito.Mock;
import org.mockito.runners.MockitoJUnitRunner;

@RunWith(MockitoJUnitRunner.class)
//@ExtendWith(MockitoExtension.class) //for JUnit 5
public class UserLoginMockTest {

    @Mock
    private static UserLogin userLogin;
    ....
}
```

Solution2 (par annotation "**@Mock**" interprétée du fait de l'appel à **MockitoAnnotations.initMocks(this)**; lors de l'initialisation) :

```
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

//rien ou @RunWith(SpringJUnit4ClassRunner.class)
public class UserLoginMockTest {

    @Mock
    private UserLogin userLogin;

    @Before
    public void init() /* or setUp or ... */ {
        MockitoAnnotations.initMocks(this);
    }
    ...
}
```

Solution3 (*sans annotation* Mockito , avec un appel explicite à **Mockito.mock()**) :

```
import org.mockito.Mockito;

public class UserLoginMockTest {
    private static UserLogin userLogin;

    @BeforeClass
    public static void init() {
        userLogin = Mockito.mock(UserLogin.class);
    }
    ....
}
```

Soit l'interface fonctionnelle suivante :

```
public interface UserLogin {

    public boolean verifyLogin(String username,String password);
    public String goodPasswordForUser(String username);
    public void setSize(int size);
    //pour tester valeur de retour par défaut:
    public String getAuteur();
    public int getSize(); //return nbAccount ( >=0)
    public double getDoubleValue(double x); //return x * 2
}
```

et soit une classe d'implémentation basique suivante :

```
public class UserLoginImpl implements UserLogin {
    private int size=10; //par défaut
    public boolean verifyLogin(String username, String password) {
        boolean res=false;
        if(password !=null && password.equals("pwd_"+username))
            return true;
        return res;
    }
    public String goodPasswordForUser(String username) {
        return "pwd_"+username;
    }
    public void setSize(int size) { this.size=size;
    }
}
```

```

    public String getAuteur() { return "didier";
    }
    public int getSize() { return size ;
    }
    public double getDoubleValue(double x) { return 2 *x;
    }
}

```

Le comportement de Mockito est alors le suivant :

3.4. Mockito as stub

Comportement par défaut d'un mock (géré par Mockito)

Avec aussi bien

***userLogin* = Mockito.mock(*UserLogin.class*); //interface**

que

***userLogin* = Mockito.mock(*UserLoginImpl.class*); //classe d'implémentation**

tout appel de méthode sur l'objet "mock" géré par mockito retourne une valeur par défaut de type "null" , false , 0 ou 0.0 selon le type de retour :

```

public void displayReturnValues(){
    boolean pwdOk= userLogin.verifyLogin("toto", "pwd_toto");
    System.out.println("pwdOk="+pwdOk);

    String goodPwd= userLogin.goodPasswordForUser("toto");
    System.out.println("goodPwd="+goodPwd);

    String auteur = userLogin.getAuteur();
    System.out.println("auteur="+auteur);

    int taille = userLogin.getSize();
    System.out.println("taille="+taille);

    double val = userLogin.getDoubleValue(3.2);
    System.out.println("val="+val);
}

```

Comportement vraie classe:

pwdOk=true
goodPwd=pwd_toto
auteur=didier
taille=10
val=6.4

Comportement du mock:

pwdOk=false
goodPwd=null
auteur=null
taille=0
val=0.0

Préciser (forcer) une valeur de retour via Mockito :

```
Mockito.when(userLogin.getSize()).thenReturn(5);
```

```
int taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche toujours taille=5
userLogin.setSize(20);    taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche toujours taille=5 (et pas 20 !)
```

On peut forcer un **retour d'exception** selon par exemple certaines valeurs en entrée :

```
Mockito.when(userLogin.setSize(Mockito.eq(-1)))
    .thenThrow(new IllegalArgumentException("message xy"));
```

Lorsque l'on "mock" une classe (et pas une interface), on peut explicitement demander à Mockito de rétablir le comportement de la véritable classe d'implémentation sur certaines méthodes :

```
Mockito.when(userLogin.getSize()).thenCallRealMethod();
Mockito.doCallRealMethod().when(userLogin).setSize(Mockito.anyInt());
// il existe aussi Mockito.anyString() , ...
```

```
userLogin.setSize(20) ;    taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche taille=20
```

3.5. Mockito as "spy" et "mock = stub + spy"

Comportement par défaut d'un mock initialisé via Mockito.spy()

Fonctionnellement : stub = bouchon (comportement simulé) avec 0,null,0.0 par défaut
.spy() = espionnage (enregistrement des appels pour vérifications ultérieures).
.mock() = comportement "stub" + fonctionnalité de .spy()

`userLogin = Mockito.spy(UserLogin.class); //interface`
 ==> même comportement que via Mockito.mock() car pas de code par défaut

`userLogin = Mockito.spy(new UserLoginImpl()); //classe d'implémentation`

On obtient alors un **comportement normal** (identique à la classe d'origine) **sur toutes les méthodes sauf sur celles où l'on demande explicitement à redéfinir le comportement** :

`Mockito.when(userLogin.getSize()).thenReturn(5);`

*Résultats (par défaut)
 depuis code précédent :*
 pwdOk=true
 goodPwd=pwd_toto
 auteur=didier
 taille=5 (à la place de taille = 10)
 val=6.4

On peut désactiver le comportement d'un setter (ou d'une méthode en void) :

`Mockito.doNothing().when(userLogin).setSize(Mockito.anyInt());`

Autrement dit :

- sémantiquement **mock** = "stub + spy"
- pragmatiquement pas de différence notable entre Mockito.mock() et Mockito.spy() en partant d'une interface car si pas de code d'implémentation ---> toujours comportement "mock=stub+spy".

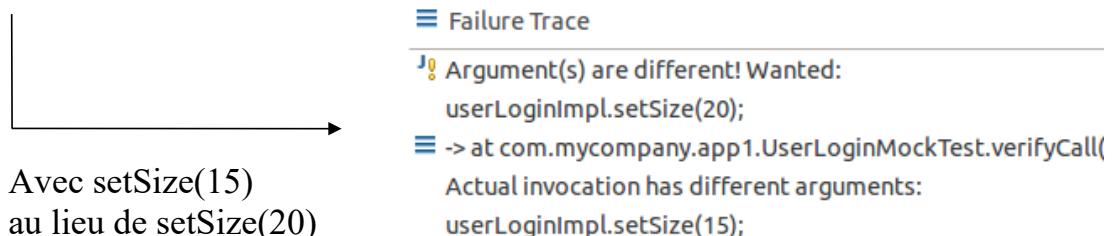
Si par contre Mockito.spy() est appelé en partant d'une réelle instance de composant alors :

- spy --> espionnage seulement pour ultérieur "verify"
 sans changer comportement de l'application
 et mock (= stub + spy)--> verify possible ET implémentation de départ
 écrasée par Mockito.when().then...

Vérification des appels effectués sur un mock (spy) :

Mockito.spy(...) porte bien son nom lorsque l'on sait que l'on peut demander à Mockito d'espionner les appels effectués sur un "mock" et vérifier si certaines méthodes ont bien été appelées (avec certaines valeurs attendues de paramètres en entrée) :

```
@Test
public void verifyCall() {
    userLogin.setSize(15); //userLogin.setSize(20);
    //appel habituellement indirect effectué depuis
    // le code caché d'un composant à tester
    // vers le "spy" ou "stub+spy=mock" d'un sous composant
    Mockito.verify(userLogin).setSize(Mockito.eq(20));
}
```



Quelques exemples de vérifications via Mockito

```
// vérifie que la méthode m1 a été appelée sur obj,  
// avec une String strictement égale à "s1" :  
Mockito.verify(obj).m1(Mockito.eq("s1"));  
// note : ici, le matcher n'est pas indispensable, la ligne suivante est équivalente :  
Mockito.verify(obj).m1("s1");  
  
// vérifie que la méthode m2 n'a jamais été appelée sur l'objet obj :  
Mockito.verify(obj, Mockito.never()).m2();  
  
// vérifie que la méthode m3 a été appelée exactement 2 fois sur l'objet obj :  
Mockito.verify(obj, Mockito.times(2)).m3();  
  
// idem avec un nombre minimum et maximum d'appels :  
Mockito.verify(obj, Mockito.atLeast(3)).m3();  
Mockito.verify(obj, Mockito.atMost(10)).m3();  
  
// vérifie que la méthode m4 a été appelée sur obj,  
// avec un objet similaire à celui passé en argument :  
Mockito.verify(obj).m4(Mockito.refEq(obj2));
```

Quelques "**matchers**" pour vérifier ou paramétrer les valeurs des paramètres :

Mockito. eq (...)	Égal à ...
Mockito. refEq (obj2)	Égal à cet objet
Mockito. anyString() , anyInt() , anyFloat() ,	Chaîne quelconque , entier quelconque, ..
Mockito. anyObject()	Objet quelconque
Mockito. any (Class<T> c)	Objet d'un certain type
Mockito. anyList()	Toute implémentation de List
Mockito. argThat (new MyMatcher())	Vérifiant matcher spécifique
...	

On peut définir de nouveaux "matcher" via des classes qui héritent de **ArgumentMatcher<T>**

Attention :

L'ancienne méthode Mockito.**anyObject()** n'existe plus .

L'ancien ~~org.mockito.Matchers~~ est maintenant obsolète ou inexistant (selon les versions).

Il faut absolument utiliser **org.mockito.ArgumentMatcher** à la place .

NB : les matchers de Mockito sont différents de ceux de hamcrest !!!!

Exemple de matcher spécifique :

```
...
import org.mockito.ArgumentMatcher;

public class MyIntegerBetween implements ArgumentMatcher<Integer>{
    private int inclusiveMini;
    private int inclusiveMaxi;

    public MyIntegerBetween(int inclusiveMini, int inclusiveMaxi) {
        this.inclusiveMini = inclusiveMini;
        this.inclusiveMaxi = inclusiveMaxi;
    }
}
```

```

public MyIntegerBetween() {
    this(0,100); //default min max values
}

@Override
public boolean matches(Integer v) {
    return (v>= inclusiveMini && v <= inclusiveMaxi) ;
}
}

```

Utilisation :

```
//Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(120)).thenReturn(3.5);
```

```

Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    Mockito.intThat( new MyIntegerBetween(61,1000))
)).thenReturn(3.5);

```

NB : **Mockito.intThat()** , **Mockito.doubleThat()** , ... pour les arguments de types primitifs et **Mockito.argThat()** pour les arguments de type "Object" .

"Strict" vs "lenient" mode :

Depuis la version 2.0 de Mockito (qui fonctionne par défaut en mode "strict") , les "stubs" inutiles sont refusés de manière à éviter une écriture inutilement compliquée des tests :

org.mockito.exceptions.misusing.UnnecessaryStubbingException:

Unnecessary stubbings detected.

Clean & maintainable test code requires zero unnecessary code.

Please remove unnecessary stubbings or use 'lenient' strictness.

De manière à tout de même laisser du code un peu inutile (mais quelquefois un peu "explicatif") , on peut éventuellement utiliser **Mockito.lenient().when(...)**... plutôt que **Mockito.when(...)**...

Exemple :

```
Mockito.lenient().when(serviceTauxInteretMock.tauxCourantAnnuelBce(Mockito.intThat( new
MyIntegerBetween(25,60))))).thenReturn(3.0);
```

```
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(Mockito.intThat( new
MyIntegerBetween(61,1000))))).thenReturn(3.5);
```

pour préparer une futur appel indirect à **serviceTauxInteretMock.tauxCourantAnnuelBce()** avec **taux=120** (pas compris entre 25 et 60) .

Eventuelle utilisation de hamcrest avec Mockito :

```

<dependency>
  <groupId>org.hamcrest</groupId>
  <artifactId>hamcrest</artifactId>
  <version>2.2</version>
  <scope>test</scope>
</dependency>

```

```

import org.mockito.hamcrest.MockitoHamcrest;
import org.hamcrest.Matchers;

...
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    MockitoHamcrest.intThat( Matchers.greaterThanOrEqualTo(61))
)).thenReturn(3.5);
/*
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    MockitoHamcrest.intThat(
        Matchers.allOf(Matchers.greaterThanOrEqualTo(61),Matchers.lessThanOrEqualTo(1000))
    )
)).thenReturn(3.5);
*/

```

4. Présentation de JMeter

JMeter est un produit **open source** de la communauté **Apache**.

JMeter est capable de **simuler une certaine charge** tout en **récoltant des mesures essentielles** (temps de réponses , débits, ...).

JMeter est développé en **java** et est donc:

- multiplateforme (windows , linux, ...)
- soumis à certains cycles de "GC/ramasse-miettes" qui ont tendance à introduire un léger niveau d'approximation dans les mesures recueillies (==> ordre de grandeur correct mais précision limitée) .
- Simple à utiliser (interface graphique perfectionnée).
-

Principales fonctionnalités du produit "JMeter":

- requêtes HTTP , SOAP , ...
- requêtes SQL (JDBC), ...
- requêtes LDAP , FTP , TCP, ...
- requêtes JMS,

- résultats en graphes , tableaux, rapports,
- vérifications via assertions, comparaisons, sauvegardes fichiers,

- Timer, threads(clients simultanés simulés)
- Pré et post-processeurs (extraction de valeurs via reg-expr, ...)
- Variables, boucles , ...

- Gestion automatique des cookies HTTP (utile entre autres pour identifier les sessions des clients simultanés simulés)
- ...

Les versions récentes de JMeter comportent maintenant un **mode "CLI"** (lignes de commandes en mode texte) permettant de déclencher et effectuer des mesures sans interface graphique. Ça améliore les performances et ça offre plus de souplesse.

Restriction importante :

JMeter n'est pas un navigateur internet et fonctionne d'un point de vue protocole réseau (ex : TCP , HTTP, ...) .

JMeter peut être très utile pour tester une montée en charge vers une api REST ou tout autre élément du coté backend .

JMeter ne peut pas être utiliser pour tester les performances d'un frondEnd javascript (ex : angular, react, vue, ...) s'exécutant au sein d'un navigateur .

5. Configuration et démarrage du produit JMeter

NB : au sein des versions récentes de JMeter on aura souvent intérêt à fixer la propriété `resultcollector.action_if_file_exists` à la valeur `APPEND` au sein de `apache-jmeter-5.6.3/bin/jmeter.properties` (pas très loin de la ligne 1050 ou autre):

```
resultcollector.action_if_file_exists=APPEND
```

car la valeur `DELETE` provoque souvent des erreurs/blocages et la valeur par défaut `ASK` déclenche trop souvent une popup assez pénible .

D'autre part , un lancement de jMeter depuis un autre répertoire que `apache-jmeter-5.6.3/bin` est conseillé pour éviter de mélanger les fichiers de jmeter à ceux de notre plan de test .

launch_jmeter.bat pas dans ~~*apache-jmeter-5.6.3/bin*~~ mais ***ailleurs*** !!!!

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set PATH=%JAVA_HOME%\bin;%PATH%
set JMeter_HOME=C:\Prog\apache-jmeter-5.6.3
%JMeter_HOME%\bin\jmeter
```

Une fois qu'un plan de test aura été mis au point (fichier `.jmx`) , on pourra lancer l'exécution de ce plan de test en ligne de commande (sans interface graphique) via :

launch_jmeter_cli.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set PATH=%JAVA_HOME%\bin;%PATH%
set JMeter_HOME=C:\Prog\apache-jmeter-5.6.3
%JMeter_HOME%\bin\jmeter -n -t test_plan_xyz.jmx -l summary-report.jtl
REM -n for no_gui (cli mode)
REM -t test_plan.jmx
REM -l log_file.jtl
```

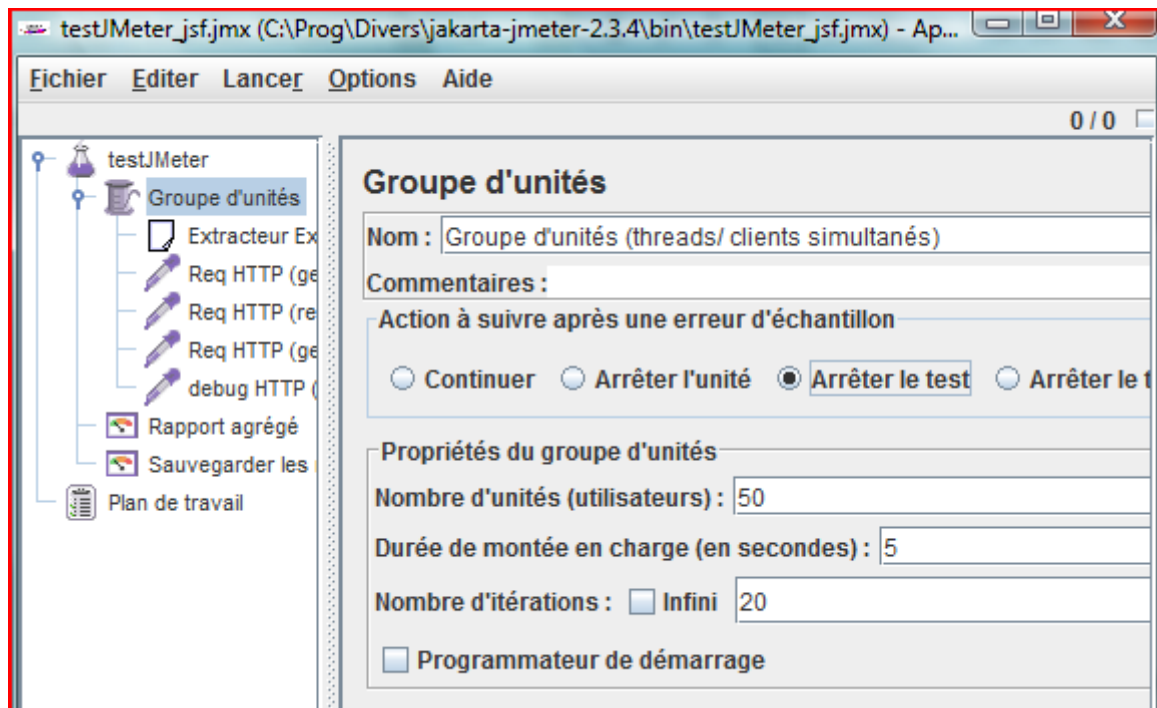

6. Configuration élémentaire d'un plan de test JMeter

Commencer par renommer le plan de test (ex: MyApp ou TestXY)

NB : les plans (configurations) de tests de jmeter sont stockés dans des fichiers XML avec l'extension **.jmx**

6.1. Configuration d'un groupe d'unités (clients simultanés)

Ajouter / Groupe d'unités [*Add / Thread group*]



10 Threads = 10 clients simultanés

Période de chauffe [*Ramp-up period*] = 5

==> les threads démarrent un par un toutes les $5 / 10 = 0,5$ s.

Une fois démarré, chaque thread enverra 20 requêtes (*Loop Count*) . Entre chaque requête , chacun des threads attendra par défaut **0** ms.

Pour paramétrer (si besoin) une pause éventuelle (ex: **1000 ms**) entre les requêtes consécutives envoyées par un même thread il faut alors incorporer (sous le groupe de Thread) un **Compteur de temps fixe**[*Timer/ConstantTimer*].

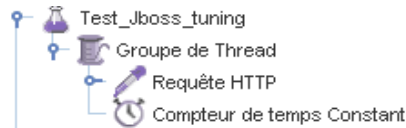
Compteur de temps Constant

Nom:

Temporisation Thread (en millisecondes):

6.2. Configuration d'une requête à lancer (plusieurs fois)

Sous ce "Groupe d'unités" ==> **Ajouter / Echantillon / Requête HTTP**
 "Thread Group" Add / Sampler / HTTP Request



Requête HTTP

Nom: Requête HTTP / Conversion

Serveur Web

Nom ou IP du Serveur: localhost

Numéro de Port: 8080

Requête HTTP

Protocole: Méthode: ☒ GET ☐ POST

Chemin: deviseWeb/convlt.jsp ☐ Rediriger Automatiquement ☒ Suivre

Envoyer les Paramètres Avec la Requête:

Nom:	Valeur
somme1	100
monnaie1	Euro
monnaie2	Yen

6.3. configuration de la récupération des résultats (synthèse, courbe, ...)

==> Sous la "Requête HTTP" ou bien sous "le groupe d'unités"

==> *Ajouter / Récepteur / Résultats Graphiques mélangés (pas très clair)*
Add / Listener / Graph Result

====>

*Add / Listener / **Response Time Graph** (bien en réglant interval = 100ms ou autre)*

====> **Ajouter / Récepteur / Rapport Agrégé (tableau récapitulatif)**
*Add / Listener / **Aggregate Report***

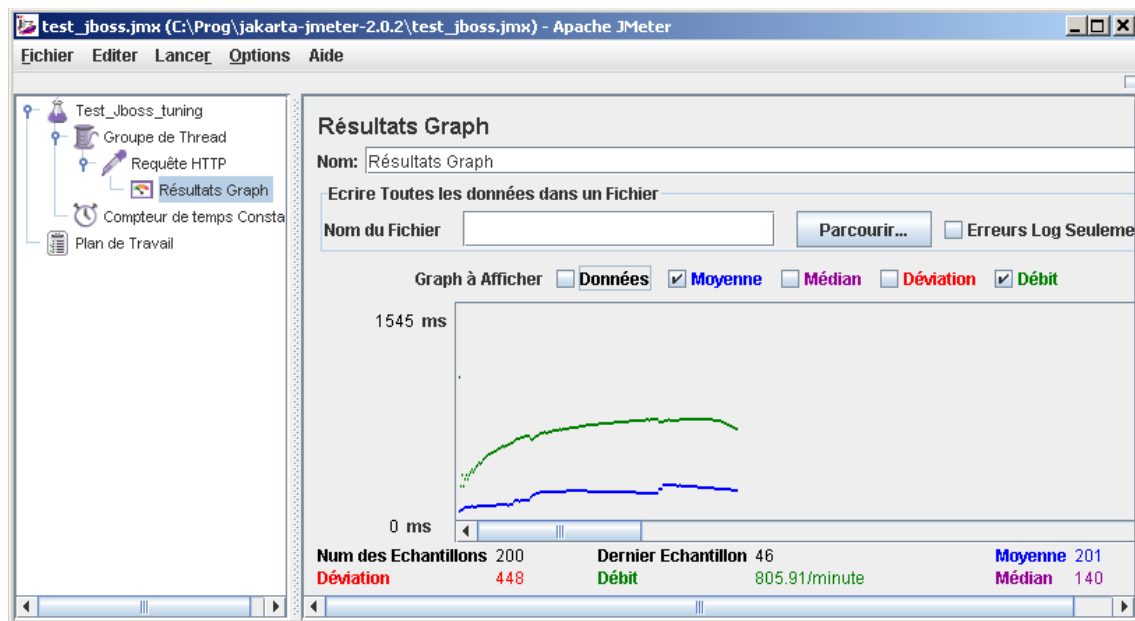
6.4. (re-)lancement du test et observation des résultats

Sauvegarder le "Test Plan" .

Ré-initialiser à zéro des courbes et résultats via "**Lancer / Nettoyer tout** [Run / Clear All]"

Activer la simulation de montée en charge via "**Lancer / Démarrer** [Run / Start]"

==>



Rapport Agrégé						
Nom: Rapport Agrégé						
Ecrire Toutes les données dans un Fichier						
Nom du Fichier		Parcourir...		☐	Erreurs Log Seulement	
URL	Count	Average	Min	Max	Error%	Rate
Requête HTTP	40	41	0	140	0,00%	4,6/sec
TOTAL	40	41	0	140	0,00%	4,6/sec

Quelques principes importants à respecter :

Si l'on souhaite obtenir des mesures/résultats corrects il est absolument indispensable de respecter cette règle fondamentale :

Faire fonctionner JMeter sur un autre ordinateur que celui où s'exécute le logiciel à tester

sauf peut être durant les toutes premières phases de mise au point .

Donc idéalement pas de **localhost** au sein des URLs à tester .

Si l'on ne respecte pas cette règle , JMeter se stresse lui même et les mesures ne seront pas fiables.

D'autre part , si en phase de "développement" ou "pré-prod" , certains logs du logiciel à tester sont mal réglés (ex : affichage de debug pour chaque requête sur la console texte) , les temps de réponse seront altérés et pas significatifs .

6.5. Response Time Graph

Add / Listener / Response time Graph

Réglage important :

interval (ms) entre 2 points du graphique

Response Time Graph

Name:

Comments:

Write results to file / Read from file

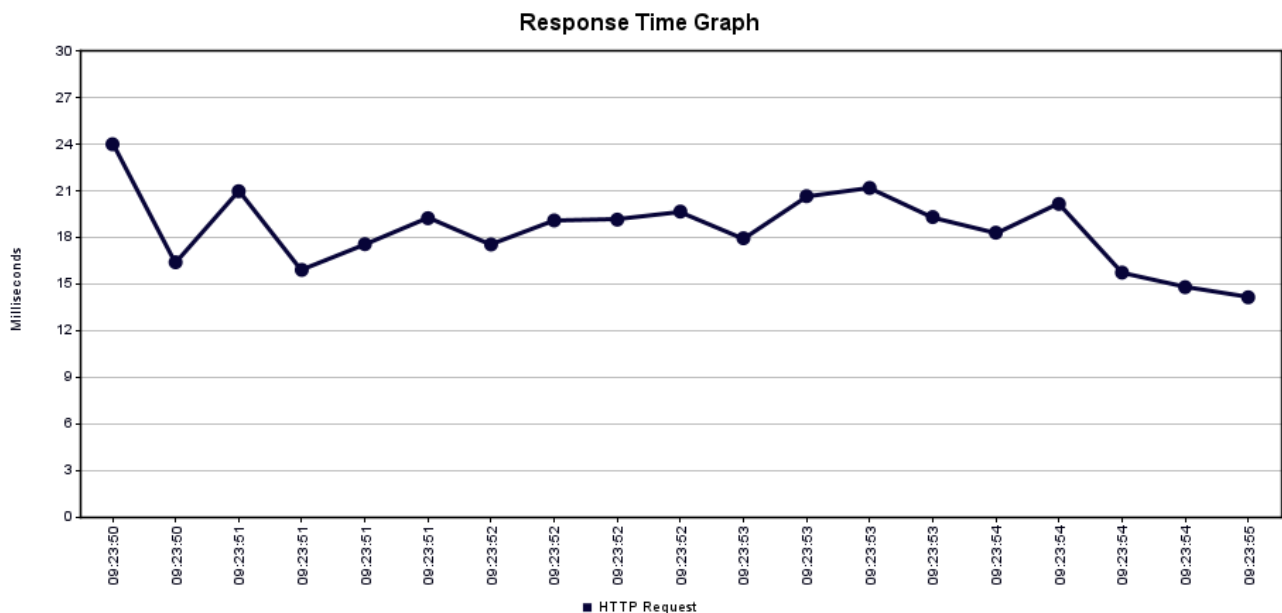
Filename

Settings **Graph**

Graph settings

Interval (ms):

Exemple de résultat :



7. Très important: tester tout d'abord le test !!!

Si l'on ne vérifie pas l'exactitude des paramétrages au niveau des tests, on peut alors confondre une réponse normale avec une réponse de type "message d'erreur" retournée par le serveur. Des temps de réponses liés à des réponses négatives (messages d'erreurs) ne sont évidemment pas significatifs.

Pour être certain que la réponse retournée par le serveur est correcte/normale, il suffit de procéder de la manière suivante (durant la phase de mise au point):

- 1) diminuer (temporairement) au maximum le nombre de requêtes qui seront envoyées:

Groupe d'unités

Nom : Groupe d'unités (threads/ clients simultanés)

Commentaires :

Action à suivre après une erreur d'échantillon

☒ Continuer ☐ Arrêter l'unité

Propriétés du groupe d'unités

Nombre d'unités (utilisateurs) : 1

Durée de montée en charge (en secondes) : 1

Nombre d'itérations : ☐ Infini 1

- 2) ajouter un récepteur de type "*Sauvegarder les réponses vers un fichier*"

Sauvegarder les réponses vers un fichier

Nom : Sauvegarder les réponses vers un fichier

Commentaires :

Préfixe du nom de fichier : res_jsf

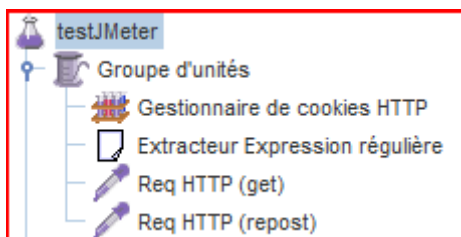
Nom de variable :

- 3) **lancer le test et vérifier le contenu du ou des fichier(s) généré(s)**
- à refaire à chaque nouveau re-paramétrage des tests (pour vérifier une non-régression)
- 4) une fois la phase de mise au point terminée, il suffit de **rendre inactif** le récepteur de type "*Sauvegarder les réponses vers un fichier*" pour ne pas ralentir inutilement les tests. En jouant ainsi sur le menu contextuel "activer/désactiver" ou "enable/disable", on peut basculer rapidement d'un mode "debug" vers un mode "test réaliste" et vice-versa. Il faudra également penser à ré-augmenter les nombres de boucles et d'utilisateurs simultanés au sein du groupe d'unité après avoir achever la phase de "debug".

8. Gestion correcte des sessions HTTP (cookies)

Si l'on souhaite tester une génération dynamique de pages web du côté serveur (ex : pages PHP ou JSP ou ASP ou autres) , de façon à ce que JMeter se comporte exactement comme un navigateur internet au niveau de la gestion des sessions utilisateurs, il faut **absolument ajouter** la configuration "**Gestionnaire de cookies HTTP**" en dessous du niveau "groupe d'unités" .

Rien d'important n'est à paramétrer à ce niveau, mais le "**Gestionnaire de cookies HTTP**" doit simplement être présent .



9. Successions de requêtes http liées entre elles

Un utilisateur utilise généralement une application web en passant par une suite logique de pages (accueil , menu1, menu2 , sélection/recherche , résultats , action , statut ,).

Ces différentes pages activées sont souvent liées entre elles par un ou plusieurs champs cachés (véhiculant un id "utilisateur" ou bien "un état de session" ou "...").

Ces "id" sont générés dynamiquement et ne sont donc pas constants (différents pour chaque utilisateur et pour chaque session).

JMeter offre heureusement un moyen pour "récupérer et ré-injecter" ces "id" véhiculés sous forme de champs cachés **via un processeur d'extraction d'expressions régulières** (ou bien Xpath).

9.1. Principe du "repost-hidden" (sur pages jsp)

repost_hidden.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>repost_hidden</title>
</head>
<%
String chX=request.getParameter("x");
String chLast=request.getParameter("last");
int x=1;
int last=1;
if(chX!=null) x=Integer.parseInt(chX);
if(chLast!=null)
```

```

    try{ last=Integer.parseInt(chLast);}
    catch (NumberFormatException ex){ last=0; }
int y=last*x;
%>
<body>
    <form>
        x=<input type="text" name="x" /> <br/>
        <input type="hidden" name="last" value="<%=y%>" />
        <input type="submit" value="mult by last value"/>
    </form>
</body>
</html>
x=<%=chX%><br/>
last=<%=chLast%><br/>
y=last*x=<%=y%> (new last)
</body>
</html>

```

Cet exemple de page jsp montre l'utilisation d'un champ caché permettant de mémoriser une certaine valeur entre plusieurs appels consécutifs émis depuis le même utilisateur.

Résultats des ré-appels successifs:

x=3 mult by last value x=null last=null y=last*x=1 (new last)	x=2 mult by last value x=3 last=1 y=last*x=3 (new last)	x= mult by last value x=2 last=3 y=last*x=6 (new last)
--	--	---

Nouvelle valeur de y = ancienne valeur multipliée par le x saisi .
L'ancienne valeur (last) est véhiculée via un champ caché (aller/retour "serveur-client-serveur").

```
<input type="hidden" name="last" value="1 , 3 puis 6"/>
```

Test JMeter associé:



Paramétrage de la première requête (en mode GET)

Requête HTTP

Nom : Req HTTP (get)

Commentaires :

Serveur web

Nom ou adresse IP : localhost Port : 8080

Requête HTTP

Protocole (défaut http) : Méthode : GET Encodage du c

Chemin : testJmeter/repost_hidden.jsp

☒ Rediriger automatiquement ☐ Suivre les redirections ☒ Connexions persistantes

Envoyer les paramètres avec la requête :

Nom :	Valeur :

Paramétrage de l'extracteur d'expression régulière (post response processor) :

Extracteur Expression régulière

Nom : Extracteur Expression régulière

Commentaires :

Champs réponse à cocher:

☒ Corps ☐ Corps (non échappé) ☐ Entêtes ☐ URL ☐ Code de

Nom de référence : last

Expression régulière : <input type="hidden" name="last" value="(.)"/>

Canevas : \$1\$

Correspond au num. (0 pour Aléatoire) : 1

Valeur par défaut : 0

==> la valeur du champ caché est ainsi récupérée dans la variable "last" ==> \${last}

Cette valeur est ensuite ré-injectée dans les requêtes ultérieures:

Requête HTTP

Nom : Req HTTP (repost)

Commentaires :

Serveur web

Nom ou adresse IP : localhost Port : 8080

Requête HTTP

Protocole (défaut http) : Méthode : POST Encodage :

Chemin : testJmeter/repost_hidden.jsp

☒ Rediriger automatiquement ☐ Suivre les redirections ☒ Connexions persistantes

Envoyer les paramètres avec la requête :

Nom :	Valeur :
x	2
last	\${last}

10. Configurations diverses

10.1. Test de charge (SQL) vers un serveur de base de données

NB: **JMeter** peut également effectuer des tests de montée en charge sur un SGBDR (MySQL , Oracle , ...) . Il faut penser à ajouter le driver JDBC adéquat au CLASSPATH de JMeter [*Archives "jar" à déposer dans jakarta-jmeter-2.x.y\lib*]. Ce type de tests (**JDBC Request**) est très pratique pour effectuer des ajustements au niveau des pools de connexions JDBC .

10.2. Variété au niveau des requêtes pour plus de réalisme

Si l'on déclenche en boucle une requête avec toujours les mêmes valeurs de paramètres en entrée, le serveur peut éventuellement optimiser ses traitements internes (via des caches) et retourner extrêmement rapidement la même réponse. Pour mieux simuler des clients simultanés envoyant des requêtes un peu différentes on a alors besoin d'injecter des valeurs variables dans les requêtes.

L'introduction d'un **contrôleur logique de type forEach** (en tant que *parent d'une requête*) permet de répéter celle-ci avec une valeur de paramètre variable (exemple: `${forEachOutputVar}`).

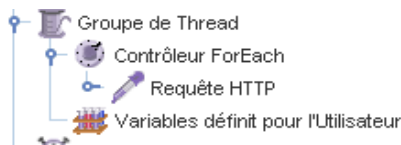
Requête HTTP

Protocole: Méthode: ☒ GET ☐ POST

Chemin : testTuningWeb/get_log_list ☐ Rediriger Automatiquement ☒ Suivre

Envoyer les Paramètres Avec la Requête:

Nom:	Valeur
log_level	\${forEachOutputVar}



Contrôleur ForEach

Nom:

Préfix du paramètre en entrée

Nom du paramètre en sortie

La boucle ForEach sera effectuée sur les variables suivantes (*_1, _2, _3, ..., _n*) :

Variables définit pour l'Utilisateur

Nom:

Variables définit pour l'Utilisateur

Nom:	Valeur
forEachInputVar_1	TRACE
forEachInputVar_2	WARN

Le Listener "**voir l'arbre des résultats**" est dans ce cas très pratique pour contrôler les requêtes et les réponses .

10.3. Test JDBC

Copier le(s) driver(s) JDBC nécessaires (ex : mysql-connector-j....jar , postgresql-....jar) dans le sous répertoire **lib** de JMeter .

Thread group , JdbcConfiguration avec url jdbc et name of datasource/connectionPool
JdbcRequest avec same name of datasource/connectionPool ...

Ordre de grandeur pour mysql : min=5ms , average: 15ms

Ordre de grandeur pour postgresql :

10.4. Plugins utiles

JMeter peut être enrichi par certains plugins.

Le site officiel de JMeter comporte quelques paquets de plugins.

Le zip "**JmeterPlugins-Extras-1.2.0.zip**" (qu'il suffit d'extraire (en ajout) dans le répertoire de JMeter) comporte quelques plugins intéressants (dont "**TPS vs Threads**" et "**ResponseTime vs Threads**" .

NB : bien penser à cliquer sur "*configurer*" et cocher "*nombre d'unités actives*"

et dimensionner au mieux la période de chauffe (ramp up period) du groupe d'unités pour que les threads puissent démarrer progressivement (via "plugin "*Stepping Thread Group*" ou ...).

10.5. JMeter à la source:

Pour télécharger **JMeter** ou approfondir certains de ses aspects , l'url de départ est :
<http://jmeter.apache.org/>

ANNEXES

XXVIII - Introspection, annotations , ...

1. Rappels sur énumérations

Il est maintenant possible de créer des énumérations comme en C/C++.

Exemple de déclaration :

```
public class Cxx {
    public enum Jour { DIMANCHE , LUNDI, MARDI, ..., SAMEDI };
```

```
    private Jour jour = Jour.DIMANCHE; // valeur par défaut ici.
```

```
    public Jour getJour(){ return this.jour; }
    public void setJour(Jour jour) { this.jour = jour; }
}
```

Le mot clef **enum** est maintenant réservé. Son utilisation nécessite l'option de compilation **-source 1.5** (ou 1.6 , 1.7) du compilateur javac .

- **Jour** est ainsi considéré comme un **type** de données . Il s'agit d'une classe implémentant implicitement les interfaces **Serializable** et **Comparable**. Cette classe Java d'énumération héritant implicitement de **Object** surcharge de façon adéquat les méthodes *toString()* , *equals()* et *hashCode()* .
- Les éléments internes de l'énumération sont codés comme des constantes entières statiques (valeurs = 0 ,1 ,) ==> méthode **ordinal()**
- Toute classe d'énumération (comme ici Jour) comporte une méthode statique **.values()** retournant le tableau des valeurs énumérables ainsi qu'une méthode **valueOf (String name)** retournant la valeur d'une constante selon son nom sous forme de chaîne de caractères.

exemple:

```
for (Jour j : Jour.values() )
    System.out.println(j + " [" + j.ordinal() + "]" );
```

```
....
objX.setJour(Jour.LUNDI);

Jour j = objX.getJour();

switch(j){
    //NB: bizarrement pas de case Jour.LUNDI  mais case LUNDI
    case LUNDI:  System.out.println("Lundi c'est ravioli"); break;
    case MARDI: System.out.println("Mardi c'est brocoli"); break;
    ...
}
```

2. Introspection (java.lang.reflect)

2.1. Potentiel de l'introspection

Introspection

Le package `java.lang.reflect` permet d'effectuer une **introspection** des classes java. L'introspection consiste à *demander aux classes de dresser la liste de leurs attributs et méthodes*.

Cette **faculté d'auto-analyse** qu'offre les mécanismes internes du langage Java est pour l'instant inexistante en C++ .

➔ L'introspection est un **gros point fort de Java** .

➔ Ceci *permet d'automatiser entièrement des opérations de bas niveaux (sauvegarde / restauration de valeurs, ...)*.

```
import java.lang.reflect.*;

Class c = Class.forName(nomClasse);
Field[] tabChamps = c.getDeclaredFields();
Method[] tabMethods = c.getDeclaredMethods();
+ boucle "for(i=0; i < tabXxxx.length; i++) ..."
```

Deux grandes variantes du point de départ :

```
Class<?> c = Class.forName("nomPackage.NomClasseJava") ;
```

ou bien

```
Class<?> c = objectInstance.getClass() ;
```

`java.lang.Class<T>` est une **méta-classe** : une classe très spéciale dont une instance décrit la structure d'une classe ordinaire.

2.2. Exemple d'introspection

Cet exemple montre comment générer automatiquement des lignes de fichier .csv depuis un objet java quelconque (Devise, Produit, Strat, Personne, ...) :

```
package util;
import java.io.FileOutputStream; import java.io.IOException;
import java.io.PrintStream; import java.lang.reflect.Field;
import java.util.List;

public class MyCsvUtil {

    public static String writeValuesAsCsvString(Object obj){
        String csvString=null;
        Class<?> c = obj.getClass(); //meta description de la classe de l'objet obj
        try {
            Field[] tabChamps = c.getDeclaredFields();
            for(Field f : tabChamps) {
                f.setAccessible(true); //pour accéder aux parties privées
                String fieldValue= f.get(obj)==null?null:f.get(obj).toString();
                csvString=(csvString==null)?fieldValue : csvString + ";" +fieldValue;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return csvString;
    }

    public static String writeFieldNamesAsCsvString(Object obj){
        String csvString=null;
        Class<?> c = obj.getClass(); //meta description de la classe de l'objet obj
        try {
            Field[] tabChamps = c.getDeclaredFields();
            for(Field f : tabChamps) {
                String fieldName= f.getName();
                csvString=(csvString==null)?fieldName:csvString + ";" +fieldName;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return csvString;
    }

    public static void writeValuesAsCsvFile(List<?> col,String fileName){
        if(col.isEmpty()) return;//fin fonction sans rien faire
        try (PrintStream ps = new PrintStream(new FileOutputStream(fileName))){
            Object firstObj = col.get(0);
            String ligneEnteteAuFormatCsv= writeFieldNamesAsCsvString(firstObj);
            ps.println(ligneEnteteAuFormatCsv);
            for(Object obj:col) {
                String ligneValeursAuFormatCsv= writeValuesAsCsvString(obj);
                ps.println(ligneValeursAuFormatCsv);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        //try_with_autoCloseable resource
    }
}
```

Le code ci dessus analyse automatiquement les structures des classes java *Produit* et *Stat* héritant toutes les deux de *Object* .

```
MyCsvUtil.writeValuesAsCsvFile(listeProduits,"produits.csv");
```

génère automatiquement

```
id;label;prixHt;tauxTva  
1;stylo bille rouge;1.6;20  
2;grand classeur;4.6;20  
3;produitXyz;4.4;10  
4;gomme;2.1;20
```

```
MyCsvUtil.writeValuesAsCsvFile(listeStats,"stats.csv");
```

génère automatiquement

```
label;somme;moyenne;ecartType  
statHt;12.7;3.175;1.3386093530227552  
statTtc;14.8;3.7;1.5143315356948757
```


2.3. Introspection avec annotations

L'annexe sur les annotations permettra de mieux comprendre l'exemple ci-dessous.

CsvIgnore.java

```
package util;
import java.lang.annotation.Documented; import java.lang.annotation.ElementType;
import java.lang.annotation.Retention; import java.lang.annotation.RetentionPolicy; import java.lang.annotation.Target;

@Documented
@Target(ElementType.FIELD) //utilisable au dessus d'un field/attribut
@Retention(RetentionPolicy.RUNTIME) //conservee dans code compile et accessible au runtime
public @interface CsvIgnore {
    //...
}
```

```
...
import com.fasterxml.jackson.annotation.JsonIgnore;
import util.CsvIgnore;

public class Stat {
    private String label;
    private Double somme;
    private Double moyenne;

    @CsvIgnore
    @JsonIgnore
    private Double ecartType;
    ...
}
```

```
...
Field[] tabChamps = c.getDeclaredFields();
for(Field f : tabChamps) {
    CsvIgnore annotCsvIgnore = f.getAnnotation(CsvIgnore.class);
    //si annotCsvIgnore==null , @CsvIgnore n'a pas été placé au dessus de f
    if(annotCsvIgnore==null) {
        String fieldName= f.getName();
        csvString=(csvString==null)?fieldName:csvString + ";" +fieldName;
    }
}
....
```

Fichier **stats.csv** généré en ignorant la partie ecartType :

```
label;somme;moyenne
statHt;12.7;3.175
statTtc;14.8;3.7
```

3. Annotations : Présentation et intérêts

Les **annotations** (disponibles depuis java 5) sont des *informations textuelles (ressemblant un peu à des commentaires)* qui sont *insérées au sein du code source (juste au dessus d'une classe , d'une méthode ou d'un attribut)* .

Avant java 5 , dans les anciennes versions 1.3 , 1.4 on utilisait des commentaires xdoclet .

Ces **annotations** sont quelquefois bien **utiles** car elles **pourront** (selon leurs portées) **être ultérieurement analysées par** :

- un **processeur** spécialisé (à écrire) et qui **va générer dynamiquement de nouvelles classes java durant la phase de compilation**
- l'**api de réflexion/introspection** de façon à *paramétrer certains mécanismes génériques* (persistance?, cryptage?,) au runtime .
-

On parle quelquefois en terme de **méta-programmation** lorsque l'on à recours aux annotations dans le cycle de développement.

Syntaxe des annotations:

NB: L'annotation porte toujours sur l'élément qui suit (sur la ligne suivante!).

@MonAnnotationSansParametre

```
public class Cxx { ...}
```

@AutreAnnotationSansParametre

```
public int méthodeX { ...}
```

@AnnotationAvecUneSeuleValeur("valeur_unique_parametre")

```
public class Cxx { ...}
```

@AnnotationAvecParamtres(param1="valeur_param1", param2=VALEUR2_CONTANTE)

```
public class Cxx { ...}
```

4. Annotations et méta-annotations prédéfinies

4.1. Annotations standards (interprétées par le compilateur):

<i>Annotations standards</i>	<i>Significations</i>
@Deprecated	Éléments (méthode , attribut, ...) devenu obsolète et qui ne devrait plus être utilisé. <u>NB</u> : l'utilisation conjointe du <i>commentaire javadoc @deprecated</i> permet en plus d'indiquer la raison et une suggestion de remplacement:

Annotations standards	Significations
	<pre>/** * @deprecated pas bien car ... , utiliser à la place **/ @Deprecated</pre>
@Override	Pour que le compilateur vérifie que la méthode qui suit est bien une redéfinition d'une méthode héritée existante (et pas une nouvelle opération) ==> ceci permet de détecter des erreurs sur la signature.
@SuppressWarnings	Pour demander au compilateur d'ignorer certains Warnings sur l'élément (classe,...) qui suit sans pour autant ignorer les les warnings sur le reste de l'application --> ex: <code>@SuppressWarnings({"deprecation","unchecked"})</code>

4.2. Méta-annotations (Annotations sur annotations):

Nécessite =====> `import java.lang.annotation.*;`

exemple:

@Documented

`public interface @MyDocumentedAnnotation { ...}`

Méta-Annotations	Significations
@Documented	L'annotation doit apparaître dans la documentation générée par javadoc ou ...
@Inherit	L'annotation sera automatiquement héritée par les sous classes (@Inherit ne peut être utilisée que sur annotation de classe – ce n'est pas utilisable sur une annotation d'interface).
@Retention(...)	Pour indiquer la portée (ou durée de vie) de l'annotation. RetentionPolicy.SOURCE (code source uniquement) RetentionPolicy.CLASS (dans .class également mais pas pris en compte par la JVM , c'est la valeur par défaut). RetentionPolicy.RUNTIME (vu par la JVM et l'api "reflection").
@Target(...)	Pour que l'annotation ne soit utilisable que devant une ou plusieurs catégories d'éléments (ex: classe , méthode ,). Si une annotation n'est pas bien utilisée (erreur de @Target) ==> Erreur du compilateur. Si la méta-annotation @Target n'est pas précisée , l'annotation peut être utilisée sur n'importe quel élément . ElementType.ANNOTATION_TYPE (devant annotation) ElementType.CONSTRUCTOR , ElementType.FIELD , ElementType.LOCAL_VARIABLE , ElementType.METHOD , ElementType.PACKAGE

Méta-Annotations	Significations
	ElementType.PARAMETER
	ElementType.TYPE (devant classe , interface ou enum).

5. Création de nouvelles annotations

Une **nouvelle annotation** se code comme une **interface spéciale** ayant un **nom commençant par le caractère @** .

Une interface d'annotation est très souvent précédée par une ou plusieurs méta-annotations.

D'autre part, les éventuelles propriétés (implicitement "public") d'une annotation se codent comme des méthodes (implicitement abstraites) de l'interface d'annotation.

Le nom implicite d'une unique propriété est "**value**=" ==> **value()** ;

Exemples:

```
import java.lang.annotation.Documented;
import java.lang.annotation.Retention;
import static java.lang.annotation.RetentionPolicy.SOURCE;
```

```
@Documented
@Retention(SOURCE)
public interface @Important {
}
```

```
@Documented
@Retention(SOURCE)
public interface @TODO {
    /** Message décrivant la tâche à effectuer. */
    String value();
    /** Niveau de criticité de la tâche (défaut : NORMAL). */
    Level level() default Level.NORMAL;
    /** Enumération des différents niveaux de criticités. */
    public static enum Level { MINEUR, NORMAL, IMPORTANT };
}
```

Attention ! Les attributs d'une annotation n'acceptent que les éléments suivants :

- Un type primitif (**boolean**, **int**, **float**, etc.).
- Une chaîne de caractères (**java.lang.String**).
- Une référence de classe (**java.lang.Class**).
- Une Annotation (**java.lang.annotation.Annotation**).
- Un type énuméré (**enum**).
- Un tableau à une dimension d'un des types ci-dessus.

Toutes les annotations héritent implicitement de l'interface **java.lang.annotation.Annotation**.

6. Insertion d'annotations au sein d'un code source

```
...
@Important
public class Cxx {
    /* ... */
}
```

```
..
@TODO(value="La gestion des exceptions est ...", level=NORMAL)
public void methode1() {
    /* ... */
}

// étant donnée que Level.NORMAL est la valeur par défaut de la propriété level
// on peut également écrire:

@TODO(value="La gestion des exceptions est à améliorer ...")
public void methode1() {
    /* ... */
}

// étant donnée que value est le nom (par défaut) d'une unique propriété, on peut également écrire:

@TODO("La gestion des exceptions est à améliorer ...")
public void methode1() {
    /* ... */
}
```

7. Accès aux annotations (de rétention RUNTIME) via l'introspection de java >=5

L'API de Réflexion (java.lang.reflect) a été étendue en version 1.5 de façon à supporter les annotations. Pour cela, les classes **Package**, **Class**, **Constructor**, **Method** et **Field** possèdent quatre nouvelles méthodes décrites dans l'interface **AnnotatedElement** :

- **getAnnotation(Class<A>)** qui retourne l'annotation dont le type est passé en paramètre (ou *null* si cette annotation n'est pas présente).
- **getAnnotations()** qui retourne un tableau comportant une instance de toutes les annotations de l'élément.
- **getDeclaredAnnotations()** qui retourne un tableau comportant une instance de toutes les annotations directement présentes sur l'élément (c'est à dire sans les annotations héritées de la classe parente). Rappel: Seules les **Class** peuvent hériter d'une annotation.
- **isAnnotationPresent(Class<A>)** qui retourne un booléen indiquant si l'annotation dont le type est passé en paramètre est présente sur l'élément ou non. Cette méthode est surtout utile pour les annotations marqueurs (sans attribut).

Exemple:

```
// Instanciation de l'objet:  
Exemple objet = new Exemple();  
  
// On récupère la classe de l'objet :  
Class<Exemple> classInstance = objet.getClass();  
  
// On regarde si la classe possède une annotation :  
MonAnnotation annotation = classInstance.getAnnotation(MonAnnotation.class);  
  
if (annotation!=null) {  
    System.out.println ("MonAnnotation : " + annotation.value() );  
}
```

8. Analyse et traitement des annotations de rétention

« SOURCE »

NB : l'ancien outil **APT** (**A**nnotation **P**rocessing **T**ool) du **JDK 5.0** a été remplacé dès java 6, java 7 par JSR 269 et APT est maintenant **deprecated** .

Exemple (packagé comme 3 projets distincts / maven multi-modules) :
le code généré sera une classe héritant de **AbstractSimpleSingleton** permettant d'accéder à certains singletons (pour classe annotées via *@BasicSingleton*)

projet my_annot :

```
package tp.annotations;
import java.lang.annotation.ElementType; import java.lang.annotation.Target;
import java.lang.annotation.Retention;      import java.lang.annotation.RetentionPolicy;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface BasicSingleton {
}
```

```
package tp.basic;
/* interface SimpleSingleton au sens SimpleSingleton_Access */
public interface SimpleSingleton {
    <T> T getInstance(Class<T> c) ; //accès à l'unique instance
    <T> int getCount(Class<T> c) ; // nombre d'accès demandés/effectués

    static SimpleSingleton implementation() {
        return AbstractSimpleSingleton.uniqueImplementationInstance();
    }
}
```

```
package tp.basic;
import java.util.HashMap; import java.util.Map;

public abstract class AbstractSimpleSingleton implements SimpleSingleton{

    protected Map<String,Object> mapClassInstance = new HashMap<>();
    protected Map<String,Integer> mapClassCount = new HashMap<>();

    public abstract void initMapClassInstance();
    /* //typical code in subclass:
    {
        //mapClassInstance.put("...Xxx",new Xxx());
        //mapClassInstance.put("...Yyy",new Yyy());
    } */
```

```

AbstractSimpleSingleton(){ this.initMapClassInstance(); }

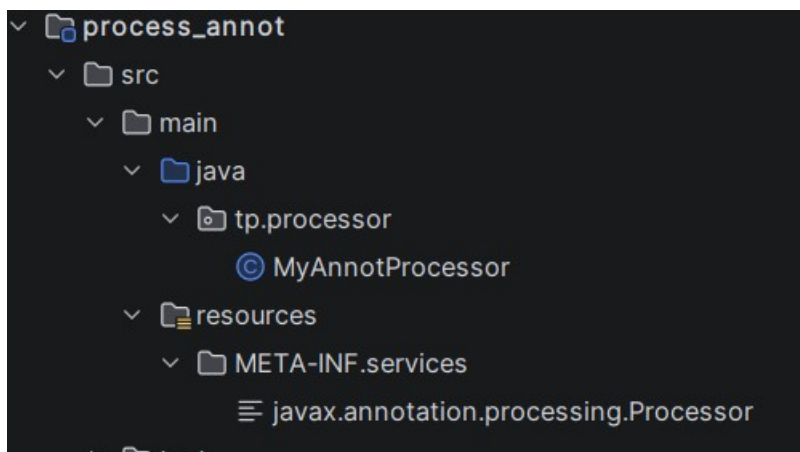
static SimpleSingleton uniqueSimpleSingleton=null;
static SimpleSingleton uniqueImplementationInstance() {
    if(uniqueSimpleSingleton==null) {
        try {
            //NB: tp.basic.MySimpleSingleton will be generated by annotations processor
            // at compile time
            Class<SimpleSingleton> c = (Class<SimpleSingleton>)
                Class.forName("tp.basic.MySimpleSingleton");
            uniqueSimpleSingleton = c.getDeclaredConstructor().newInstance();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
    return uniqueSimpleSingleton;
}

@Override
public <T> T getInstance(Class<T> c) {
    String className = c.getName();
    Integer count = mapClassCount.get(className);
    if(count==null) count=new Integer(1);
    else count=new Integer(count+1);
    mapClassCount.put(className,count);

    return (T) mapClassInstance.get(className);
}

@Override
public <T> int getCount(Class<T> c) {
    Integer count = mapClassCount.get(c.getName());
    return (count!=null)?count.intValue():0;
}
}

```

projet process_annot :

Fichier indiquant la classe du projet (futur .jar) qui implémente un processeur d'annotations :

META-INF/services/javax.annotation.processing.Processor

```
tp.processor.MyAnnotProcessor
```

```
package tp.processor;

import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.TypeElement;
import javax.tools.JavaFileObject;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

/*
NB: this processor fully qualified class name must/may be register in
META-INF/services/javax.annotation.processing.Processor text file
(in src/main/resources)

NB: tp.basic.MySimpleSingleton will be generated by this annotations processor at compile time
*/

//@SupportedAnnotationTypes("tp.annotations.*")
@SupportedAnnotationTypes("tp.annotations.BasicSingleton")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
//@SupportedOptions: to register allowed custom options that may be passed through the command line.
public class MyAnnotProcessor extends AbstractProcessor {

    public MyAnnotProcessor() {
        super();
    }

    public List<String> singletonClassNames = new ArrayList();

    @Override
    public boolean process(Set<? extends TypeElement> annotations,
        RoundEnvironment roundEnv) {

        //boucle sur annotations supportées (1 ou plusieurs) :
        for (TypeElement annotation : annotations) {

            //récupération de l'ensemble des éléments sur lesquels l'annotation à été placée dans le code source :
```

```

Set<? extends Element> annotatedElements
    = roundEnv.getElementsAnnotatedWith(annotation);

System.out.println("for annotation= " + annotation.toString()
    + " annotatedElements="+annotatedElements);
for(var e : annotatedElements){
    this.singletonClassNames.add(e.toString());
}
}

try {
    this.writeMySingletonImplementationJavaClass();
} catch (Exception e) {
    //throw new RuntimeException(e);
    //e.printStackTrace();
    System.err.println(e.getMessage());
}

return true;
}

private void writeMySingletonImplementationJavaClass() throws IOException {
    String mySingletonClassName = "tp.basic.MySimpleSingleton";
    File filer = this.processingEnv.getFiler();
    //javax.annotation.processing.Filer is useful for creating java source file
    JavaFileObject mySimpleSingletonFile = filer
        .createSourceFile(mySingletonClassName);
    try (PrintWriter out = new PrintWriter(mySimpleSingletonFile.openWriter())) {
        out.println("package tp.basic;\n");

        out.println("public class MySimpleSingleton extends AbstractSimpleSingleton {}");

        out.println("\t MySimpleSingleton() { super(); \n\t} \n\n");

        out.println("\t public void initMapClassInstance(){}");
        for(String singletonClassName : this.singletonClassNames) {
            out.println("\t\t mapClassInstance.put(\"\" + singletonClassName + "\",new " +
singletonClassName + "());");
        }
        out.println("\t");

        out.println("}");
    }
}
}

```

Projet using annot:***pom.xml***

```

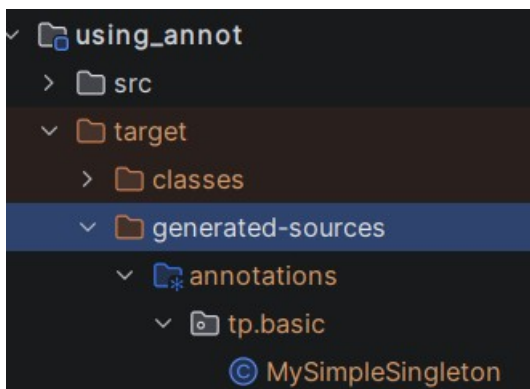
<?xml version="1.0" encoding="UTF-8"?>
<project ....>
  <dependencies>
    <dependency>
      <groupId>tp</groupId>
      <artifactId>my_annot</artifactId>
      <version>0.0.1-SNAPSHOT</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>${project.artifactId}</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.1</version>
        <configuration>
          <annotationProcessorPaths>
            <path>
              <groupId>tp</groupId>
              <artifactId>process_annot</artifactId>
              <version>0.0.1-SNAPSHOT</version>
            </path>
          </annotationProcessorPaths>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>

```

après **mvn clean package**

→



code généré automatiquement durant la phase de compilation :

```
package tp.basic;

public class MySimpleSingleton extends AbstractSimpleSingleton {
    MySimpleSingleton(){ super(); }
    public void initMapClassInstance(){
        mapClassInstance.put("tp.computing.AverageComputing",new tp.computing.AverageComputing());
        mapClassInstance.put("tp.computing.SumComputing",new tp.computing.SumComputing());
    }
}
```

*Utilisation :***@BasicSingleton**

```
public class AverageComputing {
    public double average(double a , double b){ return (a+b)/2; }
}
```

```
public class MinComputing {
    public double min(double a , double b){ return Math.min(a,b); }
}
```

@BasicSingleton

```
public class SumComputing {
    public double add(double a , double b){ return a+b; }
}
```

```
public class MyApp {
    public static void main(String[] args) {
        SumComputing sc =
            SimpleSingleton.implementation().getInstance(SumComputing.class);
        System.out.println("5+6=" + sc.add(5,6));
        m2();    m3();
        System.out.println("nombre d'appels à getInstance() pour SumComputing: "
            +SimpleSingleton.implementation().getCount(SumComputing.class));
    }

    public static void m2(){
        SumComputing sc = SimpleSingleton.implementation().getInstance(SumComputing.class);
        System.out.println("3+4=" + sc.add(3,4));
    }
    public static void m3(){
        AverageComputing ac =
            SimpleSingleton.implementation().getInstance(AverageComputing.class);
        System.out.println("moyenne(3,5)=" + ac.average(3,5));
    }
}
```

=> 5+6=11.0 3+4=7.0 moyenne(3,5)=4.0

nombre d'appels à getInstance() pour SumComputing: 2

XXIX - Switch expression, pattern matching, record

1. Switch expressions

La nouvelle forme du **switch()** avec des "**lambda**" était apparue dès les jdk 1.12 et 1.13 en mode "preview". C'est enfin devenu une nouvelle fonctionnalité "standard" à partir de la version **1.14**

1.1. expression d'un switch/case avec des "lambda"

```
int dayOfWeek = 4;
System.out.print("dayOfWeek="+dayOfWeek + " -> ");
switch(dayOfWeek) {
    case 1 -> System.out.println("monday");
    case 2 -> System.out.println("tuesday");
    case 3 -> System.out.println("wednesday");
    case 4 -> System.out.println("thursday");
    case 5 -> System.out.println("friday");
    case 6 -> System.out.println("saturday");
    case 0 , 7 -> System.out.println("sunday");
    default -> System.out.println("unknown");
}
```

Résultat:

```
dayOfWeek=4 -> thursday
```

1.2. switch en tant qu'expression retournant un résultat

```
String dayName="vendredi";
System.out.print("dayName="+dayName + " -> ");
int dayNumber =
    switch(dayName) {
        case "lundi" , "monday" -> 1;
        case "mardi" , "tuesday" -> 2;
        case "mercredi" , "wednesday" -> 3;
        case "jeudi" , "thursday" -> 4;
        case "vendredi" , "friday" -> 5;
        case "samedi" , "saturday" -> 6;
        case "dimanche" , "sunday" -> 7;
        default -> 0;
    };
System.out.println("dayNumber="+dayNumber);
```

Résultat :

```
dayName=vendredi -> dayNumber=5
```

1.3. switch avec expressions complexes et mot clef yield

Lorsqu'une expression doit nécessiter plusieurs instructions pour être calculée, on peut englober celles-ci par un bloc d'accolades et préciser la valeur calculée/retournée via le mot clef "yield" (ressemblant à un "return" mais de niveau switch et pas de niveau fonction) .

```
int value = (int) (Math.random() * 14);
String result = switch( value ) {
    case 0, 2, 4, 6, 8 -> {
        double racine = Math.sqrt( value );
        yield "chiffre pair dont la racine carré vaut " + racine;
    }
    case 1, 3, 5, 7, 9 -> {
        double carre = value * value;
        yield "chiffre impair dont le carré vaut " + carre;
    }
    default -> "ce n'est plus un chiffre, mais un nombre";
};
System.out.println("value=" + value + "->" + result );
```

Résultats :

value=2->chiffre pair dont la racine carré vaut 1.4142135623730951

value=13->ce n'est plus un chiffre, mais un nombre

value=3->chiffre impair dont le carré vaut 9.0

...

2. Pattern Matching instanceof

En preview en version 14, 15 et standardisé en version 16 et 17/LTS .

```
public class TestPatternMatchingInstanceOfApp {

    public static void main(String[] args) {
        String s = "abc";
        oldStyleWithoutPatternMatchingInstanceOf(s);
        newStyleWithPatternMatchingInstanceOf(s);
    }

    public static void oldStyleWithoutPatternMatchingInstanceOf(Object obj) {
        if (obj instanceof String) {
        String str = (String) obj;
        int len = str.length();
        System.out.println("obj is as string of length="+len);
        }
    }

    public static void newStyleWithPatternMatchingInstanceOf(Object obj) {
        if (obj instanceof String str) {
        int len = str.length();
        System.out.println("obj is as string of length="+len);
        }
    }
}
```

C'est un **sucre syntaxique** (simplification de syntaxe) **bien pratique** .

3. TextBloc

Un **"TextBloc"** (depuis java 15) est une chaîne de caractère formulée comme un **bloc de texte sur plusieurs lignes**. Encadré par des `"""` (triple guillemet), un *textBloc* peut comporter des caractères quelconques qui n'ont plus besoin d'échappement.

Ceci est surtout utile pour le format JSON (nécessitant des `"` et pas de `'`) à l'intérieur d'une chaîne java délimitée par des `"`.

écriture sans textbloc et très peu lisible :

```
public static void oldStyleWithoutTextBloc() {
    String myJsonString
        = "{\r\n" + "\"username\" : \"JeanBon\", \r\n" + "\"country\" : \"France\" \r\n" + "}";
    System.out.println("myJsonString="+myJsonString);
}
```

équivalent bien plus lisible avec textBloc :

```
public static void newStyleWithTextBloc() {
    String myJsonStringAsTextBloc = """
        {
            "username" : "AlexTherieur",
            "country" : "Belgique"
        }
        """;
    System.out.println("myJsonStringAsTextBloc="+myJsonStringAsTextBloc);
}
```

→ affiche

```
myJsonStringAsTextBloc={
"username" : "AlexTherieur",
"country" : "Belgique"
}
```

```
public static void someTextBlocs() {
    //au minimum 2 lignes de code :
    //  """ suivi de newligne pour commencer
    //  texte quelconque au milieu
    //  """; pour finir
    String simpleTextBloc0 = """
        blabla""";
    System.out.println("simpleTextBloc0="+simpleTextBloc0);
    //affiche simpleTextBloc0=blabla
}
```



```

//pour bien controller l'intentation , il faut savoir que lorsque le compilateur va analyser
//le code source, il va supprimer le nombre minimum de tabulations qu'il y a
//au niveau de toutes les lignes de code (du textBloc sans tenir compte du tout début = "")
//Bonne pratique : au sens IDE (eclipse/IntelliJ/...) placer des tabulations pour indenter le code
//          au sens contenu final du textBloc , utiliser plutôt des espaces et sauts de lignes
//          placer la fin "" sur une ligne séparée
//          (comportant le nombre de tabulations qui seront explicitement supprimées du texte)
String textBloc3AvecIndentationControlee = ""
    {
        "username" : "AlexTherieur",
        "country" : "Belgique"
    }
    """;

System.out.println("textBloc3AvecIndentationControlee="+textBloc3AvecIndentationControlee);

//par défaut les espaces en fin de ligne sont considérés inutiles et ne sont pas conservés
//le caractère d'échappement \s permet d'explicitement demander à conserver les espaces en fin de ligne
String textBloc4AvecEspacesConserveEnFinDeLignes = ""
    blabla  \s""";

System.out.println("textBloc4AvecEspacesConserveEnFinDeLignes="
    +textBloc4AvecEspacesConserveEnFinDeLignes + "suiteApres");

}

```

```

public static void textBlocsWithFormattedParams() {
    String username="toto";
    int age=40;
    double height = 1.73;
    boolean ok=true;
    String jsonString = ""
        {
            "username" : "%s",
            "age" : %d,
            "height" : %s,
            "ok" : %b
        }
        """.formatted(username,age,String.valueOf(height),ok);
    //same syntax of String.format
    System.out.println("jsonString="+jsonString);
}

```

affiche

```

{ "username" : "toto",
  "age" : 40,
  "height" : 1.73,
  "ok" : true
}

```

4. Record (classe de données simplifiée pour DTO)

"Preview" en version 14 et 15 , les "record" sont standardisés depuis java 16 (et 17 LTS) .

//exemple de classe POJO/JavaBean classique (pour comparaison) :

```
public class OldCustomer{

    private Integer id;
    private String firstName;
    private String lastName;

    public OldCustomer() {      super();
    }

    public OldCustomer(Integer id, String firstName, String lastName) {
        super();
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
    }

    @Override
    public String toString() {
        return "OldCustomer [id=" + id + ", firstName=" + firstName
            + ", lastName=" + lastName + "]";
    }

    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }

    //+ autres get/set
    //+ hashCode()
    //+ equals()
}
```

classe équivalente avec annotations de lombok :

```
import lombok.*;

@Getter @Setter @ToString
@NoArgsConstructor @AllArgsConstructor
@EqualsAndHashCode
public class LombokCustomer {
    private Integer id;
    private String firstName;
    private String lastName;
}
```

Utilisation classique d'un javaBean codé avec ou sans lombok:

```
OldCustomer oldC1 = new OldCustomer(1,"jean","Bon");
System.out.println("oldC1 as POJO =" +oldC1.toString());
System.out.println("id of oldC1="+oldC1.getId());
oldC1.setFirstName("luc");
System.out.println("firstName of oldC1="+oldC1.getFirstName());
```

```
OldCustomer oldC2 = new OldCustomer();
System.out.println("oldC2="+oldC2.toString());
```

```
System.out.println("-----");
```

```
LombokCustomer lC1 = new LombokCustomer(1,"jean","Bon");
System.out.println("olC1 as Lombok POJO =" +lC1.toString());
System.out.println("id of lC1="+lC1.getId());
lC1.setFirstName("luc");
System.out.println("firstName of lC1="+lC1.getFirstName());
```

```
LombokCustomer lC2 = new LombokCustomer();
System.out.println("lC2="+lC2.toString());
```

Nouveau mot clef "record"

Le nouveau mot clef **record** permet de demander au compilateur de construire automatiquement une sorte de pseudo **POJO IMMUTABLE** (sans Setter possible !) et avec des accesseurs de type .xxx() ne respectant même pas les conventions .getXxx() !!!

Dans les détails : classe construite "final" héritant de **java.lang.Record** et avec des champs privés "final" et avec .equals()/.hashCode()/.toString() et des accesseurs de type .xxx() et allArgsConstructor .

NB :

- record ne peut pas être vu comme un remplacement de lombok car IMMUTABLE/final/read-only et car .xxx() plutôt que .getXxx()
- record doit plutôt être vu comme une sorte de petit DTO/VO très léger

Minimalist v1 (with no default constructor):

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
}
```

Utilisation :

```
CustomerRecord c1 = new CustomerRecord(1,"jean","Bon");
System.out.println("c1 as record =" +c1.toString());
System.out.println("id of c1="+c1.id()); //mais pas .getId() ni .id
c1.setFirstName("luc");
System.out.println("firstName of c1="+c1.firstName()); //mais pas .getFirstName() ni .firstName
```

```
public void setFirstName(String firstName) { this.firstName = firstName; }
est interdit/impossible car this.firstName est final/immutable !!!
```

// V2 (with explicit default constructor) :

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    public CustomerRecord() { this(0,null,null);}
    //possible mais pas souvent utile si final/immutable !!!
}
```

Utilisation :

```
CustomerRecord c2 = new CustomerRecord();
System.out.println("c2="+c2.toString());
//affiche ici CustomerRecord[id=0, firstName=null, lastName=null]
```

V3 (V2 + quelques *redefinitions* & *ajout de methodes*)

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    public CustomerRecord() { this(0,null,null);}
    public String fullName() { return firstName + " " + lastName;}
    public String lastName() { return lastName.toUpperCase(); }
}
```

V4 (V3 + new record constructor checking syntax)

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    //not useful default constructor:
    public CustomerRecord() { this(0,"unknown","unknown");}

    //new method:
    public String fullName() { return firstName + " " + lastName;}

    //old pojo style getter as a new complementary method :
    public String getFirstName() { return firstName; }

    //accessor redefinition :
    public String lastName() { return lastName.toUpperCase(); }

    //specific record new syntax for allArgsConstructor
    //partial redefinition with value(s) checking:
    public CustomerRecord {
        if (lastName == null || lastName.isBlank()) {
            throw new IllegalArgumentException("lastName is required");
        }
    }
}
```

```

public class TestRecordApp {

    public static void main(String[] args) {
        testUsefulRecordV1();
        testUsefulRecordV2();
        testUsefulRecordV3();
    }

    //private or public locally RECORD seems to be a good use case for "record" new concept :

    private record AddressV1(Integer number,String street,String zipCode,String town) {
        public String toJsonString() {
            return ""
                {
                    "number" : %d,
                    "street" : "%s",
                    "zipCode" : "%s",
                    "town" : "%s"
                }
                ""formatted(number,street,zipCode,town);
        }
    };

    public static void testUsefulRecordV1() {
        AddressV1 a1 = new AddressV1(12,"rueElle","75000" ,"Paris");
        System.out.println("a1="+a1.toString());
        System.out.println("a1="+a1.toJsonString());
    }

    //V2 avec compatibilité avec api jackson-databind (souvent utilisé par JEE , Spring, ...)
    //pratique pour définition de DTO/VO (à la volée)
    public record AddressV2(Integer number,String street,String zipCode,String town) {
    };

    public static void testUsefulRecordV2() {
        AddressV2 a1 = new AddressV2(12,"rueElle","75000" ,"Paris");
        System.out.println("a1="+a1.toString());
        try {
            ObjectMapper jacksonObjectMapper = new ObjectMapper();
            String a1JsonString = jacksonObjectMapper.writeValueAsString(a1);
            System.out.println("via jackson, a1JsonString="+a1JsonString);
        } catch (JsonProcessingException e) {
            e.printStackTrace();
        }
    }
}

```

Résultats :

```

a1=AddressV1[number=12, street=rueElle, zipCode=75000, town=Paris]
a1={

```

```
"number" : 12,
"street" : "rueElle",
"zipCode" : "75000",
"town" : "Paris"
}
```

```
a1=AddressV2[number=12, street=rueElle, zipCode=75000, town=Paris]
via jackson, a1JsonString={"number":12,"street":"rueElle","zipCode":"75000","town":"Paris"}
```

*//NB : Les records de java ne sont bien gérés qu'avec des versions récentes de jackson-databind .
 // la version 2.12.5 de **jackson-databind** supporte bien les records de java 17 et est compatible
 //avec une version récente de SpringBoot et SpringMVC (sur partie @RestController)*

```
public class Dto {
    public record Address(Integer number,String street,String zipCode,String town) {
    };

    public record Person(Integer id,String firstName,String lastName,Address address) {
        public Person(Integer id,String firstName,String lastName) {
            this(id,firstName,lastName,null);
        }
    };
}
```

```
public static void testUsefulRecordV3() {
    Dto.Person p1Dto = new Dto.Person(1,"jean","Bon" ,
                                     new Dto.Address(12,"rueElle","75000" ,"Paris"));
    System.out.println("\np1Dto="+p1Dto.toString());

    try {
        ObjectMapper jacksonObjectMapper = new ObjectMapper();
        String p1JsonString = jacksonObjectMapper.writeValueAsString(p1Dto);
        System.out.println("via jackson, p1JsonString="+p1JsonString);
        //-----
        Dto.Person p1BisDto=
            jacksonObjectMapper.readValue(p1JsonString,Dto.Person.class);
        System.out.println("via jackson, p1BisDto=clone de p1Dto="+p1BisDto.toString());
    } catch (JsonProcessingException e) {
        e.printStackTrace();
    }
}
```

Résultats :

p1Dto=Person[id=1, firstName=jean, lastName=Bon, address=Address[number=12, street=rueElle, zipCode=75000, town=Paris]]

via jackson, p1JsonString={"id":1,"firstName":"jean","lastName":"Bon","address":{"number":12,"street":"rueElle","zipCode":"75000","town":"Paris"}}

via jackson, p1BisDto=clone de p1Dto=Person[id=1, firstName=jean, lastName=Bon, address=Address[number=12, street=rueElle, zipCode=75000, town=Paris]]

Attention (restriction importante)

En interne les "record" sont codés comme des classes spéciales héritant de la classe abstraite `java.lang.Record`.

Etant donné que java ne supporte que l'héritage simple (via mot clef `extends`), **il n'est pas possible d'utiliser le mot clef `extends` de manière à ce qu'un record hérite d'un autre record**.

On peut cependant utiliser le mot clef **`implements`** entre un **record** et une (ou plusieurs) **interface(s)**.

Exemple :

```
public class MyRecords {  
    public interface Product {Long id(); String label(); Double price();}  
    public record ProductL0(Long id,String label,Double price) implements Product {  
    }  
  
    public interface ProductI1 extends Product {String features();}  
    public record ProductL1(Long id,String label,Double price,String features)  
        implements ProductI1{  
    }  
}
```

```
MyRecords.ProductL0 p0 = new MyRecords.ProductL0(1L,"productA", 12.5);
```

```
MyRecords.Product pRef = p0;
```

```
System.out.println(pRef.toString());
```

```
System.out.println("id=" + pRef.id() + " label=" + pRef.label() + " price=" + pRef.price());
```

Autre détails :

La méta classe **`java.lang.Class<T>`** comporte maintenant une nouvelle méthode booléenne **`isRecord()`**.

5. Sealed classes

"Preview" en versions 15,16 et standardisé en version 17 .

```
//NB: "SEALED" signifiant scellé en francais
//est une nouveauté du langage java permettant d'indiquer que seules certaines classes
//(précisées par permits ... , ... )
//auront le droit d'hériter de la classe ou interface actuelle
//cela ressemble à final class qui interdit carrément tout héritage

//sealed keyword can be used if option "enabled preview features for java 16"
public sealed interface AnimalDomestique permits Chat, Chien {
    public void sayHello();
    //...
}
// effet : seules les classes Chat et Chien auront le droit d'hériter de AnimalDomestique .

/*
//HERITAGE via extends ou implements INTERDIT via sealed sur AnimalDomestique :
final class Ce implements AnimalDomestique{
//...
}
*/
```

//déclarée final , la classe Chat ne peut pas avoir de sous classe

```
public final class Chat implements AnimalDomestique{

    @Override
    public void sayHello() {
        System.out.print("chat miaou - ");
    }

    public void ronronner() {
        System.out.println(" ronron ");
    }

}
```

//déclarée non-sealed , la classe Chien peut pas avoir des sous classes

```
public non-sealed class Chien implements AnimalDomestique{
    @Override
    public void sayHello() {
        System.out.print("chien wouf wouf - ");
    }

    public void aLaNiche() {
        System.out.println(" dodo niche ");
    }

}
```



```
public class ChienFou extends Chien {
    @Override
    public void sayHello() {
        System.out.print("chienFou wouf wouf grrr wouf grrr - ");
    }

    public void tournerEnRond() {
        System.out.println(" tourne pas rond dans sa tete ");
    }
}
```

```
public static void operationOnSealed(AnimalDomestique a) {
    if(a instanceof Chien chien) {
        chien.aLaNiche();
        if(chien instanceof ChienFou chienFou) {
            chienFou.tournerEnRond();
        }
    }
    if(a instanceof Chat chat) {
        chat.ronronner();
    }
}

public static void testSealed() {

    var animauxDomestiques = new ArrayList<AnimalDomestique>();
    animauxDomestiques.add(new Chat());
    animauxDomestiques.add(new Chien());
    animauxDomestiques.add(new ChienFou());

    animauxDomestiques.stream().forEach(a-> { a.sayHello(); operationOnSealed(a);});
}
```

affiche

```
chat miaou - ronron
chien wouf wouf - dodo niche
chienFou wouf wouf grrr wouf grrr - dodo niche
tourne pas rond dans sa tete
```

NB : Une interface scellée est implémentable comme un paquets de "record" .

Exemple :

```
public sealed interface Vivant {
    record Vertebre(String name, String p1 , String p2) implements Vivant {}
    record Invertebre(String name, String p3) implements Vivant {}
}
```

NB: Le principal intérêt des classes scellées de java 16/17 tient dans la possibilité d'utiliser un pattern matching au sein d'un switch/case de type d'objet sans partie "default" .

6. Pattern matching sur switch/case of object

//NB: still in "preview mode" in java 17-LTS !!!

//java --enable-preview --source 17

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <release>${java.version}</release>
    <source>${java.version}</source>
    <target>${java.version}</target>
    <compilerArgs>--enable-preview</compilerArgs>
  </configuration>
</plugin>
```

Exemple :

```
package tp.j15_16_17;

public class TestPatternMatchingSwitchPreviewApp {

  static double getDoubleUsingSwitch(Object o) {
    return switch (o) {
      case Integer i -> i.doubleValue();
      case Float f -> f.doubleValue();
      case Double d -> d.doubleValue();
      case String s -> Double.parseDouble(s);
      case null -> 0d;
      default -> 0d;
    };
  }

  static String getTypeAnimalDomestiqueAsString(AnimalDomestique a) {
    return switch (a) {
      case Chat chat -> "chat";
      case Chien chien -> "chien";
      //default -> "ni chat , ni chien";
    };
  }

  /*
```

Remarque importante :

*Ce switch/case avec pattern-matching de type nécessite absolument une partie "default" si AnimalDomestique n'est pas scellé (sans le mot clef sealed et ...)
et ne nécessite pas de partie "default" si AnimalDomestique est scellé*

Autrement dit l'intérêt principal des classes scellées tient dans la possibilité d'utiliser directement les différents types de classes concrètes (dérivant d'un même type abstrait scellé) au sein d'un switch/case sans default sans avoir besoin de gérer en parallèle une énumération à valeurs possibles fixes/finies.

```

*/
}

static String getTypeVivantFromRecord(Vivant v) {
    return switch (v) {
        case Vivant.Invertebre ive -> "invertebre";
        case Vivant.Vertebre ve -> "vertebre";
        //with default if Vivant is not sealed
    };
}

public static void main(String[] args) {
    System.out.println(getDoubleUsingSwitch("12.5")); //12.5
    System.out.println(getDoubleUsingSwitch(12.6)); //12.6
    System.out.println(getDoubleUsingSwitch(12.7f)); //12.699999809265137
    System.out.println(getDoubleUsingSwitch(12)); //12.0
    System.out.println(getDoubleUsingSwitch(null)); //0.0

    System.out.println(getTypeAnimalDomesticAsString(new Chat())); //chat
    System.out.println(getTypeAnimalDomesticAsString(new Chien())); //chien
    System.out.println(getTypeAnimalDomesticAsString(new ChienFou())); //chien

    System.out.println(getTypeVivantFromRecord(
        new Vivant.Invertebre("limace", "pas rapide"))); // invertebre
    System.out.println(getTypeVivantFromRecord(
        new Vivant.Vertebre("homme", "intelligent", "pas toujours sage"))); // vertebre
    }
}

```

7. utilitaire jpackage

L'utilitaire **jpackage** sert à construire des installeurs d'applications pour windows, linux ou mac.

Ça peut construire des fichier ".msi" pour windows.

package_with_jpackage.bat

```

cd /d "%~dp0"
REM this script should be run after mvn package or ...

set JAVA_HOME=C:\Program Files\Java\jdk-17
REM NB: on windows10 , with jdk17 , wix tools set is a required dependency of jpackage
REM wix311-binaries.zip can be download from https://github.com/wixtoolset
set WIX_HOME=C:\Prog\wix311
set PATH=%PATH%;%JAVA_HOME%\bin;%WIX_HOME%
REM type=msi or exe for windows , deb or rpm for windows , dmg or pkg on mac
set TYPE=msi
set NAME=JPackagedemoApp

```

```

set MAIN_CLASS=tp.MainClassForJPackageTestApp
set INPUT_JAR_DIR=./target
set MAIN_JAR=tp_java_8_14-0.0.1-SNAPSHOT.jar
set OPTIONS=--win-console --java-options '--enable-preview'

jpackage --input %INPUT_JAR_DIR% --name %NAME% --main-jar %MAIN_JAR%
--main-class %MAIN_CLASS% --type %TYPE% %OPTIONS%
REM fichier construit : JPackageDemoApp.msi

REM apres installation de JPackageDemoApp.msi
REM C:\Program Files\JPackageDemoApp\JPackageDemoApp.exe et runtime java

REM https://www.baeldung.com/java14-jpackage
REM https://www.devdungeon.com/content/use-jpackage-create-native-java-app-installers

```

8. Autres apports des versions récentes (12,...,17)

8.1. Helpful *NullPointerException* (JEP 358)

Depuis le java 14, les traces générées autour de *NullPointerException* sont beaucoup plus claires/explicites .

Exemple de code avec bug:

```

int[] arr = null;
arr[0] = 1;

```

Avec les jdk précédent (ex : v11) :

```

Exception in thread "main" java.lang.NullPointerException
at tp.MyClass.main(MyClass.java:27)

```

Maintenant, la log d'erreur indique plus clairement la source du problème:

```

java.lang.NullPointerException: Cannot store to int array because "arr" is null

```

9. Nouveau type de licence depuis java 17

Le JDK 17 D'Oracle De Nouveau Gratuit Pour Une Utilisation Commerciale

Le JDK Oracle est à nouveau disponible gratuitement pour une utilisation en production - sous la nouvelle "[Oracle No-Fee Terms et conditions](#)" (NFTC). Cette décision annule une [décision de 2018](#) de facturer l'utilisation en production du JDK d'Oracle et n'affecte pas la [distribution OpenJDK d'Oracle](#). La NFTC s'applique à la version 17 récemment publiée d'Oracle JDK et aux versions futures.

Autre raison de passer à java 17 : **java 17 est plus rapide/performant que java 11 .**

XXX - ClassLoader , weak references , modules

1. ClassLoader

1.1. Rôles d'un chargeur de classes

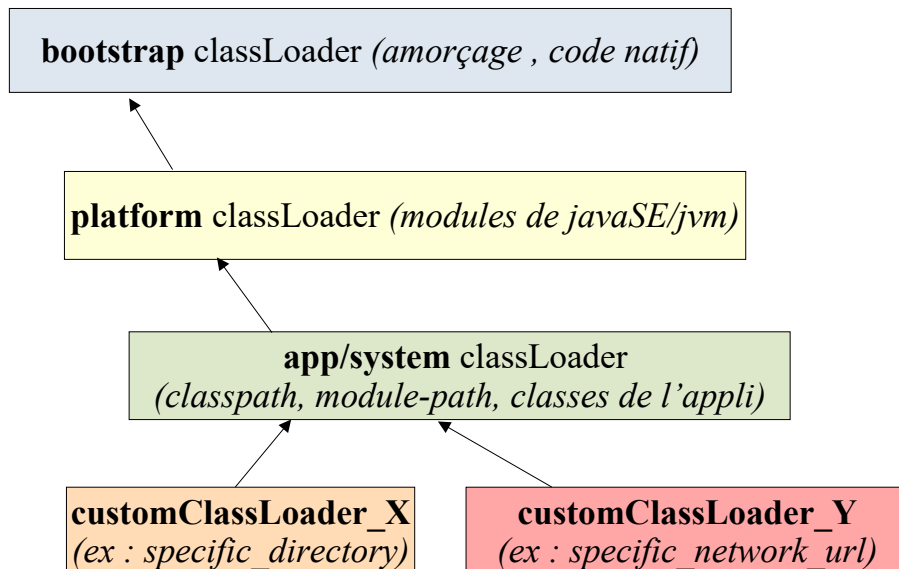
Un chargeur de classes (**classLoader**) est un élément technique de la JVM qui sert à **dynamiquement charger des classes java en mémoire** lorsque l'application java en a besoin.

Un chargeur de classes à deux grandes fonctions :

- **charger les classes en mémoire**
- **localiser une ressource à charger en mémoire** (Xyz.class , image jpeg , fichier .properties ,)

1.2. Trois grands types de classLoaders :

Hierarchie des « classLoader » (java)



Bootstrap class loader	Class loader racine/primordial , fondamental de la JVM (vu comme null car codé en code natif (pas java)) – amorce du chargement des classes
-------------------------------	---

Platform class loader	Chargeur des classes prédéfinies de la JVM (une des Api de java-SE) Avant java 9 , on appelait cela le « extension class loader » A partir de java 9, ça charge des classes via un des modules de la JVM
System class loader	Chargeur des classes d'une application (selon réglages du classpath, module path , ...)

Ces 3 chargeurs de classes sont hiérarchisés :

- le parent de PlatformClassLoader est null (le bootstrap class loader)
- le parent de SystemClassLoader (alias AppClassLoader) est PlatformClassLoader

Exemple de code visualisant certains classLoaders :

```
public static void printClassLoaders() throws ClassNotFoundException {
System.out.println("ClassLoader of ArrayList (fundamental jvm class):"
+ ArrayList.class.getClassLoader());
ClassLoader platformClassLoader = ClassLoader.getPlatformClassLoader();
System.out.println("Platform Classloader:"
+ platformClassLoader + " with parent = " + platformClassLoader.getParent());
System.out.println("ClassLoader of DriverManager (java-se api):"
+ DriverManager.class.getClassLoader());

ClassLoader systemClassLoader = ClassLoader.getSystemClassLoader();
System.out.println("System Classloader:"
+ systemClassLoader + " with parent = " + systemClassLoader.getParent());
System.out.println("ClassLoader of this applicative class:" + MyApp.class.getClassLoader());
}
```

Exemple de résultat (affichages sur la console) :

```
ClassLoader of ArrayList (fundamental jvm class):null
Platform Classloader: jdk.internal.loader.ClassLoaders$PlatformClassLoader@27716f4 with
parent = null
ClassLoader of DriverManager (java-se
api):jdk.internal.loader.ClassLoaders$PlatformClassLoader@27716f4
System Classloader: jdk.internal.loader.ClassLoaders$AppClassLoader@14dad5dc with parent =
jdk.internal.loader.ClassLoaders$PlatformClassLoader@27716f4
ClassLoader of this applicative class:jdk.internal.loader.ClassLoaders$AppClassLoader@14dad5dc
```

Comportement du classLoader java :

Soit *name* le nom complet (qualifié par le package d'une classe à charger)
et *resolve* le paramètre précisant si l'on souhaite charger le code (true) ou simplement vérifier si la classe existe (false)

Class<?> java.lang.ClassLoader.loadClass(String name, boolean resolve) appel dans l'ordre :

1. ***findLoadedClass(String name)*** vérifiant si la classe n'est pas déjà chargée en mémoire et qui retourne null si le chargement n'a pas encore été effectué.
2. ***loadClass(String name)*** sur le **classLoader parent** et ainsi de suite jusqu'au sommet de la hiérarchie des classLoaders
3. ***findClass(String name)*** sur le **classLoader courant** pour trouver la classe à charger

Si tous les niveaux de recherche ne mènent à rien , alors `java.lang.ClassNotFoundException`

En d'autres termes, le modèle de délégation donne d'abord au classLoader parent l'opportunité de trouver et charger la classe recherchée .

Cette délégation est avant tout basée sur un modèle hiérarchique et peut dans certains cas pointus comporter quelques traitements parallélisés .

L'avantage de cet algorithme de recherche est de pouvoir s'assurer qu'à partir d'un nom complet/qualifié bien précis , on va récupérer une classe (code binaire) **unique** .

Visibilités :

- Un classLoader enfant peut voir les classes chargées par un classLoader parent (et grand parent)
- Inversement un classLoader parent ne peut pas voir les classes chargées par un classLoader enfant (et ceci assure une bonne stabilité/fiabilité du coeur de la JVM) .

1.3. ClassLoader personnalisé

Le chargeur de classe prédéfini de la JVM suffit généralement à charger les classes accessibles sur le système de fichier local .

On peut néanmoins s'appuyer sur un chargeur de classe personnalisé de manière à par exemple :

- charger dynamiquement certaines classes à travers le réseau
- charger dynamiquement plusieurs variantes d'une classe de même nom (depuis différents endroits) puis choisir la version la plus appropriée en fonction du contexte.

Explications préliminaires :

```
protected final Class<?> defineClass(  
String name, byte[] b, int off, int len) throws ClassFormatError
```

est une méthode prédéfinie et finale (pas de @override possible) permettant de transformer un paquet d'octets binaires en une classe java chargée en mémoire .

NB : si le code binaire est invalide , l'exception `ClassFormatError` est alors soulevée .

```
protected Class<?> findClass( String name) throws ClassNotFoundException
```

est la méthode qu'il faut généralement implémenter sur un `ClassLoader` personnalisé de manière à dynamiquement charger quelque part le code binaire d'une classe java selon son nom qualifié .

Exemple simple :

```
package tp;  
import java.io.*;  
  
public class CustomDirectoryClassLoader extends ClassLoader {  
  
    private String complementaryClassDirectoryPath=".";   
  
    public CustomDirectoryClassLoader(String complementaryClassDirectoryPath) {  
        super();  
        this.complementaryClassDirectoryPath = complementaryClassDirectoryPath;  
    }  
}
```



```
@Override
public Class findClass(String name) throws ClassNotFoundException {
    byte[] b = loadClassFromFile(name);
    //System.out.println("found code for " + name + " have length=" + b.length);
    return defineClass(name, b, 0, b.length);
}

private byte[] loadClassFromFile(String fileName) {
    byte[] buffer;
    ByteArrayOutputStream byteStream = new ByteArrayOutputStream();
    int nextValue = 0;
    try {
        String completeFileName = this.complementaryClassDirectoryPath + File.separatorChar
            + fileName.replace('.', File.separatorChar) + ".class";
        InputStream inputStream = new FileInputStream(completeFileName);
        while ( (nextValue = inputStream.read()) != -1 ) {
            byteStream.write(nextValue);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    buffer = byteStream.toByteArray();
    return buffer;
}
}
```

1.4. Utilisation d'un chargeur de classe spécifique

À coté du classique `Class.forName(String name)` il existe cette variante surchargée :

`Class.forName(String name, boolean initialize, ClassLoader loader)`

Soit divers un projet maven multi-modules contenant :

- le sous projet `abstract_ext` comportant lui même l'interface `tp.Display` avec une méthode `print()`
- le sous projet `ext1` comportant une classe `tp.MyDisplay` implémentant l'interface `tp.Display`
- le sous projet `using_classloader` comportant le code suivant :

```
package tp;

public class MyApp {
    public static void main(String[] args) {
        String complementaryClassDirectoryPath = "C:\\.....\\divers\\ext1\\target\\classes";
        ClassLoader customDirectoryClassLoader = new
        CustomDirectoryClassLoader(complementaryClassDirectoryPath);
        System.out.println("Custom Classloader:" + customDirectoryClassLoader
            + " with parent = " + customDirectoryClassLoader.getParent());
        try {
            //ClassLoader alternativeClassLoader = ClassLoader.getSystemClassLoader();
            // → java.lang.ClassNotFoundException: tp.MyDisplay
            ClassLoader alternativeClassLoader = customDirectoryClassLoader; //ok
            Class<?> displayClass = Class.forName("tp.MyDisplay", true, alternativeClassLoader);
            System.out.println("loaded class : " + displayClass.getName()
                + " from " + displayClass.getClassLoader());
            Display display = (Display) displayClass.getDeclaredConstructor().newInstance();
            display.print("azerty");
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

⇒

Custom Classloader: *tp.CustomDirectoryClassLoader@4a574795* with parent = *jdk.internal.loader.ClassLoaders\$AppClassLoader@14dad5dc*

CustomDirectoryClassLoader.loadClassFromFile , *completeFileName=C:\tp\local-git-mycontrib-repositories\java_8_9_1x\divers\ext1\target\classes\tp\MyDisplay.class*

found code for tp.MyDisplay have length=994

loaded class : tp.MyDisplay from tp.CustomDirectoryClassLoader@4a574795

MyDisplay.print : azerty

2. WeakReference et SoftReference

2.1. Références faibles, douces et fortes

Types de références	comportements/caractéristiques
Forte (ordinaire , par défaut)	Un objet n'est pas libéré/détruit par le ramasse miettes tant qu'il est encore référence par au moins une référence forte/ordinaire.
Faible (WeakReference)	Une référence faible est une référence qui n'est pas suffisamment forte pour forcer un objet à rester en mémoire suite au passage du garbage collector. Autrement dit , un objet uniquement référencé par une weak référence aura une durée de vie très éphémère = tant que pas déclenchement (souvent automatique) du GC.
Douce (SoftReference)	Une « SoftReference » est une variante intéressante de WeakReference au sein de laquelle la libération de l'espace mémoire est laissée à la discrétion du garbage collector en fonction de l'espace mémoire dont il dispose. Autrement dit , un objet uniquement référencé par une soft référence ne sera maintenu en mémoire que si le GC n'a pas absolument besoin de libérer ce bloc de mémoire référencé pour que l'application puisse fonctionner

- Une mauvaise utilisation des références fortes/ordinaires peut quelquefois conduire à des fuites de mémoires.
- Les « **SoftReference** » peuvent quelquefois être utilisées pour un système de **cache ultra simple** (pas très perfectionné) .

WeakReference et **SoftReference** héritent de `java.lang.ref.Reference`

2.2. Exemple d'utilisation de WeakReference

```
public class IdValue {
    private Long id;
    private String value;
    ...
}
```

```
import java.lang.ref.Reference;
```

```

import java.lang.ref.SoftReference;
import java.lang.ref.WeakReference;
...
public static void test_simple_weak_reference() {
    WeakReference<IdValue> wr1 = new WeakReference<>(new IdValue(1, "janvier"));
    System.out.println(wr1 + " " + wr1.get());
    //IdValue refForte1 = wr1.get(); //casse l'intérêt de la référence faible
    for (int i=0; i<6; i++) {
        System.out.println(wr1.get());
        if (i == 3) System.gc();
        //si que référence faible , alors objet potentiellement détruit lors de la phase du GC
    }
}

```

⇒ comportement/résultats :

java.lang.ref.WeakReference@2a84aee7 IdValue{id=1, value='janvier'}

IdValue{id=1, value='janvier'}

IdValue{id=1, value='janvier'}

IdValue{id=1, value='janvier'}

IdValue{id=1, value='janvier'}

null

null

2.3. Exemple d'utilisation de SoftReference :

```

package tp;

public class MemoryBlock {
    private int id;
    private int size;
    private byte[] block;

    public MemoryBlock( int id, int size ) {
        this.id = id;
        this.size = size;
        block = new byte[size];
        //System.out.println("MemoryBlock created: " + this);
    }

    @Override

```

```

public String toString() {
    return "{id="+id+",size="+size+"}";
}

/*
@Override @SuppressWarnings("deprecation")
protected void finalize() {
    System.out.println( "MemoryBlock finalized: "+this );
}
*/
}

```

```

...
public static void test_big_block_weak_reference() {
    List<Reference<MemoryBlock>> list = new ArrayList<>();
    int size = 65536;
    int id = 0;
    for (int i=0;i<=20;i++) {
        //list.add(new WeakReference<>(new MemoryBlock(id++, size)));
        list.add(new SoftReference<>(new MemoryBlock(id++, size)));
        if(size < Integer.MAX_VALUE /2 )
            size *= 2;
        displayMemoryBlockList(">>> i=" + i ,list);
        System.gc();
    }
}

public static void displayMemoryBlockList(String message , List<Reference<MemoryBlock>>
list){
    System.out.println(message);
    for (Reference<MemoryBlock> r : list) {
        System.out.print(" " + r.get());
    }
    System.out.print("\n");
}
}

```

⇒

```

>>> i=0
{id=0,size=65536}
>>> i=1
{id=0,size=65536} {id=1,size=131072}
...
>>> i=16
null null null null null null null null null null null null null null {id=16,size=1073741824}
>>> i=17

```

```

null null null null null null null null null null null null null null null null {id=16,size=1073741824}
{id=17,size=1073741824}

```

```

...

```

2.4. Cache élémentaire basé sur WeakHashMap

Un objet de la classe **WeakHashMap** est une table (map) dont les clés sont des références faibles (weak références) et telle que lorsque la référence n'est plus disponible, l'entrée est supprimée de la map.

```

public static void very_simple_cache_with_weakHashMap ( ) {
    WeakHashMap<String,MemoryBlock> cache = new WeakHashMap<>();
    //it works with key of type String , but not with key of type Integer , ???
    int n = 0;
    int size = 1048*1048*64;
    while (n<20) {
        cache.put(""+n++, new MemoryBlock(n, size));

        System.out.printf("Keys in cache: ");
        for (String k : cache.keySet()) {
            System.out.printf(" %s ", k);
        }
        // System.out.println("cache="+cache);
        System.out.printf("\n");
    }
}

```

⇒

```

...
Keys in cache: 5 6
Keys in cache: 5 6 7
Keys in cache: 8
Keys in cache: 9
Keys in cache: 9 10
Keys in cache: 11 9 10
Keys in cache: 12
Keys in cache: 13
Keys in cache: 14 13
...

```

3. Modules java 9+ (essentiel)

3.1. Problèmes et limitations en java 8

En Java 8 et antérieur, à l'exécution d'une application, la JVM recherche les classes utilisées par l'application dans :

- Les classes prédéfinies de la plate-forme Java : stockées dans l'**énorme** fichier **monolithique** `rt.jar` (*jusqu'à 53 Mo en Java 8 !*).
- Le **classpath** paramétré au lancement (un ensemble de chemins relatifs ou absolus vers des .jars ou des répertoires contenant des arborescences en package contenant des fichiers ".class")

Selon d'éventuelles différences de contextes (ordinateurs différents , "jdk" différents, ...) , *le classpath n'est pas forcément le même à la compilation et à l'exécution* et cela peut engendrer `"java.lang.NoClassDefFoundError"` !!!!

En Java 8 et antérieur, à l'exécution d'une application, le "classloader" de la JVM a le comportement suivant :

- il charge les classes sur demande lors d'un new ou d'un appel à une méthode statique.
- il charge les classes linéairement (selon l'ordre des jars dans le classpath qui est donc très important) et s'arrête à la première classe correspondant au nom complet demandé.
- aucune vérification n'est faite à l'exécution sur l'existence de plusieurs occurrences d'une même classe.
- aucune vérification n'est faite au démarrage de l'application sur la présence de tous les jars / classes nécessaires au bon fonctionnement de l'application.
- le classpath est plat sans aucune notion de dépendances entre jar.

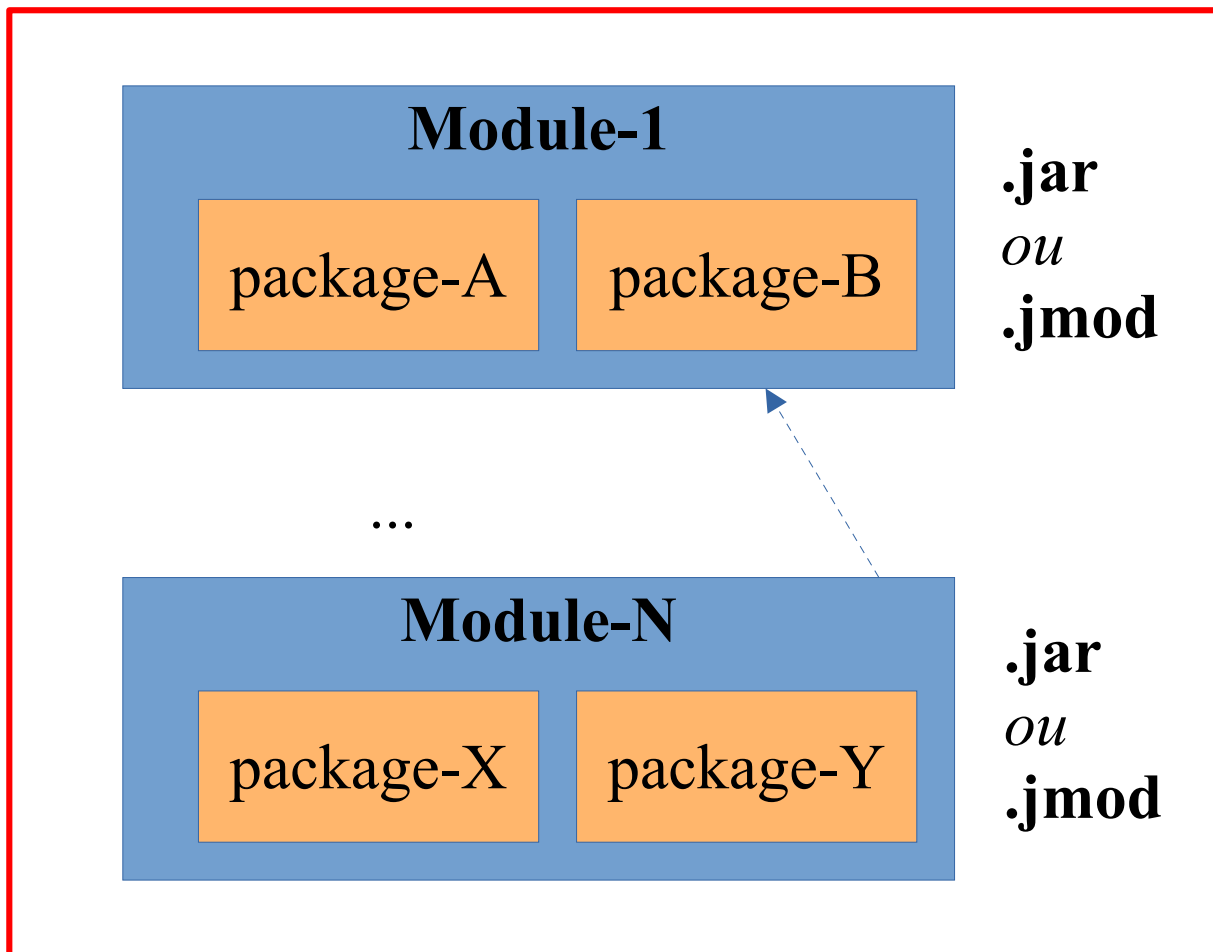
3.2. Projet JIGSAW (modularisation de la jvm depuis java >=9)

Principaux **apports** de la **modularisation** de la JVM (depuis V9) :

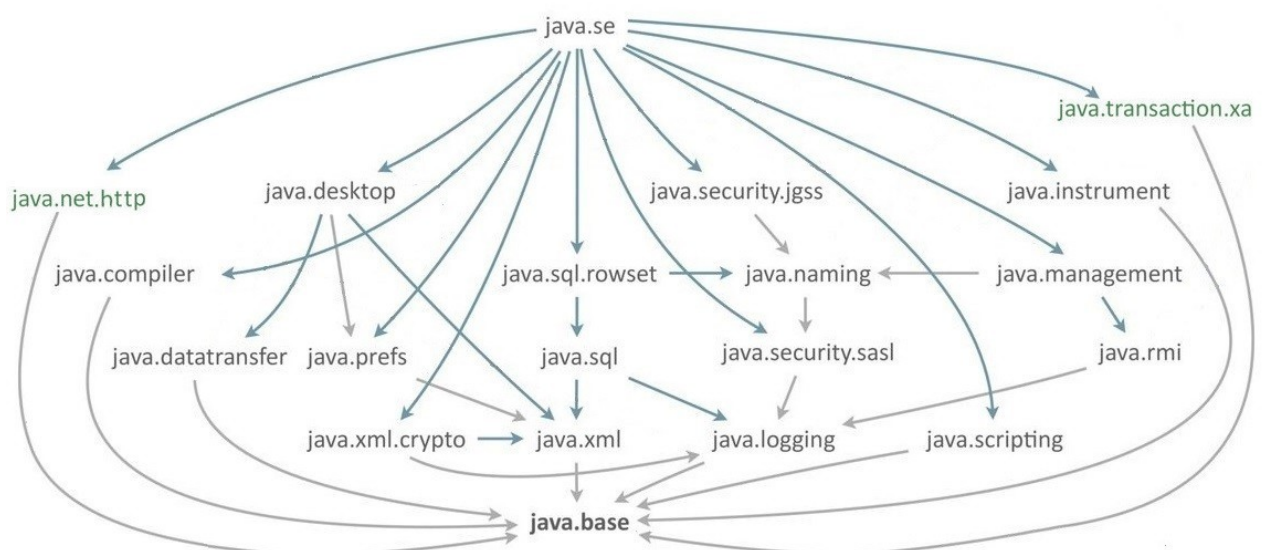
- **un classpath sous forme d'arbre de dépendances**
- **une vérification de la présence de tous les modules nécessaires à notre application au démarrage**, sans quoi l'application ne démarre pas.
- **un renforcement de la sécurité**, seuls les packages exportés explicitement par un module sont visibles par un autre
- la JVM elle-même est **modulaire** (*l'énorme fichier `rt.jar` n'existe plus depuis java 9*)
(NB : la taille du JRE a sensiblement diminué à partir de java 11)

NB : Un module "java >=9" est un **.jar** comportant le fichier ***module-info.java*** (compilé en *module-info.class*) à la racine du module .

Appli ou JVM modulaire depuis java 9



Exemple de graphe de dépendances entre modules d'un jdk récent :



Modules du jdk 11 :

Java > jdk-11.0.4 > jmods

Nom	Modifié le	Type	Taille
 java.base.jmod	19/08/2019 20:57	Fichier JMOD	18 021 Ko
 java.compiler.jmod	19/08/2019 20:57	Fichier JMOD	109 Ko
 java.datatransfer.jmod	19/08/2019 20:57	Fichier JMOD	50 Ko
 java.desktop.jmod	19/08/2019 20:57	Fichier JMOD	11 799 Ko
 java.instrument.jmod	19/08/2019 20:57	Fichier JMOD	116 Ko
 java.logging.jmod	19/08/2019 20:57	Fichier JMOD	110 Ko
 java.management.jmod	19/08/2019 20:57	Fichier JMOD	867 Ko
 java.management.rmi.jmod	19/08/2019 20:57	Fichier JMOD	87 Ko
 java.naming.jmod	19/08/2019 20:57	Fichier JMOD	436 Ko
 java.net.http.jmod	19/08/2019 20:57	Fichier JMOD	680 Ko
 java.prefs.jmod	19/08/2019 20:57	Fichier JMOD	53 Ko
 java.rmi.jmod	19/08/2019 20:57	Fichier JMOD	369 Ko
 java.scripting.jmod	19/08/2019 20:57	Fichier JMOD	44 Ko
 java.se.jmod	19/08/2019 20:57	Fichier JMOD	3 Ko
 java.security.jgss.jmod	19/08/2019 20:57	Fichier JMOD	638 Ko
 java.security.sasl.jmod	19/08/2019 20:57	Fichier JMOD	79 Ko
 java.smartcardio.jmod	19/08/2019 20:57	Fichier JMOD	57 Ko
 java.sql.jmod	19/08/2019 20:57	Fichier JMOD	74 Ko
 java.sql.rowset.jmod	19/08/2019 20:57	Fichier JMOD	184 Ko
 java.transaction.xa.jmod	19/08/2019 20:57	Fichier JMOD	4 Ko
 java.xml.crypto.jmod	19/08/2019 20:57	Fichier JMOD	635 Ko
 java.xml.jmod	19/08/2019 20:57	Fichier JMOD	4 380 Ko
 jdk.accessibility.jmod	19/08/2019 20:57	Fichier JMOD	484 Ko
 jdk.aot.jmod	19/08/2019 20:57	Fichier JMOD	274 Ko
 jdk.attach.jmod	19/08/2019 20:57	Fichier JMOD	38 Ko
 jdk.charsets.jmod	19/08/2019 20:57	Fichier JMOD	1 451 Ko
 jdk.compiler.jmod	19/08/2019 20:57	Fichier JMOD	6 904 Ko
⋮			
 jdk.crypto.cryptoki.jmod	19/08/2019 20:57	Fichier JMOD	305 Ko
 jdk.crypto.ec.jmod	19/08/2019 20:57	Fichier JMOD	148 Ko
 jdk.crypto.mscapi.jmod	19/08/2019 20:57	Fichier JMOD	61 Ko
 jdk.dynalink.jmod	19/08/2019 20:57	Fichier JMOD	159 Ko
 jdk.editpad.jmod	19/08/2019 20:57	Fichier JMOD	7 Ko
 jdk.hotspot.agent.jmod	19/08/2019 20:57	Fichier JMOD	2 298 Ko
 jdk.httpserver.jmod	19/08/2019 20:57	Fichier JMOD	98 Ko
 jdk.internal.ed.jmod	19/08/2019 20:57	Fichier JMOD	8 Ko
 jdk.internal.jvmstat.jmod	19/08/2019 20:57	Fichier JMOD	88 Ko
 jdk.internal.le.jmod	19/08/2019 20:57	Fichier JMOD	181 Ko
 jdk.internal.opt.jmod	19/08/2019 20:57	Fichier JMOD	80 Ko
 jdk.internal.vm.ci.jmod	19/08/2019 20:57	Fichier JMOD	399 Ko
 jdk.internal.vm.compiler.jmod	19/08/2019 20:57	Fichier JMOD	6 025 Ko
 jdk.internal.vm.compiler.manag...	19/08/2019 20:57	Fichier JMOD	12 Ko
 jdk.jartool.jmod	19/08/2019 20:57	Fichier JMOD	195 Ko
 jdk.javadoc.jmod	19/08/2019 20:57	Fichier JMOD	1 594 Ko
 jdk.jcmd.jmod	19/08/2019 20:57	Fichier JMOD	143 Ko
 jdk.jconsole.jmod	19/08/2019 20:57	Fichier JMOD	456 Ko
 jdk.jdeps.jmod	19/08/2019 20:57	Fichier JMOD	710 Ko
 jdk.jdi.jmod	19/08/2019 20:57	Fichier JMOD	841 Ko
 jdk.jdp.agent.jmod	19/08/2019 20:57	Fichier JMOD	121 Ko
 jdk.jfr.jmod	19/08/2019 20:57	Fichier JMOD	414 Ko
 jdk.jlink.jmod	19/08/2019 20:57	Fichier JMOD	388 Ko
 jdk.jshell.jmod	19/08/2019 20:57	Fichier JMOD	626 Ko
 jdk.jsobject.jmod	19/08/2019 20:57	Fichier JMOD	6 Ko
 jdk.jstatd.jmod	19/08/2019 20:57	Fichier JMOD	33 Ko
⋮			
 jdk.localedata.jmod	19/08/2019 20:57	Fichier JMOD	9 327 Ko
 jdk.management.agent.jmod	19/08/2019 20:57	Fichier JMOD	80 Ko
 jdk.management.jfr.jmod	19/08/2019 20:57	Fichier JMOD	34 Ko
 jdk.management.jmod	19/08/2019 20:57	Fichier JMOD	68 Ko
 jdk.naming.dns.jmod	19/08/2019 20:57	Fichier JMOD	57 Ko
 jdk.naming.rmi.jmod	19/08/2019 20:57	Fichier JMOD	19 Ko
 jdk.net.jmod	19/08/2019 20:57	Fichier JMOD	21 Ko
 jdk.pack.jmod	19/08/2019 20:57	Fichier JMOD	131 Ko
 jdk.rmic.jmod	19/08/2019 20:57	Fichier JMOD	516 Ko
 jdk.scripting.nashorn.jmod	19/08/2019 20:57	Fichier JMOD	2 143 Ko
 jdk.scripting.nashorn.shell.jmod	19/08/2019 20:57	Fichier JMOD	56 Ko
 jdk.sctp.jmod	19/08/2019 20:57	Fichier JMOD	23 Ko
 jdk.security.auth.jmod	19/08/2019 20:57	Fichier JMOD	73 Ko
 jdk.security.jgss.jmod	19/08/2019 20:57	Fichier JMOD	24 Ko
 jdk.unsupported.desktop.jmod	19/08/2019 20:57	Fichier JMOD	14 Ko
 jdk.unsupported.jmod	19/08/2019 20:57	Fichier JMOD	18 Ko
 jdk.xml.dom.jmod	19/08/2019 20:57	Fichier JMOD	42 Ko
 jdk.zipfs.jmod	19/08/2019 20:57	Fichier JMOD	87 Ko

3.3. option "--list-modules" de java

print_list_modules.bat

```
set JAVA_HOME=D:\Prog\java\jdk-14.0.1_windows-x64_bin\jdk-14.0.1
"%JAVA_HOME%\bin\java" --list-modules
pause
```

--> affiche la liste des modules systèmes de la JRE (et du JDK) , soit par exemple :

```
java.base@14.0.1
java.compiler@14.0.1
java.datatransfer@14.0.1
java.desktop@14.0.1
...
jdk.xml.dom@14.0.1
jdk.zipfs@14.0.1
```

3.4. option "--module-path" de java

L'option **--module-path** d'une ligne de commande java permet de spécifier le "module path". C'est une **liste de un ou plusieurs répertoires qui comportent des modules** .

Exemple :

prepare_modules.bat

```
cd /d "%~dp0"
REM cd ..
REM mvn package
REM cd /scripts
copy ..\mod_xx\target\mod_xx.jar .\my_modules
copy ..\mod_yy\target\mod_yy.jar .\my_modules
pause
```

launch_app_withmodules.bat

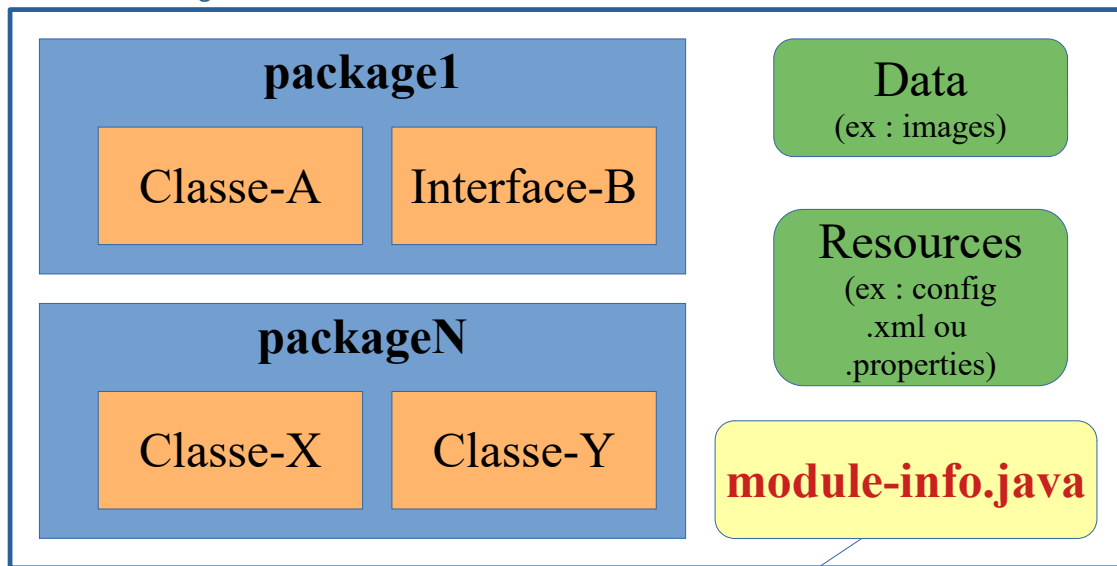
```
cd /d "%~dp0"
REM java --module-path modules_directory -m mainModuleName/mainClassFullName
java --module-path .\my_modules -m tp.module.modyy/tp.mod.mod_yy.app.MyApp
pause
```

3.5. Structure d'un module

Un module java (packagé en un ".jar" éventuellement renommé) est **l'association** de :

- un **paquet de packages java** (un sous package précis "xxx.yyy.zzz" ne peut être rangé que dans un seul module)
- un **paquet d'éventuelles ressources** (fichiers annexes : configuration_xml , images, ...)
- un fichier de configuration **module-info.java**

Module java 9+



*NB: si module-info.java pas spécifié , alors module automatique (exports * , requires *)*

```
module m2 {
    exports package1;
    requires m1;
}
```

3.6. Types de modules

Types de modules	Caractéristiques
System Modules	modules prédéfinis livrés avec jre/jdk (listés via java --list-modules)
Application Modules	modules applicatifs (.jar fabriqués avec module-info.class)
Automatic Modules	unofficial module (.jar fabriqués sans module-info.class) (with names based on ".jar" name) chargés depuis le --module-path . <i>Attention : sans module-info.class , ces modules ont par défaut un accès complet en lecture vers tous les autres modules chargés .</i>
Unnamed Module	" module fourre tout sans nom " comportant toutes les classes et packages chargés depuis des ".jar" via le --classpath (et pas le nouveau --module-path) -> pour récupérer tout les éléments codés en java <= 8 .

3.7. Descripteur d'un module (module-info.java)

name	nom de notre module (avec d'éventuel(s) "." mais pas de "-")
dependencies (<i>requires</i>)	liste des modules dont on dépend
public packages (<i>exported</i>)	liste des packages que l'on rend accessibles aux autres modules. Normalement, un bon module ne doit exposer publiquement qu'une petite partie de son API (interfaces , pas l'implémentation)
services offered (<i>provides</i>)	implémentation de services (à injecter/utiliser dans d'autres modules)
services consumed (<i>uses</i>)	autoriser le module courant à utiliser des services d'autres modules implémentant une certaine interface ou classe abstraite
reflection permissions (<i>opens</i>)	autoriser ou pas d'autres classes à utiliser la "Reflection java" pour accéder aux éléments privés d'un de nos packages

3.8. Variantes de dépendances (requires)

requires	module my.module { requires module.name; }	dépendance à la fois à la compilation et à l'exécution ("runtime")
requires static	module my.module { requires static module.name; }	dépendance à la compilation seulement , facultatif à l'exécution ("runtime")
requires transitive	module my.module { requires transitive module.name; }	dépendance automatique transitive (au niveau du futur module client qui n'aura donc pas besoin d'explicitement les dépendances indirectes)

comportement "requires transitive"

Si yy utilise xx utilisant lui même zz et que xx expose une interface de zz
alors yy est alors obligé d'utiliser zz
Si le module xx comporte "requires transitive zz"
alors yy aura juste besoin d'exprimer "requires xx" (pas besoin de requires zz")

comportement "requires static"

Le (rare) "**requires static** moduleOptionnelPasToujoursUtilise"
signifie "obligatoire à la compilation" et "facultatif à l'exécution"
avec éventuelle NoClassDefFoundError à gérer au runtime .

3.9. Variantes d'exportations / expositions

exports	module my.module { exports com.my.package.name; }	exportation explicite
exports ... to ...	module my.module { export com.my.package.name to com.specific.package; }	(rare) exportation limitée/restreinte à certain(s) autres package(s) ami(s).

Comportement des "exports"

NB: **exports p1** n'exporte pas les sous packages **p1.aa** et **p1.bb**

Un même nom complet de package ne peut pas être placé dans plusieurs modules (mais dans un unique module)

3.10. interfaces et implémentations de Services

uses	module my.module { uses interfaceOrAbstractClass.name; }	besoin de consommer/injecter un certain type de service
provides ... with ...	module my.module { provides MyInterface with MyImpl; }	Notre module fournit l'implémentation d'un certain type de service

provides <type> **with** <type> et **uses** <type> ont été conçus pour mettre en oeuvre une sorte d'injection de dépendances (selon packages d'interfaces) et avec implémentations internes cachées mais cependant chargées au runtime (selon le paramétrage de --module-path)

Exemple de module abstrait (packagé en my_modules/mod_aa_itf.jar) :

```
module tp.module.modaa.itf {
    exports tp.mod.mod_aa.itf;
}
```

et

```
package tp.mod.mod_aa.itf;

public interface MyDisplayApi {
    void display(String msg);
    void displayEx(String msg);
}
```

Exemple de module d'implémentation (packagé en my_modules/mod_aa_impl.jar) :

```

module tp.module.modaa.aa.impl1 {
    requires tp.module.modaa.itf;
    provides tp.mod.mod_aa.itf.MyDisplayApi with tp.mod.mod_aa.impl1.MyDisplayImplV1;
}

```

et

```

package tp.mod.mod_aa.impl1;
import tp.mod.mod_aa.itf.MyDisplayApi;

public class MyDisplayImplV1 implements MyDisplayApi{
    public void display(String msg) {    System.out.println(">>>" + msg);    }
    public void displayEx(String msg) { System.out.println(":::" + msg);    }
}

```

Exemple d'utilisation de service :

```

module tp.module.modxx {
    ...
    exports tp.mod.mod_xx_ext;
    requires transitive tp.module.modaa.itf;
    uses tp.mod.mod_aa.itf.MyDisplayApi;
}

// pour uses tp.mod.mod_aa.itf.MyDisplayApi;
// besoin de requires transitive tp.module.modaa.itf;
// mais pas besoin de requires transitive tp.module.modaa.impl1 ou impl2;

```

```

package tp.mod.mod_xx_ext;
import java.util.Iterator;
import java.util.ServiceLoader;
import tp.mod.mod_aa.itf.MyDisplayApi;

public class CxExt {
    private static MyDisplayApi displayService = null;

    public static void displayViaMyDisplayApi(String msg) {
        if(displayService==null) {
            Iterable<MyDisplayApi> myDisplayServices = ServiceLoader.load(MyDisplayApi.class);
            //list of found implementations
            Iterator<MyDisplayApi> myDisplayServicesIterator = myDisplayServices.iterator();
            if(myDisplayServicesIterator.hasNext()) {
                displayService = myDisplayServicesIterator.next(); //first implementation is choosen
            }
        }
        if(displayService!=null) {
            displayService.display(msg);
        }
    }
}

```

```

        }else{
            System.out.println("no MyDisplayApi implementation found !!!");
        }
    }
}

```

----> comportement d'un appel à `CxExt.displayViaMyDisplayApi("my top secret message");`

- si aucune implémentation n'est présente dans le `-module-path` ça affiche *"no MyDisplayApi implementation found !!!"*
- si au moins une implémentation (ex : `mod_aa_impl1.jar` ou `mod_aa_impl2.jar`) est présente dans le `-module-path` la première trouvée est alors utilisée et ça affiche *">>> my top secret message"*

3.11. Variantes d'ouvertures à l'introspection :

open	<code>open module my.module { }</code>	<i>on autorise tous les autres packages à effectuer de l'introspection sur des éléments privés de l'ensemble des éléments de notre module</i>
opens	<code>module my.module { opens com.my.package; }</code>	Seul(s) un ou plusieurs(s) de nos packages sont ouvert à de l'introspection avec "accès aux éléments privés"
opens ... to	<code>module my.module { opens com.my.package to com.specific.package; }</code>	introspection ouverte mais limitée/restreinte à certain(s) autres package(s) ami(s).

opens <package> (à ajouter à côté de exports <package>) permet d'autoriser une introspection évoluée (ex: avec `setAccessible(true)`) .

L'autorisation doit être explicitement donnée (ce qui renforce la sécurité)

Exemple :

Du côté "module ...modxx" :

```

package tp.mod.mod_xx.pub;

public class Cx {
    private String secret="007";
    ...
}

```

et

```

module tp.module.modxx {
    ...
    exports tp.mod.mod_xx.pub;
    opens tp.mod.mod_xx.pub;
}

```

Du côté module client ...modyy :

```
module tp.module.modyy {
    requires tp.module.modxx;
    ...
}
```

et

```
package tp.mod.mod_yy.app;
import java.lang.reflect.Field;
import tp.mod.mod_xx.pub.Cx;

public class MyApp {
    private static void testOpenReflection(Cx objX) {
        Class classX = objX.getClass();
        for(Field f : classX.getDeclaredFields()) {
            f.setAccessible(true);
            String sVal="?";
            try {
                sVal = f.get(objX).toString();
            } catch (IllegalAccessException e) { e.printStackTrace();
                //Exception in thread "main" java.lang.reflect.InaccessibleObjectException:
                //Unable to make field private java.lang.String tp.mod.mod_xx.pub.Cx.secret
                //accessible: module tp.module.modxx does not "opens tp.mod.mod_xx.pub" to module tp.module.modyy
            }
            System.out.println( f.getType().getSimpleName()+ " " + f.getName() + "=" + sVal);
        }
    }

    public static void main(String[] args) {
        Cx objX = new Cx();        testOpenReflection(objX);
    }
}
```

--> affiche une exception de type **java.lang.reflect.InaccessibleObjectException** si pas de opens tp.mod.mod_xx.pub; dansmodxx

--> affiche "**String secret=007**" si **opens** tp.mod.mod_xx.pub; dansmodxx

3.12. Exemple (partiel) de quelques modules java 9+ :

NB : l'aspect "**multi-modules maven**" n'est pas indispensable mais cela introduit une cohérence

structurelle :

pom.xml (multi-modules)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>tp.mod</groupId>
  <artifactId>mod</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>pom</packaging>
  <description>modules=nouveautes de java 9 </description>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <java.version>11</java.version>
    <maven.compiler.release>${java.version}</maven.compiler.release>
  </properties>

  <modules>
    <module>mod_xx</module>
    <module>mod_yy</module>
  </modules>

  <build>
    <pluginManagement>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.8.0</version>
          <configuration>
            <release>${java.version}</release>
            <source>${java.version}</source> <target>${java.version}</target>
          </configuration>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

module "mod_xx" (sera utilisé par "mod_yy")

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>tp.mod</groupId>
```

```

<artifactId>mod</artifactId>
<version>0.0.1-SNAPSHOT</version>
</parent>
<artifactId>mod_xx</artifactId>
<description>mod_xx java>=9 sera utilisé par mod_yy</description>
<build>
  <finalName>${project.artifactId}</finalName>
</build>
</project>

```

module-info.java (à la racine de src/main/java)

```

module tp.module.modxx {
    exports tp.mod.mod_xx.pub;
    exports tp.mod.mod_xx_ext;
}

```

```

package tp.mod.mod_xx.pub;

import tp.mod.mod_xx.internal.InternalCx;

public class Cx {
    public void fl(String s) {
        InternalCx icx = new InternalCx();
        icx.fli("***" + s);
    }
}

```

```

package tp.mod.mod_xx.internal;

public class InternalCx {
    public void fli(String s) { System.out.println("fl:" + s); }
}

```

```

package tp.mod.mod_xx_ext;

import tp.mod.mod_xx_ext.internal.InternalCxExt;

public class CxExt {
    public static double add(double x, double y) { return InternalCxExt.addition(x, y); }
}

```

```

package tp.mod.mod_xx_ext.internal;

public class InternalCxExt {
    public static double addition(double x, double y) { return x+y; }
}

```

module "mod_yy" (utilisant "mod_xx")**pom.xml**

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>tp.mod</groupId>
    <artifactId>mod</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </parent>
  <artifactId>mod_yy</artifactId>
  <description>mod_yy (java=>9) utilisant mod_xx</description>

  <dependencies>
    <dependency>
      <groupId>tp.mod</groupId>
      <artifactId>mod_xx</artifactId>
      <version>0.0.1-SNAPSHOT</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>${project.artifactId}</finalName>
  </build>
</project>

```

module-info.java (à la racine de src/main/java)

```

module tp.module.modyy {
  requires tp.module.modxx;
  requires java.desktop;
  exports tp.mod.mod_yy.pub;
}

```

```

package tp.mod.mod_yy.pub;
import tp.mod.mod_yy.internal.InternalCy;

```

```

public class Cy {
  public void f2(String s) {
    InternalCy icy = new InternalCy();
    icy.f2i("##" + s);
  }
}

```

```

package tp.mod.mod_yy.internal;
//import tp.mod.mod_xx.internal.InternalCx; //pas accessible car package pas exporté
//import tp.mod.mod_xx_ext.internal.InternalCxExt; //pas accessible car package pas exporté

```

```
import tp.mod.mod_xx.pub.Cx;
import tp.mod.mod_xx_ext.CxExt;

public class InternalCy {
    public void f2i(String s) { System.out.println("f2:"+s); }
    public void f1i(String s) { Cx cx =new Cx(); cx.f1(s); }
    public static double ajouter(double a,double b) { return CxExt.add(a, b); }
    //code impossible (basé sur classe InternalCx pas accessible / pas exportée ):
    //public void forbidden_f1(String s) { InternalCx icx =new InternalCx(); icx.f1i(s); }
}
```

```
package tp.mod.mod_yy.app;
import javax.swing.JOptionPane;
import tp.mod.mod_yy.internal.InternalCy;
public class MyApp {
    public static void main(String[] args) {
        InternalCy icy = new InternalCy();
        icy.f2i("java");icy.f1i("java");
        JOptionPane.showMessageDialog(null, "ok with requires java.desktop");
    }
}
```

3.13. Modules explicites , automatiques et "unnamed"

Modules explicites gérés rigoureusement :

accès contrôlés si `module.info.java` est présent

Si dans le module `mod_yy` (utilisant `mod_xx`) , le fichier **`module.info.java`** est présent
il faut alors tout rendre cohérent (exports , requires) :
et alors **seuls les éléments bien exportés par `mod_xx`**
et bien importés (via requires) dans `mod_yy` seront accessibles dans `mod_yy`.

Modules automatiques sans gestion rigoureuse :

accès libres mais pas rigoureux si pas de `module.info.java` (modules "automatiques")

NB: si jamais de `module.info.java` (pour chaque module)
alors tout ce qui est publique est accessible --> comme ancien comportement de java <=8 .

requires * par défaut si `module.info.java` est absent (du coté module utilisateur)

Si le module `mod_xx` comporte `module.info.java` (avec certains éléments exportés),
alors le module `mod_yy` (utilisant `mod_xx`) pourra alors éventuellement ne pas comporter de
fichier `module.info.info` et dans ce cas le comportement semble équivalent à
requires * (vérifié)
*et (à priori) exports **

NB : On appel **module automatique** un *.jar ne comportement pas de fichier `module-info.class`* et pourtant **placé** dans un des répertoires du `--module-path` .

Un tel module se voit alors attribué un nom automatiquement basé sur le nom du .jar et quelques ajustements (ex : `libs-legacy/xxx-yyy-legacy-1.0-SNAPSHOT.jar` est automatiquement nommé `xxx.yyy.legacy`) et un `requires xxx.yyy.legacy` est donc envisageable depuis un autre module .

Les modules automatiques voient tous leurs types publics exportés ("`exports *`" automatique). Un module utilisant un module automatique peut donc utiliser n'importe quel type public

Attention, maven et eclipse ne gèrent pas très bien les modules automatiques lors de l'exécution d'une application java.

Modules "**unnamed**" pour les .jar chargés depuis le `--class-path` plutôt que `--module-path`:

NB : On appelle **module "unnamed"** le méga module constitué de tous les **.jar** qui ne sont pas chargés via le `--module-path` mais qui sont (à l'ancienne) chargés via l'option `--class-path` ayant `-classpath` et `-cp` comme *alias* .

Cet "unnamed module" a la particularité d'avoir accès à :

- tous les packages exportés par tous les autres modules disponibles dans le module-path
- tous les jars du classpath (ie: tous les autres types présents dans cet unnamed module)

Le module par défaut (sans nom) "unnamed module" exporte tous ses packages.

Cependant les classes de cet "unnamed module" ne sont accessibles que depuis ce même "unnamed module".

NB : Par défaut, aucun "named module" ne peut accéder aux classes du "unnamed module".

(that was done on purpose to prevent modular JARs from depending on "the chaos of the class path".)

Eventuel ajout de modules (chargés via `--module-path`) dans la sphere de visibilité du "unnamed module"

Il est possible d'ajouter un fichier ".jar" (hors jdk récent mais chargé via le `--module-path`) au "unnamed / default module" via l'option de `--add-modules` de java :

Exemple :

```
java .... --add-modules java.xml.bind
```

ou bien(via maven) :

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <source>11</source><target>11</target>
    <release>11</release>
    <compilerArgs>
      <arg>--add-modules</arg>
      <arg>java.xml.bind</arg>
    </compilerArgs>
  </configuration>
</plugin>
```

Ceci est essentiellement utile pour l'ancien l'écosystème JEE .

3.14. Options (très pointues) disponibles en ligne de commande pour ré-ajuster la configuration des modules

--add-reads <module>=<target-module>(,<target-module>)*

met à jour <module> pour lire <target-module>, sans tenir compte de la déclaration de module.

<target-module> peut être ALL-UNNAMED pour lire tous les modules sans nom.

--add-exports <module>/<package>=<target-module>(,<target-module>)*

met à jour <module> pour exporter <package> vers <target-module>, sans tenir compte de la déclaration de module. <target-module> peut être ALL-UNNAMED pour effectuer un export vers tous les modules sans nom.

--add-opens <module>/<package>=<target-module>(,<target-module>)*

met à jour <module> pour ouvrir <package> vers <target-module>, sans tenir compte de la déclaration de module

--patch-module <module>=<file>(;<file>)*

Remplacement ou augmentation d'un module avec des classes et des ressources dans des fichiers ou des répertoires JAR.

XXXI - Threads virtuels , etc

1. Pattern and record matching

Rappel du pattern matching "if/instanceof" (déjà possible en java 16,17)

```
if (obj instanceof String str) {
    ...
}
```

1.1. Pattern matching switch (preview java 17 , finalisé java 21)

```
static double getDoubleUsingSwitch(Object o) {
    return switch (o) {
        case Integer i -> i.doubleValue();
        case Float f -> f.doubleValue();
        case Double d -> d.doubleValue();
        case String s -> Double.parseDouble(s);
        case null -> 0d;
        default -> 0d;
    };
}
```

```
static String getTypeAnimalDomesticAsString(AnimalDomestique a) {
    return switch (a) {
        case Chat chat -> "chat";
        case Chien chien -> "chien";
        //default -> "ni chat , ni chien"; //default not mandatory if AnimalDomestique is "sealed"
    };
}
```

1.2. Destructuration via "record pattern"

```
public class MyRecords {
    public record Point(int x,int y) {};
    public record GPS_Point(double latitude,double longitude) {};
    public record Location(String name,GPS_Point position) {};
}
```



```

import tp.java_new_21plus.record_pattern.MyRecords.*;

public class RecordDeconstructApp {

    public static void main(String[] args) {
        Double d1=23.5;
        print(d1);

        Point pt1 = new Point(20,30);
        print(pt1);
        print_destructuring(pt1);
        switch_destructuring(pt1);

        Location loc1 = new Location("Paris" , new GPS_Point(48.866667, 2.333333));
        print(loc1);
        print_destructuring(loc1);
        switch_destructuring(loc1);
    }

    public static void print(Object o) {
        //Pattern matching for instance of (since java 16/17):
        if(o instanceof Double d) {System.out.println("double d="+d);}
        if(o instanceof Point pt) { System.out.println("Point pt="+pt);}
        if(o instanceof Location loc) { System.out.println("Location loc="+loc);}
    }

    public static void print_destructuring(Object o) {
        //record pattern = destructure (like javascript) in Pattern matching for instance of
        if(o instanceof Point (int x,int y)) {
            System.out.println("Destructuring Point x="+x + " y="+y);
        }
        if(o instanceof Location (String name,GPS_Point(double latitude,double longitude))) {
            System.out.println("Destructuring Location name="+name
                               + " latitude="+latitude + " longitude="+longitude );
        }
    }

    public static void switch_destructuring(Object o) {
        //record pattern = destructure (like javascript) in switch expression
        double maxi = switch(o) {
            case Point (int x,int y) -> (double) Math.max(x, y);
            case Location (String name,GPS_Point(double latitude,double longitude))
                -> Math.max(latitude, longitude);
            default -> 0;
        };
        System.out.println("switch_destructuring, maxi="+maxi);
    }
}

```

2. Virtual Thread

2.1. Threads natifs et virtuels

Un **thread natif** est géré par le système d'exploitation (ex : linux ou windows)

Un **thread virtuel** n'est géré que par la machine virtuelle java . C'est possible depuis les versions 18,19,20,21 de java. En étant plus léger, un thread virtuel peut démarrer plus rapidement et le changement de contexte d'un thread à l'autre est également plus rapide.

2.2. Démarrage d'un thread virtuel

La classe *java.lang.Thread* comporte maintenant une méthode *.ofVirtual()* permettant de créer un nouveau thread virtuel .

Thread.ofVirtual()

```
.name(virtualThreadName)
//.start(runnable); //code automatiquement démarré
.unstarted(runnable); //code à démarrer via appel à .start() ou équivalent
```

Exemple :

```
import java.time.Duration;

public class MyThreadUtil {

    static void log(String message) {
        System.out.println(message + " by " + Thread.currentThread());
    }

    static void sleep(Duration duration) {
        try { Thread.sleep(duration); } catch (InterruptedException e) { e.printStackTrace(); }
    }

    static void sleep(long nbMs) {
        try { Thread.sleep(nbMs); } catch (InterruptedException e) { e.printStackTrace(); }
    }
}
```

```
public class MyRunnableUtil {
    public static Runnable prepareCoffeeRunnable() {
        return ()->{
            MyThreadUtil.log("prepare coffee");
            MyThreadUtil.sleep(Duration.ofMillis(1000));
            MyThreadUtil.log("coffee is ready");
        };
    }
}
```

```

public static void testSimpleVirtualThread() throws Exception {
    Thread vt1 = Thread.ofVirtual().name("vt1")
                .unstarted(MyRunnableUtil.prepareCoffeeRunnable());
    vt1.start();// démarrage de vt1
    vt1.join();//attente de la fin de vt1 depuis main Thread
}

```

==>

prepare coffee by VirtualThread[#21,vt1]/runnable@ForkJoinPool-1-worker-1

coffee is ready by VirtualThread[#21,vt1]/[runnable@ForkJoinPool-1-worker-1](#)

2.3. Threads virtuels via Executors

```

//one unnamed virtualThread for each concurrent task submitted to the executor
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    ....
}

```

```

public static void testViaExecutors() throws Exception {
    //one named virtualThread (generated by factory)
    //for each concurrent task submitted to the executor
    final ThreadFactory virtualThreadFactory =
        Thread.ofVirtual().name("routine-", 0).factory();
    //names of virtual threads will be "routine-0", "routine-1", ...
    try (var executor = Executors.newThreadPerTaskExecutor(virtualThreadFactory)) {
        Future<?> f1 = executor.submit(MyRunnableUtil.prepareJokeRunnable());
        Future<?> f2 = executor.submit(MyRunnableUtil.prepareCoffeeRunnable());

        Future<Integer> f3 = executor.submit(MyRunnableUtil.longComputationCallable());
        f1.get();// attente fin de f1
        f2.get();// attente fin de
        Integer resF3 = f3.get();// attente resultat de f3
        System.out.println("resF3="+resF3);
    }
}

```

Résultat :

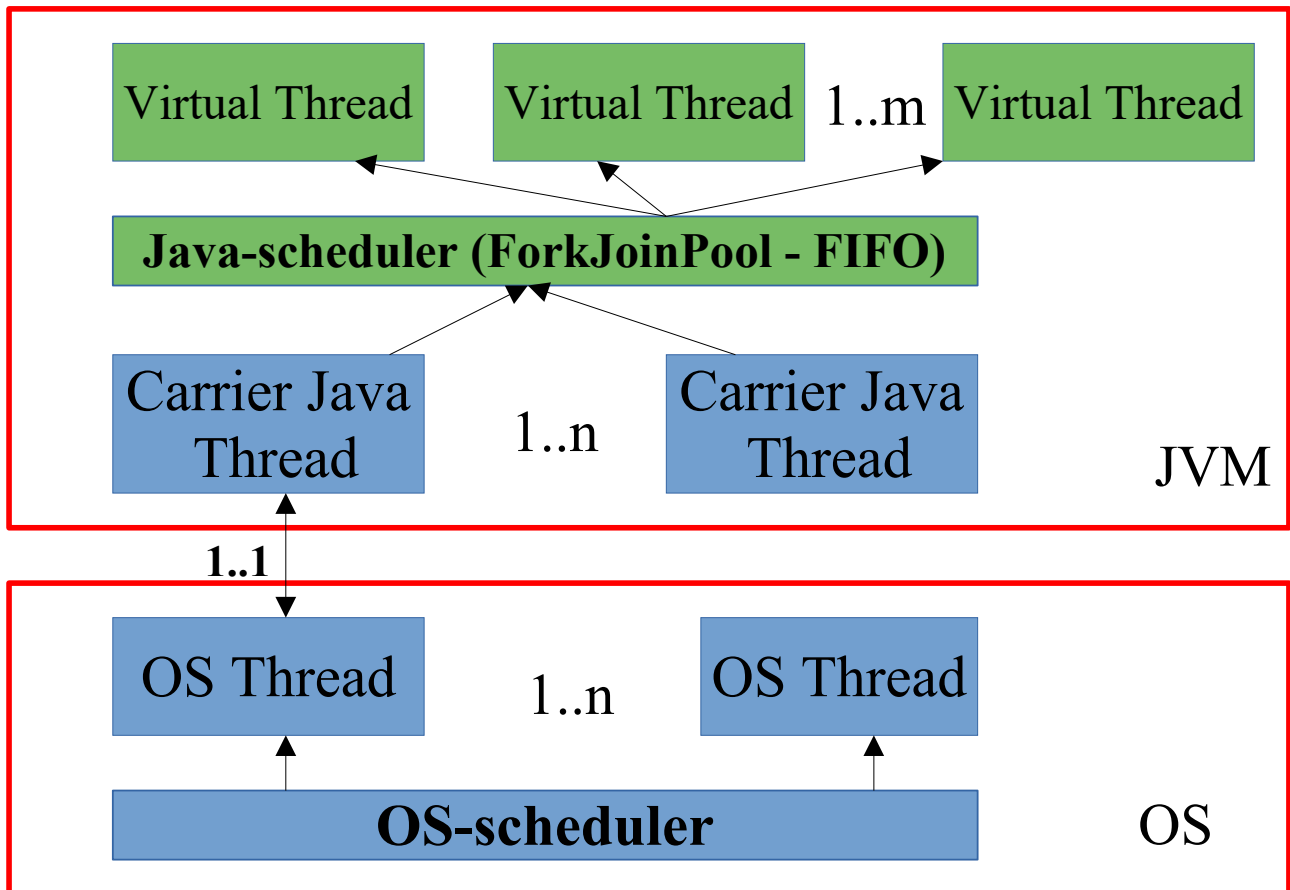
prepare a joke by VirtualThread[#21,routine-0]/runnable@ForkJoinPool-1-worker-1
 prepare coffee by VirtualThread[#23,routine-1]/runnable@ForkJoinPool-1-worker-2
 start computing by VirtualThread[#25,routine-2]/runnable@ForkJoinPool-1-worker-3
 joke is ready by VirtualThread[#21,routine-0]/runnable@ForkJoinPool-1-worker-1
 coffee is ready by VirtualThread[#23,routine-1]/runnable@ForkJoinPool-1-worker-1
 computing is ready by VirtualThread[#25,routine-2]/runnable@ForkJoinPool-1-worker-3
 resF3=5

2.4. CarrierPlatformThread

```
//[#virtualThreadId,virtualThreadName]/... carrierPlatformThread
```

```
//exemple : VirtualThread[#37,routine-8]/runnable@ForkJoinPool-1-worker-4
```

VirtualThreads and CarrierPlatformThread



La machine virtuelle java s'appuie en interne sur un pool de "carrier java thread" (mappé chacun avec un thread natif de l'OS) pour prendre en charge un paquet de threads virtuels.

Lorsqu'un thread virtuel est bloqué (en attente , en someil , ...) , celui ci peut être temporairement dissocié du "carrier java thread" porteur de manière à ce que ce " carrier java thread" puisse exécuter d'autres tâches plus utiles (autre "virtual thread") .

Le lien entre un "virtual_thread" et un " carrier java thread" peut évoluer dans le temps.

Exemple :

```
VirtualThread[#24,routine-2]/runnable@ForkJoinPool-1-worker-7
```

```
VirtualThread[#24,routine-2]/runnable@ForkJoinPool-1-worker-6
```

Par défaut sur une machine à n processeurs/coeurs , le "ForkJoinPool" comportera n "carrier java

thread" (ex : *ForkJoinPool-1-worker-1* , ... , *ForkJoinPool-1-worker-8*)

Cas d'utilisation :

Les threads virtuels apportent un plus dans le cas où beaucoup d'opérations concurrentes sont potentiellement bloquées (en attentes , exemple = appel synchrone vers api REST).

Par contre , les `VirtualThreads` n'apportent rien pour un usage intensif du CPU . Pour cela l'algorithme du `ForkJoin` et les `ParallelStream` sont déjà assez bien optimisés.

3. Structured Concurrency (preview java 21)

L'api "**Structured Concurrency**" (toujours en mode "preview" en java 21) sert à déclencher des sous tâches en parallèle (qui seront exécutées par des threads virtuels) tout en ayant la possibilité d'attendre (en mode "bloquant avec timeout") selon les besoins :

- soit la fin de toutes les sous-tâches
- soit le résultat de la sous-tâches la plus rapide

Exemple :

```
public class FakeLongTaskService {
    public static FakeLongTaskService INSTANCE = new FakeLongTaskService();

    public double slowSquareRoot(double x,int nbMs) throws InterruptedException{
        try {
            Thread.currentThread().sleep(nbMs);
        } catch (InterruptedException e) {
            System.err.println(Thread.currentThread()+"was interrupted before end of sleep="+nbMs);
            throw e;
        }
        if(nbMs<50) throw new RuntimeException("lunatic exception because nbMs<50");
        double res = Math.sqrt(x);
        System.out.println("slowSquareRoot(x="+x + ") returning res="+res
            + " after sleep="+ nbMs + " "+ Thread.currentThread());
        return res;
    }
}
```

En mode attente du résultat de la sous-tâches la plus rapide :

```
...
import java.time.Instant;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.TimeoutException;

public class WithFirstFinishedSubTask {

    public static WithFirstFinishedSubTask INSTANCE = new WithFirstFinishedSubTask();
    /*
     ShutdownOnSuccess captures the first result and shuts down the task scope
     to interrupt unfinished threads and wakeup the owner.
     This class is intended for cases where the result of any subtask will do ("invoke any")
     and where there is no need to wait for results of other unfinished tasks.
     It defines methods to get the first result or throw an exception if all subtasks fail.
     */

    public Double quickerOfSlowSquareRoot(double x) {
        Double firstFinishedResult=0.0;
        try (var scope = new StructuredTaskScope.ShutdownOnSuccess<Double>()) {
            //int extra = -200; //causing all subtaskToFail because xxx + extra < 50
            //int extra = 1000; //causing timeoutException
            int extra = 10; //no global error
            StructuredTaskScope.Subtask<Double> subtask1 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 200+extra);//200ms
            StructuredTaskScope.Subtask<Double> subtask2 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 100+extra);
            StructuredTaskScope.Subtask<Double> subtask3 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 50+extra);//50ms
        }
```

```

StructuredTaskScope.Subtask<Double> subtask4 =
    scope.fork(() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 40));
    //if less than 50 with negative extra ->lunatic Exception
//NB: sometimes futurRes4 throws a RuntimeException , it fails ,
//but "no problem" if at least another // subtask succeed

//scope.join();
scope.joinUntil(Instant.now().plusSeconds(1)); //with TimeoutException if all is not finish Is later now

//firstFinishedResult = scope.result(); //without specific exception
firstFinishedResult = scope.result(e -> new RuntimeException("cannot compute squareRoot"));
    //with specific exception if all failed (but not timeout)
System.out.println("firstFinishedResult="+firstFinishedResult);
} catch (InterruptedException e) {
    throw new RuntimeException(e);
} catch (TimeoutException e) {
    throw new RuntimeException(e);
} catch (/*ExecutionException*/ Exception e) {
    throw new RuntimeException(e);
}
}
return firstFinishedResult;
}
}

public static void testQuickerSubTask() {
    try {
        double res = WithFirstFinishedSubTask.INSTANCE.quickerOfSlowSquareRoot(9.0);
        System.out.println("quickerOfSlowSquareRoot() returning "+res);
    } catch (Exception e) {
        System.err.println("quickerOfSlowSquareRoot() throwing "+e.getMessage());
    }
}
}

```

Résultats :

slowSquareRoot(x=9.0) returning res=3.0 after sleep=60

VirtualThread[#200]/runnable@ForkJoinPool-1-worker-1

VirtualThread[#199]/runnable@ForkJoinPool-1-worker-8was interrupted before end of sleep=110

VirtualThread[#198]/runnable@ForkJoinPool-1-worker-7was interrupted before end of sleep=210

firstFinishedResult=3.0

quickerOfSlowSquareRoot() returning 3.0

En mode "attente de la fin de toutes les sous-tâches parallèles"

```

...
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.TimeoutException;

public class AllFinishedSubTasks {

    public static AllFinishedSubTasks INSTANCE = new AllFinishedSubTasks();
    /*
    ShutdownOnSuccess captures the first result and shuts down the task scope
    to interrupt unfinished threads and wakeup the owner.
    This class is intended for cases where the result of any subtask will do ("invoke any")
    and where there is no need to wait for results of other unfinished tasks.

```

It defines methods to get the first result or throw an exception if all subtasks fail.
 */

```
public List<Double> doubleListSlowSquareRoot(List<Double> doubleList) {
    List<Double> doubleResultList = new ArrayList<>();
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        List<StructuredTaskScope.Subtask<Double>> subtaskList =
            new ArrayList<StructuredTaskScope.Subtask<Double>>();
        //int extra = 1000; //causing timeoutException
        int extra = 10; //no global error
        for(Double val : doubleList) {
            StructuredTaskScope.Subtask<Double> subtask = scope.fork(
                () -> (Double) FakeLongTaskService.INSTANCE.slowSquareRoot(val, 100+extra)
            );
            subtaskList.add(subtask);
        }
        //scope.join();
        scope.joinUntil(Instant.now().plusSeconds(1)); //with timeout

        for(StructuredTaskScope.Subtask<Double> subtask : subtaskList) {
            doubleResultList.add(subtask.get());
        }
    } catch (InterruptedException e) { throw new RuntimeException(e);
    } catch (TimeoutException e) { throw new RuntimeException(e);
    } catch (/*ExecutionException*/ Exception e) { throw new RuntimeException(e); }
    return doubleResultList;
}

public static void testAllFinishedParallelSubTask() {
    try {
        List<Double> doubleList = Arrays.asList(9.0 , 4.0 , 25.0 , 16.0);
        System.out.println("doubleList="+doubleList);
        List<Double> resultList =
            AllFinishedSubTasks.INSTANCE.doubleListSlowSquareRoot(doubleList);
        System.out.println("doubleListSlowSquareRoot() returning "+resultList);
    } catch (Exception e) {
        System.err.println("doubleListSlowSquareRoot() throwing "+e.getMessage());
    }
}
}
```

Résultats :

doubleList=[9.0, 4.0, 25.0, 16.0]

*slowSquareRoot(x=4.0) returning res=2.0 after sleep=110
 VirtualThread[#203]/runnable@ForkJoinPool-1-worker-5
 slowSquareRoot(x=16.0) returning res=4.0 after sleep=110
 VirtualThread[#205]/runnable@ForkJoinPool-1-worker-1
 slowSquareRoot(x=25.0) returning res=5.0 after sleep=110
 VirtualThread[#204]/runnable@ForkJoinPool-1-worker-8
 slowSquareRoot(x=9.0) returning res=3.0 after sleep=110
 VirtualThread[#202]/runnable@ForkJoinPool-1-worker-7
doubleListSlowSquareRoot() returning [3.0, 2.0, 5.0, 4.0]*

4. Foreign Function and Memory (preview java 21)

L'api FFM (Foreign Function and Memory) est toujours en mode **preview** en java 21.

Cette api permet d'accéder à des fonctions et blocs mémoires externes à la JVM (ex : fonction native du langage "C") de manière efficace et rigoureuse (mieux que JNI ou JNA).

Attention:

sans arrêt , de petits changements au niveau de l'api FFM (Foreign Function and Memory) entre les previews des versions 18,19,20,21 (PAS STABLE DU TOUT)

Le code ci-dessous est pour le jdk 21

<https://openjdk.org/jeps/442> pour java 21 (third preview of FFM)

```
...
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.SymbolLookup;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;

public class FFMTestApp {
    public static void main(String[] args) throws Throwable {
        // 1. Get a lookup object for commonly used libraries
        SymbolLookup stdlib = Linker.nativeLinker().defaultLookup();

        // 2. Get a handle to the "strlen" function in the C standard library
        MethodHandle strlen = Linker.nativeLinker().downcallHandle(
            stdlib.find("strlen").orElseThrow(),
            FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));

        try (Arena offHeap = Arena.ofConfined()) { //version java 21

            // 3. Convert Java String to C string and store it in off-heap memory
            MemorySegment str = offHeap.allocateUtf8String("Happy Coding!");
                                                    //13 caracteres

            // 4. Invoke the foreign function
            long len = (long) strlen.invoke(str);

            System.out.println("len = " + len); //len=13 (ok)

        }
    }
}
```

5. Vector api (incubator java 21)

"**Vector api**" (toujours en mode "incubator" en java 21) sert à effectuer d'un seul coup des opérations mathématiques (ex : addition, multiplication,...) sur des tableaux de valeurs en utilisant certaines fonctionnalités pointues de certains micro-processeurs récents.

```
package tp.java_new_21plus.vector_api;
import jdk.incubator.vector.IntVector;

public class TestVectorApi {
    //in run configuration : VM arguments
    //--add-modules=jdk.incubator.vector

    public static void main(String[] args) {
        smallVectorAddition();
        largeVectorAdditionV1(); //with IntVector.SPECIES_512
        largeVectorAdditionV2(); //with IntVector.SPECIES_128 , 4 iterations
    }

    public static void display_intTab(String tabName, int[] values) {
        System.out.print(tabName+"={");
        for(int v : values)
            System.out.print(" " + v);
        System.out.print("}\n");
    }
}
//...
}
```

```
public static void smallVectorAddition() {
    int[] a = {1, 2, 3, 4};
    int[] b = {17, 18, 19, 20};
    int[] c = new int[4]; //4fois 32bits = 128bits

    // Scalar (ordinary) computation:
    //c[0] = a[0] + b[0];
    //...
    //c[15] = a[15] + b[15];
    for (int i = 0; i < c.length;i++)
        c[i] = a[i] + b[i];
    display_intTab("c",c);

    // Vector computation:
    IntVector va = IntVector.fromArray(IntVector.SPECIES_128, a, 0);
    IntVector vb = IntVector.fromArray(IntVector.SPECIES_128, b, 0);
    IntVector vc = va.add(vb); //add operation between 2 vector

    int[] c2 = new int[4];
    vc.toArray(c2, 0); //store vc in ordinary array of scalar
    display_intTab("c2",c2);
}
```

variante sans boucle :

```
public static void largeVectorAdditionV1() {
    int[] a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int[] b = {17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

    // Vector computation:
    int[] c2 = new int[16];
    IntVector va = IntVector.fromArray(IntVector.SPECIES_512, a, 0);
    IntVector vb = IntVector.fromArray(IntVector.SPECIES_512, b, 0);
    IntVector vc = va.add(vb); //add operation between 2 vector
    vc.toArray(c2, 0); //store vc in ordinary array of scalar
    display_intTab("c2", c2);
}
```

16 fois 32 = 512 (tableaux de 16 cases de int (32bits))

variante avec boucle et offset:

```
public static void largeVectorAdditionV2() {
    int[] a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int[] b = {17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

    // Vector computation:
    int[] c2 = new int[16];
    for (int offset = 0; offset < c.length; offset += IntVector.SPECIES_128.length()) {
        IntVector va = IntVector.fromArray(IntVector.SPECIES_128, a, offset);
        IntVector vb = IntVector.fromArray(IntVector.SPECIES_128, b, offset);
        IntVector vc = va.add(vb); //add operation between 2 vector
        vc.toArray(c2, offset); //store vc in ordinary array of scalar
    }
    display_intTab("c2", c2);
}
```

Résultats :

WARNING: Using incubator modules: jdk.incubator.vector

c={ 18 20 22 24}

c2={ 18 20 22 24}

c={ 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48}

c2={ 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48}

XXXII - Annexe – JShell

1. JShell (RPEL java)

1.1. Principe RPEL

REPL (Read-Eval-Print Loop)

Un langage interprété (ex : "lisp" , "basic" ou "javascript") peut (à un haut niveau , quelque fois proche d'une interaction utilisateur) effectuer une boucle dite "REPL" de ce type :

```
loop({
- read expression (ex : read "2*4+5" in variable expr )
- eval result (ex : res=eval(expr) = 13)
- print result (ex : console.log("res="+res) ; )
})
```

vision fonctionnelle :

```
(loop (print (eval (read()))))
```

```
expression: 2+3
result=5
expression: 4+4
result=8
expression: 2*6+1
result=13
expression: fin
end
```

1.2. Présentation de JShell java

Depuis java9 , JShell permet d'interpréter et exécuter du code à la volée (sans devoir absolument écrire du code dans une classe java) .

(exemple : C:\Program Files\Java\jdk-11.0.4\bin\jshell.exe)

```
Ctrl Invite de commandes - jshell
Microsoft Windows [version 10.0.18362.900]
(c) 2019 Microsoft Corporation. Tous droits réservés.

C:\Users\didier>jshell
| Welcome to JShell -- Version 11.0.4
| For an introduction type: /help intro

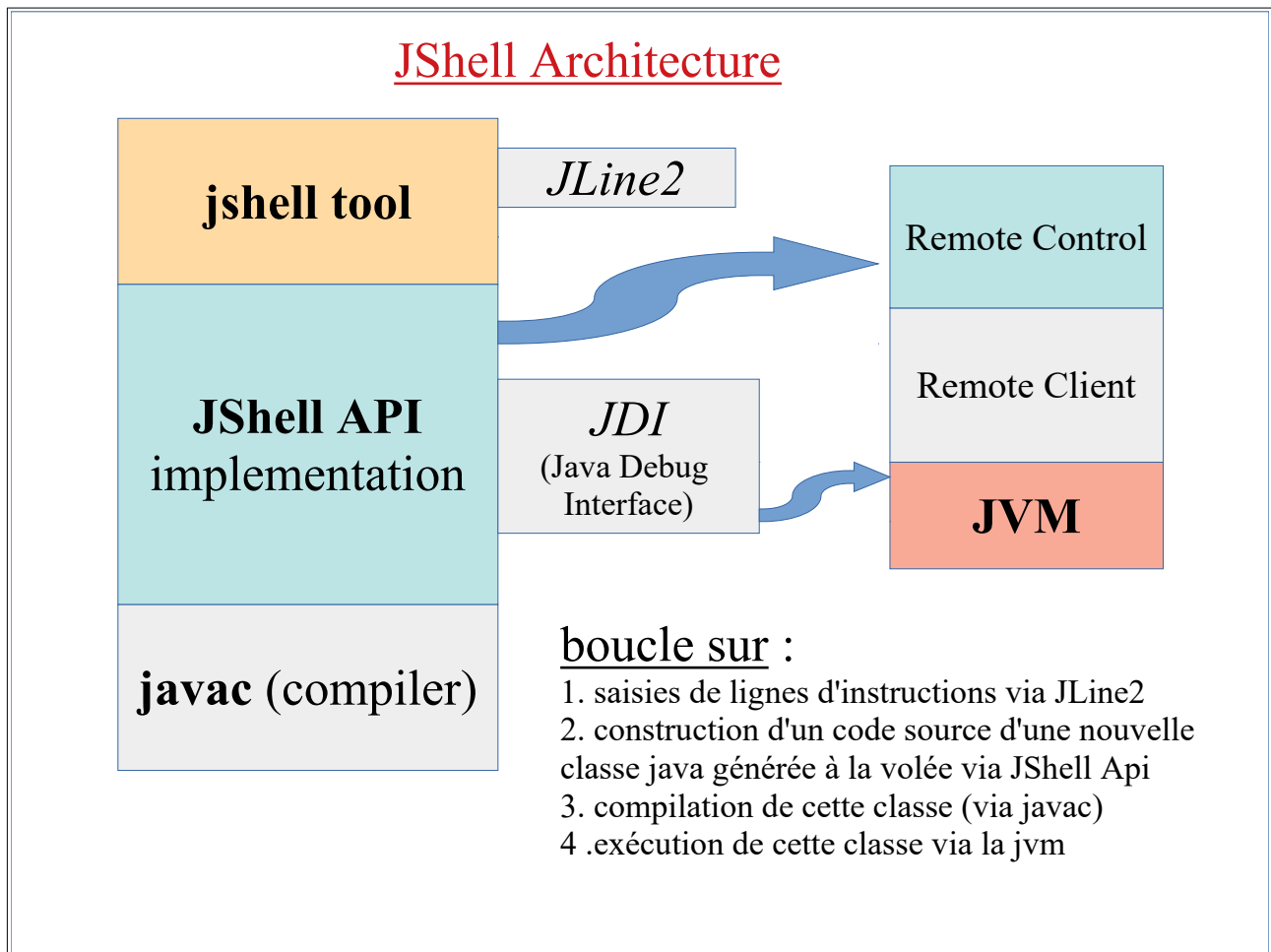
jshell> 2+2
$1 ==> 4

jshell> Math.sqrt(16)
$2 ==> 4.0

jshell> "fic1"+".txt"
$3 ==> "fic1.txt"

jshell>
```

1.3. Fonctionnement interne de JShell



1.4. Utilisation de JShell (exemples)

```
jshell> System.out.println("ok");
ok
```

```
jshell> 1+
...> 3
$2 ==> 4
```

```
jshell> for(int i=0;i<3;i++){
...> System.out.println("i="+i);
...> }
i=0
i=1
i=2
```

```
jshell> 2+2
$4 ==> 4
```

```
jshell> $4*3
$5 ==> 12
```

```
jshell> "abc"+"def"
$6 ==> "abcdef"
jshell> $6.toUpperCase()
$7 ==> "ABCDEF"
```

```
jshell> double x=5.5
x ==> 5.5

jshell> double y=6.6;
y ==> 6.6

jshell> x+y
$10 ==> 12.1
```

```
jshell> /vars
| int $2 = 4
| int $4 = 4
| int $5 = 12
| String $6 = "abcdef"
| String $7 = "ABCDEF"
| double x = 5.5
| double y = 6.6
| double $10 = 12.1
```

/vars affiche la liste des variables .

```
jshell> public int add(int a,int b){
...> return a+b;
...> }
| created method add(int,int)

jshell> add(5,6)
$12 ==> 11
```

```
jshell> /imports
| import java.io.*
| import java.math.*
| import java.net.*
| import java.nio.file.*
| import java.util.*
| import java.util.concurrent.*
| import java.util.function.*
| import java.util.prefs.*
| import java.util.regex.*
| import java.util.stream.*
```

```
jshell> class User{
...> String nom;
...> public User(){ nom="default name"; }
...> public void sayHello(){ System.out.println("Hello, my name is " + nom); }
...> }
| created class User

jshell> User u1=new User();
u1 ==> User@6e1567f1

jshell> u1.nom="toto"
$15 ==> "toto"

jshell> u1.sayHello()
Hello, my name is toto
```

```
jshell> /types
| class User
```

1.5. Pour approfondir le sujet jshell

https://blog.soat.fr/2018/03/java9_decouverte_jshell_1

...

XXXIII - Annexe – Optional , DateTime, ...

1. Optional , DateTime , ...

1.1. Optional<T>

`java.util.Optional` (depuis le jdk 1.8) correspond à un **conteneur (jamais "null") de valeur "objet" optionnelle** . Ceci un sémantiquement plus précis qu'une valeur "null" .

Depuis java 8 , le **type de retour** `Optional<T>` plutôt que T éventuellement null ne fait qu'**expliquer clairement** (*via le type , sans autre commentaire ou documentation supplémentaire*) **l'aspect optionnel d'une référence vers un objet en valeur de retour**

Cela pourrait correspondre à la **multiplicité 0..1** en UML mais malheureusement `Optional<T>` n'est prévu que pour n'être utilisé qu'au niveau des types de retour (pas sur un type d'attribut/propriété).
Principale raison : `Optional<T>` n'est pas bien gérée par la sérialisation JSON (valeur null préférée) ni par la persistance automatique de JPA/Hibernate (valeur null préférée).

Autrement dit, **`Optional<T>` est surtout utile pour éviter un `NullPointerException` en plein milieu d'un enchaînement de traitements via les streams de java >=8 .**

Exemple :

```
...
public static List<String> initWinnerList(){
    String[] winnerArray = {"bob", "anna", "alice"};
    List<String> winnerList = Arrays.asList(winnerArray);
    return winnerList;
}
public static Optional<String> getWinnerByName(String name){
    for(String s : initWinnerList()){
        if(s.equals(name))
            return Optional.of(s);
    } /*else*/
    return Optional.empty(); //instead of return null
}
public static void testOptional(){
    Optional<String> opStr = getWinnerByName("alice");
    System.out.println("Optional<String> opStr = " + opStr );
    if(opStr.isPresent())
        System.out.println(opStr.get());
    //with lambda expression:
    opStr.ifPresent( (s) -> System.out.println(s) );
}
```

```

opStr = getWinnerByName("looser");
System.out.println("Optional<String> opStr = " + opStr );
//System.out.println(opStr.get());
//java.util.NoSuchElementException instead of NullPointerException
opStr.ifPresent((s)->System.out.println(s));

//String str = "abc";
String str = null;
Optional<String> opS = Optional.ofNullable(str); //build .empty() if null
System.out.println("Optional<String> opS = " + opS );

String msg = opS.map((notNullStr)-> "not null string value : " + notNullStr)
                .orElse("empty optional");
System.out.println(msg);
}

```

NB :

optional.get(); retourne la valeur interne (non nulle) ou bien retourne une **exception** si vide/empty.

optional.orElse(null); retourne la valeur interne (non nulle) ou bien retourne **null** si vide/empty.

```

public static void testDiversAvecNouvellesMethodesDeOptionalDepuisJava9() {
    Optional<String> opS1 = Optional.of("s1");
    Optional<String> opS2 = /*Optional.ofNullable(null); */ Optional.empty();

    //op.or(...) permet de construire et fournir un autre optional (valeur par défaut
    // ou plan B) si l'optionnel original est null/empty :
    System.out.println(opS1.or(()->Optional.of("default_string"))); //affiche Optional[s1]
    System.out.println(opS2.or(()->Optional.of("default_string")));
    //affiche Optional[default_string]

    //op.ifPresentOrElse(nomEmptyConsumer_as_lambda , emptyActionLambda)
    //permet de déclencher alternativement une lambda ou une autre en fonction d'un
    // contenu vide ou pas , pas d'enchaînement après (opération terminale en "void")
    opS1.ifPresentOrElse( (value) -> System.out.println("opS1="+value),
                        () -> System.out.println("value of opS1 is empty"));
    //affiche opS1=s1
    opS2.ifPresentOrElse( (value) -> System.out.println("opS2="+value),
                        () -> System.out.println("value of opS2 is empty"));
    //affiche value of opS2 is empty
}

```

1.2. LocalTime , LocalDate (de java.time)

java.time. **LocalTime** , **LocalDate** (d'inspiration JodaTime) sont plus simples à utiliser que **Date** et **Calendar**

Exemple :

```
public static void testLocalTimeAndLocalDate(){
LocalTime now = LocalTime.now();      System.out.println("now is " + now);
LocalTime later = now.plus(8, ChronoUnit.HOURS);
System.out.println("later (now+8h) is " + later);

LocalDate today = LocalDate.now();      System.out.println("today is " + today);
LocalDate thirtyDaysFromNow = today.plusDays(30);
System.out.println("thirtyDaysFromNow is " + thirtyDaysFromNow);
LocalDate nextMonth = today.plusMonths(1); System.out.println("nextMonth is " + nextMonth);
LocalDate aMonthAgo = today.minusMonths(1); System.out.println("aMonthAgo is " + aMonthAgo);

//LocalDate date14July2015 = LocalDate.of(2015, 7, 14);
LocalDate date14July2015 = LocalDate.of(2015, Month.JULY, 14);

LocalTime time = LocalTime.of(14 /*h*/, 15 /*m*/, 0 /*s*/);
LocalDateTime datetime = date14July2015.atTime(time);
System.out.println("le 14 juillet 2015 à 14h15 : " + datetime);

LocalDate date1=today , date2=nextMonth;
Period p1 = Period.between(date1, date2) ;   System.out.println("periode p1:" + p1);

LocalTime time1= time;
LocalTime time2= LocalTime.of(14 /*h*/, 30 /*m*/, 0 /*s*/);
Duration d = Duration.between(time1, time2);   System.out.println("durée d:" + d);

Duration twoHours = Duration.ofHours(2); System.out.println("durée de 2 heures:" + twoHours);
Duration tenMinutes = Duration.ofMinutes(10); System.out.println("durée de 10 minutes:" + tenMinutes);
Duration thirtySecs = Duration.ofSeconds(30); System.out.println("durée de 30 secondes:" + thirtySecs)
LocalTime t2 = time.plus(twoHours);      System.out.println("14h15 plus 2 heures:" + t2);

//.with(temporalAdjuster) :
LocalDate premierJourDeCetteAnnee=
```

```

LocalDate.now().with(TemporalAdjusters.firstDayOfYear());
System.out.println("premierJourDeCetteAnnee="+premierJourDeCetteAnnee);
LocalDate dernierJourDuMois= LocalDate.now().with(TemporalAdjusters.lastDayOfMonth());
System.out.println("dernierJourDuMois="+dernierJourDuMois);

ZoneId myZone = ZoneId.systemDefault();
System.out.println("my (local) zoneId is:" + myZone);

//lien entre java.util.Date et java.time... :
Date date = new Date();
Instant nowInstant = date.toInstant();
LocalDateTime dateTime = LocalDateTime.ofInstant(nowInstant, myZone);
System.out.println("today/now from Date:" + dateTime);
}

```

```

LocalDateTime now = LocalDateTime.now();
System.out.println("basic/default display of LocalDateTime.now() :"+ now);
//ex :2021-11-23T09:28:12.289915600

Instant instantT = now.atZone(ZoneId.systemDefault()).toInstant();
long nbMsSinceFirstJanuary1970GMT = instantT.toEpochMilli();
System.out.println("instantT.toEpochMilli() , timestamp , nb ms since 1970-01-01 00:00:00
GMT=" + nbMsSinceFirstJanuary1970GMT); //ex : 1637656092289

```

```

LocalDate nowDate = LocalDate.now();
System.out.println("basic/default display : today (nowDate local) is " + nowDate); //ex :2021-11-23
String sdate_fr_2=nowDate.format(DateTimeFormatter.ofPattern("EEEE, dd MMMM
yyyy",Locale.FRENCH)); // EEEE means fullname of weekday
System.out.println("sdate_fr_2="+sdate_fr_2); //exemple: dimanche, 13 septembre 2020

```

1.3. Encodage base64 (java.util.Base64)

```

public static void testBase64(){
    try {
        // Encode using basic encoder :
        String base64encodedString =
            Base64.getEncoder().encodeToString("Myjava8String".getBytes("utf-8"));
        System.out.println("Base64 Encoded String (Basic) :"+ base64encodedString);
    }
}

```

```
// Decode :  
byte[] base64decodedBytes = Base64.getDecoder().decode(base64encodedString);  
System.out.println("Original String: " + new String(base64decodedBytes, "utf-8"));  
} catch (UnsupportedEncodingException e) {  
    e.printStackTrace();  
}  
}
```

1.4. Diverses autres nouveautés de java 8

Amélioration de l'introspection si option "-arguments" au lancement du compilateur "javac"

→ `parameter.getName()` retourne véritable nom du paramètre (stocké dans `byteCode`) plutôt que "arg0", "arg1", ...

`@Repeatable` ([java.lang.annotation](#)) , ...

XXXIV - Annexe – Lombok

1. Lombok

1.1. Fonctionnalités de lombok et principes de fonctionnement

Beaucoup de frameworks java nécessitent que l'on programme des classes java avec tout un tas de choses classiques mais répétitives (getter/setter , constructeurs , méthode toString() ,).

De façon à gagner du temps dans la programmation et à aérer le code source on peut éventuellement s'appuyer sur des annotations telles que @Getter , @Setter de la technologie lombok de façon à ce que les méthodes getXxx() et setXxx() soient automatiquement générées avec le code basique par défaut . Il est heureusement toujours possible de personnaliser quelques getters/setters .

Depuis le jdk 1.6 (et Pluggable Annotation Processing API (JSR 269)) , les annotations de rétention "SOURCE" , telles de celles de lombok sont détectées dans le code source à compiler et traitées automatiquement durant la phase de compilation à partir du moment où le classpath comporte les ".jar" des traitements associés aux annotations. Il n'est donc plus nécessaire de lancer explicitement l'utilitaire apt du jdk 1.5 .

Lorsque la compilation du code est gérée par la technologie maven, les annotations de lombok (@Getter , @Setter , ...) sont interprétées et traitées automatiquement (si la dépendance maven "lombok" est présente dans pom.xml) .

1.2. Dépendances "maven" pour lombok

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.30</version> <!-- or via spring boot version -->
</dependency>
```

1.3. Installation de l'agent lombok dans un IDE (ex : eclipse)

Dans le cas particulier d'une compilation lancée par eclipse , une synchronisation doit être établie entre le code source et le code compilé (dès chaque enregistrement du code source modifié).

De façon à ce que les annotations de lombok soient bien prises en compte par l'IDE eclipse (ou autre), une configuration doit être effectuée. Celle-ci peut s'effectuer de la façon suivante :

<https://projectlombok.org/p2> (menu **help**, **install new software** , **add site**)

ou bien

java -jar lombok-1.18.30.jar (ou double click sous windows)
pour modifier **eclipse.ini** (avant un redémarrage)



→ un redémarrage de l'IDE (ex : eclipse) est nécessaire.

1.4. Paramétrages (@Getter, @Setter,...) et utilisation

```
import lombok.AllArgsConstructor; import lombok.NoArgsConstructor;
import lombok.Getter; import lombok.Setter; import lombok.ToString;
```

@Getter @Setter

@ToString

//@EqualsAndHashCode

@NoArgsConstructor @AllArgsConstructor

//-----

@Entity @Table(name="Customer")

```
public class Customer {
    @Id
    private Long id;
    private String name;
}
```

@NoArgsConstructor	Génère un constructeur par défaut (avec aucun argument)
@AllArgsConstructor	Génère un constructeur avec tous les arguments/champs Attention : si d'une version à l'autre le nombre de champs évolue ça peut rendre le code d'utilisation instable .
@RequiredArgsConstructor	Génère un constructeur avec tous les arguments/champs qui sont soit déclarés "final" soit préfixé par @NotNull C'est utilisé au sein du framework Spring pour générer un constructeur opérant des injections de dépendances

Attention :

@ToString génère par défaut un code prenant en compte tous les champs . Dans le cas de relation bi-directionnelle (en JPA) , ça peut éventuellement conduire à des boucles infinies .

@Slf4j (à placer juste au dessus d'une classe Cxyz) permet de générer automatiquement la ligne de code suivante souvent utile :

```
private final Logger log= LoggerFactory.getLogger(Cxyz.class);
```

avec `import org.slf4j.Logger; import org.slf4j.LoggerFactory;`

Le développeur n'a plus qu'à utiliser l'objet log via `log.error(...)` , `log.warning(...)` `log.debug(...)`

@Builder quelquefois pratique :

```
@Getter
@Builder
public class Produit {
    private final String name;
    private final int id;
}
```

⇒

```
Produit p1 = Produit.builder().name("p1").id(1).build();
```

L'annotation **@Data** de Lombok est une annotation de commodité regroupant les annotations **@ToString**, **@EqualsAndHashCode**, **@Getter**, **@Setter** et **@RequiredArgsConstructor**. Elle permet de générer tout le code-boilerplate en une seule ligne.

C'est généralement utilisable sur des classes simples (ex : DTO) où il n'y a pas de relation bidirectionnelle .

XXXV - Annexe – MapStruct

1. MapStruct (efficient Dto converter)

1.1. Configuration maven pour MapStruct

pom.xml

```
...
<properties>
    <mapstruct.version>1.5.2.Final</mapstruct.version>
    <m2e.apt.activation>jdt_apt</m2e.apt.activation>
</properties>
<dependencies>
    ...
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>${mapstruct.version}</version>
    </dependency>
    ...
</dependencies>
<build>
<plugins>
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.10.1</version>
        <configuration>
            <source>${java.version}</source>
            <target>${java.version}</target>
            <release>${java.version}</release>
            <annotationProcessorPaths>
                <path>
                    <groupId>org.projectlombok</groupId>
                    <artifactId>lombok</artifactId>
                    <version>1.18.24</version>
                </path>
                <!-- Mapstruct should follow the lombok path(s) -->
                <path>
                    <groupId>org.mapstruct</groupId>
                    <artifactId>mapstruct-processor</artifactId>
```

```

        <version>${mapstruct.version}</version>
        </path>
    </annotationProcessorPaths>
</configuration>
</plugin>    ...
</plugins></build></project>

```

1.2. Intégration de MapStruct dans l'IDE eclipse

pour que mapStruct s'intègre bien à eclipse:

- 1) installer le plugin *m2e-apt* via **Help / Eclipse marketPlace**
- 2) **project/properties .../**
maven /annotation processing
/enable specific
/ automatically configure JDT APT

1.3. Utilisation de MapStruct

MyMapper.java

```

package tp.appliSpring.converter;

import org.mapstruct.InheritConfiguration;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.factory.Mappers;
import ....

//@Mapper // MyMapper.INSTANCE...
@Mapper(componentModel = "spring") //for @Autowired
public interface MyMapper {

    MyMapper INSTANCE = Mappers.getMapper( MyMapper.class );

    OperationDto operationToOperationDto(Operation source);
    Operation operationDtoToOperation(OperationDto source);

    @Mapping(target="number", source="numero")
    @Mapping(target="firstName", source="prenom")
    @Mapping(target="lastName", source="nom")
    @Mapping(target="address", source="adresse")
    ClientDto clientToClientDto(Client source);

    @InheritConfiguration
    @Mapping(target="comptes", source="comptes" , resultType = CompteDto.class)
    ClientDtoEx clientToClientDtoEx(Client source);

```

```

    @Mapping(target="numero", source="number")
    @Mapping(target="prenom", source="firstName")
    @Mapping(target="nom", source="lastName")
    @Mapping(target="adresse", source="address")
    @Mapping(ignore = true, target="comptes" )
    Client clientDtoToClient(ClientDto source);

    @InheritConfiguration
    @Mapping(ignore = false, target="comptes" )
    Client clientDtoExToClient(ClientDtoEx source);

    CompteDto compteToCompteDto(Compte compte);
    Compte compteDtoToCompte(CompteDto compteDto);
}

```

La classe d'implémentation est automatiquement générée à partir de l'interface lors du build (maven et/ou eclipse)

Exemple de code généré (dans `\target\generated-sources\annotations\tp\apliSpring\converter\MyMapperImpl.java`) :

```

public class MyMapperImpl implements MyMapper {
    ..
    public ClientDto clientToClientDto(Client source) {
        if ( source == null ) {
            return null;
        }
        ClientDto clientDto = new ClientDto();
        clientDto.setNumber( source.getNumero() );
        clientDto.setFirstName( source.getPrenom() );
        clientDto.setLastName( source.getNom() );
        clientDto.setAddress( source.getAdresse() );
        clientDto.setEmail( source.getEmail() );
        return clientDto;
    } ...
}

```

Utilisation sans injection Spring :

```

return MyMapper.INSTANCE.clientToClientDtoEx(client);

```

Utilisation avec injection Spring :

```

...
@Autowired //ou @Resource ou autre
private MyMapper myMapper;

public ClientDtoEx searchCustomerWithAccountsById(Long numClient) {
    Client client = daoClient.findWithAccountById(numClient);
    return myMapper.clientToClientDtoEx(client);
}

```

1.4. MyGenericMapper s'appuyant sur MapStruct

MyGenericMapper.java

```
package tp.appliSpring.converter;

import java.lang.reflect.Method; import java.util.List; import java.util.stream.Collectors;
import org.springframework.beans.BeanUtils;

public class MyGenericMapper {

    static MyMapper mapper = MyMapper.INSTANCE;

    static String withFirstLowerCase(String s){
        return Character.toLowerCase(s.charAt(0)) + s.substring(1);
    }

    @SuppressWarnings("unchecked")
    public static <S,D> D map(S source,Class<D> destinationClass) {
        D destination = null;
        try {
            //With mapStruct
            String convertMethodName=withFirstLowerCase(source.getClass().getSimpleName()
+ "To" + destinationClass.getSimpleName());
            //System.out.println("convertMethodName="+convertMethodName);
            Method convertMethod=mapper.getClass().getDeclaredMethod(convertMethodName,
source.getClass());
            if(convertMethod!=null) {
                destination = (D) convertMethod.invoke(mapper, source);
            }else {
                //without mapStruct (as fault back)
                destination = destinationClass.getDeclaredConstructor().newInstance();
                BeanUtils.copyProperties(source, destination);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return destination;
    }

    public static <S,D> List<D> map(List<S> sourceList , Class<D> destinationClass){
        return sourceList.stream()
        .map((source)->map(source,destinationClass))
        .collect(Collectors.toList());
    }
}
```

Exemple d'utilisation :

```
return MyGenericMapper.map(client,ClientDtoEx.class);
```

XXXVI - Annexe – NIO

1. NIO2 (new IO)

1.1. Introduction à nio , nio2

Dès l'origine du langage java , la classe File du package java.io était disponible pour accéder aux éléments du "file system" (fichier , répertoire, ...) et pour effectuer des opérations dessus (tester l'existence d'un fichier , supprimer des fichiers , ...).

Bien qu'opérationnel , le package java.io (et la classe File) ont quelques limitations importantes :

- La classe File manque de fonctionnalité importante telle qu'une méthode "copy".
- Beaucoup de méthode retourne des booléens plutôt que des exceptions . Ce qui rend assez délicate l'analyse des problèmes (raison exacte de l'erreur?).
- Pas de bonne gestion des liens symboliques .
- Un ensemble très limité d'attributs sur les fichiers sont disponibles sur java.io.File.

Pour surmonter certaines lacunes , un nouveau package "java.nio" a été introduit dès java4 de façon à apporter quelques améliorations très techniques telles que les suivantes :

- *Channels et Selectors*: un "channel" est une abstraction d'une caractéristiques de bas niveau telle qu'un fichier mappé en mémoire .
- *Buffers*: Buffering pour toutes les classes wrapper/primitives (sauf pour Boolean).
- *Jeu de caractères*: **Charset** (*java.nio.charset*), avec encodeurs et décodeurs entre bytes[] et symboles "Unicode".

La version 7 de java a introduit un nouveau (gros) package "**java.nio.file**" (alias nio2) .

Ce package apporte à la fois de nouvelles fonctionnalités techniques telles que :

- gestion des liens symboliques
- bonne gestions des "Path" (aspect bien séparé).
- meilleur gestion des "File attributes"

et de nouvelles syntaxes (plus concises , plus "orienté objet") telles que les remontées d'exceptions.

Attention : Entre java1, java4 , java7 , des ajouts , mais pas de suppression (pour garder une compatibilité avec les anciennes versions).

Bien que la classe "java.io.File" n'est pas officiellement considérée comme obsolète / "deprecated" , il est très conseillé d'utiliser les nouvelles classes de nio2 (sachant qu'il existe des passerelles entre "java.io" et "java.nio.file") .

2. Principales classes et interfaces de nio2

NIO 2 (associé aux packages "**java.nio.file**" et "**java.nio.file.attribute**") repose sur plusieurs classes et interfaces dont les principales sont :

- **Path** : encapsule un chemin dans le système de fichiers

- **Files** : avec méthodes statiques pour manipuler les éléments du système de fichiers
- *FileSystemProvider* : fournisseur technique (implémentation) de FS
- **FileSystem** : encapsule un système de fichiers
- FileSystems : fabrique permettant entre autres de créer une instance de FileSystem
- **FileStorage** : système de stockage (ex : partition / volume logique ,)

Quelques équivalences/transpositions de fonctionnalités basiques entre "java.io" et "nio2" :

Fonctionnalité	java.io	NIO 2
Encapsuler un chemin	java.io.File	java.nio.file.Path
Vérifier les permissions	File.canRead(), File.canCrite() et File.canExecute()	Files.isReadable(), Files.isWritable() et Files.isExecutable().
Vérifier le type d'élément	File.isDirectory(), File.isFile()	Files.isDirectory(Path, LinkOption...), Files.isRegularFile(Path, LinkOption...),
Taille d'un fichier	File.length()	Files.size(Path)
Obtenir ou modifier la date de dernière mise à jour	File.lastModified() , File.setLastModified(long)	Files.getLastModifiedTime(Path, LinkOption...), Files.setLastModifiedTime(Path, FileTime)
Modifier les attributs	File.setExecutable(), File.setReadable(), File.setReadOnly(), File.setWritable()	Files.setAttribute(Path, String, Object, LinkOption...)
Déplacer un fichier	File.renameTo()	Files.move()
Supprimer un fichier	File.delete()	Files.delete()
Créer un fichier	File.createNewFile()	Files.createFile()
Créer un fichier temporaire	File.createTempFile()	Files.createTempFile(Path, String, FileAttributes<?>), Files.createTempFile(Path, String, String, FileAttributes<?>)
Tester l'existence d'un fichier	File.exists	Files.exists() ou Files.notExists()
Obtenir le chemin absolu	File.getAbsolutePath() ou File.getAbsolutePath()	Path.toAbsolutePath()
Chemin canonique (sans ".", "..")	File.getCanonicalPath() ou File.getCanonicalFile()	Path.toRealPath() ou Path.normalize()
Convertir en URI	File.toURI()	Path.toURI()
L'élément est-il caché	File.isHidden()	Files.isHidden()
Obtenir le contenu d'un répertoire	File.list() ou File.listFiles()	Path.newDirectoryStream()
Créer un répertoire	File.mkdir() ou File.mkdirs()	Path.createDirectory()
Obtenir le contenu du répertoire racine	File.listRoots()	FileSystem.getRootDirectories()
Place totale , libre ,	File.getTotalSpace()	FileStore.getTotalSpace()

... sur FS	File.getFreeSpace()	FileStore.getUnallocatedSpace()
------------	---------------------	---------------------------------

3. Gestion des chemins (Path)

L'interface "java.nio.file.**Path**" est une représentation abstraite d'un chemin (relatif ou absolu) au sein d'un système de fichiers.

Un chemin peut référencer un fichier , un répertoire , un lien symbolique , un sous-chemin , ...

Les instances de "Path" sont "immutables" et peuvent être utilisées dans un contexte multi-threads.

3.1. Obtention d'une instance de "Path"

Récupération d'un chemin en appelant explicitement **getPath()** sur une **instance de "FileSystem"** :

```
Path chemin = FileSystems.getDefault().getPath(
    "C:/Users/powerUser/Temp/monfichier.txt");
```

équivalent indirect via la méthode (utilitaire / "helper") statique **Paths.get()** :

```
Path chemin = Paths.get("C:/Users/powerUser/Temp/monfichier.txt");
Path chemin2 = Paths.get(URI.create("file:///C:/Users/powerUser/Temp/monfichier.txt"));
Path chemin3 = Paths.get(System.getProperty("java.io.tmpdir"), "monfichier.txt");
```

NB : **Path.of()** existe depuis java 11 et se comporte comme **Paths.get()**

NB : bien que (sous windows) on puisse utiliser le séparateur "\" (ex : C:\\RepXy\\monfichier.txt) , il vaut mieux utiliser le séparateur "/" (plus portable et plus simple) .

Passerelle "io / nio2" (depuis java 7) :

En partant d'une instance de l'ancienne classe **java.io.File** , on peut appeler la nouvelle méthode **".toPath()** pour récupérer une instance de "java.nio.file.Path".

3.2. Obtention des éléments d'un chemin

Méthode	Rôle
String getFileName()	Retourne le nom du dernier élément du chemin. Si le chemin concerne un fichier alors c'est le nom du fichier qui est retourné
Path getName(int index)	Retourne l'élément du chemin dont l'index est fourni en paramètre. Le premier élément (hors racine) possède l'index 0 et correspond à la première partie sous la racine (ex : "windows" sous racine " <u>c:/</u> ").
int getNameCount()	Retourne le nombre d'éléments du chemin (hors racine)
Path getParent()	Retourne le chemin parent ou null s'il n'existe pas
Path getRoot()	Retourne la racine d'un chemin absolu (par exemple C:\ sous windows ou / sous Unix) ou null pour un chemin relatif
String toString()	Retourne le chemin sous la forme d'une chaîne de caractères

Path subPath (int <i>beginInclusiveIndex</i> , int <i>endExclusiveIndex</i>)	Retourne un sous-chemin (hors racine) [beginIndex,endIndex[
--	---

Exemple avec un chemin absolu :	Exemple avec un chemin relatif :
path=C:\Windows\Fonts\arial.ttf toString() = C:\Windows\Fonts\arial.ttf getFileName() = arial.ttf getRoot() = C:\ getName(0) = Windows getNameCount() = 3 getParent() = C:\Windows\Fonts subpath(0,2) = Windows\Fonts	path=Fonts\arial.ttf toString() = Fonts\arial.ttf getFileName() = arial.ttf getRoot() = null getName(0) = Fonts getNameCount() = 2 getParent() = Fonts subpath(0,2) = Fonts\arial.ttf

3.3. Manipulation , combinaison et conversions de chemins

Méthode	Rôle
Path normalize ()	Normaliser (ou rendre "canonique") un chemin en supprimant les éléments non indispensables « . » et « .. » qu'il contient.
Path relativize (Path other)	Retourner le chemin relatif permettant d'aller du "path courant" vers celui fourni en paramètres .
Path resolve (Path relative)	Combiner deux chemins (courant + relatif_en_arg) pour former global
Path toAbsolutePath ()	Retourne un chemin absolu (en <i>tenant compte du répertoire courant du contexte d'exécution</i> (idem à "pwd")).
Path toRealPath (LinkOption...)	Retourner le chemin physique du chemin notamment en résolvant les liens symboliques selon les options fournies . Peut lever une exception si le fichier au bout du chemin courant (absolu ou relatif) n'existe pas .
URI toUri ()	Retourner le chemin sous la forme d'une URI (file:///)

Exemple1 :

```
path.toString() = C:\Utilisateurs\..\Windows\..\Fonts\arial.ttf
path.normalize() = C:\Windows\Fonts\arial.ttf
```

```
path.toString() = ..\..\Fonts\..\arial.ttf
path.normalize() = ..\..\Fonts\arial.ttf      (seul le ../ inutile supprimé)
```

Exemple2 :

```
Path windowsFontPath = Paths.get("c:/Windows/Fonts");
Path usersPath = Paths.get("c:/Utilisateurs");
//relative path from current to arg
Path relativePathFromUsersToWindowsFonts= usersPath.relativize(windowsFontPath);
System.out.println("relative path=" + relativePathFromUsersToWindowsFonts) ;
```



```
relative path=..\Windows\Fonts
```

Exemple3 :

```
Path windowsPath = Paths.get("c:/Windows");
Path arialRelativePath = Paths.get("Fonts/arial.ttf");
Path globalPath = windowsPath.resolve(arialRelativePath);
System.out.println("globalPath="+globalPath);
```

```
globalPath=c:\Windows\Fonts\arial.ttf
```

```
Path windowsFontsPath = Paths.get("C:/Windows/Fonts");
System.out.println("uri=" + windowsFontsPath.toUri()); //java.net.URI
```

```
uri=file:///C:/Windows/Fonts/
```

```
Path relativePath = Paths.get(".");
Path absolutePath = relativePath.toAbsolutePath(); //selon rep courant (pwd)
System.out.println("absolutePath=" + absolutePath);
```

```
absolutePath=C:\Users\didier\workspace\test_j7_j8\.
```

Rappel : sous linux , un lien symbolique se construit via la commande

\$ ln -s /nom_du_dossier_source nom_du_lien

Attention, sous windows , un lien symbolique ne doit pas être confondu avec un raccourci .
A l'époque de windows XP , un lien symbolique se construisait avec la commande "*junction.exe*" (à installer).

Depuis "Vista" , et encore aujourd'hui avec windows 7, 8 et 10, un lien symbolique se construit avec la commande **mklink** suivante :

MKLINK [[/D] | [/H] | [/J]] Lien Cible

/D : Crée un lien symbolique vers un répertoire. Par défaut, il s'agit d'un lien symbolique vers un fichier.

/H : Crée un lien réel à la place d'un lien symbolique.

/J : Crée une jonction de répertoires.

Lien : Spécifie le nom du nouveau lien symbolique.

Cible : Spécifie le chemin d'accès (relatif ou absolu) auquel le nouveau lien fait référence.

Exemple (à lancer avec des droits "administrateur"):

```
cd c:\tmp\bb
```

```
mklink /J lien_vers_aa c:\tmp\aa
```

jonction créée pour lien_vers_aa <==> c:\tmp\aa

```
Path path = Paths.get("C:/tmp/bb/raccourci_vers_aa/f1.txt");
→ java.nio.file.NoSuchFileException: C:\tmp\bb\raccourci_vers_aa\f1.txt
```

```
Path path = Paths.get("C:/tmp/bb/lien_vers_aa/f1.txt");
try {
    System.out.println("chemin indirect sans suivre résolution lien symbolique="
        + path.toRealPath(LinkOption.NOFOLLOW_LINKS));
    System.out.println("chemin direct suivant résolution lien symbolique="
        + path.toRealPath());
} catch (IOException e) {
    e.printStackTrace();
}
```

```
chemin indirect sans suivre résol lien symbolique=C:\tmp\bb\lien_vers_aa\f1.txt
chemin direct suivant résolution lien symbolique=C:\tmp\aa\f1.txt
```

3.4. Comparaisons de chemins

La méthode `.equals()` permet de tester si deux chemins ont des valeurs identiques.

L'interface **Path** hérite de **Comparable**. Les chemins peuvent donc être automatiquement **triés**.

L'interface **Path** comporte en outre les méthodes de comparaison spécifiques suivantes :

Méthode	Rôle
int compareTo (Path other) (Comparable<Path>)	Comparer le chemin avec celui fourni en paramètre et retourne 0 si identiques, <0 avant, >0 si après
boolean endsWith (Path other) boolean endsWith (String other)	Comparer la fin du chemin avec celui fourni en paramètre
boolean startsWith (Path other) boolean startsWith (String other)	Comparer le début du chemin avec celui fourni en paramètre

3.5. interface PathMatcher avec méthode matches(path) et "glob"

Un "glob" est une expression basée sur certains méta-caractères qui seront comparés à des parties de "path" .

```
Path path = Paths.get("C:/tmp/aa/fl.txt");
PathMatcher txtMatcher = FileSystems.getDefault().getPathMatcher("glob:*.txt");
if (txtMatcher.matches(path.getFileName())) {
    System.out.println(path + " reference un fichier texte");
}
```

Motif (dans glob)	Rôle associé / correspondance
*	Aucun ou plusieurs caractères
**	Aucun ou plusieurs sous-répertoires
?	Un caractère quelconque
{ }	Un ensemble de motifs exemple : {htm, html}
[]	Un ensemble de caractères. Exemple : [A-Z] : toutes les lettres majuscules [0-9] : tous les chiffres [a-z,A-Z] : toutes les lettres indépendamment de la casse Chaque élément de l'ensemble est séparé par un caractère virgule Le caractère - permet de définir une plage de caractères A l'intérieur des crochets, les caractères *, ? et / ne sont pas interprétés

\	Il permet d'échapper des caractères pour éviter qu'ils ne soient interprétés. Il sert notamment à échapper le caractère \ lui-même
Les autres caractères	Ils se représentent eux-mêmes sans être interprétés

Quelques exemples :

<i>Glob</i>	<i>Correspondances</i>
*.html	tous les fichiers ayant l'extension .html
???	trois caractères quelconques
[0-9]	tous les fichiers qui contiennent au moins un chiffre
*.{htm, html}	tous les fichiers dont l'extension est htm ou html
Test*.java	tous les fichiers dont le nom commence par un Test et possède une extension .java

NB : L'interface **PathMatcher** (existant depuis java 7) comporte une unique méthode "*Boolean matches(Path path)*". Elle peut donc être utilisée au sein de "lambda expression" depuis java 8.

Exemple (fonctionnant depuis java8) :

...
....

4. Classe utilitaire Files ("*helper*" avec 50 méthodes statiques)

4.1. Vérifications sur fichiers ou répertoires

Méthode	Rôle
boolean exists (Path)	vérifier l'existence sur le système de fichiers de l'élément dont le chemin est encapsulé dans le paramètre de type Path fourni
boolean notExists (Path)	
boolean isReadable (Path path)	peut être lu (droits en lecture) ?
boolean isWritable (Path path)	peut être modifié (droits en écriture) ?
boolean isHidden (Path path)	est caché ?
boolean isExecutable (Path path)	est exécutable ?
boolean isRegularFile (Path path)	est un fichier ?
boolean isDirectory (Path path)	est un répertoire ?
boolean isSymbolicLink (Path path)	est un lien symbolique ?
String probeContentType (Path path)	retourne type MIME d'un fichier (ex: "text/plain") (ou null si indéterminé). S'appuie par défaut sur l'OS sous jacent.

Exemple:

```
Path path = Paths.get("C:/tmp/aa/fl.txt");
if(Files.isRegularFile(path)){
    System.out.println(path + " est un chemin vers un fichier");
}
```

4.2. Création d'un fichier ou d'un répertoire

Méthode	Rôle ou bien exemple
Path createFile (Path path, FileAttribute<?>... attrs)	Créer un fichier dont le chemin est encapsulé par l'instance de type Path fournie en paramètre
Path createDirectory (Path dir, FileAttribute<?>... attrs)	Path monRepertoire = Paths.get("C:/temp/mon_repertoire"); Files.createDirectory (monRepertoire);
Path createDirectories (Path dir, FileAttribute<?>... attrs)	Créer d'un seul coup plusieurs niveaux de sous répertoire selon le chemin exprimé "C:/temp/ niveau1/niveau2/niv3 "
Path createTempDirectory (Path dir, String prefix , FileAttribute<?>... attrs) Path createTempDirectory (String prefix , FileAttribute<?>... attrs)	Créer un répertoire temporaire de nom " préfixe indiqué " + numéro_calculé_par_syst (ex : rep_1245643) au sein du répertoire indiqué ou (à défaut) au sein du répertoire système (par défaut) prévu pour les temporaires.
Path createTempFile (Path dir, String prefix , String suffix , FileAttribute<?>... attrs) Path createTempFile (String prefix , String suffix , FileAttribute<?>... attrs)	Créer un fichier temporaire de nom " préfixe indiqué " + numéro_calculé_par_syst + " suffixe indiqué " (ex : fic_1245643.txt) au sein du répertoire indiqué ou (à défaut) au sein du répertoire système (par défaut) prévu pour les temporaires. Le suffix peut éventuellement être à null .

Les **attributs** (de type *FileAttribute*) sont **facultatifs** (des valeurs par défaut existent).

Les attributs possibles seront étudiés ultérieurement .

NB : **createDirectory()** créer un seul niveau de sous répertoire à la fois (*contrairement à createDirectories*) .

Si le répertoire existe déjà , l'exception *FileAlreadyExists* est remontée.

Si le répertoire parent n'existe pas , *NoSuchFileException* est remontée.

4.3. Copie d'un fichier ou d'un répertoire

Méthode	Rôle / fonctionnalité
Path copy (Path source , Path target , CopyOption... options)	Copier un élément avec les options précisées (StandardCopyOption.REPLACE_EXISTING , StandardCopyOption.COPY_ATTRIBUTES)

long copy (<i>InputStream</i> in, <i>Path</i> target, CopyOption... options)	Copier tous les octets d'un flux de type <i>InputStream</i> vers un fichier
long copy (<i>Path</i> source, <i>OutputStream</i> out)	Copier tous les octets d'un fichier dans un flux de type <i>OutputStream</i>

NB : L'option pointue `LinkOption.NOFOLLOW_LINKS` permet de recopier si besoin un lien symbolique au bout du path indiqué (plutôt que de suivre le lien).

NB2: il est possible d'utiliser la méthode `copy()` sur un répertoire . Cependant, le répertoire sera créé sans que les fichiers contenus et .. ne soient eux aussi copiés .

Quoi que contienne le répertoire, la méthode `copy` ne crée qu'un répertoire vide. Pour copier le contenu du répertoire, il faut parcourir son contenu et copier chacun des éléments un par un.

4.4. Déplacement et suppression d'un fichier ou d'un répertoire

Méthode	Rôle
move (<i>Path</i> source, <i>Path</i> target, CopyOption... options)	Déplacer ou renommer un élément avec les options précisées (<code>StandardCopyOption.REPLACE_EXISTING</code> , <code>StandardCopyOption.ATOMIC_MOVE</code>)
void delete (<i>Path</i> path)	Supprimer un élément du système de fichiers (avec exception s'il n'existe pas ou si répertoire pas vide)
boolean deleteIfExists (<i>Path</i> path)	Supprimer un élément du système de fichiers s'il existe (sans exception s'il existe pas)

Si un déplacement efficace/performant demandé en mode "ATOMIC_MOVE" est impossible (par exemple déplacement de "`c:/repXy`" vers "`d:/repXy`") , une exception est alors remontée. On peut alors éventuellement ré-essayer sans l'option .

Des exceptions peuvent potentiellement remonter si un répertoire à déplacer n'est pas vide et que certains fichiers contenus sont en cours d'utilisation.

5. Parcours des éléments d'un répertoire

La solution de parcours proposée par NIO2 est plus performante que `java.io.File.list(...)` .

La méthode `newDirectoryStream()` de la classe utilitaire **Files** attend en paramètre un objet de type *Path* qui correspond au répertoire à parcourir et permet d'obtenir une instance de "**stream**" de type **DirectoryStream<Path>** (à parcourir avec un itérateur ou autre) .

Attention: il est très important d'invoquer la méthode `close()` de l'instance de type **DirectoryStream** pour libérer les ressources utilisées.

Exemple :

```
Path tmpAaPath = Paths.get("C:/tmp/aa");
DirectoryStream<Path> stream = null;
try {
    stream=Files.newDirectoryStream(tmpAaPath);
    Iterator<Path> iterator = stream.iterator();
```

```

while(iterator.hasNext()) {
    Path p = iterator.next();
    System.out.println(p);
}
catch(IOException ex){    ex.printStackTrace();
}
finally {
    try {stream.close();
    } catch (IOException e) {e.printStackTrace();
    }
}

```

Exemple amélioré et simplifié :

```

Path tmpAaPath = Paths.get("C:/tmp/aa");

//NB1 : le second paramètre facultatif de Files.newDirectoryStream()
// sert à filtrer les éléments à parcourir (sans besoin de préfixe glob:)

//NB2: L'interface DirectoryStream implémente hérite de Closable et
//le try(avec ressource implémentant interface Closable)
//sera automatiquement associé à un finally implicite déclenchant .close()

try (DirectoryStream<Path> stream = Files.newDirectoryStream(tmpAaPath, "*.txt")){
    for(Path p : stream){
        System.out.println(p);
    }
}
catch(IOException ex){
    ex.printStackTrace();
}

```

On peut également paramétrer et utiliser un filtre spécifique lors du parcours :

```

Path tmpAaPath = Paths.get("C:/tmp/aa");
DirectoryStream.Filter<Path> littleSizeFilter = new DirectoryStream.Filter<Path>() {
    public static final long MEGABYTE = 1024*1024;
    @Override
    public boolean accept(Path element) throws IOException {
        return (Files.size(element) <= MEGABYTE );
    }
}; //fin de classe anonyme imbriquée implémentant DirectoryStream.Filter<Path>

try (DirectoryStream<Path> stream = Files.newDirectoryStream(tmpAaPath, littleSizeFilter)){
    for(Path p : stream){
        System.out.println(p);
    }
}

```

NB : ce code (java7) pourra être amélioré/simplifié via une lambda expression de java8 .

6. Parcours d'une hiérarchie de répertoires (visiteur)

La méthode **Files.walkFileTree()** permet de parcourir une (sous-)arborescence de répertoires en utilisant le **design pattern "visiteur"**. Ce type de parcours peut être utilisé pour rechercher, copier, déplacer, supprimer, ... des éléments de la hiérarchie parcourue.

Il faut écrire une classe qui implémente l'interface **java.nio.file.FileVisitor<T>**. Cette interface définit des méthodes qui seront des callbacks appelées lors du parcours de la hiérarchie.

Méthode	Rôle / fonctionnalité
FileVisitResult postVisitDirectory (T dir, IOException exc)	Le parcours sort d'un répertoire qui vient d'être parcouru ou une exception est survenue durant le parcours
FileVisitResult preVisitDirectory (T dir, BasicFileAttributes attrs)	Le parcours rencontre un répertoire, cette méthode est invoquée avant de parcourir son contenu
FileVisitResult visitFile (T file, BasicFileAttributes attrs)	Le parcours rencontre un fichier
FileVisitResult visitFileFailed (T file, IOException exc)	La visite d'un des fichiers durant le parcours n'est pas possible et une exception a été levée

Les méthodes de l'interface **FileVisitor** renvoient toutes une valeur qui appartient à l'énumération **FileVisitResult**. Cette valeur permet de contrôler le processus de parcours de l'arborescence :

- **CONTINUE** : poursuite du parcours
- **TERMINATE** : arrêt immédiat du parcours
- **SKIP_SUBTREE** : inhibe le parcours de la sous-arborescence.
- **SKIP_SIBLING** : inhibe le parcours des répertoires frères.

Exemple(s) à ajouter ici plus tard .

7. FileSystem (par défaut et "personnalisé")

7.1. Fabrique FileSystems

FileSystems est une **fabrique** (avec **méthodes statiques**) pour obtenir des objets **FileSystem**.

.getDefault() renvoie l'instance de type **FileSystem** qui encapsule le F.S. de la JVM.

.getFileSystem(fsUri) renvoie un **FileSystem** selon l'URI est fourni en paramètre.

.newFileSystem() surchargée permet de créer une instance spécifique de type **FileSystem** (cas pointu)

7.2. FileSystem

String separator = FileSystems.getDefault().**getSeparator()**; // / sous linux ou \ sous windows

```

Iterable<Path> dirs = FileSystems.getDefault().getRootDirectories();
for (Path name: dirs) {
    System.err.println(name); // C:\ , D:\ , ...
}

```

7.3. FileSystem spécifique

...

8. Lecture et écriture dans un fichier

8.1. Vue d'ensemble

Principaux apports de NIO et NIO2 :

IO	NIO	NIO2
Depuis Java 1.0 et 1.1	Depuis Java 1.4 (JSR 151)	Depuis Java 7 (JSR 203)
Synchrone bloquant	Synchrone non bloquant	Asynchrone non bloquant
File		Path
InputStream	FileChannel	AsynchronousFileChannel
OutputStream	SocketChannel	AsynchronousByteChannel
Reader (Java 1.1)	ServerSocketChannel	AsynchronousSocketChannel
Writer (Java 1.1)	(Charset, Selector, ByteBuffer)	AsynchronousServerSocketChannel
Socket		1
RandomAccessFile		SeekableByteChannel

8.2. Options sur l'ouverture d'un fichier

L'énumération **StandardOpenOption** implémente l'interface *OpenOption* et définit les options d'ouverture standard d'un fichier :

Valeur	Signification
APPEND	Si le fichier est ouvert en écriture alors les données sont ajoutées au fichier. Cette option doit être utilisée avec les options CREATE ou WRITE
CREATE	Créer un nouveau fichier s'il n'existe pas sinon le fichier est ouvert
CREATE_NEW	Créer un nouveau fichier : si le fichier existe déjà alors une exception est levée
DELETE_ON_CLOSE	Supprimer le fichier lorsque son flux associé est fermé : cette option

	est utile pour des fichiers temporaires
DSYNC	Demander l'écriture synchronisée des données dans le système de stockage sous-jacent (pas d'utilisation des tampons du système) ???
READ	Ouvrir le fichier en lecture
SPARSE	Indiquer au système que le fichier est clairsemé ce qui peut lui permettre de réaliser certaines optimisations si l'option est supportée par le système de fichiers (c'est notamment le cas avec NTFS)
SYNC	Demander l'écriture synchronisée des données et des métadonnées dans le système de stockage sous-jacent
TRUNCATE_EXISTING	Si le fichier existe et qu'il est ouvert en écriture alors il est vidé. Cette option doit être utilisée avec l'option WRITE
WRITE	Ouvrir le fichier en écriture

8.3. Lecture de l'intégralité d'un fichier / Files.readAllLines()

Lecture (en boucle) de toutes les lignes d'un fichier texte :

```
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
...
List<String> lignes = Files.readAllLines(
    FileSystems.getDefault().getPath("c:/tmp/aa/fl.txt"), StandardCharsets.UTF_8);
for (String ligne : lignes)
    System.out.println(ligne);
```

Lecture d'un bloc de tout un (petit) fichier binaire :

```
byte[] binaryContent = Files.readAllBytes(binaryFilePath);
```

XXXVII - Process Api, try with resource,Http2 api, ...

1. Process Api

1.1. Lancement d'un processus depuis une application java

```
public static void startNodePad() {
    ProcessBuilder builder = new ProcessBuilder("notepad.exe"); //de java.lang depuis java 1.5
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

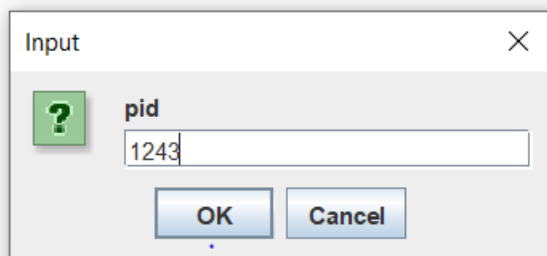
1.2. L'interface ProcessHandle (java.lang) depuis java 9

return type	method	fonctionnalités
static Stream < ProcessHandle >	allProcesses()	Returns a snapshot of all processes visible to the current process.
Stream < ProcessHandle >	children()	Returns a snapshot of the current direct children of the process.
int	compareTo (ProcessHandle other)	Compares this ProcessHandle with the specified ProcessHandle for order.
static ProcessHandle	current()	Returns a ProcessHandle for the current process.
Stream < ProcessHandle >	descendants()	Returns a snapshot of the descendants of the process.
boolean	destroy()	Requests the process to be killed.
boolean	destroyForcibly()	Requests the process to be killed forcibly.
boolean	equals (Object other)	Returns true if other object is non-null, is of the same implementation, and represents the same system process; otherwise it returns false.
int	hashCode()	Returns a hash code value for this ProcessHandle.
ProcessHandle.Info	info()	Returns a snapshot of information about the process.
boolean	isAlive()	Tests whether the process represented by this ProcessHandle is alive.

static Optional<ProcessHandle>	of(long pid)	Returns an Optional<ProcessHandle> for an existing native process.
CompletableFuture<ProcessHandle>	onExit()	Returns a CompletableFuture<ProcessHandle> for the termination of the process.
Optional<ProcessHandle>	parent()	Returns an Optional<ProcessHandle> for the parent process.
long	pid()	Returns the native process ID of the process.
boolean	supportsNormalTermination()	Returns true if the implementation of destroy() normally terminates the process.

1.3. Exemples de gestion de processus

```
import java.io.IOException;
import java.util.Optional;
import javax.swing.JOptionPane;
....
public static void destroyProcessById() {
    long pid=Long.parseLong(JOptionPane.showInputDialog(null, "pid"));
    Optional<ProcessHandle> optionalProcessHandle = ProcessHandle.of(pid);
    optionalProcessHandle.ifPresent(processHandle -> processHandle.destroy());
}
```



```
public static void startAndDestroyProcess() {
    //ProcessBuilder builder = new ProcessBuilder("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
    ProcessBuilder builder = new ProcessBuilder("notepad.exe");
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
        Thread.sleep(5000); //5000ms
        process.destroy();
        if (process.isAlive()) {
            System.out.println("process still alive ...");
            process.destroyForcibly(); //a utiliser que si process.destroy() ne suffit pas
        }else {
            System.out.println("process is no more alive ...");
        }
    }
```

```

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

...
public static String processHandleAsString(ProcessHandle ph) {
    return "pid="+ph.pid()+" - "+ph.info().toString();
}

public static void displayingAllProcess() {
    ProcessHandle.allProcesses().forEach(ph -> System.out.println(processHandleAsString(ph)));
}
-->

```

```

...
pid=21964 - [user: Optional[LAPTOP-DDC\didier], cmd: C:\Program Files (x86)\Google\Chrome\Application\chrome.exe,
startTime: Optional[2020-07-02T14:04:20.296Z], totalTime: Optional[PT0.078125S]]
pid=4840 - []
pid=5696 - [user: Optional[LAPTOP-DDC\didier], cmd: C:\Program Files\Java\jdk-11.0.4\bin\javaw.exe, startTime: Optional[2020-
07-02T14:47:38.376Z], totalTime: Optional[PT0.34375S]]

```

```

package org.mycontrib.tp;
public class MySubProcess {
    public static void main(String[] args) {
        System.out.println("MySubProcess ...");
        try { Thread.sleep(5000); /*5000ms*/ } catch (InterruptedException e) { e.printStackTrace(); }
        System.exit(0);
    }
}

```

```

public static void startAndWaitingProcess() {
    ProcessBuilder builder = new ProcessBuilder();
    /*
    final String javaHome = "C:\\Program Files\\Java\\jdk-11.0.4" ; // " " need around directory name with space
    final String javaCmdeWithQuote = "\"" + javaHome + "\\bin\\java.exe" + "\"";
    //final String cmde = javaCmdeWithQuote + " -version";
    final String cmde = javaCmdeWithQuote + " -cp .\\target\\myProcessApp.jar org.mycontrib.tp.MySubProcess";
    System.out.println("cmde="+cmde);
    //builder.command("cmd.exe", "/c", cmde); // ok , "/c" = terminate after this run
    */
    //NB: .\\target\\myProcessApp.jar will be ready after mvn install or ...
    builder.command("java", "-cp", ".\\target\\myProcessApp.jar",
        "org.mycontrib.tp.MySubProcess"); //ok with java in PATH
    boolean isStopped = false;
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
        BufferedReader outputReaderOfSubProcess = new BufferedReader(new
            InputStreamReader(process.getInputStream()));
        System.out.println("output of process (first line) =" +
            outputReaderOfSubProcess.readLine());
    }
}

```

```

        isStopped=process.waitFor(3, TimeUnit.SECONDS);
        if(isStopped) {
            System.out.println("process is terminated");
        }else {
            System.out.println("process not terminated after 3s");
            isStopped=process.waitFor(3, TimeUnit.SECONDS);//re-wait
        }
        if (isStopped) {
            System.out.println("process is stopped with exit value="
                               +process.exitValue());
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

2. try with resources (AutoClosable)

Ce code opérationnel

```

public void lireFichierWithScanner(String fileName) {
    Scanner scanner = null;
    try {
        //scanner = new Scanner(new File(fileName));
        scanner = new Scanner(Paths.get(fileName));
        while(scanner.hasNext()) {
            String ligne = scanner.nextLine();
            System.out.println(ligne) ;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    finally {
        if (scanner != null) {
            scanner.close();
        }
    }
}

```

peut se simplifier en

```

public void lireFichierWithScannerWithTryWithResources(String fileName) {
    //try (Scanner scanner = new Scanner(new File(fileName)) ){
    try (Scanner scanner = new Scanner(Path.of(fileName)) ){
        while(scanner.hasNext()) {
            String ligne = scanner.nextLine();
            System.out.println(ligne) ;
        }
    }
}

```

```

    } catch (IOException e) {
        e.printStackTrace();
    }
    //Automatic finally with call
    //of .close() of AutoCloseable interface implementation in Scanner class
}

```

Variante possible depuis java9 :

```

final Scanner scanner = new Scanner(new File(fileName)) ;
try (scanner){ ...}
catch(...) { ... }

```

try/catch avec plusieurs ressources :

```

try (Scanner scanner = new Scanner(new File("testRead.txt")) ;
    PrintWriter writer = new PrintWriter(new File("testWrite.txt"))) {
    while (scanner.hasNext()) {
        writer.print(scanner.nextLine());
    }
}

```

Avec ordre d'appels des .close() = inverse de l'ordre dans try(...)

Ressource personnalisée compatible avec "try with resources" :

```

public class MyResource implements AutoCloseable {
    @Override
    public void close() throws Exception {
        System.out.println("Close MyResource");
    }
}

```

Principales classes ou Interfaces Java ayant déjà implémenté l'interface AutoCloseable :

- Scanner
- PrintWriter et beaucoup d'autres Writer
- Reader , InputStreamReader , BufferedReader,
- InputStream , FileInputStream , ObjectInputStream, ...
- OutputStream , FileOutputStream , DataOutputStream , ObjectOutputStream,
- Pipe, Socket , ...
- Connection, Statement , ResultSet , RowSet de jdbc

3. Http2 Client

3.1. Préambule sur le protocole HTTP/2 :

La version **1.1** du protocole Http date de **1997**.

La version 2 du protocole Http a été standardisée en **2015** (en s'inspirant du protocole expérimental SPDY de google commencé en 2009)

La plupart des navigateurs web sont aujourd'hui compatible avec **http/2** .

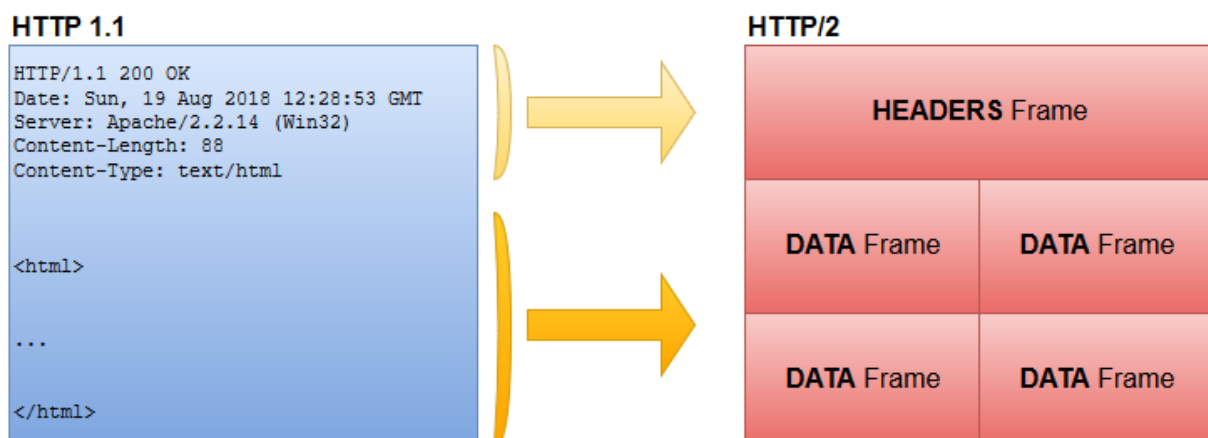
HTTP/2 conserve une rétrocompatibilité complète avec HTTP 1.1 . Mêmes syntaxes .

http/1 échange la plupart des données en mode texte.

http/2 échange (en interne) les données en mode binaire (même si du côté "client" et "serveur" une vision "mode texte" est toujours de mise pour les types MIMES "text/html" , ...)

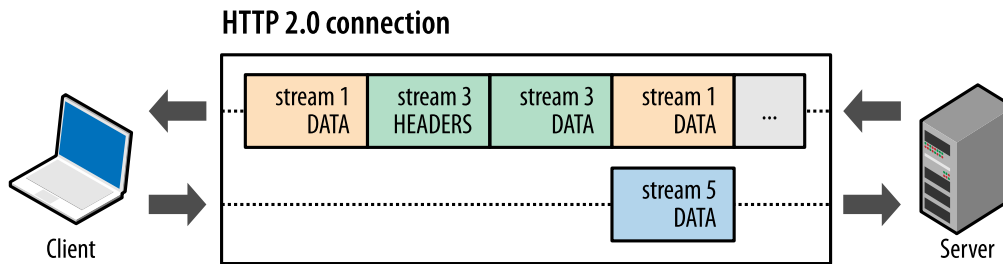
Une connexion http/1 ne pouvait récupérer qu'une seule ressource à la fois . Malgré la réutilisation de connexion (keep-alive) , et l'éventuelle parallélisation des connexions tcp/ip vers un même domaine (limitées à 6) , un chargement d'un grand nombre de ressources via http est plutôt séquentiel en http/1 et peut donc prendre beaucoup de temps.

Une connexion **http/2** peut être utilisée en mode "**multiplexage de flux**" et on peut ainsi récupérer plusieurs ressources en parallèle . Ceci permet de beaucoup améliorer la vitesse de certains téléchargement .



En interne, le protocole **http/2** véhicule des données dans des blocs binaires appelées "**Frames**" (avec parties "**header**" et "**data**") .

Plusieurs "frames" peuvent ainsi transitées (éventuellement dans les 2 sens en mode "bi-directionnel" au sein d'une même connexion http/2 . Les "frames" peuvent éventuellement être priorisées .



NB : au sein d'une même connexion HTTP2 établie, le client et le serveur peuvent échanger (dans les 2 sens) **des frames** (HEADERS + DATA) comme s'il s'agissait de requêtes **indépendantes** .

Chaque frame (HEADERS ou DATA) comporte un attribut important **.streamId** .

Ceci permet au client de comprendre qu'une frame de réponse est associée à telle frame de requête (par analyse de **corrélation** du **stream ID**) .

Le protocole **http/2** autorise également un mode **push** (où le serveur peut prendre l'initiative de renvoyer spontanément certaines pièces attachées par encore demandées) .

Le protocole **http/2** est capable d'effectuer automatiquement **une compression** (en binaire optimisé) **des entêtes HTTP** ce qui permet d'**optimiser la bande passante** .

Pour approfondir --> <https://oxiane.developpez.com/tutoriels/java/http2-java/>

3.2. Présentation de l'api java HttpClient

La nouvelle API **HTTP Client** (en pré-version "incubator" en v9 puis standard depuis v 11) propose **un support des versions 1.1 et 2 du protocole HTTP** ainsi que les WebSockets côté client.

Cette **api** (beaucoup plus moderne que l'ancienne classe `HttpURLConnection` datant du `jdk 1.1`) **peut fonctionner en mode asynchrone** et permet (si nécessaire) l'utilisation de l'API `Flow` (reactive streams) pour fournir les données du body d'une requête (`Flow.Publisher`) et consommer le body d'une réponse (`Flow.Subscriber`).

L'API est contenue dans le package **`java.net.http`** du module `java.net.http`. Elle contient plusieurs types dont les principaux sont **`HttpClient`** , **`HttpClient.Builder`** , **`HttpRequest`** , **`HttpRequest.Builder`** , **`HttpRequest.BodyPublisher`** , **`HttpRequest.BodyPublishers`** , **`HttpResponse`** , `HttpResponse.ResponseInfo` , **`HttpResponse.BodySubscriber`** , **`HttpResponse.BodySubscribers`** , `HttpResponse.BodyHandler` , `HttpResponse.BodyHandlers` .

Généralités : Un **`XxxBuilder`** sert à construire une instance immuable de type **`Xxx`**

Les **`XxxYyyys`** sont des fabriques de **`XxxYyy`**

Un "**`Handler`**" sert à décortiquer la réponse (et la récupérer dans un certain type) en s'aidant des informations d'entêtes trouvées dans `HttpResponse.ResponseInfo`

3.3. Exemples avec api HttpClient

Exemple en mode synchrone :

```
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpResponse.BodyHandlers;
...
public static void test_new_http2_client_since_java9_standard_since_java11() {
    HttpClient client = HttpClient.newHttpClient();

    HttpRequest req =
        HttpRequest.newBuilder(URI.create("http://www.google.com"))
            .header("User-Agent", "Java")
            .GET()
            .build();

    try {
        HttpResponse<String> resp = client.send(req, BodyHandlers.ofString());
        System.out.println("reponse status:" + resp.statusCode()); //200 for ok
        System.out.println("reponse uri:" + resp.uri().toString()); //http://www.google.com
        System.out.println("reponse type:" + resp.headers().map().get("Content-Type"));
            //--> [text/html; charset=ISO-8859-1]
        System.out.println("reponse text:" + resp.body()); //texte html
    } catch (IOException e) {
        e.printStackTrace();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

```
HttpClient client = HttpClient.newHttpClient(); //Http2 par défaut si possible
```

ou bien

```
HttpClient httpClient = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_1_1)
    //.proxy(ProxySelector.of(new InetSocketAddress("proxy.xyz.com", 80)))
    .connectTimeout(Duration.ofSeconds(10))
    .build();
```

Préparation d'une requête à envoyer en mode POST :

```
HttpRequest requetePost = HttpRequest.newBuilder()
```

```
.uri(URI.create("https://www.xyz.com/api-xyz"))
.setHeader("Content-Type", "application/json")
.POST(BodyPublishers.ofString(dataAsJsonString))
.build();
```

Quelques Variantes :

```
.POST(HttpRequest.BodyProcessor.fromByteArray(sampleData))
```

ou encore

```
.POST(HttpRequest.BodyProcessor.fromFile(
    Paths.get("src/test/resources/sample.json")))
```

Variante du premier exemple en **mode asynchrone** :

```
..
//En mode asynchrone , sendAsync retournant un CompletableFuture :
client.sendAsync(req, BodyHandlers.ofString())
    .thenAccept(resp -> {
        System.out.println("recuperation réponse asynchrone / interpreted by "
            + Thread.currentThread().getName());
        System.out.println("reponse status:" + resp.statusCode());
        System.out.println("reponse uri:" + resp.uri().toString());
        System.out.println("reponse type:" + resp.headers().map().get("Content-Type"));
        System.out.println("reponse text size:" + resp.body().length());
    });

System.out.println("suite synchrone interpreted by " + Thread.currentThread().getName());
try { Thread.sleep(2000); //pause ici pour eviter arrêt complet du programme
    // avant la fin des taches de fond asynchrones
} catch (InterruptedException e) { e.printStackTrace(); }
System.out.println("fin synchrone / interpreted by " + Thread.currentThread().getName());
```

Résultats :

```
suite synchrone interpreted by main
recuperation réponse asynchrone / interpreted by ForkJoinPool.commonPool-worker-3
reponse status:200
reponse uri:http://www.google.com
reponse type:[text/html; charset=ISO-8859-1]
reponse text size:12791
fin synchrone / interpreted by main
```

Avec subscriber/reactive stream (pour gérer réponse) :

```

public static void test_new_httpClient_withSubscriber() {
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest req =
        HttpRequest.newBuilder(URI.create("http://www.google.com"))
            .header("User-Agent", "Java").GET().build();
    MySimplePrintSubscriber subscriber1 = new MySimplePrintSubscriber();
    //NB ; MySimplePrintSubscriber est une implémentation simple de Flow.Subscriber<Object>

    // BodyHandlers.fromLineSubscriber(s) for registering a subscriber that will receive response text line by line
    // the return CompletableFuture contains no body but status and other infos
    client.sendAsync(req, BodyHandlers.fromLineSubscriber(subscriber1))
        .thenApply(HttpResponse::statusCode) //extract statusCode from httpResponse
        .thenAccept((status) -> {
            if (status != 200) { System.err.printf("ERROR: %d status %n", status); }
        });
    System.out.println("suite synchrone interpreted by " + Thread.currentThread().getName());
    try {
        Thread.sleep(2000); //pause ici pour éviter arrêt complet du programme
        // avant la fin des taches de fond asynchrones
    } catch (InterruptedException e) { e.printStackTrace(); }
    System.out.println("fin synchrone / interpreted by " + Thread.currentThread().getName());
}

```

Résultat de type :

```

mySimplePrintSubscriber>> Received onNext(item) with item=ligne1
mySimplePrintSubscriber>> Received onNext(item) with item=ligne2
mySimplePrintSubscriber>> Received onNext(item) with item=ligne3

```

XXXVIII - Communication par messages / JMS

1. Présentation de JMS

JMS (Java Message Service)

JMS est une API permettant de faire **dialoguer des applications** de façon **asynchrone**.

Architecture associée: **MOM** (Message Oriented MiddleWare).

NB: JMS n'est qu'une API qui sert à accéder à un véritable fournisseur de Files de messages (ex: MQSeries/Websphere_MQ d'IBM, ActiveMQ d'apache, ...)

Dans la terminologie JMS, les Clients JMS sont des programmes Java qui envoient et reçoivent des messages dans/depuis une file (**message queue**).

Une file de message sera gérée par un "Provider JMS".

Les clients utiliseront **JNDI** pour accéder à une file.

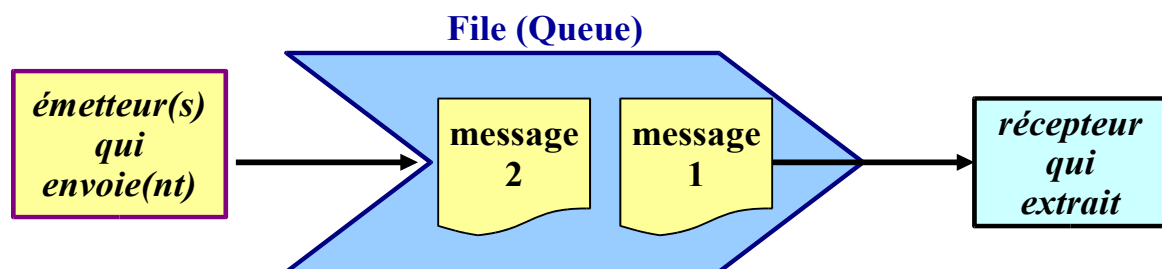
L'objet **ConnectionFactory** sera utilisé pour établir une connexion avec une file.

L'objet **Destination** (File ou Topic) sert à préciser la destination d'un message que l'on envoie ou bien la source d'un message que l'on souhaite récupérer.

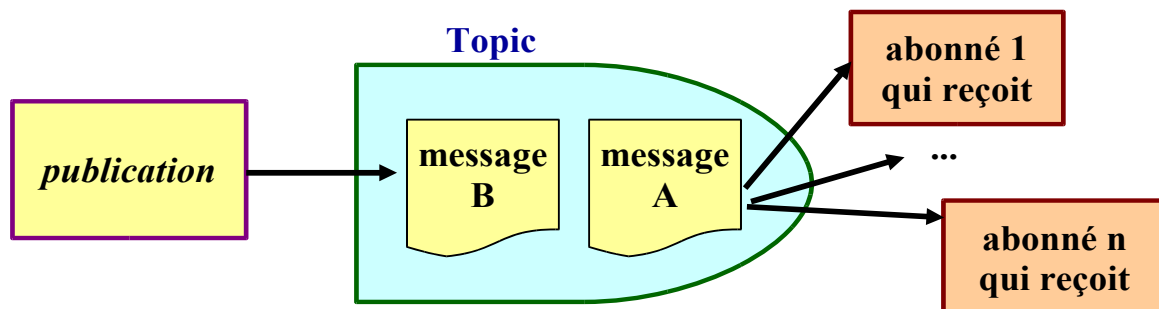
JMS permet de mettre en oeuvre les 2 modèles suivants:

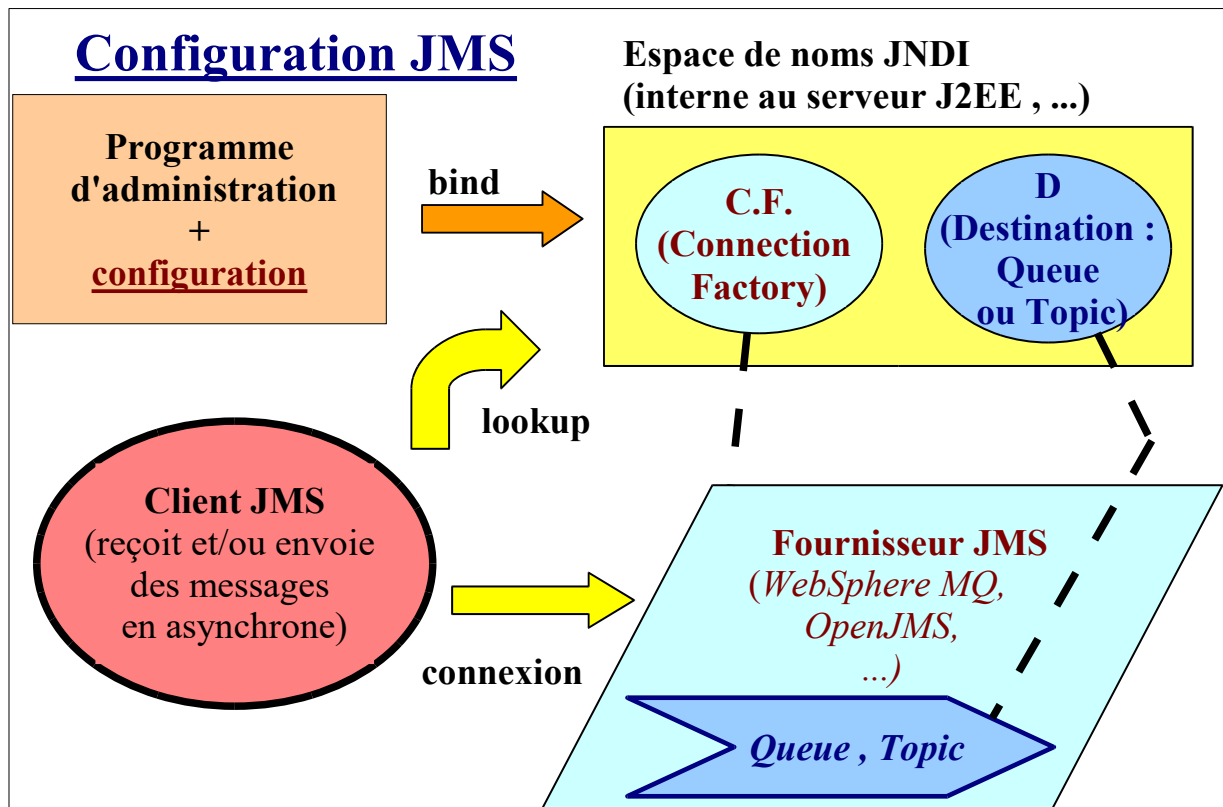
- **PTP** (Point To Point)
- **Pub/Sub** (Published & Subscribe) .../...

JMS Queue : Point To Point



JMS Topic : Publish / Subscribe





2. Queue & Topic

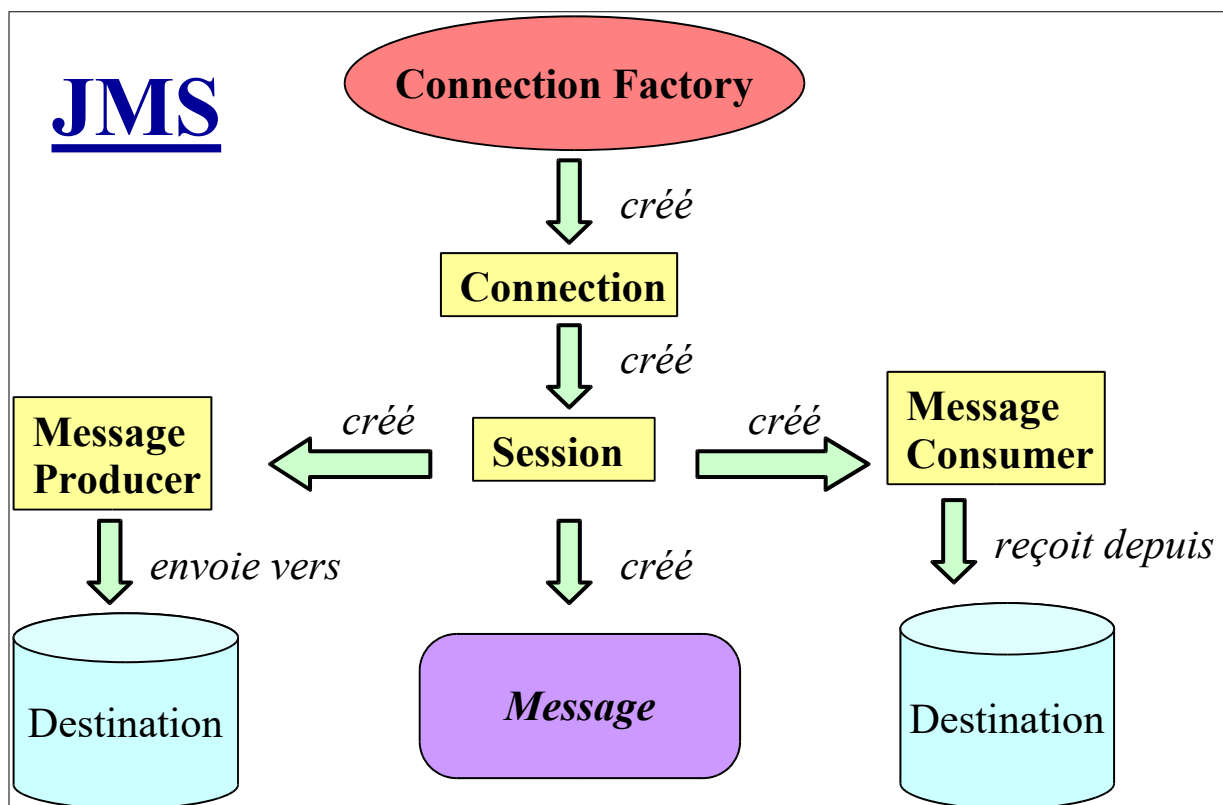
2 modèles de destination :

- **PTP** (Point To Point) - *File de message spécifique à une entité (comparable à une boîte à lettres).*
- **Pub/Sub** (Published & Subscribe) - *Les clients et les serveurs utilisent une même file logique dont le contenu est organisé de façon hiérarchique. On publiera des messages au niveau d'un certain noeud et ceux qui se sont inscrits (abonnés) vis à vis de ce noeud recevront alors ces messages.*

Le tableau ci-dessous résume les différentes **interfaces** utilisées au niveau de l'api JMS:

JSM (Interface générique)	PTP Domain	Pub/Sub Domain
ConnectionFactory (mt)	QueueConnectionFactory	TopicConnectionFactory
Connection (mt)	QueueConnection	TopicConnection
Destination (mt)	Queue	Topic
Session	QueueSession	TopicSession
MessageProducer	QueueSender	TopicPublisher
MessageConsumer	QueueReceiver, QueueBrowser	TopicSubscriber

(mt) : multi-threading support.



3. Exemples (fragments) de code en JMS 1.1

3.1. Obtention de l'objet ConnectionFactory via JNDI:

```

javax.jms.ConnectionFactory connectionFactory;
Properties props = new Properties();
//props.put("...", "...") au cas par cas selon le contexte (activeMq, Artemis, ...)
Context ic = new javax.naming.InitialContext(props);
connectionFactory = (ConnectionFactory) ic.lookup("ConnectionFactory");

```

3.2. Eventuelle obtention d'une file de message via JNDI:

```

Queue stockQueue;
stockQueue = (Queue) ic.lookup("StockQueue");
ou bien

```

```

Topic stockTopic;
stockTopic = (Topic) ic.lookup("StockTopic");

```

Attention: un lookup JNDI ne fonctionne généralement bien que si la file ou le topic a déjà été préparé/créé coté server/broker et il faut quelquefois ajouter un préfixe (ex: "jms/" ou "queues/" ou "dynamicQueues/" ou autre).

3.3. Création d'un objet Connection via la fabrique:

```

Connection connection = connectionFactory.createConnection();

```

3.4. Création d'une Session à partir de l'objet Connexion:

```

Session session= connection.createSession(false /*trasact. */,
Session.AUTO_ACKNOWLEDGE);

```

3.5. Eventuelle obtention ou création dynamique de file :

```

String queueName = "queue.req_xyz";
//String queueName = "queue.resp_xyz";
Queue queue=session.createQueue(queueName);
//NB: createQueue() create a new queue or open an existing one

```

3.6. Obtention de l'objet "MessageProducer" pour envois

```
MessageProducer messageProducer = session.createProducer(queue_or_topic);
...
messageProducer.close();
```

3.7. Obtention de l'objet "MessageConsumer" pour réceptions

```
MessageConsumer messageConsumer = session.createConsumer(queue);
et
long receivingTimeout = 1000; //1000ms = 1s
Message message = messageConsumer.receive(receivingTimeout); //return null if no message
// Message message = messageConsumer.receiveNoWait();
```

ou bien

```
XYZListener xyzListener= new XYZListener(...);
messageConsumer.setMessageListener(xyzListener);
    avec
    public class XYZListener implements javax.jms.MessageListener {
        void onMessage(Message message) {
            //unpack and handle the messages we receive.
        }
    }
```

3.8. Déclencher le début possible des réceptions de messages:

```
connection.start(); //don't forget to call .start() for receiving message !!!
```

3.9. Création de messages

Création d'un message binaire:

```
byte[] stockData; // stock information as a byte array
BytesMessage message;
message = session.createByteMessage();
message.writeBytes(stockData);
```

Création d'un message en mode texte:

```
String stockData; // information as a String (ex: jsonString built via jackson-databind)
TextMessage message;
message = session.createTextMessage();
message.setText(stockData);
```

Création d'un message structuré (avec différents champs nommés):

```
MapMessage message;
message = session.createMapMessage();
```



```
message.setString("Name", stockName); message.setDouble("Value", stockValue);
message.setLong("Time", stockTime); message.setDouble("Diff", stockDiff);
message.setString("Info", stockInfo);
```

Création d'un message à données à lire séquentiellement:

```
StreamMessage message;
message = session.createStreamMessage();
message.writeString(stockName); ...message.writeDouble(stockValue);
message.writeLong(stockTime); ...
```

Création d'un message contenant les valeurs d'un objet Java (Sérialisation):

```
ObjectMessage message = session.createObjectMessage();
message.setObject(stockObject);
```

3.10. Envoi et réception

```
messageProducer.send(message);
```

et

```
Message message = messageConsumer.receiveNoWait(); //or .receive(timeout)
```

ou ...

3.11. Extraction des valeurs d'un message

Extraction des valeurs d'un message binaire:

```
byte[] stockData; // stock information as a byte array
int length;
length = message.readBytes(stockData);
```

Extractions des valeurs d'un message texte:

```
String stockData;
stockData = message.getText();
```

Extractions des valeurs d'un message structuré:

```
stockName = message.getString("Name");
stockValue = message.getDouble("Value");
stockTime = message.getLong("Time");
stockDiff = message.getDouble("Diff");
stockInfo = message.getString("Info");
```

Extractions des valeurs d'un message à valeurs séquentielles:

```
stockName = message.readString(); stockValue = message.readDouble();
```

```
stockTime = message.readLong(); stockDiff = message.readDouble();
stockInfo = message.readString();
```

Extractions des valeurs d'un message objet:

```
stockObject = message.getObject();
```

3.12. Filtrage éventuel des messages que l'on souhaite récupérer:

```
String selector;
selector = new String("(name = 'SUNW') OR (name = 'IBM')");
```

et

```
MessageConsumer messageConsumer = session.createConsumer(queue, selector);
```

Nb: le package **javax.jms** est quelquefois intégré dans **j2ee**

4. Champs des entêtes de message

Champ de l'entête	signification	fixé (affecté) par
JMSDestination	File de destination	méthode send()
JMSDeliveryMode	PERSISTENT ou NON_PERSISTENT	méthode send()
JMSExpiration	0 : pas d'expiration. sinon <i>n ms</i> à vivre.	méthode send()
JMSPriority	priorité de 0 à 9 (0-4: normal) (5-9: high)	méthode send()
JMSMessageID	ID:xxx identifiant du message	méthode send()
JMSTimestamp	estampillage de temps	méthode send()
JMSCorrelationID	identifiant de la requête associée à la réponse	Client
JMSReplyTo	File où il faut placer la réponse.	Client
JMSType	selon le contexte , catégorie , ...	Client
JMSRedelivered	si réception multiple d'un même message	Provider

Le champ **JMSReplyTo** peut comporter le nom d'une file (éventuellement temporaire) que l'émetteur de la requête a préalablement créé pour récupérer la réponse..

5. Acquittement des messages reçus.

Principe de base (*lié aux mécanismes internes*): Il faut **acquitter un message une fois consommé pour ne pas le recevoir une nouvelle fois** .

mode d'acquittement	caractéristiques
Session.CLIENT_ACKNOWLEDGE	il faut appeler explicitement message.acknowledgde() .
Session.AUTO_ACKNOWLEDGE	automatique en fin de OnMessage ou en fin de receive()
Session.DUPS_OK_ACKNOWLEDGDE	Si recevoir un message dupliqué n'est pas 1 pb

Nb: L'acquittement ne revient pas vers l'émetteur . Il n'est pris en compte que par les mécanismes internes qui doivent normalement garantir qu'un même message (identifié via un ID) envoyés plusieurs fois ou bien dupliqué ne sera reçu qu'une seule fois.

6. Liste des principaux "Provider JMS"

Parmi les implémentations les plus connues, on peut citer:

- **MQSeries** (→ renommé **WebSphere_MQ**) d' IBM
- **RabbitMQ** (deux versions : *community et commerciale*)
- compatibilité de "**Kafka**" avec l'api JMS
- Services "JMS" intégrés dans les serveurs JEE (JBoss, WebLogic, WebSphere).

Produits (FreeWare - OpenSource):

- ~~OpenJMS (dernière version: 2007)~~
- **ActiveMQ classic** (version open source de référence d'Apache Software)
- **ActiveMQ Artemis** (version plus élaborée/récente , maintenant gérée par Apache Software)
- **Joram** de OW2
- ...

7. ActiveMQ / Artemis

7.1. Présentation de ActiveMQ / Artemis

ActiveMQ est la principale implémentation d'un serveur **JMS** mise en œuvre par le groupe **Apache**.

ActiveMQ classic correspond à l'implémentation historique du groupe open source apache.

ActiveMQ Artemis est une implémentation plus récente (d'origine Jboss) qui a été reprise par le groupe apache .

7.2. Agent (broker) de ActiveMQ / Artemis

<https://artemis.apache.org/components/artemis/download/>

→ téléchargement de apache-artemis-2.52.0-bin.zip et extraction de l'archive

create_my_broker.bat (à lancer dans *MY_BROKER_HOME=C:\Prog\artemis-brokers\mybroker*)

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set ARTEMIS_HOME=C:\Prog\apache-artemis-2.52.0
%ARTEMIS_HOME%\bin\artemis create mybroker
REM admin/admin
```

run_my_broker.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set ARTEMIS_HOME=C:\Prog\apache-artemis-2.52.0
set MY_BROKER_HOME=C:\Prog\artemis-brokers\mybroker
%MY_BROKER_HOME%\bin\artemis run
```

L'URL de la console d'administration du broker ActiveMq/Artemis est

<http://localhost:8161/console> avec admin/admin ou autre comme login.

L'URL par défaut pour se connecter au "broker ActiveMQ" depuis l'api JMS en java est **"tcp://localhost:61616"**

Pour éventuellement créer (de manière anticipée) une file d'attente au niveau du broker on peut :

- 1) cliquer sur mybroker\bin\artemis.cmd
- 2) lancer la commande
queue create --name=my_queue
- 3) faire quelques choix (par exemple Y pour --anycast , Y pour --durable , ...)

7.3. Connexion (via JNDI) à ActiveMQ/Artemis.

```
import javax.jms.*; import javax.naming.*; import java.util.Properties;

ConnectionFactory connectionFactory = null;
Properties props = new Properties();
props.put("java.naming.factory.initial",
"org.apache.activemq.artemis.jndi.ActiveMQInitialContextFactory");
//5445 is port number for hornetq/artemis , 61616 is port number for actimemq/artemis
//props.put(Context.PROVIDER_URL, "tcp://localhost:5445");
props.put(Context.PROVIDER_URL, "tcp://localhost:61616?type=CF");
props.put(Context.SECURITY_PRINCIPAL, "admin");//username , "..."
props.put(Context.SECURITY_CREDENTIALS, "admin");//"pwd", "..."

InitialContext ic = new InitialContext(props);
connectionFactory = (ConnectionFactory) ic.lookup("ConnectionFactory");
Connection connection = connectionFactory.createConnection();
Session session = connection.createSession(false, Session.AUTO_ACKNOWLEDGE);
```

7.4. Exemple de Code JMS

Exemple de code complémentaire pour envoyer des messages :

```
Queue queue=session.createQueue("queue.req_xyz");
MessageProducer messageProducer = session.createProducer(queue);
TextMessage message = session.createTextMessage();
message.setText("text of message ...");
System.out.println("Sending message: " + message.getText());
messageProducer.send(message);
messageProducer.close(); session.close(); connection.close();
//+try/catch
```

Exemple de code complémentaire pour recevoir des messages :

```
String queueName = "queue.resp_xyz";
Queue queue=session.createQueue(queueName);
MessageConsumer messageConsumer = session.createConsumer(queue);
connection.start(); //don't forget to call .start() for receiving message !!!

long receivingTimeout = 1000; //1000ms = 1s
Message message = messageConsumer.receive(receivingTimeout);
while(message != null) {
    if(message instanceof TextMessage txtMessage) {
        System.out.println("received message=" + txtMessage.getText());
    }
    message = messageConsumer.receiveNoWait();
}
System.out.println("no more message to retrieve in =" + queueName);
messageConsumer.close(); session.close(); connection.close();
//+try/catch
```

Variante avec listener :

```
...
XyzListener xyzListener= new XyzListener(session);
messageConsumer.setMessageListener(xyzListener);
connection.start(); //don't forget to call .start() for receiving message !!!
while(!stopServ) {
    Thread.sleep(1000 * 15); //15s
    //do to: set stopServ to true , v1 : server always running
}
messageConsumer.close(); ....
```

et

```
public class XyzListener implements MessageListener { ...
    public XyzListener(Session session){ this.session=session; }

    public void sendXyzInResponseQueue(Xyz xyz){
        try {Queue queue=session.createQueue("queue.resp_xyz"); ...} catch(...) {...}
    }

    @Override
    public void onMessage(Message message) {
        try { TextMessage textMessage = (TextMessage) message;
            String xyzRequestText = textMessage.getText();
            Xyz xyz = ... //build response
            sendXyzInResponseQueue(xyz);
        } catch(Exception ex){ ex.printStackTrace(); }
    }
}
```


XXXIX - Annexe – énoncés des Tps (java avancé)

1. TP sur les threads (java avancé)

1.1. Création et gestion de Threads

Programmer une petite application qui va démarrer et lancer 2 threads en parallèle :

- un premier thread va afficher un par un les nombres pairs de 0 à n (ex : ≤ 25)
- un second thread va afficher un par un les nombres impairs de 1 à n (ex : ≤ 25)

NB : on pourra structurer l'application avec une classe CounterApp (avec méthode main() qui va créer et démarrer les threads) et une classe CounterTaskV1 implémentant Runnable et qui va compter de 2 en 2 en partant de 0 ou de 1 (selon booléen "even/pair" du constructeur).

Exemple de résultat attendu :

```
0 2 4 6 8 10 1 3 5 7 9 11 13 15 17 19 21 23 25 12 14 16 18 20 22 24
```

affichage mélangé des nombres pairs et impairs

1.2. Accès concurrents avec ou sans synchronized

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules `tp/tp_java_avance` , sous module `exos_threads` .

La classe **MyNumberConcurrentListV1WithBugs** est sensée démarrer trois threads en parallèle :

- un premier thread va générer (environ toutes les 2ms) 10 nombres aléatoires et les placer dans une liste "numbers"
- un second thread va régulièrement (environ toutes les 2ms) extraire, retirer et afficher les nombres pairs disponibles au sein de la liste "numbers"
- un troisième thread va régulièrement (environ toutes les 2ms) extraire, retirer et afficher les nombres impairs disponibles au sein de la liste "numbers"

Résultats attendus (après correction des bugs) :

```
extracted even numbers : []
extracted odd numbers : []
extracted even numbers : [234, 120, 116, 80]
extracted even numbers : [82, 94, 192, 52, 254, 134]
extracted odd numbers : [173, 133, 241, 97, 179, 121, 229, 41, 33, 9]
extracted even numbers : []
extracted odd numbers : [71, 213, 185, 243, 43, 215, 183]
extracted even numbers : [30, 44, 204]
extracted even numbers : [52, 2, 206, 88, 198]
extracted odd numbers : [73, 93, 199, 57, 99]
extracted odd numbers : []
extracted even numbers : [122, 38, 190, 218, 214, 122, 242, 160]
extracted odd numbers : [217, 119, 161, 233, 19]
...
```


Gérer l'exception `ConcurrentModificationException` avec le pattern "collect & removeAll after loop" et en introduisant `synchronized(...)` { ... } aux endroits appropriés .

1.3. Gestion de la concurrence (avec sémaphore ou autres)

Reprendre le premier exercice (avec `CounterApp`) .

Créer une version améliorée *MyCounterTaskV2* qui va **alterner les affichages des valeurs impaires et paires** . On pourra par exemple se baser sur des **sémaphores** *readyEvenMutexAsSemaphore* et *readyOddMutexAsSemaphore* et des sous méthodes *waitOurTurn()* et *notifyYourTurn()* .

Exemple de résultat attendu :

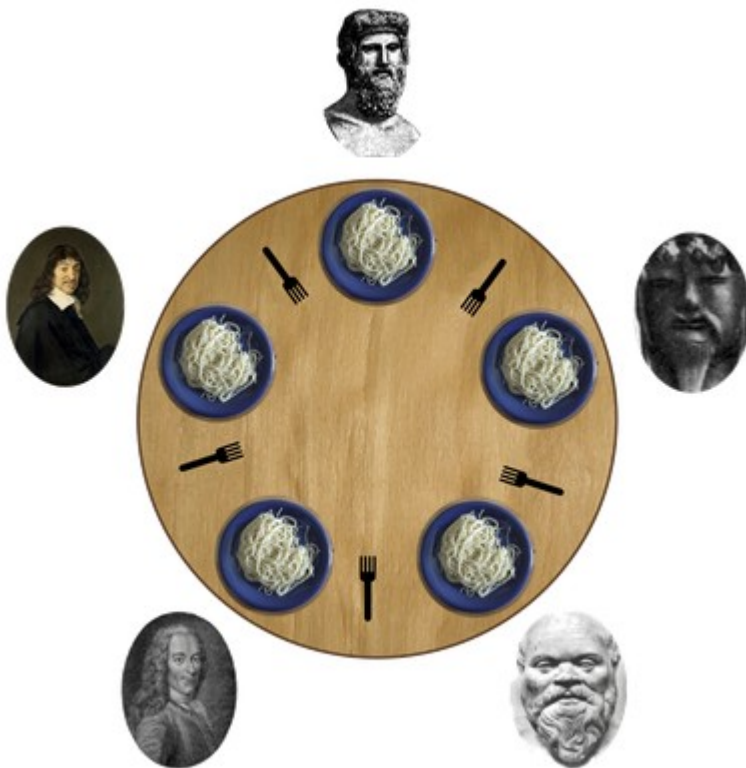
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

1.4. Gestion des DeadLocks (dîner des philosophes)

Référentiel https://github.com/didier-tp/java_avance ,

projet java/maven multi-modules *tp/tp_java_avance* , sous module *exos_threads* .

Problème classique "dîner des philosophes" pour montrer un cas de "blocage mutuel" / "deadlock" et une manière de résoudre le problème .



N(ex:5) philosophes sont dans un cycle :

- penser (en un temps fini et potentiellement un peu aléatoire)
- avoir faim et attendre deux fourchettes libres (à gauche puis à droite)
- manger (en un temps fini et potentiellement un peu aléatoire)

- rendre les fourchettes

Code de départ :

DinerPhilosophesApp.main() avec PhilosopherTaskWithDeadLock

A faire en Tp :

- dupliquer PhilosopherTaskWithDeadLock en PhilosopherTaskWithoutDeadLock
- utiliser PhilosopherTaskWithoutDeadLock dans DinerPhilosophesApp.main()
- ajouter dans le code de PhilosopherTaskWithoutDeadLock un nouveau sémaphore (ex : roomSemaphore) de taille initiale N-1 et ajouter au sein de la méthode run :
 - * une acquisition d'un jeton de "roomSemaphore" pour pouvoir rentrer dans la salle (réfectoire) de manière à pouvoir ensuite tenter de prendre des fourchettes et manger
 - * une restitution d'un jeton dans "roomSemaphore" après "repas pris" et "fourchettes rendues"

Solution :

DinerPhilosophesApp.main() avec PhilosopherTask

1.5. Monitoring , détection et correction de deadlock

- 1) Refaire fonctionner l'application DinerPhilosophesApp avec PhilosopherTaskWithDeadLock
- 2) lancer **jps** et **jstack** (ou autre) pour visualiser les 5 threads (Thread-0 , ... Thread-4) bloqués en attentes mutuelles
- 3) dupliquer PhilosopherTaskWithDeadLock en PhilosopherTaskWithoutDeadLockV2 et utiliser PhilosopherTaskWithoutDeadLockV2 dans DinerPhilosophesApp.main()
- 4) au lieu d'utiliser un sémaphore complémentaire "roomSemaphore" , tenter de résoudre le problème du "deadlock" via tryLock() soit via **tryAcquire()** avec des sémaphores.

1.6. Etude de cas "marché boursier"

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules tp/tp_java_avance , sous module market_core .

Point d'entrée du code : tp.market.app.BasicMarketAppV1

...

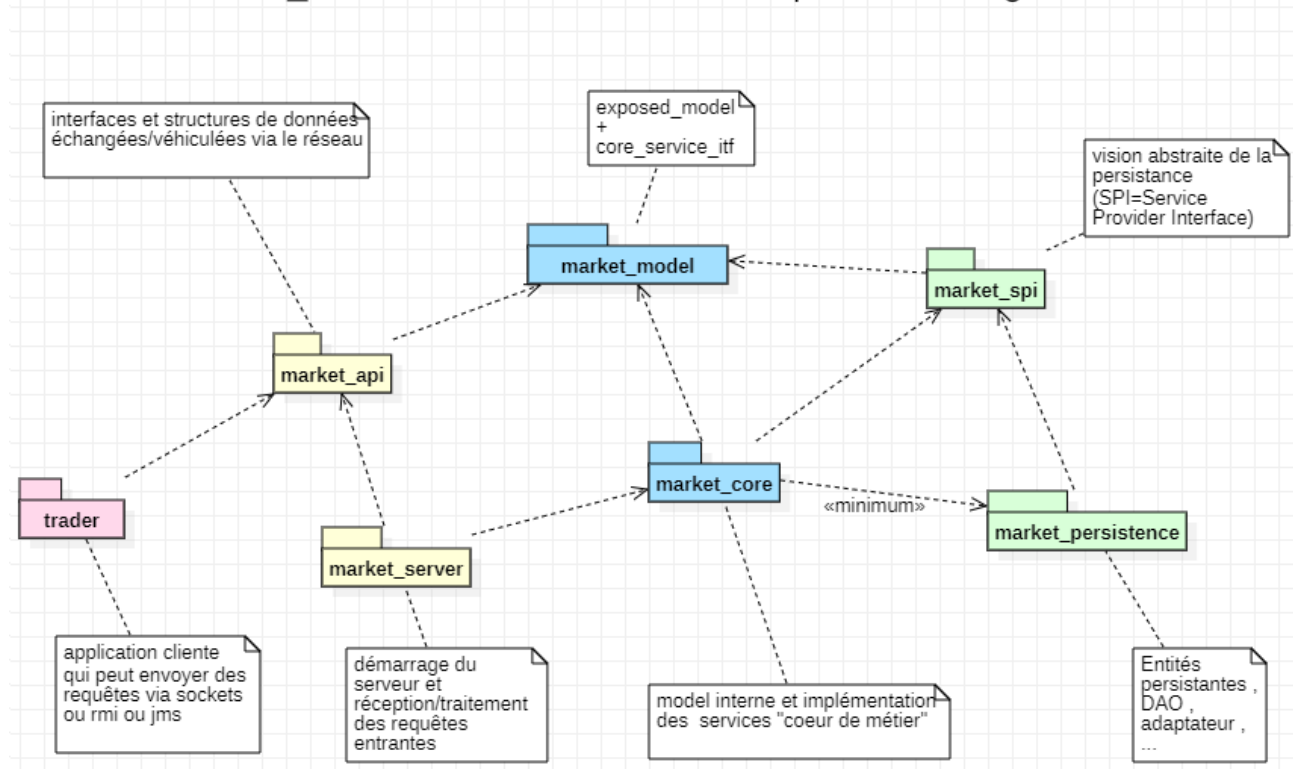
2. TP sur les communications réseau (java avancé)

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules **tp/tp_java_avance**

Structure du projet (à compléter) :

market_core et market_model	Coeur de métier (services et données internes en partie exposés)
market_api	Sous projet avec structures de données communes entre les cotés "serveur" et "client"
market_server	Serveur (place de marché boursier)
trader	Application cliente vis à vis du server "market"

Stock_market en mode "architecture pseudo-hexagonale"



Point d'entrée du code (coté marker_server):

tp.market.my_server.**MyBasicMarketServerAppV2** (début de TP à compléter)
tp.market.server.BasicMarketServerAppV2 (solution possible du TP)

But commun des Tps (aussi bien en version "socket" , "RMI" ou "JMS") :

En s'appuyant principalement sur le sous projet "market_api" , coder les cotés serveur et client de manière à pouvoir :

- envoyer une requête de type "récupérer liste des actions/stocks" .
- récupérer la réponse du coté application cliente "trader"

- générer des ordres d'opérations boursières au sein de l'application "trader" en s'appuyant sur `tp.market.core.service.OrderServiceImpl(List<Stock> stockList)`
- envoyer du client "trader" vers "market_server" un paquet d'ordre (via une boucle adéquate)
- appliquer les ordres reçus du côté "market_server" en déléguant "sur partie modele/core /".

2.1. Communications via sockets

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorSocketTask` avec `MyBasicMarketServerAppV2.main()` qui déclenche `my_server.my_task.MyOrderCollectorTaskSocketV2` (démarrage de la partie "serveur_socket") qui délègue lui même à `my_server.my_task.MyOrderCollectorSocketTask` la gestion des requêtes.
- solution possible dans `tp.market.server`

Conseils :

On pourra s'appuyer sur les classes du package `tp.market.message` du sous projet "market_api"

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderSocketClient`
- solution possible dans `tp.market.trader.TraderSocketClient`

2.2. Communications via RPC/RMI

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorTaskRmiV2` avec `MyBasicMarketServerAppV2.main()` qui déclenche `my_server.my_task. MyOrderCollectorTaskRmiV2` (démarrage de la partie "serveur_rmi")
- solution possible dans `tp.market.server`

Conseils :

- interfaces "remote" dans `tp.market.rmi_api` du projet "market_api"
- implémentations prévues : classes dans `tp.market.my_server.my_rmi` du projet "market"

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderRmiClient`
- solution possible dans `tp.market.trader.TraderRmiClient`

2.3. Communications asynchrones via JMS

Coté serveur (projet "market_server") :

- votre code à coder et placer dans `tp.market.my_server.my_task.MyOrderCollectorTaskJmsV2` avec `MyBasicMarketServerAppV2.main()` qui déclenche `my_server.my_task. MyOrderCollectorTaskJmsV2` (démarrage de la partie "serveur_jms")
- solution possible dans `tp.market.server`

Conseils :

- les valeurs des messages JMS (en mode texte) pourront éventuellement être au format JSON en s'appuyant sur `tp.market.util.MyJsonUtil` du sous projet "market_model"
- les types fonctionnels des messages JMS échangés pourront éventuellement être basés sur `tp.market.jms.MarketMessage` et véhiculés via `message.get_or_setStringProperty(..)`

Coté client (projet "trader") :

- votre code à coder et placer dans `tp.market.my_trader.MyTraderJmsClient`
- solution possible dans `tp.market.trader.TraderJmsClient`

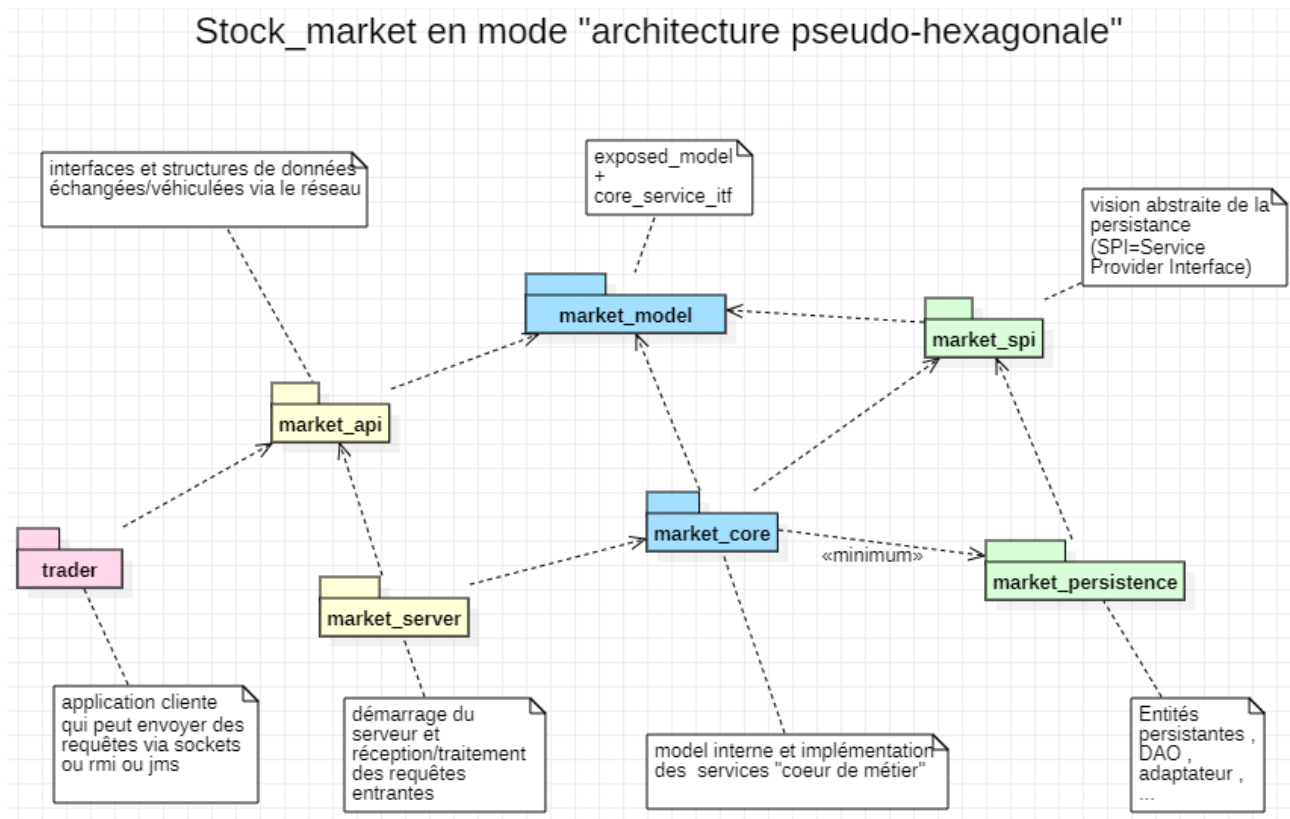
3. TP sur la persistance (JPA/Hibernate) (java avancé)

Référentiel https://github.com/didier-tp/java_avance ,
projet java/maven multi-modules **tp/tp_java_avance**

Structure du projet (à compléter) :

market_spi	Vision abstraite de la persistance vue par le coeur de métier (SPI, ...)
market_core et market_model	Coeur de métier (services et données internes en partie exposés)
market_persistence	Lien avec la base de données (code basé sur jpa/hibernate)

Stock_market en mode "architecture pseudo-hexagonale"



Point d'entrée du code des TPs:

projet "**market_persistence**" (début de TP à compléter)

3.1. Analyse de la configuration java/maven de JPA/Hibernate

* Repérer toutes les parties "jpa/hibernate" dans **pom.xml** du projet "market_persistence" (ainsi que le driver jdbc "mysql-connector-j").

* Visualiser le contenu de src/main/resources/META-INF/persistence.xml et si besoin adaptez ce fichier (mot de passe , url , ...)

* Au sein d'une application java d'entreprise basée sur le framework SpringBoot ou sur le framework JakartaEE , JPA/Hibernate est déjà bien intégré (par exemple via @PersistenceContext et @Transactional).

Dans le projet "market_persistence" (volontairement basé que sur le coeur du langage java) , on n'a pas de transaction automatique ni d'ouverture/fermeture automatique du "EntityManager" et l'on pourra s'appuyer sur la classe tp.market.persistence.jpa.**MyJpaUtil** pour intégrer JPA/hibernate dans notre application java .

* Visualiser (en diagonale) le contenu de src/main/java/tp.market.persistence.generic.**GenericDAO** et **GenericDaoJpa** .

=====

Via le driver jdbc "mysql-connector-j" on pourra se connecter à une base de données **MariaDB** (plus facile à installer que MySQL).

Lors d'une éventuelle rapide installation de MariaDB , on pourra choisir root/root comme username/password .

3.2. Persistance élémentaire

* Compléter tp.market.persistence.my_entity.**MyStockEntity** avec avec @Entity , @Table , @Id , @Column , ...

NB : noms java en CamelCase et nom de tables/colonnes en snake_case à préciser via @Table et @Column .

* ajouter cette ligne (en version "My...") dans src/main/resources/META-INF/persistence.xml (sans enlever l'autre version "solution" sans "My") .

```
<class>tp.market.persistence.my_entity.MyStockEntity</class>
```

* Visualiser le contenu de src/main/java/tp.market.persistence.my_dao.**MyStockDAO** et **MyStockDaoJpa** .

* compléter le code de la classe src/main/test/tp.market.persistence.my_dao.**TestMyStockDAO**

* lancer ce test et corriger les éventuelles erreurs jusqu'à ce que les tests fonctionnent tous correctement.

3.3. @ManyToOne , @OneToMany , transactions et cascades

* Compléter `tp.market.persistence.my_entity.MyStockExchangeEntity` avec avec `@Entity` , `@Table` , `@Id` , `@Column` , ... et `@ManyToOne` , `@JoinColumn`

NB : noms java en CamelCase et nom de tables/colonnes en snake_case à préciser via `@Table` et `@Column` .

* ajouter cette ligne (en version "My...") dans `src/main/resources/META-INF/persistence.xml` (sans enlever l'autre version "solution" sans "My") .

```
<class>tp.market.persistence.my_entity.MyStockExchangeEntity</class>
```

* Compléter le code de `tp.market.persistence.my_entity.MyStockEntity` en y ajout la relation inverse (avec `@OneToMany`)

* Ajouter au sein de `src/main/java/tp.market.persistence.my_dao` l'interface `MyStockExchangeDAO` et la classe `MyStockExchangeDaoJpa` .

* compléter le code de la classe

`src/main/test/tp.market.persistence.my_dao.TestMyStockExchangeDAO`

* lancer ce test et corriger les éventuelles erreurs jusqu'à ce que les tests fonctionnent tous correctement.

* ajouter un déclenchement d'une suppression de la deuxième action (stock) dans la méthode de test.

* ajouter si besoin une **cascade** appropriée près de `@OneToMany` de manière à ce que la suppression de passe bien lors du déclenchement du test .

3.4. Requêtes (JPQL , api Criteria, LAZY, ...)

* Ajouter au sein de `my_dao.MyStockExchangeDAO` et `my_dao.MyStockExchangeDaoJpa` plein de variantes de la méthode de recherche des échanges associés à un `stockId` spécifique :

```
List<MyStockExchangeEntity> findByStockId(String stockId); //JPQL
```

```
List<MyStockExchangeEntity> findByCriteriaStockId(String stockId); //criteria api
```

```
List<MyExchangeEssential> findExchangeEssentialListByStockId(String stockId);
```

```
//with projection/aggregation
```

* tester toutes ces variantes (via changement de lignes en commentaire) au sein de la classe `my_dao.TestMyStockExchangeDAO`

* ajouter dans **MyStockDAO** et **MyStockDaoJpa** et coder une nouvelle méthode telle que

public MyStockEntity findByIdWithExchanges(String stockId)

retournant d'un coup une action et ses échanges rattachés

* ajouter dans **TestMyStockExchangeDAO** une nouvelle variante basée sur un appel de cette nouvelle méthode et tester plein de combinaisons fonctionnelles (EAGER , LAZY et SELECT avec LEFT JOIN FETCH) en privilégiant à la fin celle basée sur LAZY .

3.5. Sécurité et optimisations

* Ajouter au sein de `my_dao.MyStockExchangeDAO` et `my_dao.MyStockExchangeDaoJpa` plein de variantes de la méthode de recherche des échanges associés à un `stockId` spécifique :

```
List<MyStockExchangeEntity> findByStockId(String stockId); //good JPQL with setParameter()
```

```
List<MyStockExchangeEntity> badFindByStockIdWithSqlInjectionProblem(String stockId);
```

```
//wrong JPQL query without .setParameter()
```

Au sein de `TestMyStockExchangeDAO` tester la robustesse des différentes versions via des appels temporaires de ce genre :

```
List<MyStockExchangeEntity> exchanges =
stockExchangeDAO.badFindByStockIdWithSqlInjectionProblem(stockId1+" OR 1=1");
//avec injection SQL !!!
```

* Retirer temporairement le paramétrage `indexes = @Index(name="idx_isin", columnList = "isin", unique = true)` au sein de `@Table` dans la solution `entity.StockEntity`

et lancer un test

* Visualiser la structure de la table "stock" de la base de données via un utilitaire adéquat (ex : "HeidiSql" ou autre).

* ajouter `indexes = @Index(name="idx_isin", columnList = "isin", unique = true)` au sein au sein de `@Table` dans `my_entity.MyStockEntity` et relancer un test.

* Visualiser la nouvelle structure de la table "stock" de la base de données via un utilitaire adéquat (ex : "HeidiSql" ou autre) :

Sans index	Avec index

- * Coder la nouvelle méthode stockDAO.**findByIsin(...)** dans la partie my_dao
- * Appeler cette méthode via une petite variante de code au sein du my_dao.**TestMyStockDAO.testCrudStock()**
- * Repérer toutes les parties "**cache**" dans **pom.xml** du projet "market" ...
- * Visualiser le fichier src/main/resources/**ehcache.xml**
- * Visualiser les propriétés hibernate nécessaires au cache dans META-INF/persistence.xml
- * coder une nouvelle classe de test my_dao.MyTestCache avec une méthode de ce genre :

```
@Test
public void testSaveAndFindWithCache(){
    String pk="sxx.PA";
    stockDAO.saveNew(new MyStockEntity("Sxx" , pk , "FR12367890" , 3.03 ));
    String pk2="syy.PA";
    stockDAO.saveNew(new MyStockEntity("Syy" , pk2 , "FR1236780" , 4.03 ));
    long t1 = System.currentTimeMillis();
    MyStockEntity sRelu = stockDAO.findById(pk);
    long t2 = System.currentTimeMillis();
    Cache cache = emf.getCache();//2nd Level cache (shared by all users/threads/sessions/entityManager)
    boolean wasCached = cache.contains(StockEntity.class, pk);
    log.info("findById execution time (with or without cache)="+t2-t1));
    //ex: 12ms with cache , 35ms without cache
    log.info("sRelu="+sRelu);
    log.info("wasCached="+wasCached);
    assertEquals("Sxx",sRelu.getName());
    assertTrue(wasCached);
}
```

- * Lancer plusieurs fois ce test sans ou avec l'ajout de

```
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
dans my_entity.MyStockEntity
```

et visualiser d'éventuelles différences .

- * éventuelles autres expérimentations selon l'inspiration du moment ...

4. TP sur coeur langage java (fonctionnalités avancés)

NB : tous ces énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

4.1. TP sur réflexivité

Coder la méthode **generateLineOfCsv()** de la classe `tp.util.TpCsvGenerate` en s'appuyant sur la réflexivité du langage java.

Phase 1 : le fichier .csv (ou bien une de ses lignes) comportera tous les champs existants dans la classe java (ex : `Product` ou `Person`).

Phase 2 : on effectuera le test "est que le champ comporte l'annotation `@CsvIgnore` ?" et si oui ce champ ne sera pas placé dans la sortie (de type `String`) au format csv .

Tester tout cela via `TpCsvGenerate.main()` .

Type de résultat attendu (à la console) pour la phase 1 :

```
id;firstName;lastName;email;age;taille
```

```
1;jean;Bon;jb@gmail.com;40;180
```

```
-----
```

```
id;label;price;features
```

```
1;prod 1;123.0;Bon produit
```

Fichier `products.csv` (généré par défaut à la racine des projets) pour la phase 2 :

```
id;label;price
```

```
1;printer z1;156.8
```

```
2;usb wire;6.8
```

```
3;computer x2;976.5
```

```
4;webcam;96.83
```

```
5;computer y6;896.83
```

5. TP sur fonctionnalités récentes (java avancé)

NB : tous ces énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

5.1. Eventuelles révisions sur lambda et streams

Si besoin de réviser lambda et stream , coder une partie d'application qui va enchaîner filtrage , tri et transformation sur une liste d'objets structurés de type Produit.

Point de départ du code (à compléter) :

tp.avec_lambda.**AppGestionProduitsAvecStream**
en s'appuyant tp.data.**Product** et tp.util.**ProductUtil**

5.2. Tp sur principales syntaxes récentes (switch , yield , record , textBloc , ...)

Compléter le code de **tp.j15_16_17.SyntheseTestApp** en utilisant au maximum les nouvelles syntaxes des versions 8,9,10,11,12,13,14,15,16,17 de java

Phase 1 :

switch/case avec lambda expression au sein de **tp.j15_16_17.SyntheseTestApp**

Résultat attendu :

LIGNE
RECTANGLE
CERCLE
LIGNE
TYPE DE FIGURE INCONNU

Phase2 :

switch/case avec lambda expression et yield au sein de **tp.j15_16_17.SyntheseTestApp**

Résultat attendu :

LIGNE
resTransformation=Line from Figure(type=line, x1=10, y1=10, x2=100, y2=100, color=red)
RECTANGLE
resTransformation=Rectangle from Figure(type=rectangle, x1=5, y1=5, x2=120, y2=120, color=blue)
CERCLE
resTransformation=Circle from Figure(type=circle, x1=40, y1=40, x2=80, y2=80, color=green)
LIGNE
resTransformation=Line from Figure(type=line, x1=10, y1=10, x2=100, y2=10, color=black)
TYPE DE FIGURE INCONNU

resTransformation=Unknown from Figure(type=ellipse, x1=10, y1=10, x2=100, y2=30, color=orange)

Phase3 :

1. lire les **indications** dans les méthodes figureTo...() de **tp.util.FigureUtil**
2. **coder** les **...Record** nécessaires au sein de **tp.j15_16_17.Dto**
3. **coder les méthodes figureTo...() de tp.util.FigureUtil** en utilisant le mot clef var si besoin de variables locales
4. relancer **tp.j15_16_17.SyntheseTestApp.main()**

Résultat attendu :

LIGNE

resTransformation=LineRecord[x1=10, y1=10, x2=100, y2=100, lineColor=red]

RECTANGLE

resTransformation=RectangleRecord[x1=5, y1=5, largeur=115, hauteur=115, fillColor=blue]

CERCLE

resTransformation=CircleRecord[xc=60, yc=60, radius=28, fillColor=green]

LIGNE

resTransformation=LineRecord[x1=10, y1=10, x2=100, y2=10, lineColor=black]

TYPE DE FIGURE INCONNU

resTransformation=FigureRecord[type=ellipse, x1=10, y1=10, x2=100, y2=30, color=orange]

Phase4 :

- lire les **indications** dans les méthodes figureTo...() de **tp.j15_16_17.Dto**
- au niveau de chaque record , redéfinir la méthode toString() en s'appuyant sur les **TextBlocs** et de manière à générer des chaines de caractères au format JSON.
- **relancer** **tp.j15_16_17.SyntheseTestApp.main()**

résultat attendu :

LIGNE

```
resTransformation={
  "x1" : 10,
  "y1" : 10,
  "x2" : 100,
  "y2" : 100,
  "lineColor" : "red"
}
```

RECTANGLE

```
resTransformation={
  "x1" : 5,
  "y1" : 5,
  "largeur" : 115,
  "hauteur" : 115,
  "fillColor" : "blue"
}
```

CERCLE

```
resTransformation={
```

```
"xc" : 60,
"yc" : 60,
"radius" : 28,
"fillColor" : "green"
}
```

LIGNE

```
resTransformation={
  "x1" : 10,
  "y1" : 10,
  "x2" : 100,
  "y2" : 10,
  "lineColor" : "black"
}
```

TYPE DE FIGURE INCONNU

```
resTransformation={
  "type" : "ellipse",
  "x1" : 10,
  "y1" : 10,
  "x2" : 100,
  "y2" : 30,
  "color" : "orange"
}
```

5.3. Exo synthèse (HttpClient + Record + Json/Jackson , ...)

Compléter le code de la classe `tp.j15_16_17.HttpJsonRecordTestApp` selon les indications suivantes :

Phase 1 :

Via l'api **HttpClient** , appeler en mode **GET** le web service REST dont l'URL est

https://catfact.ninja/fact

Ce WS facile à appeler (sans api_key) retourne une réponse au format **JSON** de type
{ "fact" : "un fait sur les chats" , "length" : 21_ou_autre }

On pourra effectuer l'appel en mode synchrone ou bien en asynchrone via
 client.sendAsync(req, BodyHandlers.ofString()).thenAccept(resp -> ...);
 si mode asynchrone choisie prévoir pause d'attente en fin du main()

Phase 2 :

Via une structure de code de ce type :

```
client.sendAsync(req, BodyHandlers.ofString())
    .thenApply(resp -> {
        System.out.println(.....);
        System.out.println(.....);
        return resp.body();
    })
    .thenApply(*****lambda transformant jsonString en Dto.CatFact*****)
    .thenAccept(*****lambda affichant le record de type Dto.CatFact*****);
```

on pourra:

- transformer la réponse du format "jsonString" vers le format instance de **DTO.CatFact** (record à coder avec propriétés .fact et .length) en s'appuyant sur l'api jackson databind en version >= 2.12.5 gérant bien les "record"
catFact = jacksonObjectMapper.readValue(catFactAsString, CatFact.class);
- afficher finalement les valeurs du DTO/record java construit viatoString()

Exemple de résultats affichés dans la console:

suite synchrone interpreted by main

recuperation reponse asynchrone / interpreted by ForkJoinPool.commonPool-worker-1

reponse status:200

reponse uri:https://catfact.ninja/fact

reponse type:[application/json]

reponse text:{"fact":"Approximately 1/3 of cat owners think their pets are able to read their minds.", "length":78}

catFact as java record:CatFact[fact=Approximately 1/3 of cat owners think their pets are able to read their minds., length=78]

fin synchrone / interpreted by main

6. TP sur Tests,Mock,Profiling,... (java avancé)

NB : la plupart des énoncés de TP sont basés sur le point de départ suivant :

sous projet "exos_java" de tp/tp_java_avance (https://github.com/didier-tp/java_avance)

6.1. Test unitaire avec assertions

Compléter le code de la classe src/test/java/tp.data.TestPersonne

Partie facultative du Tp :

Compléter le code de la classe src/test/java/tp.calculs.TestParametreCalculsFinanciers en s'appuyant sur @ParameterizedTest et @CsvSource

6.2. Suite de tests

Compléter le code de la classe src/test/java/tp.calculs.CalculsSuite de manière à lancer TestCalculsSimple , TestEmptySerie et TestSerie

Lancer cette suite de tests depuis un IDE (ex : eclipse ou intelliJ)

Lancer si possible (si le temps le permet) cette suite de tests via maven ou via un script (.bat ou .sh ou autre).

6.3. Mockito

- Charger le sous projet "with_mockito" de tp/tp_java_avance (https://github.com/didier-tp/java_avance) au sein d'un IDE java (ex : eclipse ou intelliJ) .
- Analyser et faire tourner l'exemple `tp.service.TestServiceDevise` .
- Au sein du package `tp.emprunt` s'inspirer de `TestServiceEmpruntSansMock` pour coder **`TestServiceEmpruntAvecMock`**

NB : coder idéalement 2 versions/variantes :

- une variante sans annotations de Mockito
- une seconde variante avec quelques annotations de Mockito

6.4. Vérification de la couverture de code

Installer le plugin maven jaCoCo et s'en servir pour mesurer la couverture des tests

6.5. Test de charge avec JMeter (tp facultatif)

6.6. Profiling

Simulation de fuite mémoire, interprétation de métriques et amélioration ciblée.

